

Enterprise Transport API Java Edition 3.10.1.L1

DEVELOPERS GUIDE

Document Version: 3.10.1.L1
Date of issue: September 2026
Document ID: ETAJ3101L1UM.260



LSEG DATA &
ANALYTICS

© **LSEG 2015 - 2026**. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any software, including but not limited to: the code, screen, structure, sequence, and organization thereof, and its documentation are protected by national copyright laws and international treaty provisions. This manual is subject to U.S. and other national export regulations.

LSEG Data & Analytics, by publishing this document, does not guarantee that any information contained herein is and will remain accurate or that use of the information will ensure correct and faultless operation of the relevant service or equipment. LSEG Data & Analytics, its agents, and its employees, shall not be held liable to or through any user for any loss or damage whatsoever resulting from reliance on the information contained herein.

Contents

1	Introduction	1
1.1	About this Manual	1
1.2	Audience	1
1.3	Programming Language.....	1
1.4	Acronyms and Abbreviations	1
1.5	References	3
1.6	Documentation Feedback	3
1.7	Document Conventions.....	3
1.7.1	<i>Typographic</i>	3
1.7.2	<i>Diagrams</i>	4
2	Product Description	5
2.1	What is the Enterprise Transport API?	5
2.2	Enterprise Transport API Features	7
2.2.1	<i>General Capabilities</i>	7
2.2.2	<i>Consumer Applications</i>	7
2.2.3	<i>Provider Applications: Interactive</i>	7
2.2.4	<i>Provider Applications: Non-Interactive</i>	8
2.3	Performance and Feature Comparison	8
2.3.1	<i>Java Garbage</i>	9
2.3.2	<i>Use of Assertions</i>	9
2.4	Functionality: Which API to Choose?	10
3	Consumers and Providers	14
3.1	Overview	14
3.2	Consumers	15
3.2.1	<i>Subscriptions: Request/Response</i>	16
3.2.2	<i>Batches</i>	16
3.2.3	<i>Views</i>	17
3.2.4	<i>Pause and Resume</i>	18
3.2.5	<i>Symbol Lists</i>	19
3.2.6	<i>Posting</i>	21
3.2.7	<i>Generic Message</i>	22
3.2.8	<i>Private Streams</i>	23
3.2.9	<i>Update Filtering</i>	24
3.3	Providers	25
3.3.1	<i>Interactive Providers</i>	26
3.3.2	<i>Non-Interactive Providers</i>	27
4	System View	29
4.1	System Architecture Overview	29
4.2	LSEG Real-Time Advanced Distribution Server	30
4.3	LSEG Real-Time Advanced Distribution Hub	31
4.4	Elektron	32
4.5	Data Feed Direct.....	33
4.6	Internet Connectivity via HTTP and HTTPS.....	34
4.7	Direct Connect	35
5	Model and Package Overviews	36
5.1	Enterprise Transport API Models	36

5.1.1	<i>Open Message Model</i>	36
5.1.2	<i>Rssl Wire Format</i>	36
5.1.3	<i>Domain Message Model</i>	36
5.2	Packages	37
5.2.1	<i>Transport Package</i>	37
5.2.2	<i>Codec Package</i>	37
6	Building an Open Message Model Consumer	38
6.1	Overview	38
6.2	Establish Network Communication	38
6.3	Perform Login Process.....	38
6.4	Obtain Source Directory Information	39
6.5	Load or Download Necessary Dictionary Information	39
6.6	Issue Requests, Forward Generic Messages, and/or Post Information.....	40
6.7	Log Out and Shut Down.....	40
6.8	Additional Consumer Details.....	40
7	Building an Open Message Model Interactive Provider	41
7.1	Overview	41
7.2	Establish Network Communication	41
7.3	Perform Login Process.....	41
7.4	Provide Source Directory Information	42
7.5	Provide or Download Necessary Dictionaries	42
7.6	Handle Requests and Post Messages	43
7.7	Dispatch Round Trip Time Messages	43
7.8	Disconnect Consumers and Shut Down	43
7.9	Additional Interactive Provider Details	43
8	Building an Open Message Model Non-Interactive Provider	45
8.1	Overview	45
8.2	Establish Network Communication	45
8.3	Perform Login Process.....	45
8.4	Perform Dictionary Download	46
8.5	Provide Source Directory Information	46
8.6	Provide Content	46
8.7	Log Out and Shut Down.....	47
8.8	Additional Non-Interactive Provider Details.....	47
9	Encoding and Decoding Conventions	48
9.1	Concepts	48
9.1.1	<i>Data Types</i>	48
9.1.2	<i>Composite Pattern and Nesting</i>	49
9.2	Encoding Semantics	50
9.2.1	<i>Init and Complete Suffixes</i>	50
9.2.2	<i>The Encode Iterator: EncoderIterator</i>	50
9.2.3	<i>Content Roll Back with Example</i>	52
9.3	Decoding Semantics	53
9.3.1	<i>The Decode Iterator: DecoderIterator</i>	53
9.3.2	<i>Functions for Use with DecoderIterator</i>	53
9.3.3	<i>DecoderIterator: Basic Use Example</i>	54
9.4	Return Code Values	55
9.4.1	<i>Success Codes</i>	55
9.4.2	<i>Failure Codes</i>	56
9.4.3	<i>CodecReturnCodes Methods</i>	57

9.5	Versioning	58
9.5.1	<i>Protocol Versioning</i>	58
9.5.2	<i>Library Versioning</i>	59
10	Transport Package Detailed View.....	60
10.1	Concepts	60
10.1.1	<i>Transport Types</i>	60
10.1.2	<i>Channel Object</i>	61
10.1.3	<i>Server Object</i>	65
10.1.4	<i>Transport Error Handling</i>	67
10.1.5	<i>General Transport Return Codes</i>	68
10.1.6	<i>Application Lifecycle</i>	68
10.2	Initializing and Uninitializing the Transport.....	69
10.2.1	<i>Initialization and Uninitialization Methods</i>	69
10.2.2	<i>Initialization Arguments Methods</i>	69
10.2.3	<i>Initialization Reference Counting with Example</i>	70
10.2.4	<i>Transport Locking Models</i>	70
10.3	Creating the Connection	72
10.3.1	<i>Network Topologies</i>	72
10.3.2	<i>Creating the Outbound Connection: Transport.connect Method</i>	75
10.3.3	<i>Transport.connect Outbound Connection Creation Example</i>	81
10.3.4	<i>Tunneling Connection Keep Alive</i>	82
10.4	Server Creation and Accepting Connections	83
10.4.1	<i>Creating a Listening Socket</i>	83
10.4.2	<i>Accepting Connection Requests</i>	89
10.4.3	<i>Compression Support</i>	91
10.5	Channel Initialization	91
10.5.1	<i>Channel.init Method</i>	92
10.5.2	<i>InProgInfo Object</i>	92
10.5.3	<i>Calling Channel.init</i>	92
10.5.4	<i>Channel.init Return Codes</i>	93
10.5.5	<i>Channel.init Example</i>	93
10.6	Reading Data	95
10.6.1	<i>Channel.read Method</i>	95
10.6.2	<i>ReadFlags Values</i>	96
10.6.3	<i>Channel.read Return Codes</i>	96
10.6.4	<i>Channel.read Example</i>	98
10.7	Writing Data: Overview	100
10.8	Writing Data: Obtaining a Buffer	101
10.8.1	<i>Transport Buffer Management Channel Methods</i>	101
10.8.2	<i>Transport Buffer Management Server Method</i>	102
10.8.3	<i>Channel.getBuffer Return Values</i>	102
10.9	Writing Data to a Buffer.....	102
10.9.1	<i>Channel.write Method</i>	103
10.9.2	<i>WriteFlags Values</i>	103
10.9.3	<i>Compression</i>	104
10.9.4	<i>Fragmentation</i>	104
10.9.5	<i>Channel.write Return Codes</i>	104
10.9.6	<i>Channel.getBuffer and Channel.write Example</i>	106
10.10	Managing Outbound Queues	108
10.10.1	<i>Ordering Queued Data: WritePriorities</i>	108
10.10.2	<i>Channel.flush Method</i>	109
10.10.3	<i>Channel.flush Return Codes</i>	109
10.10.4	<i>Channel.flush Example</i>	110
10.11	Packing Additional Data into a Buffer.....	111

10.11.1	<i>Channel.packBuffer Return Values</i>	111
10.11.2	<i>Example: Channel.getBuffer, Channel.packBuffer, and Channel.write</i>	111
10.12	Ping Management	113
10.12.1	<i>Ping Timeout</i>	113
10.12.2	<i>Channel.ping Function</i>	114
10.12.3	<i>Channel.ping Return Values</i>	114
10.12.4	<i>Channel.ping Example</i>	114
10.13	Closing Connections	115
10.13.1	<i>Functions for Closing Connections</i>	115
10.13.2	<i>Close Connections Example</i>	116
10.14	Utility Methods	116
10.14.1	<i>General Transport Utility Methods</i>	116
10.14.2	<i>ChannelInfo Methods</i>	117
10.14.3	<i>multicastStats Methods</i>	118
10.14.4	<i>componentInfo Method</i>	119
10.14.5	<i>ServerInfo Methods</i>	119
10.14.6	<i>Channel.ioctl ioctlCodes</i>	119
10.14.7	<i>Server.ioctl ioctlCodes</i>	120
10.15	Encrypted and Proxy Connections	121
10.15.1	<i>Configuring HTTPS, HTTP, and Proxy Connections</i>	121
10.15.2	<i>Proxy Authentication</i>	122
10.15.3	<i>Encrypted SOCKET and WEBSOCKET Connections</i>	128
10.15.4	<i>Encrypted Server</i>	128
10.15.5	<i>Debugging a Tunnel Connection</i>	129
11	Data Package Detailed View	130
11.1	Concepts	130
11.2	Primitive Types	130
11.2.1	DataTypes Methods	133
11.2.2	<i>Real</i>	134
11.2.3	<i>Date</i>	138
11.2.4	<i>Time</i>	139
11.2.5	<i>DateTime</i>	141
11.2.6	<i>Qos</i>	143
11.2.7	<i>State</i>	146
11.2.8	<i>Array</i>	151
11.2.9	<i>Buffer</i>	157
11.2.10	<i>RMTES Decoding</i>	159
11.3	Container Types	162
11.3.1	<i>FieldList</i>	165
11.3.2	<i>ElementList</i>	172
11.3.3	<i>Map</i>	179
11.3.4	<i>Series</i>	188
11.3.5	<i>Vector</i>	195
11.3.6	<i>FilterList</i>	203
11.3.7	<i>Non-LSEG Rssl Wire Format Container Types</i>	211
11.4	Permission Data	213
11.5	Summary Data	213
11.6	Set Definitions and Set-Defined Data	214
11.6.1	<i>Set-Defined Primitive Types</i>	215
11.6.2	<i>Set Definition Use</i>	218
11.6.3	<i>Set Definition Database</i>	222
12	Message Package Detailed View	232
12.1	Concepts	232

12.1.1	<i>Common Message Interface</i>	232
12.1.2	<i>Message Key</i>	235
12.1.3	<i>Stream Identification</i>	237
12.2	Messages.....	239
12.2.1	<i>Request Message Interface</i>	239
12.2.2	<i>Refresh Message Interface</i>	242
12.2.3	<i>Update Message Interface</i>	245
12.2.4	<i>Status Message Interface</i>	247
12.2.5	<i>Close Message Interface</i>	249
12.2.6	<i>Generic Message Class</i>	250
12.2.7	<i>Post Message Interface</i>	252
12.2.8	<i>Acknowledgment Message Interface</i>	255
12.2.9	<i>Msg Encoding and Decoding</i>	257
13	Advanced Messaging Concepts	266
13.1	Multi-Part Message Handling	266
13.2	Stream Priority	267
13.3	Stream Quality of Service	268
13.4	Item Group Use	268
13.4.1	<i>Item Group Buffer Contents</i>	269
13.4.2	<i>Item Group Utility Functions</i>	270
13.4.3	<i>Group Status Message Information</i>	270
13.4.4	<i>Group Status Responsibilities by Application Type</i>	270
13.5	Single Open and Allow Suspect Data Behavior	272
13.6	Pause and Resume.....	273
13.7	Batch Requesting.....	274
13.7.1	<i>Batch Request Usage</i>	274
13.7.2	<i>Batch RequestMsg Encoding Example</i>	275
13.8	Dynamic View Use	277
13.8.1	<i>RDM ViewTypes Names</i>	278
13.8.2	<i>Dynamic View RequestMsg Encoding Example</i>	278
13.9	Posting	280
13.9.1	<i>Post Message Encoding Example</i>	281
13.9.2	<i>Post Acknowledgement Encoding Example</i>	282
13.10	Round Trip Time Examples.....	283
13.10.1	<i>Round Trip Time Message Sending Example</i>	283
13.10.2	<i>Round Trip Time Message Calculation Example</i>	283
13.11	Visible Publisher Identifier.....	284
13.11.1	<i>Example: Encoding PostUserInfo into a Refresh Message</i>	284
13.11.2	<i>Example: Decoding PostUserInfo from Refresh Message</i>	285
13.11.3	<i>Example: Encoding PostUserInfo into a Post Message</i>	285
13.11.4	<i>Example: Decoding PostUserInfo from Post Message</i>	286
13.12	UserAuthn Authentication	288
13.13	Private Streams.....	288
13.14	Creating a DACSLOCK for Publishing Permission Data.....	290
	Appendix A Item and Group State Decision Table	291
	Appendix B RWF/JSON Converter	293
B.1	Overview	293
B.2	Automatic Conversion using the Transport API Reactor.....	293
B.3	Creating the Converter.....	293
B.3.1	<i>Create JsonConverterBuilder Instance</i>	293
B.3.2	<i>Setting Converter Properties</i>	294

B.3.3	<i>Properties Supported by JsonConverterBuilder</i>	294
B.3.4	<i>Converter Callbacks</i>	295
B.3.5	<i>Creating an RWF/JSON Converter Example</i>	295
B.4	Converting from JSON to RWF	296
B.4.1	<i>Methods parseJsonBuffer</i>	296
B.4.2	<i>ParseJsonOptions Interface</i>	296
B.4.3	<i>Method decodeJsonMsg()</i>	297
B.4.4	<i>DecodeJsonMsgOptions Interface</i>	297
B.4.5	<i>Example: Converting from JSON to RWF</i>	298
B.5	Converting from RWF to JSON	300
B.5.1	<i>Method JsonConverter</i>	300
B.5.2	<i>RWFTToJsonOptions Interface</i>	300
B.5.3	<i>Method JsonConverter.getJsonBuffer()</i>	301
B.5.4	<i>GetJsonMsgOptions Interface</i>	301
B.5.5	<i>Example: Converting RWF to JSON</i>	301

Contents

Figure 1.	Network Diagram Notation	4
Figure 2.	UML Diagram Notation.....	4
Figure 3.	Open Message Model-Based Product Offerings.....	5
Figure 4.	Enterprise Transport API: Core Diagram	6
Figure 5.	LSEG Real-Time Distribution System Infrastructure	14
Figure 6.	Enterprise Transport API as a Consumer	15
Figure 7.	Batch Request.....	16
Figure 8.	View Request Diagram	17
Figure 9.	Symbol List: Basic Scenario.....	19
Figure 10.	Symbol List: Accessing the Entire LSEG Real-Time Advanced Distribution Server Cache.....	19
Figure 11.	Server Symbol List	20
Figure 12.	Posting into a Cache	21
Figure 13.	Open Message Model Post with Legacy Inserts	22
Figure 14.	Private Stream Scenarios	23
Figure 15.	Provider Access Point.....	25
Figure 16.	Interactive Providers	26
Figure 17.	Non-Interactive Provider: Point-To-Point	27
Figure 18.	Non-Interactive Provider: Multicast	28
Figure 19.	Typical LSEG Real-Time Distribution System Components	29
Figure 20.	Enterprise Transport API and LSEG Real-Time Advanced Distribution Server	30
Figure 21.	Enterprise Transport API and the LSEG Real-Time Advanced Distribution Hub	31
Figure 22.	LSEG Real-Tme APIs and Delivery Platform	32
Figure 23.	Enterprise Transport API and Data Feed Direct	33
Figure 24.	Enterprise Transport API and Internet Connectivity.....	34
Figure 25.	Transport API and Direct Connections.....	35
Figure 26.	Enterprise Transport API and the Composite Pattern	49
Figure 27.	Transport Application Lifecycle	68
Figure 28.	Unified TCP Network.....	72
Figure 29.	TCP Connection Creation	73
Figure 30.	Unified Multicast Network.....	73
Figure 31.	Segmented Multicast Network	74
Figure 32.	Multicast Connection Creation	74
Figure 33.	Transport API Server Creation.....	83
Figure 34.	Enterprise Transport API Writing Flow Chart	100
Figure 35.	Channel.write Priority Scenario.....	108
Figure 36.	Enterprise Transport API Consumer Application authenticating with a Proxy Server using NTLM	126
Figure 37.	Enterprise Transport API Consumer Application Authenticating with a Proxy Server using Negotiate/Kerberos 127	
Figure 38.	Item Group Example	269

Contents

Table 1:	Acronyms and Abbreviations	1
Table 2:	API Performance Comparison	8
Table 3:	Capabilities by API	10
Table 4:	EncodeIterator Utility Methods	51
Table 5:	DecodeIterator Utility Methods	53
Table 6:	Codec Package Success CodecReturnCodes	55
Table 7:	Codec Package Failure Return Codes.....	56
Table 8:	CodecReturnCodes Methods.....	57
Table 9:	Codec Methods	58
Table 10:	Codec Package Protocol Values.....	59
Table 11:	Library Version Utility Methods	59
Table 12:	LibraryVersionInfo Methods	59
Table 13:	channel Methods	61
Table 14:	Channel State Values	63
Table 15:	ConnectionType Values	63
Table 16:	Channel Settings for Socket and Websocket Connection Types	65
Table 17:	Server Methods	66
Table 18:	Server Settings for Socket and Websocket Connection Types.....	67
Table 19:	Error Methods.....	67
Table 20:	General Transport Return Codes.....	68
Table 21:	Initialization and Uninitialization Methods	69
Table 22:	Initialization Arguments InitArgs Methods	69
Table 23:	Locking Types	71
Table 24:	Transport.connect Method	75
Table 25:	ConnectOptions Methods.....	75
Table 26:	UnifiedNetworkInfo Method Options.....	77
Table 27:	SegmentedNetworkInfo Method Options	78
Table 28:	TcpOpts Method Option	78
Table 29:	McastOpts Method Options	79
Table 30:	ShmemOpts Method Option	79
Table 31:	SeqMcastOpts Method Option.....	79
Table 32:	wSocketOpts Options	80
Table 33:	HttpMessage Methods	80
Table 34:	HttpRequestConnectionInfo Methods	80
Table 35:	Transport.bind Method.....	83
Table 36:	BindOptions Methods.....	83
Table 37:	Server.accept Method.....	89
Table 38:	AcceptOptions Methods	89
Table 39:	CompressionTypes Values	91
Table 40:	channel.init Method	92
Table 41:	InProgInfo Methods.....	92
Table 42:	channel.init Return Codes.....	93
Table 43:	channel.read Method	95
Table 44:	ReadFlags Values.....	96
Table 45:	channel.read Return Codes	96
Table 46:	Buffer Management Channel Methods	101
Table 47:	Buffer Management Server Methods	102
Table 48:	channel.getBuffer Return Codes.....	102
Table 49:	Channel.write Function	103
Table 50:	writeFlags	103
Table 51:	channel.write TransportReturnCodes.....	104

Table 52:	WritePriorities Values.....	109
Table 53:	channel.flush Method.....	109
Table 54:	channel.flush TransportReturnCodes.....	109
Table 55:	channel.packBuffer Method.....	111
Table 56:	channel.packBuffer Return Values.....	111
Table 57:	channel.ping method.....	114
Table 58:	channel.ping TransportReturnCodes.....	114
Table 59:	Connection Closing Functionality.....	115
Table 60:	Transport Utility Methods.....	116
Table 61:	channelInfo Methods.....	117
Table 62:	multicastStats Methods.....	118
Table 63:	componentInfo Options.....	119
Table 64:	serverInfo Methods.....	119
Table 65:	channel.ioctl IoctlCodes/OctlCodes.....	119
Table 66:	server.ioctl IoctlCodes/OctlCodes.....	120
Table 67:	TunnelingInfo Methods.....	121
Table 68:	CredentialsInfo Methods.....	124
Table 69:	Encrypted SOCKET and WEBSOCKET Connection Methods.....	128
Table 70:	Encrypted Server Methods.....	128
Table 71:	Enterprise Transport API Primitive Types.....	131
Table 72:	DataTypes Methods.....	133
Table 73:	Real Methods.....	134
Table 74:	RealHints Enumeration Values.....	135
Table 75:	Date Methods.....	138
Table 76:	DateTimeStringFormatTypes	139
Table 77:	Time Methods.....	139
Table 78:	DateTimeStringFormatTypes	140
Table 79:	DateTime Methods.....	141
Table 80:	DateTimeStringFormatTypes	142
Table 81:	Qos Methods.....	143
Table 82:	QosTimeliness Values.....	144
Table 83:	QosRates Values.....	145
Table 84:	State Methods.....	146
Table 85:	StreamStates Values.....	147
Table 86:	StreamStates Methods.....	147
Table 87:	DataStates Values.....	148
Table 88:	DataStates Methods.....	148
Table 89:	StateCodes Values.....	148
Table 90:	StateCodes Methods.....	150
Table 91:	Array Structure Members.....	151
Table 92:	ArrayEntry Methods.....	153
Table 93:	Buffer Methods.....	157
Table 94:	RmtesCacheBuffer Methods.....	159
Table 95:	RmtesBuffer Methods.....	160
Table 96:	RmtesDecoder Decode Methods.....	160
Table 97:	Enterprise Transport API Container Types.....	162
Table 98:	FieldList Methods.....	165
Table 99:	FieldListFlag Values.....	166
Table 100:	FieldEntry Methods.....	167
Table 101:	ElementList Methods.....	172
Table 102:	ElementListFlags Flags Values.....	173
Table 103:	ElementEntry Methods.....	174
Table 104:	Map Methods.....	179
Table 105:	MapFlags Values.....	181
Table 106:	MapEntry Methods.....	182

Table 107:	MapEntryFlags Values.....	183
Table 108:	MapEntryActions Values.....	184
Table 109:	Series Methods	188
Table 110:	SeriesFlags Values.....	190
Table 111:	SeriesEntry Methods.....	191
Table 112:	Vector Methods	195
Table 113:	VectorFlags Values	197
Table 114:	VectorEntry Methods.....	198
Table 115:	VectorEntryFlags Values.....	199
Table 116:	VectorEntryActions Values.....	199
Table 117:	FilterList Methods.....	203
Table 118:	FilterListFlags Values	204
Table 119:	FilterEntry Methods.....	204
Table 120:	FilterEntryFlags Values.....	206
Table 121:	FilterEntryActions Values.....	206
Table 122:	Non-LSEG Rssl Wire Format Type Encode Methods	211
Table 123:	Set-Defined Primitive Types.....	215
Table 124:	FieldSetDef Methods.....	218
Table 125:	FieldSetDefEntry Methods.....	219
Table 126:	ElementSetDef Methods	220
Table 127:	ElementSetDefEntry Methods.....	221
Table 128:	LocalFieldSetDefDb Methods	222
Table 129:	LocalElementSetDefDb Methods	222
Table 130:	Local Set Definition Database Encode Methods.....	223
Table 131:	Local Set Definition Database Decode Methods.....	223
Table 132:	Msg Methods	232
Table 133:	MsgClasses Values.....	234
Table 134:	MsgClasses Methods.....	235
Table 135:	msgKey Methods	235
Table 136:	MsgKeyFlags Values	237
Table 137:	RequestMsg Methods.....	239
Table 138:	RequestMsgFlags Values.....	240
Table 139:	RefreshMsg Methods.....	242
Table 140:	RefreshMsgFlags Values.....	244
Table 141:	UpdateMsg Methods.....	245
Table 142:	UpdateMsgFlags Values.....	246
Table 143:	StatusMsg Methods.....	247
Table 144:	Flags Values.....	248
Table 145:	CloseMsg Methods	249
Table 146:	CloseMsgFlags Values.....	249
Table 147:	GenericMsg Methods.....	250
Table 148:	GenericMsgFlagsValues	251
Table 149:	PostMsg Methods	252
Table 150:	Flags Values	253
Table 151:	PostUserRights Values.....	254
Table 152:	PostUserRights Methods	254
Table 153:	AckMsg Methods	255
Table 154:	Flags Values.....	256
Table 155:	AckMsgNakCodes Values.....	256
Table 156:	Msg Encode Methods.....	257
Table 157:	Msg Decode Methods.....	262
Table 158:	EncodeIterator Utility Methods	264
Table 159:	DecodeIterator Utility Methods	265
Table 160:	groupId Buffer Utility Methods	270
Table 161:	SingleOpen and AllowSuspectData Effects.....	272

Table 162:	LSEG Domain Model Viewtypes Values.....	278
Table 163:	Setting PostUserInfo in Provider Example	284
Table 164:	Getting PostUserInfo in a Consumer Example Sent by Provider	285
Table 165:	Set PostUserInfo in Consumer Example.....	285
Table 166:	Getting PostUserInfo from Post Messages in a Provider Example Sent by Consumer	286
Table 167:	Item and Group State Decision Table	291
Table 168:	JsonConverterBuilder Methods	294
Table 169:	Properties Supported by JsonConverterBuilder	294
Table 170:	Converter Callbacks	295
Table 171:	JsonConverter.parseJsonBuffer() Method Parameters	296
Table 172:	ParseJsonOptions Interface Methods	296
Table 173:	JsonConverter.decodeJsonMsg() Parameters.....	297
Table 174:	DecodeJsonMsgOptions Interface Methods	297
Table 175:	JsonConverter.convertRWFTToJson() Method Parameters	300
Table 176:	RWFTToJsonOptions Interface Methods	300
Table 177:	JsonConverter.getJsonBuffer() Method Parameters.....	301
Table 178:	RWFTToJsonOptions Interface Methods	301

1 Introduction

1.1 About this Manual

This document is authored by Enterprise Transport API architects and programmers who encountered and resolved many of the issues the reader might face. Several of its authors have designed, developed, and maintained the Enterprise Transport API product and other LSEG products which leverage it. As such, this document is concise and addresses realistic scenarios and use cases.

This guide documents the functionality and capabilities of the Enterprise Transport API Java Edition. In addition to connecting to itself, the Enterprise Transport API can also connect to and leverage many different LSEG and customer components. If you want the Enterprise Transport API to interact with other components, consult that specific component's documentation to determine the best way to configure for optimal interaction.

1.2 Audience

This manual provides information and examples that aid programmers using the Enterprise Transport API Java Edition. The level of material covered assumes that the reader is a user or a member of the programming staff involved in the design, coding, and test phases for applications which will use the Enterprise Transport API. It is assumed that the reader is familiar with the data types, classes, operational characteristics, and user requirements of real-time data delivery networks, and has experience developing products using the Java programming language in a networked environment.

1.3 Programming Language

The Enterprise Transport API Components are written to the C, Java, and C# languages. This guide discusses concepts related to the Java Edition. All code samples in this document and all example applications provided with the product are written accordingly.

1.4 Acronyms and Abbreviations

ACRONYM / TERM	MEANING
ADH	LSEG Real-Time Advanced Distribution Hub is the horizontally scalable service component within the LSEG Real-Time Distribution System providing high availability for publication and contribution messaging, subscription management with optional persistence, conflation and delay capabilities.
ADS	LSEG Real-Time Advanced Distribution Server is the horizontally scalable distribution component within the LSEG Real-Time Distribution System providing highly available services for tailored streaming and snapshot data, publication and contribution messaging with optional persistence, conflation and delay capabilities.
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
DMM	Domain Message Model
Enterprise Message API (EMA)	The Enterprise Message API is an ease of use, open source, Open Message Model API. EMA is designed to provide clients rapid development of applications, minimizing lines of code and providing a broad range of flexibility. It provides flexible configuration with default values to simplify use and deployment. EMA is written on top of the Enterprise Transport API utilizing the Value Added Reactor and Watchlist features of ETA.

Table 1: Acronyms and Abbreviations

ACRONYM / TERM	MEANING
Enterprise Transport API (ETA)	Enterprise Transport API is a high performance, low latency, foundation of the LSEG Real-Time SDK. It consists of transport, buffer management, compression, fragmentation and packing over each transport and encoders and decoders that implement the Open Message Model. Applications written to this layer achieve the highest throughput, lowest latency, low memory utilization, and low CPU utilization using a binary Rssl Wire Format when publishing or consuming content to/from LSEG Real-Time Distribution Systems.
GC	Garbage Collection
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol (Secure)
JWK	JSON Web Key. Defined by RFC 7517, a JWK is a JSON formatted public or private key.
JWKS	JSON Web Key Set, This is a set of JWK, placed in a JSON array.
JWT	JSON Web Token. Defined by RFC 7519, JWT allows users to create a signed claim token that can be used to validate a user.
OMM	Open Message Model
QoS	Quality of Service
RDM	Domain Model
DP	Delivery Platform: this platform is used for REST interactions. In the context of Real-Time APIs, an API gets authentication tokens and/or queries Service Discovery to get a list of Real-Time - Optimized endpoints using DP.
LSEG Real-Time Distribution System	LSEG Real-Time Distribution System is LSEG's financial market data distribution platform. It consists of the LSEG Real-Time Advanced Distribution Server and LSEG Real-Time Advanced Distribution Hub. Applications written to the LSEG Real-Time SDK can connect to this distribution system.
Reactor	The Reactor is a low-level, open-source, easy-to-use layer above the Enterprise Transport API. It offers heartbeat management, connection and item recovery, and many other features to help simplify application code for users.
RFA	Robust Foundation API
RMTES	A multi-lingual text encoding standard
RSSL	Source Sink Library
RTT	Round Trip Time, this definition is used for round trip latency monitoring feature.
RWF	Rssl Wire Format, an LSEG proprietary binary format for data representation.
SOA	Service Oriented Architecture
SSL	Sink Source Library
UML	Unified Modeling Language
UTF-8	8-bit Unicode Transformation Format

Table 1: Acronyms and Abbreviations

1.5 References

- Enterprise Transport API Java Edition *LSEG Domain Model Usage Guide*
- *API Concepts Guide*
- Enterprise Transport API Java Edition *Configuration Guide*
- Enterprise Transport API Java Edition *Developers Guide*
- Enterprise Transport API *ANSI Library Reference Manuals*
- Enterprise Transport API *DACS LOCK Library Reference Manuals*
- Enterprise Transport API Java Edition *Value Added Components Developers Guide*
- The [LSEG Developer Community](#)

1.6 Documentation Feedback

While we make every effort to ensure the documentation is accurate and up-to-date, if you notice any errors, or would like to see more details on a particular topic, you have the following options:

- Send us your comments via email at ProductDocumentation@lseg.com.
- Add your comments to the PDF using Adobe's **Comment** feature. After adding your comments, submit the entire PDF to LSEG by clicking **Send File** in the **File** menu. Use the ProductDocumentation@lseg.com address.

1.7 Document Conventions

1.7.1 Typographic

This document uses the following types of conventions:

- Java classes, methods, in-line code snippets, and types are shown in **Courier New** font.
- Parameters, filenames, tools, utilities, and directories are shown in **Bold** font.
- Document titles and variable values are shown in *italics*.
- When initially introduced, concepts are shown in ***Bold, Italics***.
- Longer code examples are shown in Courier New font against a gray background. For example:

```
/* decode contents into the filter list object */
if ((retVal = filterList.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* create single filter entry and reuse while decoding each entry */
    FilterEntry filterEntry = CodecFactory.createFilterEntry();
```


1.7.2 Diagrams

Diagrams that depict the interaction between components on a network use the following notation:

	Feed Handler, Real-Time server, or other application		Network of multiple servers
	Enterprise Transport API application		Point-to-point connection showing direction of primary data flow
	Application with local daemon		Point-to-point connection showing direction of client connecting to server
	Multicast network		Data from external source (e.g. consolidated network or exchange)
	Connection to Multicast network, no primary data flow direction		Connection to Multicast network showing direction of primary data flow

Figure 1. Network Diagram Notation

	Object
	Inheritance: object on left is like object on right
	Composition: object on left is made up of some number of objects on right
	Composition: object on left is made up of one object on right

Figure 2. UML Diagram Notation

2 Product Description

2.1 What is the Enterprise Transport API?

The Enterprise Transport API (also known as the Source Sink Library API) is the customer release of LSEG's low-level internal API, currently used by LSEG Data Management Solutions (LSEG Real-Time Distribution System) and its dependent APIs for optimal distribution of Open Message Model / Rssl Wire Format data. Due to its well-integrated and common usage across these products, the Enterprise Transport API allows clients to write applications for use with LSEG Data Management Solutions (LSEG Real-Time Distribution System) to achieve the highest performance, highest throughput, and lowest latency.

The Enterprise Transport API is currently used by products such as the LSEG Real-Time Advanced Distribution Server, LSEG Real-Time Advanced Distribution Hub, Robust Foundation API, Data Feed Direct, and certain Delivery Platform APIs where Enterprise Transport API serves as a foundation.

The Enterprise Transport API supports all constructs available as part of the Open Message Model. It complements the Robust Foundation API and Enterprise Message API by allowing users to choose the type of functionality and layer (Session or Transport) at which they want to access the LSEG Real-Time Distribution System. With the addition of the Enterprise Transport API, customers can choose between an easy-to-use session-level API (Enterprise Message API) and a high-performance transport-level API (Enterprise Transport API).

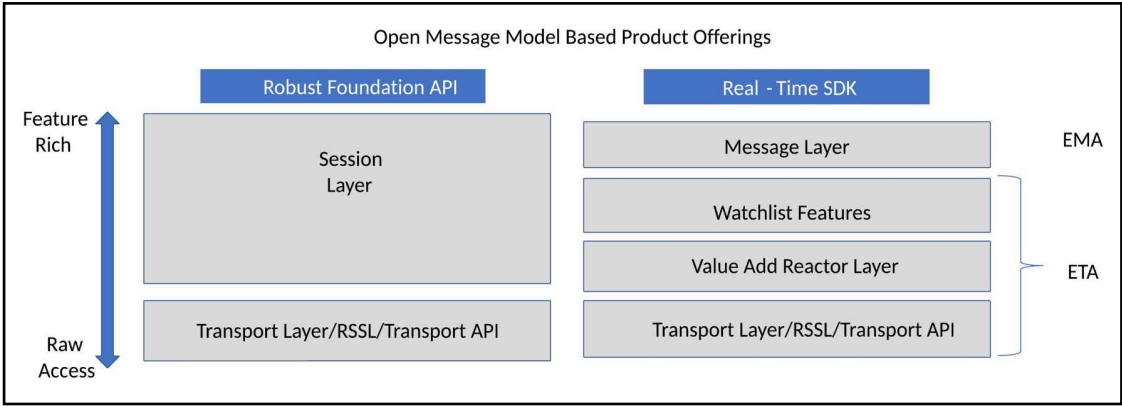


Figure 3. Open Message Model-Based Product Offerings

The Enterprise Transport API is a low-level API that provides application developers with the most flexible development environment and is the foundation on which all LSEG Open Message Model-based components are built. By utilizing an API at the transport level, a client can write to the same API as the LSEG Real-Time Advanced Distribution Server / LSEG Real-Time Advanced Distribution Hub and achieve the same levels of performance.

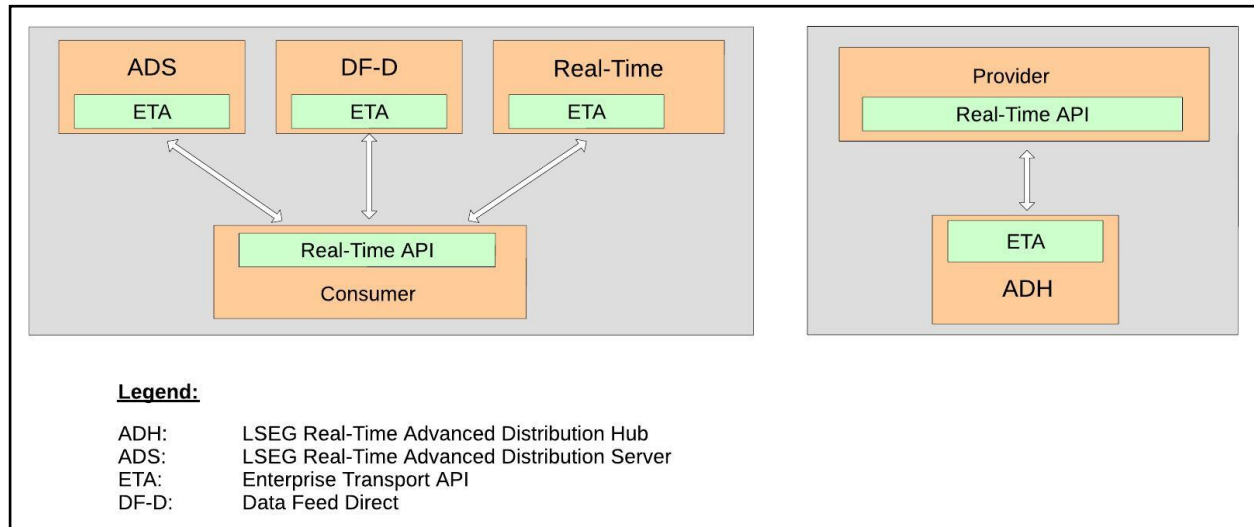


Figure 4. Enterprise Transport API: Core Diagram

2.2 Enterprise Transport API Features

The Enterprise Transport API is:

- Depending on the particular API, available in C and Java.
- 64-bit.
- Thread-safe and thread-aware.
- Capable of handling:
 - Any and all Open Message Model primitives and containers.
 - All Domain Models, including those defined by LSEG as well as other user-defined models.
- A reliable, transport-level API which includes Open Message Model encoders/decoders.

Additionally, certain the Enterprise Transport APIs provides an ANSI Page parser to encode/decode ANSI sequences and a Data Access Control System library to allow for generating DACS Locks.

2.2.1 General Capabilities

The Enterprise Transport API provides general capabilities independent of the type of application. The Enterprise Transport API:

- Supports fully connected or unified network topologies as well as segmented topologies.
- Supports multiple network session types, including TCP, HTTP, and multicast-based networks.
- Can internally fragment and reassemble large messages.
- Can pack multiple, small messages into the same network buffer.
- Can perform data compression and decompression internally.
- Can choose its locking model based on need. Locking can be enabled globally, within a connection, or disabled entirely, thus allowing clients to develop single-threaded, multi-threaded, thread-safe, or thread-aware solutions.
- Has full control over the number of message buffers and can dynamically increase or decrease this quantity during runtime.
- Does not have external configuration, log file, or message file dependencies: everything is programmatically supplied, where the user can define any external configuration or logging according to their needs.
- Allows users to write messages at different priority levels, allowing higher priority messages to be sent before lower priority messages.

2.2.2 Consumer Applications

You can use the Enterprise Transport API to create consumer-based applications that can:

- Make streaming and snapshot-based subscription requests to the LSEG Real-Time Advanced Distribution Server.
- Send batch, views, and symbol list requests to the LSEG Real-Time Advanced Distribution Server.
- Support pause and resume on active data streams with the LSEG Real-Time Advanced Distribution Server.
- Send post messages to the LSEG Real-Time Advanced Distribution Server (for consumer-based publishing and contributions).
- Send and receive generic messages with LSEG Real-Time Advanced Distribution Server.
- Establish private streams and tunnel streams.
- Transparently use HTTP to communicate with an LSEG Real-Time Advanced Distribution Server by tunneling through the Internet.

2.2.3 Provider Applications: Interactive

You can use the Enterprise Transport API to create interactive providers that can:

- Receive requests and respond to streaming and snapshot-based requests from an LSEG Real-Time Advanced Distribution Hub.
- Receive and respond to batch, views, and symbol list requests from an LSEG Real-Time Advanced Distribution Hub.
- Receive and respond to requests for a private streams and tunnel streams from an LSEG Real-Time Advanced Distribution Hub.
- Receive requests for pause and resume on active data streams.

- Receive and acknowledge post messages (used receiving consumer- based Publishing and Contributions) from an LSEG Real-Time Advanced Distribution Hub.
- Send and receive Generic Messages with an LSEG Real-Time Advanced Distribution Hub.

Additionally, you can use the Enterprise Transport API to create server-based applications that can accept multiple connections from an LSEG Real-Time Advanced Data Hub, or allows multiple LSEG Real-Time Advanced Distribution Hubs to connect to a provider.

2.2.4 Provider Applications: Non-Interactive

Using the Enterprise Transport API, you can write non-interactive applications that start up and begin publishing data to an LSEG Real-Time Advanced Distribution Hub. This includes both TCP and UDP multicast-based non-interactive provider applications.

2.3 Performance and Feature Comparison

Though LSEG Real-Time Distribution System's core infrastructure can achieve great performance numbers, such performance can suffer from bottlenecks caused by using the rich features offered in certain APIs (i.e., Robust Foundation API) when developing high-performance applications. By writing to the Enterprise Transport API, a client can leverage the full throughput and low latency of the core infrastructure while by-passing the full set of Robust Foundation API's features. For a comparison of API capabilities and features, refer to Section 2.4.

As illustrated in Figure 4, core infrastructure components (as well as their performance test tools, such as `testclient` and `sink_driven_src`) are all written to the Enterprise Transport API. An Enterprise Transport API-based application's maximum achievable performance (latency, throughput, etc) is determined by the infrastructure component to which it connects. Thus, to know performance metrics, you should look at the performance numbers for the associated infrastructure component. For example:

- If a Enterprise Transport API consumer application talks to the LSEG Real-Time Advanced Distribution Server and you want to know the maximum throughput and latency of the consumer, look at the performance numbers for the LSEG Real-Time Advanced Distribution Server configuration you use.
- If a Enterprise Transport API provider application talks to an LSEG Real-Time Advanced Distribution Hub and you want to know the maximum throughput and latency of the Enterprise Transport API provider, look at the performance numbers for the LSEG Real-Time Advanced Distribution Hub Configuration you use.



TIP: The Enterprise Transport API ships with API performance tools and additional documentation to which you can refer which you can use to arrive at more-specific results for your environment.

When referring to LSEG Real-Time Distribution System infrastructure documentation, look for Enterprise Transport API or Source Sink Library numbers (LSEG Real-Time Distribution System documentation often refers to the Enterprise Transport API as the Source Sink Library), which will give the performance and latency of the Transport API and the associated core infrastructure component.

The following table compares existing API products and their performance. Key factors are latency, throughput, memory, and thread safety. Results may vary depending on whether you use of watch lists and memory queues and according to your hardware and operating system. Typically, when measuring performance on the same hardware and operating system, these comparisons remain consistent.

API	THREAD SAFETY	THROUGHPUT	LATENCY	MEMORY FOOTPRINT
Enterprise Transport API	Safe and Aware	Very High	Lowest	Lowest
ETA Reactor ^a	Safe and Aware	Very High	Low	Medium (watch list optional)
Enterprise Message API	Safe and Aware	High	Low	Medium (watch list ^b)
Websocket API ^c	Depends on application	Medium	Medium	Depends on application

Table 2: API Performance Comparison

API	THREAD SAFETY	THROUGHPUT	LATENCY	MEMORY FOOTPRINT
Robust Foundation API	Safe and Aware	High	Low	Medium (watch list, allows optional queues)
System Foundation Classes C++	None	Medium	High	Medium – High (watch list, cache)

Table 2: API Performance Comparison (Continued)

- The Reactor is an ease-of-use layer provided with the Enterprise Transport API.
- The Enterprise Message API leverages the reactor watchlist.
- The Websocket API is a protocol specification to implement a simpler version of the Open Message Model using a JSON payload over the wire over a websocket. There are examples to show how to access content using this specification [on GitHub](#).

2.3.1 Java Garbage

Within its own implementation, the Enterprise Transport API Java Edition minimizes garbage collection. Additionally, the interface allows user applications to limit their own garbage collection, if desired.

If the performance overhead of garbage collection is a concern, you have several options in reducing its impact on your application, such as:

- Objects obtained through Enterprise Transport API Java Edition's factories are owned by the application, with some exceptions (e.g.; TransportBuffers, Channels, Servers). An application can limit garbage collection by reusing these objects (e.g., pooling).
- Maintain long term references to objects in the application until such a time as garbage collection is tolerable (i.e., after trading hours).
- Avoid the use of collections that have internal garbage collection. When using collections, ensure they are sized to allow for growth without implicit resizing, and attempt reuse where possible. Alternately, write your own variant or leverage a third-party package.
- Use a profiler to detect hot spots in your application, which you can then optimize to meet your requirements.
- Java Strings are immutable, resulting in contents being garbage collected whenever an attempt is made to modify or redefine the string's contents. Consider the use of an alternative type or use StringBuilder to avoid some garbage collection. **Object.toString** conversion methods will generally create a new string whenever it is invoked, which can also add to garbage collection overhead.

This manual and the Enterprise Transport API Java Edition Reference Manual both denote any method that internally incurs garbage collection. However, non-Enterprise Transport API libraries (i.e., Apache, Data Access Control System, ANSIPage, XPP, etc) might also collect garbage.

2.3.2 Use of Assertions

In some cases the Enterprise Transport API Java Edition library uses assertions, rather than IF statements, to verify the integrity of expected values. To aid in troubleshooting during development, we recommend that you enable Java assertions (JVM arg:

-enableassertions). When running in production, do not enable Java assertions as this adversely affects performance.

2.4 Functionality: Which API to Choose?

To make an informed decision on which API to use, you should balance consider both performance and functionality. For performance comparisons, refer to Section 2.3.

The Robust Foundation API uses information provided from the Enterprise Transport API and creates specific implementations of capabilities. Though these capabilities are not implemented in the Enterprise Transport API, Enterprise Transport API-based applications can use the information provided by the Enterprise Transport API to implement the same functionality (i.e., as provided by the Robust Foundation API). Additionally, Enterprise Transport API Value Added Components offer fully-supported reference implementations for much of this functionality.

The Enterprise Transport API Reactor is an open source component that functions within the Enterprise Transport API.

The following table lists API capabilities using the following legend:

- X: Supported in current version, natively implemented
- X*: Supported only in the C / C++ version of the software
- X**: Supported in current version, leverages lower-level capability
- Any X that is in blue: Supported only in C/C++ and Java version of the software.
- X+: Supports V2 authentication in C# and both V1 and V2 in C/C++ and Java
- Future: Planned for a future release
- Legacy: A legacy functionality

CAPABILITY TYPE	CAPABILITY	ENTERPRISE TRANSPORT API 3.X	ENTERPRISE TRANSPORT REACTOR	ENTERPRISE MESSAGE API 3.X	THE ROBUST FOUNDATION API 8.X
Transport	Compression via Open Message Model	X	X**	X**	X
	HTTP Tunneling (Rssl Wire Format)	X	X**	X**	X
	TCP/IP: Rssl Wire Format	X	X**	X**	X
	Reliable Multicast: Rssl Wire Format	X	X**	X**	X
	Sequenced Multicast	X			
	Websocket	X	X	X**	
	Unidirectional Shared Memory	X			
Application Type	Consumer	X	X	X**	X
	Provider: Interactive	X	X	X**	X
	Provider: Non-Interactive	X	X	X**	X

Table 3: Capabilities by API

CAPABILITY TYPE	CAPABILITY	ENTERPRISE TRANSPORT API 3.X	ENTERPRISE TRANSPORT REACTOR	ENTERPRISE MESSAGE API 3.X	THE ROBUST FOUNDATION API 8.X
General	Batch Request	X	X	X	X
	Batch Re-issue and Close	X	X		X
	Generic Messages	X	X	X	X
	Pause/Resume	X	X	X	X
	Posting	X	X	X	X
	Snapshot Requests	X	X	X	X
	Streaming Requests	X	X	X	X
	Private Streams	X	X	X	X
	Qualified Streams	X	X	X	
	Views	X	X	X	X
Domain Models	Custom Data Model Support	X	X	X	X
	Domain Model: Dictionary	X	X	X	X
	Domain Model: Enhanced Symbol List	X	X	X**	X
	Domain Model: Login	X	X	X	X
	Domain Model: Market Price	X	X	X	X
	Domain Model: MarketByOrder	X	X	X	X
	Domain Model: MarketByPrice	X	X	X	X
	Domain Model: Market Maker	X	X	X	X
	Domain Model: Source Directory	X	X	X	X
	Domain Model: Symbol List	X	X	X	X
	Domain Model: Yield Curve	X	X	X	X
Encoders/Decoders	AnsiPage	X	X**	X**	Legacy
	DACS Lock	X	X**	X**	X
	Open Message Model	X	X	X**	X
	RMTES	X	X	X**	X

Table 3: Capabilities by API(Continued)

CAPABILITY TYPE	CAPABILITY	ENTERPRISE TRANSPORT API 3.X	ENTERPRISE TRANSPORT REACTOR	ENTERPRISE MESSAGE API 3.X	THE ROBUST FOUNDATION API 8.X
Layer Specific	Config: file-based			X	X
	Config: programmatic	X	X	X	X
	Group fanout to items		X	X**	X
	Load balancing: API-based				X
	Logging: file-based			X	X
	Logging: programmatic	X	X		X
	Quality of Service Matching		X	X**	X
	Network Pings: automatic		X	X**	X
	NiProvider multiple connections			X	X
	Recovery: connection		X	X**	X
	Preferred Host in ConnectionList		X	X	
	Recovery: items		X	X**	X
	Request routing			X	X
	Round trip time	X	X	X	
	Session management to RTO		X+	X+	
	Service Groups				X
	Single Open: API-based		X	X**	X
	Warm Standby: API-based (must enable Watchlist)		X	X	X
	Warm Standby with Preferred Group		X	X	
	Watchlist		X	X**	X
	Controlled fragmentation and assembly of large messages	X	X**	X**	
	Controlled locking/threading model	X			

Table 3: Capabilities by API(Continued)

CAPABILITY TYPE	CAPABILITY	ENTERPRISE TRANSPORT API 3.X	ENTERPRISE TRANSPORT REACTOR	ENTERPRISE MESSAGE API 3.X	THE ROBUST FOUNDATION API 8.X
	Controlled dynamic message buffers with ability to programmatically modify during runtime	X	X**		
	Controlled message packing	X	X**	X**	
	Messages can be written at different priority levels	X	X**	X**	

Table 3: Capabilities by API(Continued)

3 Consumers and Providers

3.1 Overview

For those familiar with previous API products or concepts from LSEG Real-Time Distribution System, we map how the Enterprise Transport API implements the same functionality.

At a very high level, the LSEG Real-Time Distribution System facilitates controlled and managed interactions between many different service **providers** and **consumers**. Thus, LSEG Real-Time Distribution System is a real-time, streaming Service Oriented Architecture (SOA) used extensively as middleware integrating financial-service applications. While providers implement services and expose a certain set of capabilities (e.g. content, workflow, etc.), consumers use the capabilities offered by providers for a specific purpose (e.g., trading screen applications, black-box algorithmic trading applications, etc.). In some cases, a single application can function as both a consumer and a provider (e.g., a computation engine, value-add server, etc.).

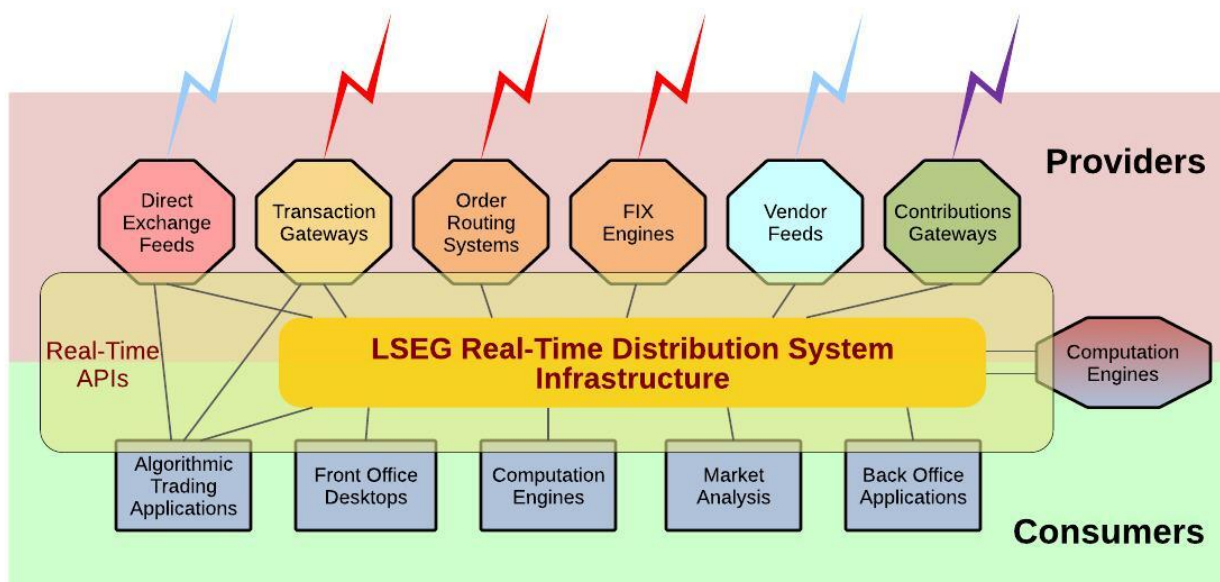


Figure 5. LSEG Real-Time Distribution System Infrastructure

To access needed capabilities, consumers always interact with a provider, either directly and/or via LSEG Real-Time Distribution System. Consumer applications that want the lowest possible latency can communicate directly via LSEG Real-Time (or LSEG Real-Time Distribution System) APIs with the appropriate service providers. However, you can implement more complex deployments (i.e., integrating multiple providers, managing local content, automated resiliency, scalability, control, and protection) by placing the LSEG Real-Time Distribution System infrastructure between provider and consumer applications.

NOTE: Enterprise Message API C# Edition supports only consumers in current release.

3.2 Consumers

Consumers make use of capabilities offered by providers through access points. To interact with a provider, the consumer must attach to a consumer access point. Access points manifest themselves in two different forms:

- A **concrete access point**. A concrete access point is implemented by the service-provider application if it supports direct connections from consumers. The right-side diagram in the following figure illustrates a Enterprise Transport API consumer connecting to LSEG Real-Time via a direct access point.
- A **proxy access point**. A proxy access point is point-to-point based and implemented by an LSEG Real-Time Distribution System Infrastructure component (i.e., an LSEG Real-Time Advanced Distribution Server). The following figure also illustrates a Enterprise Transport API consumer connecting to the provider by first passing through a proxy access point.

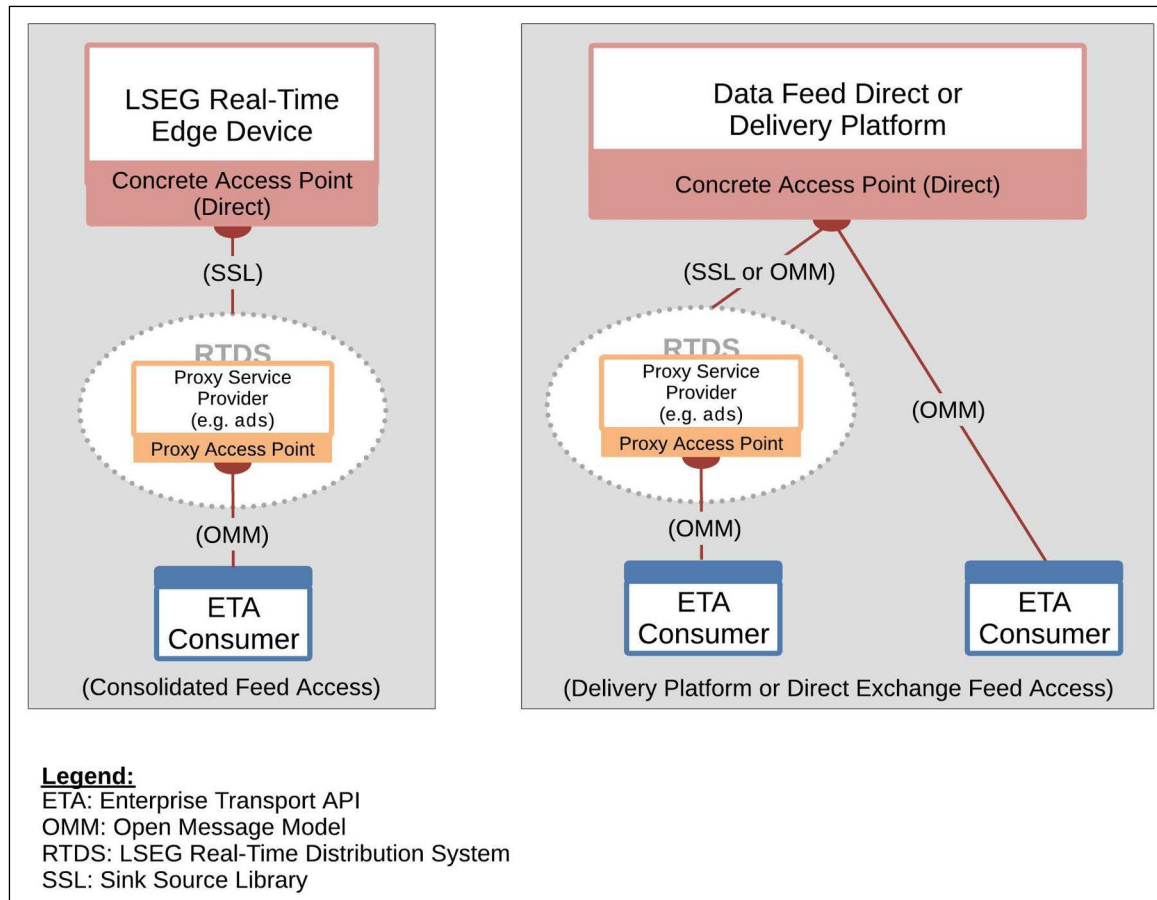


Figure 6. Enterprise Transport API as a Consumer

Examples of consumers include:

- An application that subscribes to data via LSEG Real-Time Distribution System or LSEG Real-Time.
- An application that posts data to LSEG Real-Time Distribution System or LSEG Real-Time (e.g., contributions/inserts or local publication into a cache).
- An application that communicates via generic messages with LSEG Real-Time Distribution System or LSEG Real-Time.
- An application that does any of the above via a private stream.

3.2.1 Subscriptions: Request/Response

After a consumer successfully logs into a provider (i.e., LSEG Real-Time Advanced Distribution Server or LSEG Real-Time) and obtains a list of available sources, the consumer can then subscribe and receive data for various services. A consumer subscribes to a service or service ID that in turn maps to a service name in the Source Directory. Any service or service ID provides a set of items to its clients.

- If a consumer's request does not specify interest in future changes (i.e., after receiving a full response), the request is a classic **snapshot request**. The data stream is considered closed after a full response of data (possibly delivered in multiple parts) is sent to the consumer. This is typical behavior when a user sends a non-streaming request. Because the response contains all current information, the stream is considered complete as soon as the data is sent.
- If a consumer's request specifies interest in receiving future changes (i.e., after receiving a full response), the request is considered to be a **streaming request**. After such a request, the provider sends the consumer an initial set of data and then sends additional changes or "updates" to the data as they occur. The data stream is considered open until either the consumer or provider closes it. A consumer typically sends a streaming request when a user subscribes for an item and wants to receive every change to that item for the life of the stream.

Specialized cases of request / response include:

- Batches
- Views
- Symbol Lists
- Server Symbol Lists

3.2.2 Batches

A consumer can request multiple items using a single, client-based, request called a **batch** request. After the Transport API consumer sends an optimized batch request to the LSEG Real-Time Advanced Distribution Server, the LSEG Real-Time Advanced Distribution Server responds by sending the items as if they were opened individually so the items can be managed individually.

Figure 7 illustrates a Transport API consumer issuing a batch request for "TRI", "GE", and "INTC.O" and the resulting LSEG Real-Time Advanced Distribution Server responses.

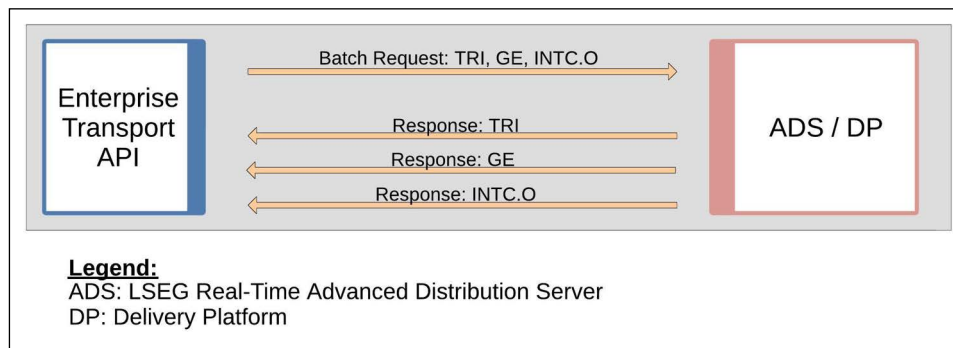


Figure 7. Batch Request

3.2.3 Views

The system reduces the amount of data that flows across the network by filtering out content in which the user is not interested. To improve performance and maximize bandwidth, you can configure the LSEG Real-Time Distribution System to filter out certain fields to downstream users. When filtering, all consumer applications see the same subset of fields for a given item.

Another way of controlling filtering is to configure the consumer application to use **Views**. Using a view, a consumer requests a subset of fields with a single, client-based request (refer to Figure 8). The API then requests (from the LSEG Real-Time Advanced Distribution Server / LSEG Real-Time) only the fields of interest. When the API receives the requested fields, it sends the subset back to the consumer. This is also called consumer-side (or request-side) filtering.

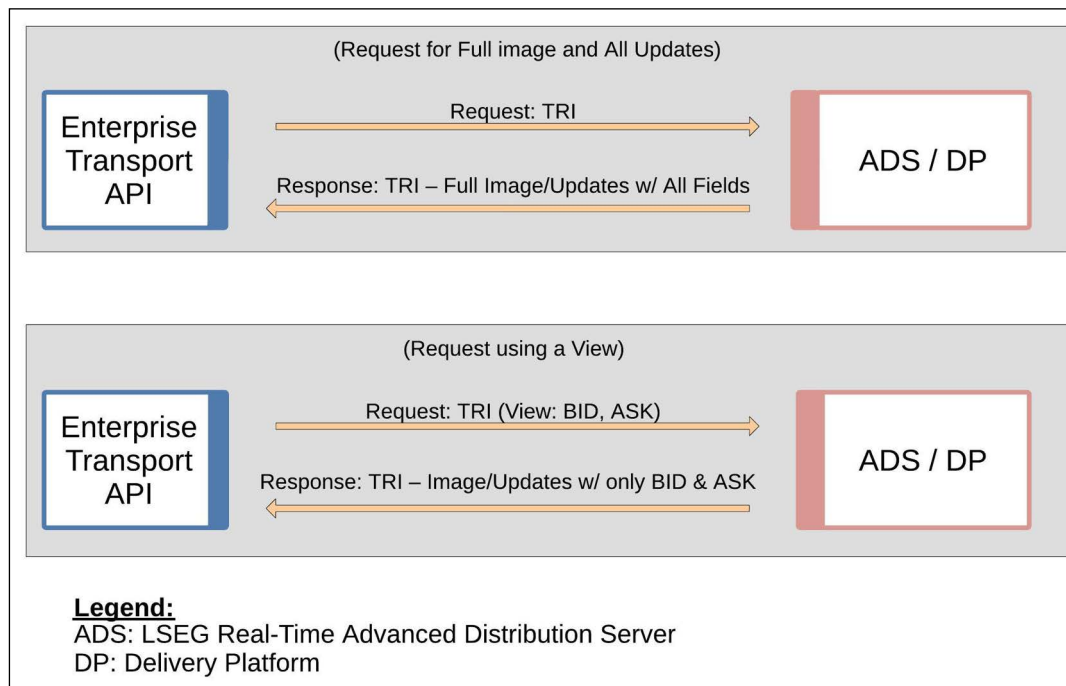


Figure 8. View Request Diagram

Views were designed to provide the same filtering functionality as the System Foundation Classes (based on its own internal cache) while optimizing network traffic.

Views, in conjunction with server-side filtering, can be a powerful tool for bandwidth optimization on a network. Users can combine a view with a batch request to send a single request to open multiple items using the same view.

3.2.4 Pause and Resume

The **Pause/Resume** feature optimizes network bandwidth. You can use Pause/Resume to reduce the amount of data flowing across the network for a single item or for many items that might already be openly streaming data to a client.

To pause/resume data, the client first sends a request to pause an item to the LSEG Real-Time Advanced Distribution Server. The LSEG Real-Time Advanced Distribution Server receives the pause request and stops sending new data to the client for that item, though the item remains open and in the LSEG Real-Time Advanced Distribution Server cache. The LSEG Real-Time Advanced Distribution Server continues to receive messages from the upstream device (or feed) and continues to update the item in its cache (but because of the client's pause request, does not send the new data to the client). When the client wants to start receiving messages for the item again, the client sends a resume to the LSEG Real-Time Advanced Distribution Server, which then responds by sending an aggregated update or a refresh (a current image) to the client. After the LSEG Real-Time Advanced Distribution Server resumes sending data, the LSEG Real-Time Advanced Distribution Server sends all subsequent messages.

By using the Pause/Resume feature a client can avoid issuing multiple open/close requests which can disrupt the LSEG Real-Time Advanced Distribution Server and prolong recovery times. There are two main use-case scenarios for this feature:

- Clients with intensive back-end processing
- Clients that display a lot of data

3.2.4.1 Pause / Resume Use Case 1: Back-end Processing

In this use-case, a client application performs heavy back-end processing and has too many items open, such that the client is at the threshold for lowering the downstream update rate. The client now needs to run a specialized report, or do some other back-end processing. Such an increase in workload on the client application will negatively impact its downstream message traffic. The client does not want to back up its messages from the LSEG Real-Time Advanced Distribution Server and risk having LSEG Real-Time Advanced Distribution Server abruptly cut its connection, nor does the client want to close its own connection (or close all the items on the LSEG Real-Time Advanced Distribution Server) which would require the client to re-open all items after finishing its back-end processing.

In this case, the client application:

- Sends a single PAUSE message to the LSEG Real-Time Advanced Distribution Server to pause all the items it has open.
- Performs all needed back-end processing.
- Sends a Resume request to resume all the items it had paused.

After receiving the Resume request, the LSEG Real-Time Advanced Distribution Server sends a refresh (i.e., current image), to the client for all paused items and then continues to send any subsequent messages.

3.2.4.2 Pause / Resume Use Case 2: Display Applications

The second use case assumes the application displays a lot of data. In this scenario, the user has two windows open. One window has item "TRI" open and is updating (Window 1). The other has "INTC.O" open and is updating (Window 2). On his screen, the user moves Window 1 to cover Window 2 and the user can no longer see the contents of Window 2. In this case, the user might not need updates for "INTC.O" because the contents are obstructed from view. In this case, the client application can:

- Pause "INTC.O" as long as Window 2 is covered and out of view.
- Resume the stream for "INTC.O" when Window 2 moves back into view.

When Window 2 is again visible, the LSEG Real-Time Advanced Distribution Server sends a refresh, or current image, to the client for the item "INTC.O" and then continues to send any subsequent messages.

3.2.5 Symbol Lists

The Symbol List domain provides access to a set of symbol names, typically from an index, service, or cache. The Consumer application can issue a Symbol List request if the Provider specifies Symbol List domain in its Capabilities list in Source Directory Refresh.

The following diagram illustrates issuing a basic symbol list request. In this diagram, the consumer issues the request using a particular key name (**FRED**). The request flows through the platform to a provider capable of resolving the symbol list name (the interactive provider with **FRED** in its cache). The provider sends back all names that map to **FRED** (**TRI** and **GE**). After receiving the response, the client can then choose whether to open items; individually or by making a batch request for multiple items. A subsequent request is resolved by the first cache that contains the data (listed in the diagram as optional caches).

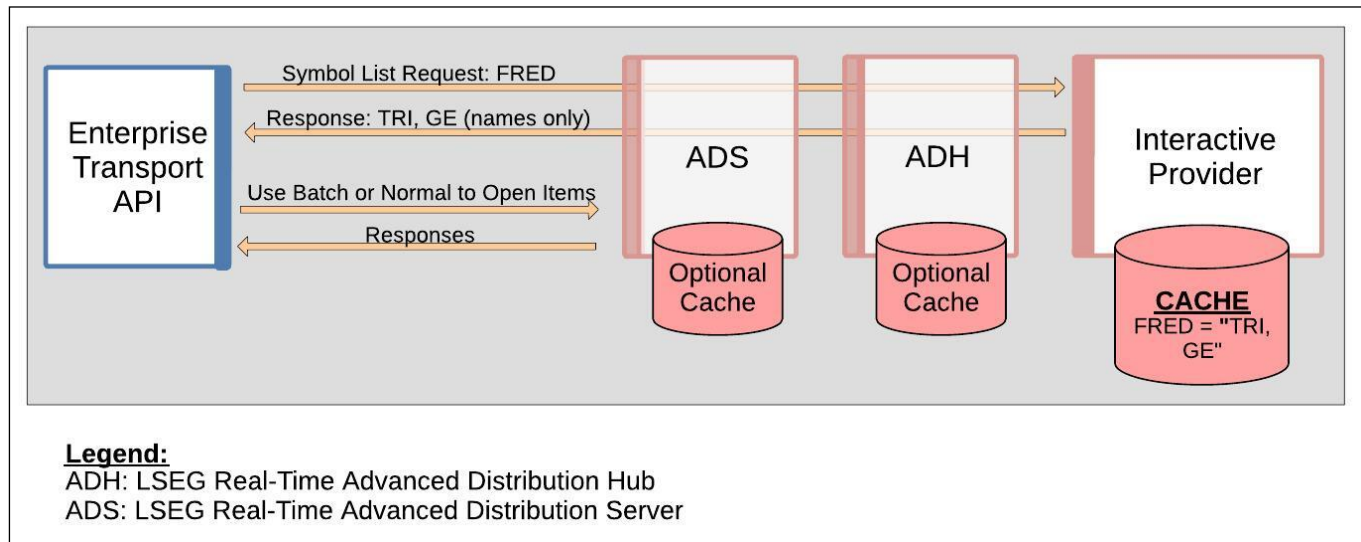


Figure 9. Symbol List: Basic Scenario

The following diagram illustrates how a consumer can access all items in the LSEG Real-Time Advanced Distribution Server cache, effectively dumping the cache to the Open Message Model client. In this scenario, the client requests the symbol list **_ADS_CACHE_LIST**. The LSEG Real-Time Advanced Distribution Server receives the request and responds with the names of all items in its cache. The client can then choose to open items individually, or make a batch request to open multiple items. The LSEG Real-Time Advanced Distribution Server provides an additional symbol list (**_SERVER_LIST**) for obtaining lists of items stored in specific LSEG Real-Time Advanced Distribution Hub instances. For details on this symbol list, refer to the *LSEG Real-Time Advanced Distribution Server* and *LSEG Real-Time Advanced Distribution Hub System Administration Manuals*.

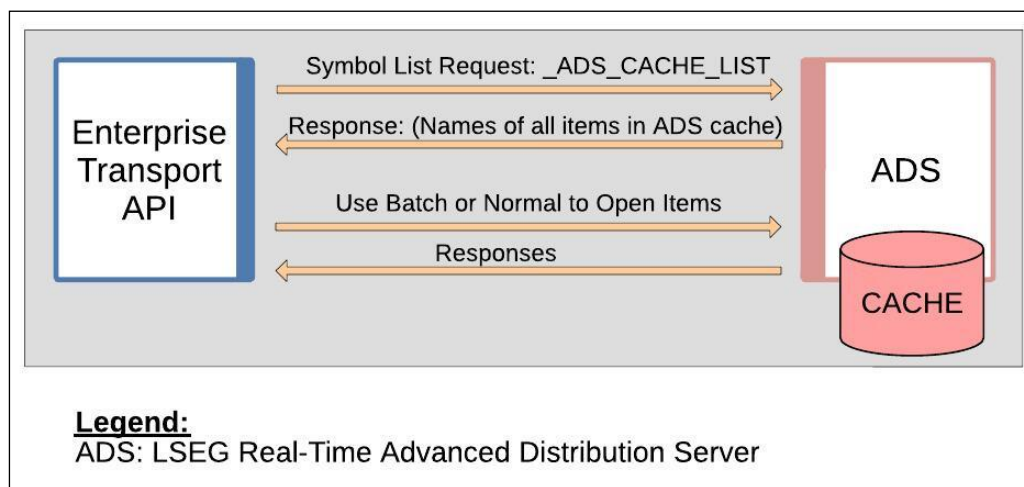


Figure 10. Symbol List: Accessing the Entire LSEG Real-Time Advanced Distribution Server Cache

3.2.5.1 Requesting Symbol List Data Streams

In case the server advertises support for Enhanced Symbol List in its Login Refresh message, the Consumer application can indicate that it wants the server to open streams for the items in the symbol list or send snapshots for these items automatically. To achieve this, the Consumer application can send the symbol list request with the payload that specifies the desired behavior. For details about the payload contents, refer to the “Data” section in the “Symbol List Domain” chapter of the *Enterprise Transport API Java Edition RDM Usage Guide*.

When the Provider opens streams for the items in symbol list automatically, the Consumer will receive the respective messages under negative **streamId** values.

3.2.5.2 Server Symbol Lists

Using certain LSEG Real-Time APIs, client applications can request a list of all symbols maintained in the cache of all LSEG Real-Time Advanced Distribution Hub servers across the network. Client applications start by first requesting a symbol list item **_SERVER_LIST** which will return a list of all servers and their supported domains. Each entry on that list is a symbol list item name formatted as follows **_CACHE_LIST.serverId.domain**. Client applications can then spawn individual symbol list requests for servers and domains of interest using the symbol name **_CACHE_LIST.serverId.domain**. If **domain** is not provided, it defaults to **6**.

The symbol list response for **_CACHE_LIST.serverId.domain** will include a list of all Level 1 or Level 2 items in the server cache. It will also include opened non-cached items but not items opened on private streams. The symbol list response will provide only item names, not item data.

The streams for **_SERVER_LIST** and **_CACHE_LIST.serverId.domain** requests will be kept open and updates will be sent to modify list of servers or list of items in server cache. These streams will be closed if a server is no longer available or it no longer supports a particular domain.

If the LSEG Real-Time Advanced Distribution Hub is configured for source mirroring, a failover will trigger a server id change and will lead to closing of the relevant **_CACHE_LIST.serverId.domain** request and updating of the **_SERVER_LIST** to show the new server id after the failover. Clients will need to make a new symbol list request to the new server.

This feature provides the symbol list of all items in the LSEG Real-Time Advanced Distribution Hub cache for both interactive and non-interactive services and is supported for both RSSL (symbol list) and SSL 4.5 (criteria) clients.

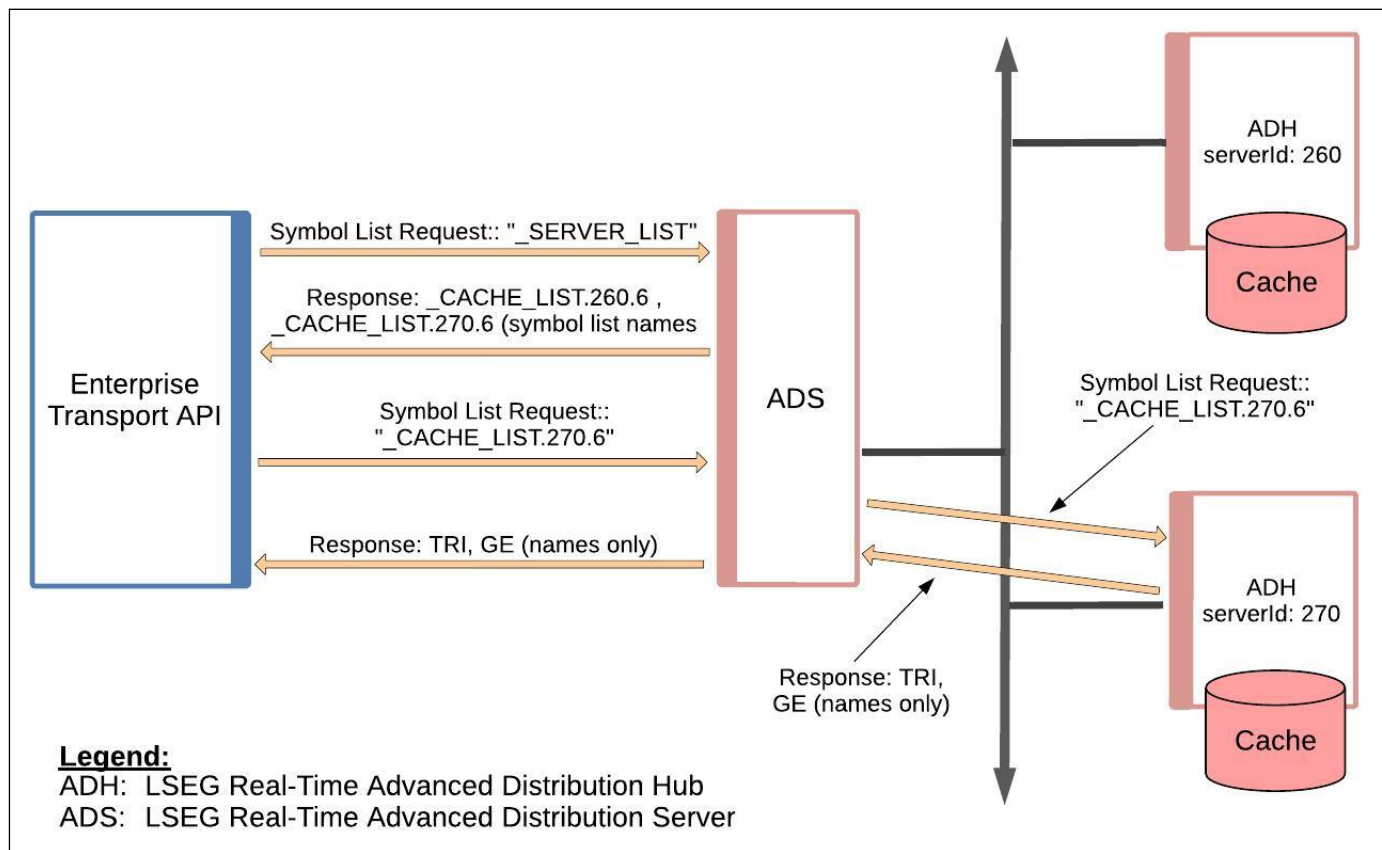


Figure 11. Server Symbol List

3.2.6 Posting

Through posting, API consumers can easily push content into any cache within the LSEG Real-Time Distribution System (i.e., an HTTP POST request). Data contributions/inserts into the ATS or publishing into a cache offer similar capabilities today. When posting, API consumer applications reuse their existing sessions to publish content to any cache(s) residing within the LSEG Real-Time Distribution System (i.e., service provider(s) and/or infrastructure components). When compared to spreadsheets or other applications, posting offers a more efficient form of publishing, because the application does not need to create a separate provider session or manage event streams. The posting capability, unlike unmanaged publishing or inserts, offers optional acknowledgments per posted message. The two types of posting are on-stream and off-stream:

- **On-Stream Post:** Before sending an on-stream post, the client must first open (request) a data stream for an item. After opening the data stream, the client application can then send a post. The route of the post is determined by the route of the data stream.
- **Off-Stream Post:** In an off-stream post, the client application can send a post for an item via a Login stream, regardless of whether a data stream first exists. The route of the post is determined by the Core Infrastructure (i.e., LSEG Real-Time Advanced Distribution Server, LSEG Real-Time Advanced Distribution Hub, etc.) configuration.

3.2.6.1 Local Publication

The following diagram illustrates the benefits of posting.

Green and Red services support internal posting and are fully implemented within the LSEG Real-Time Advanced Distribution Hub. In both cases the LSEG Real-Time Advanced Distribution Hub receives posted messages and then distributes these messages to interested consumers. In the right-side segment, the LSEG Real-Time Advanced Distribution Server component has enabled caching (for the Red service). In this case posted messages received from connected applications are cached and distributed to these local applications before being forwarded (re-posted) up into the LSEG Real-Time Advanced Distribution Hub cache. The Enterprise Transport API can even post to provider applications (i.e., the Purple service in this diagram) that support posting.

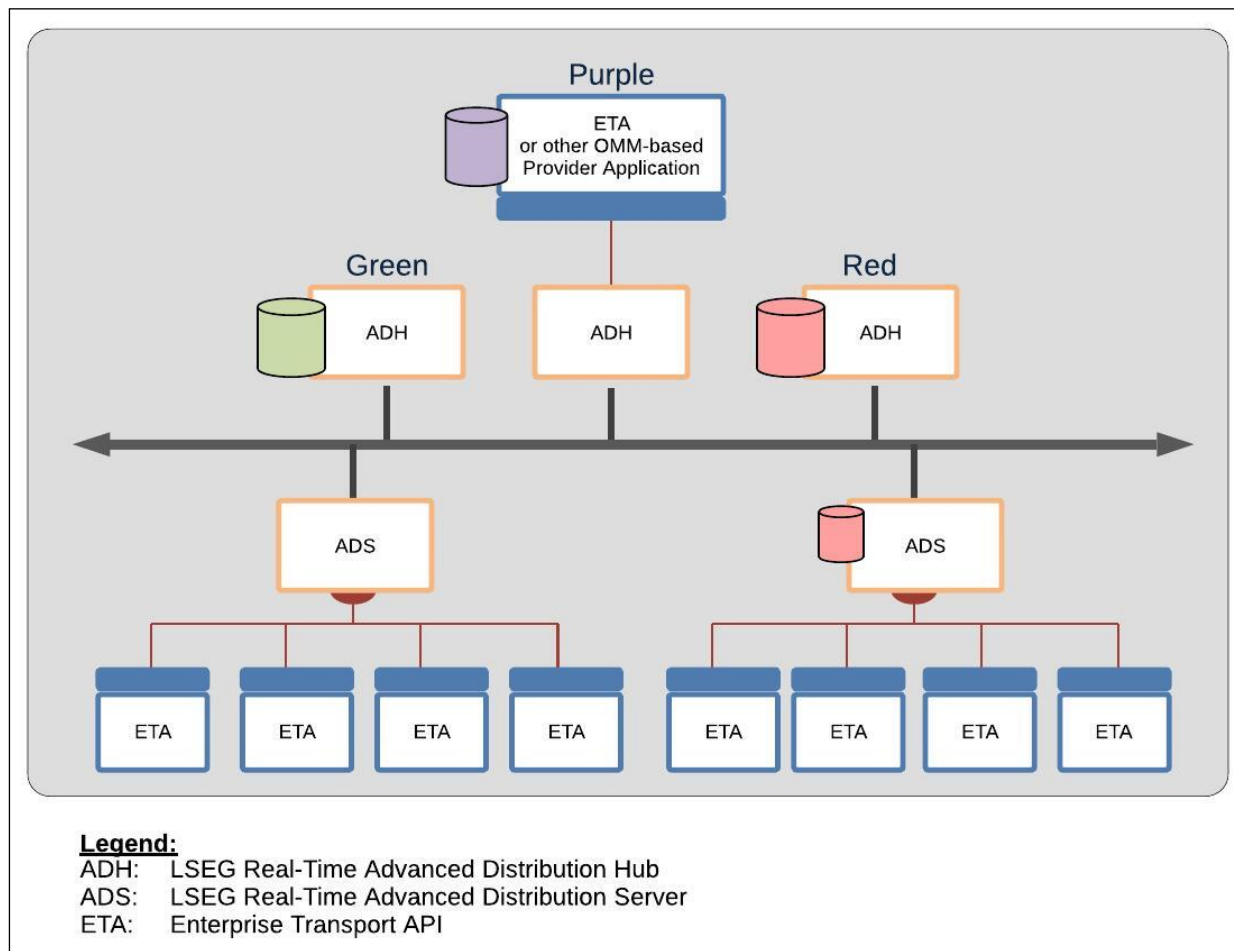


Figure 12. Posting into a Cache

You can use the Enterprise Transport API to post into an LSEG Real-Time Advanced Distribution Hub cache. If a cache exists in the LSEG Real-Time Advanced Distribution Server (the Red service), the LSEG Real-Time Advanced Distribution Server cache is also populated by responses from the LSEG Real-Time Advanced Distribution Hub cache. If you configure LSEG Real-Time Distribution System to allow such behavior, posts can be sent beyond the LSEG Real-Time Advanced Distribution Hub (to the Provider Application in the Purple service). Such posting flexibility is a good solution if one's applications are restricted to a LAN which hosts an LSEG Real-Time Advanced Distribution Server but allows publishing up the network to a cache with items to which other clients subscribe.

3.2.6.2 Contribution/Inserts

Posting also allows Open Message Model-based contributions. Through such posting, clients can contribute data to a device on the head end or to a custom-provider. In the following example, the Enterprise Transport API sends an Open Message Model post to a provider application that supports such functionality.

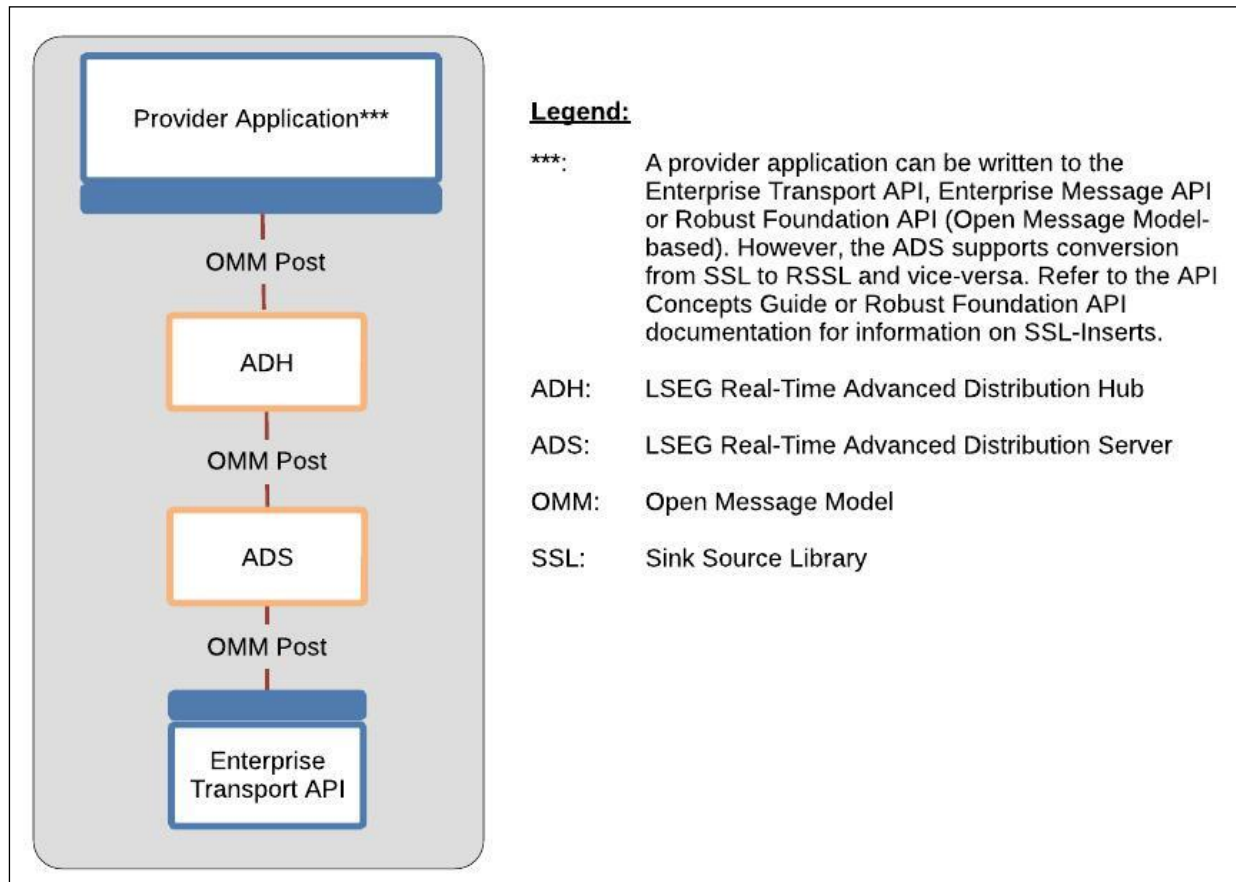


Figure 13. Open Message Model Post with Legacy Inserts

3.2.7 Generic Message

Using a **Generic Message**, an application can send or receive a bi-directional message. A generic message can contain any Open Message Model primitive type. Whereas the request/response type message flows from LSEG Real-Time Distribution System to a consumer application, a generic message can flow in any direction, and a response is not required or expected. One advantage to using generic messages is its freedom from the traditional request/response data flow.

In a generic message scenario, the consumer sends a generic message to an LSEG Real-Time Advanced Distribution Server, while the LSEG Real-Time Advanced Distribution Server also publishes a generic message to the consumer application. All domains support this type of generic message behavior, not just market data-based domains (such as Market Price, etc). If a generic message is sent to a component that does not understand generic messages, the component ignores the message.

3.2.8 Private Streams

Using a **Private Stream**, a consumer application can create a virtual private connection with an interactive provider. This virtual private connection can be either a direct connection, through the LSEG Real-Time Distribution System, or via a cascaded set of platforms. The following diagram illustrates these different configurations.

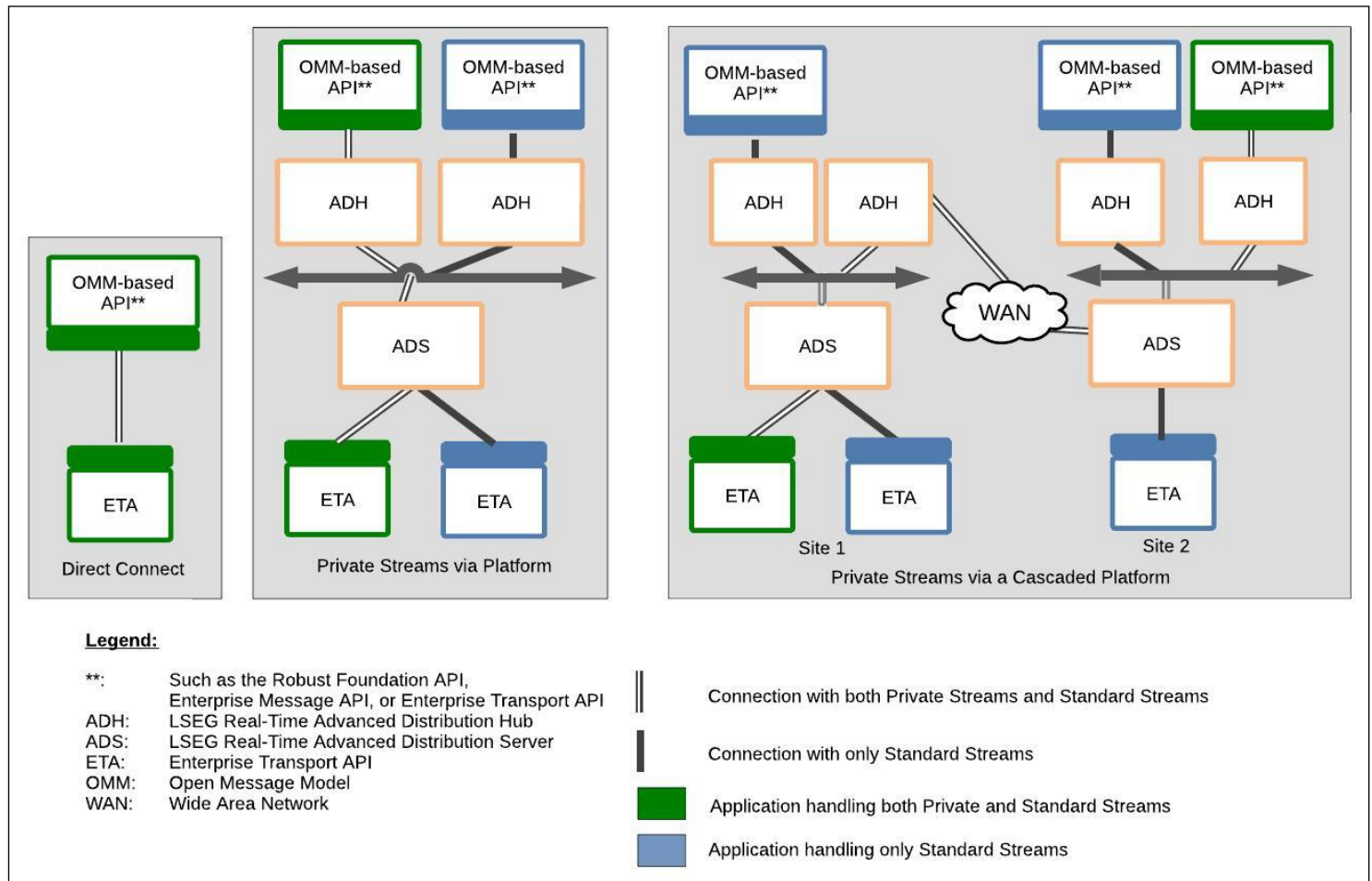


Figure 14. Private Stream Scenarios

A virtual private connection piggy backs on existing, individual point-to-point and multicast connections in the system (Figure 14 illustrates this behavior using a white connector). Messages exchanged via a Private Stream flow between a Consumer and an Interactive Provider using these existing underlying connections. However, unlike a regular stream, the Enterprise Transport API or LSEG Real-Time Distribution System components do not fan out these messages to other consumers or providers.

In the figure above representing private stream scenarios, each diagram shows a green consumer creating a private stream with a green provider. The private stream, using existing infrastructure and network connections, is illustrated as a white path in each of the diagrams. When established, communications sent on a private stream flow only between the green consumer and the green provider to which it connects. Blue providers and consumers do not see messages sent via the private stream.

Any break in a “virtual connection” causes the provider and consumer to be notified of the loss of connection. In such a scenario, the consumer is responsible for re-establishing the connection and re-requesting any data it might have missed from the provider. All types of requests, functionality, and Domain Models can flow across a private stream, including (but not limited to):

- Streaming Requests
- Snapshot Requests
- Posting
- Generic Messages
- Batch Requests

- Views
- All LSEG Domain Models & Custom Domain Models

3.2.9 Update Filtering

Update filtering allows consuming applications that request streaming content to control the volume of updates they receive based on the **updateType** parameter. This is particularly beneficial in environments where minimizing unnecessary data flow or selectively processing updates is critical.

To enable this feature, the consuming application includes specific fields (element entries), as part of the element list payload, in the Login Request message sent to a server supporting update filtering. These element entries are:

- **UpdateTypeFilter**: A bitmask indicating the types of updates the consumer **does want** to receive.
- **NegativeUpdateTypeFilter**: A bitmask indicating the types of updates the consumer **does not want** to receive.

These fields act as filters applied to incoming update messages. If both fields are present, the server will filter using the last field specified. This mechanism provides fine-grained control over update traffic, improving efficiency and clarity in data processing throughout the session.

Application using this API must implement proper encoding and decoding.

For an overview of the login process, refer to Section 6.3 of this Guide.

Refer to the *Enterprise Transport API RDM Usage Guide* for the following information:

- For Login Request definition, refer to section “Login Request Message”.
- For possible **updateType** values per domain (e.g., “Market Price”, “Market By Order”, and so on), refer to the corresponding chapters.
- For configuration of **UpdateTypeFilter** and **NegativeUpdateTypeFilter**, refer to section “Login Request Elements”.

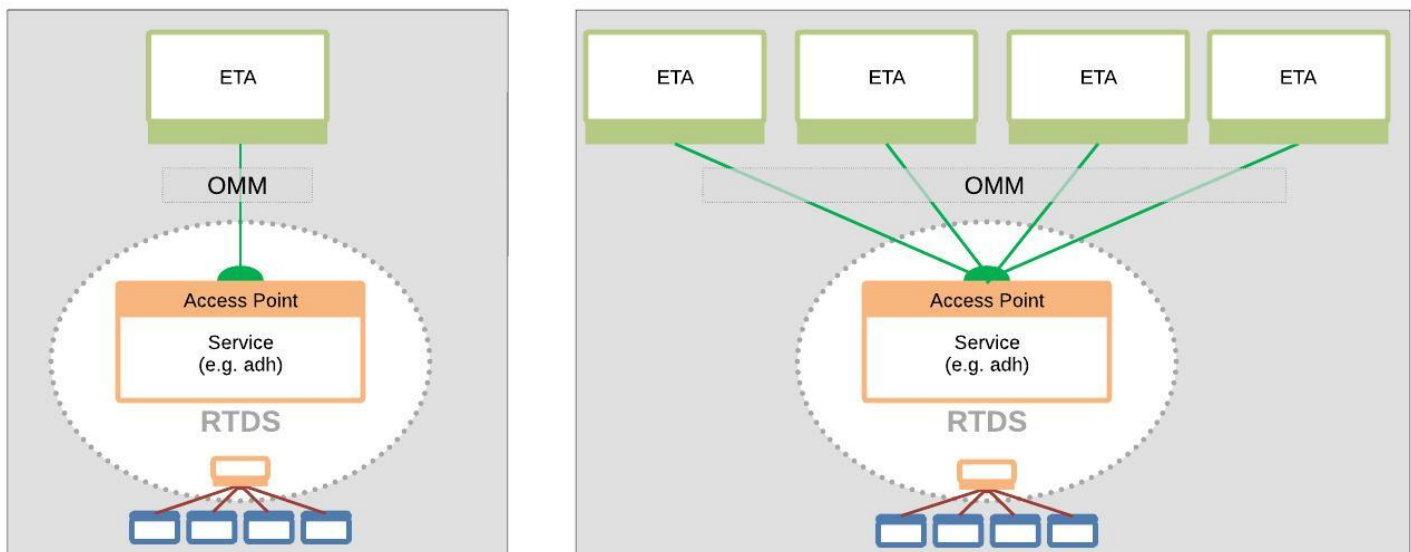
NOTE: Update filtering requires explicit support from the infrastructure. Consult your infrastructure provider to confirm compatibility.

3.3 Providers

Providers make their services available to consumers through LSEG Real-Time Distribution System infrastructure components. Every provider-based application must attach to a provider access point to inter-operate with consumers. All provider access points are considered concrete and are implemented by an LSEG Real-Time Distribution System infrastructure component (like the LSEG Real-Time Advanced Distribution Hub).

Examples of providers include:

- A user who receives a subscription request from LSEG Real-Time Distribution System.
- A user who publishes data into LSEG Real-Time Distribution System, whether in response to a request or using a broadcast-publishing style.
- A user who receives post data from LSEG Real-Time Distribution System. Providers can handle such concepts as receiving requests for contributions/inserts, or receiving publication requests.
- A user who sends and/or receives generic messages with LSEG Real-Time Distribution System.



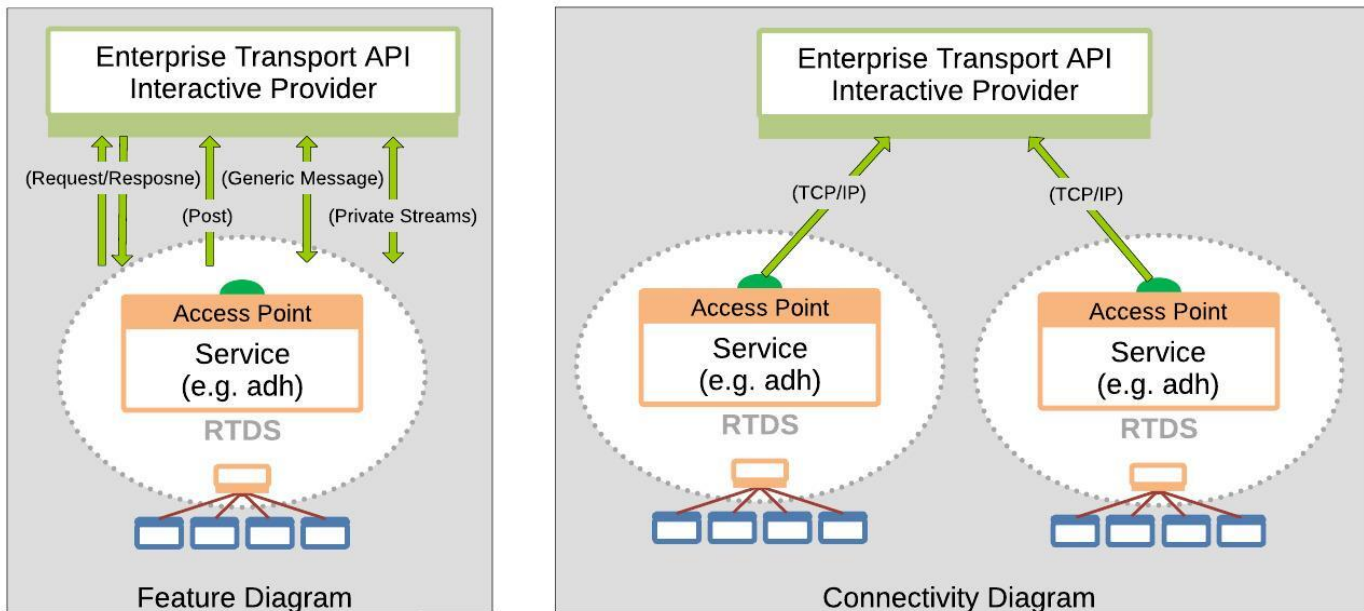
Legend:

ETA: Enterprise Transport API
 OMM: Open Message Model
 RTDS: LSEG Real-Time Distribution System

Figure 15. Provider Access Point

3.3.1 Interactive Providers

An **interactive provider** is one that communicates with the LSEG Real-Time Distribution System, accepting and managing multiple connections with LSEG Real-Time Distribution System components. The following diagram illustrates this concept.



Legend:

RTDS: LSEG Real-Time Distribution System

Figure 16. Interactive Providers

An interactive provider receives connection requests from the LSEG Real-Time Distribution System. The Interactive Provider responds to requests for information as to what services, domains, and capabilities it can provide or for which it can receive requests. It may also receive and respond to requests for information about its data dictionary, describing the format of expected data types. After this is completed, its behavior is interactive.

For the LSEG Real-Time Distribution System adopters, the Interactive Provider is similar in concept to the legacy Sink-Driven Server or Managed Server Application. Interactive Providers act like servers in a client-server relationship. A Enterprise Transport API interactive provider can accept and manage connections from multiple LSEG Real-Time Distribution System components.

3.3.1.1 Request /Response

In a standard request/response scenario, the interactive provider receives requests from consumers on LSEG Real-Time Distribution System (e.g., "Provide data for item AAPL"). The consumer then expects the interactive provider to provide a response, status, and possible updates whenever the information changes. If the item cannot be provided by the interactive provider, the consumer expects the provider to reject the request by providing an appropriate response - commonly a status message with state and text information describing the reason. Request and response behavior is supported in all domains, not simply Market-Data-based domains.

Interactive providers can receive any consumer-style request described in the consumer section of this document, including batch requests, views, symbol lists, pause/resume, etc. Provider applications should respond with a negative acknowledgment or response if the interactive application cannot provide the expected response to a request.

3.3.1.2 Posts

The interactive provider can receive post messages via LSEG Real-Time Distribution System. Post messages will state whether an acknowledgment is required. If required, LSEG Real-Time Distribution System will expect the interactive provider to provide a response, in the form of a positive or negative acknowledgment. Post behavior is supported in all domains, not simply Market-Data-based domains. Whenever an interactive provider connects to LSEG Real-Time Distribution System and publishes the supported domains, the provider states whether it supports post messages.

Further discussion on posting can be found in Section 13.9.

3.3.1.3 Generic Messages

Using generic messages, an application can send or receive bi-directional messages. Whereas a request/response type message flows from LSEG Real-Time Distribution System to an interactive provider, generic messages can flow in any direction and do not expect a response. When using generic messages, the application need not conform to the request/response flow. A generic message can contain any Open Message Model data type.

Interactive providers can receive a generic message from and publish a generic message to LSEG Real-Time Distribution System.

Generic message behavior is supported in all domains, not simply Market-Data-based domains. If a generic message is sent to a component (e.g., a legacy application) which does not understand generic messages, the component ignores it.

Additional details on generic messages can be found in Section 12.2.6.

3.3.1.4 Private Streams

In a typical private stream scenario, the interactive provider can receive requests for a private stream. Once established, interactive providers can receive any consumer-style request via a private stream, described in the consumer section of this document, including Batch requests, Views, Symbol Lists, Pause/Resume, Posting, etc. Provider applications should respond with a negative acknowledgment or response if the interactive application cannot provide the expected response to a request.

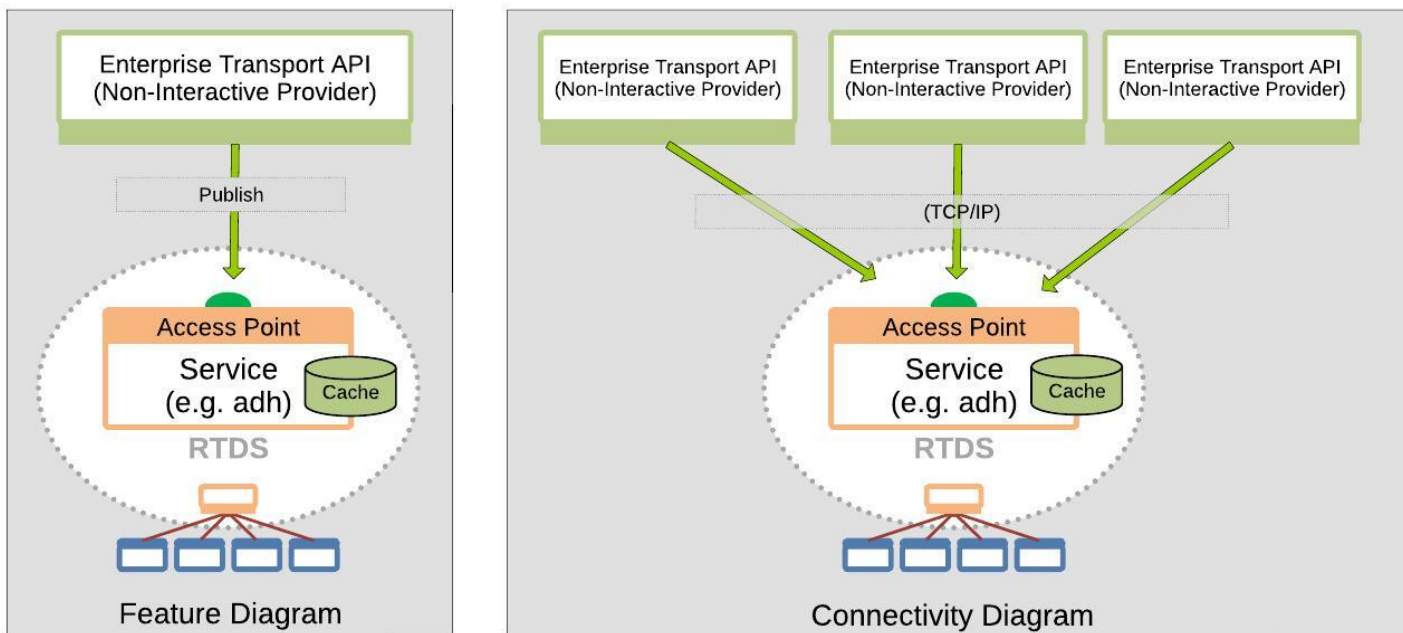
3.3.1.5 Tunnel Streams (Available Only in ETA Reactor and EMA)

An interactive provider can receive requests for a tunnel stream when using the ETA Reactor or EMA. When creating a tunnel stream, the consumer indicates any additional behaviors to enforce, which is exchanged with the provider application end point. The provider end-point acknowledges creation of the stream as well as the behaviors that it will enforce on the stream. After the stream is established, the consumer can exchange any content it wants, though the tunnel stream will enforce behaviors on the transmitted content as negotiated with the provider.

A tunnel stream allows for multiple substreams to exist, where substreams follow from the same general stream concept, except that they flow and coexist within the confines of a tunnel stream.

3.3.2 Non-Interactive Providers

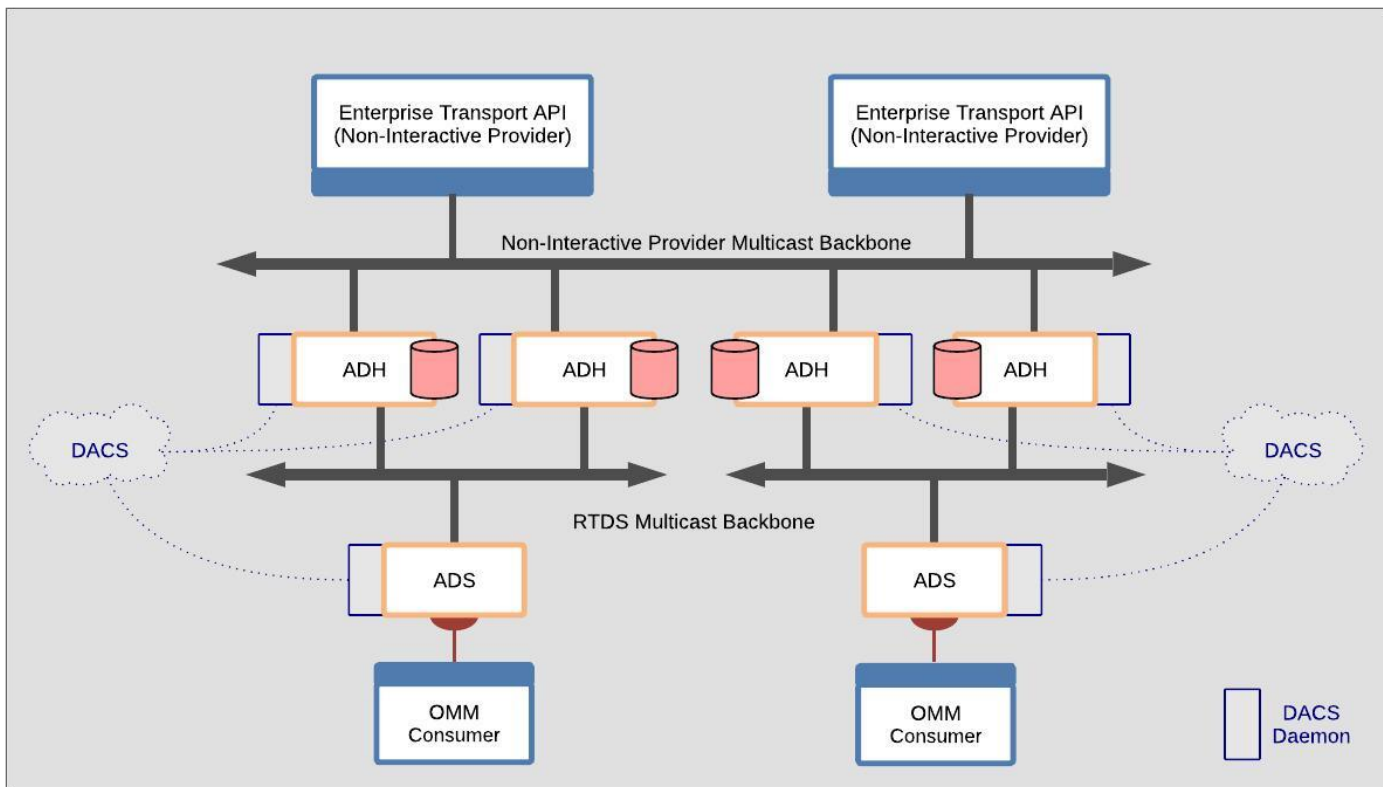
A **non-interactive provider** writes a provider application that connects to LSEG Real-Time Distribution System and sends a specific set of non-interactive data (services, domains, and capabilities).



Legend:

RTDS: LSEG Real-Time Distribution System

Figure 17. Non-Interactive Provider: Point-To-Point

**Legend:**

ADH:	LSEG Real-Time Advanced Distribution Hub
ADS:	LSEG Real-Time Advanced Distribution Server
DACS:	Data Access Control System
OMM:	Open Message Model
RTDS:	LSEG Real-Time Distribution System

Figure 18. Non-Interactive Provider: Multicast

After a non-interactive provider connects to LSEG Real-Time Distribution System, the non-interactive provider can start sending information for any supported item and domain. For the LSEG Real-Time Distribution System adopters, the non-interactive provider is similar in concept to what was once called the Src-Driven, or Broadcast Server Application.

Non-interactive providers act like clients in a client-server relationship. Multiple non-interactive providers can connect to the same LSEG Real-Time Distribution System and publish the same items and content. For example, two non-interactive providers can publish the same or different fields for the same item "INTC.O" to the same LSEG Real-Time Distribution System.

Non-interactive provider applications can connect using a point-to-point TCP-based transport as shown in Figure 17, or using a multicast transport as shown in Figure 18.

The main benefit of this scenario is that all publishing traffic flows from top to bottom: the way a system normally expects updating data to flow. In the local publishing scenario, posting is frequently done upstream and must contend with a potential Infrastructure bias in prioritization of upstream versus downstream traffic.

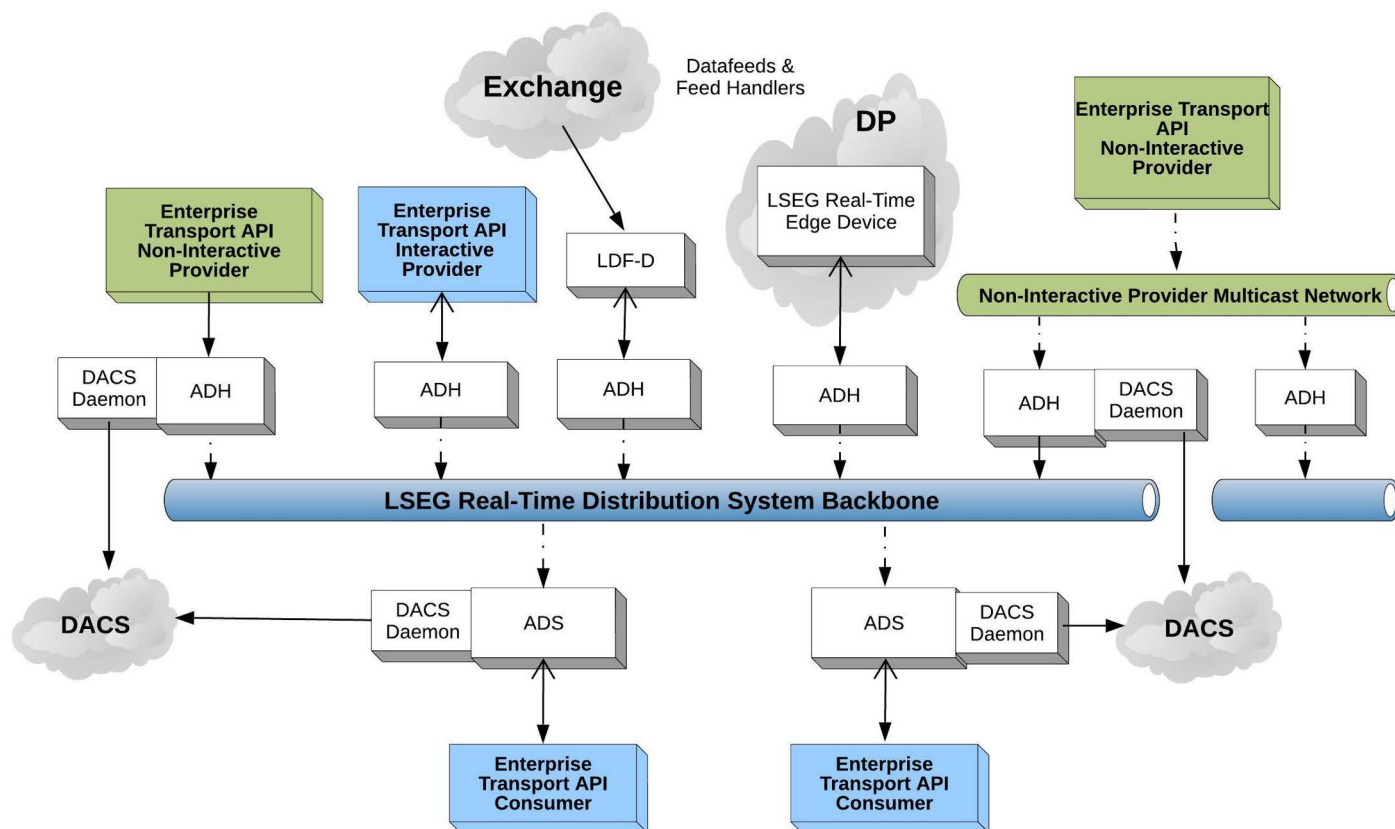
4 System View

4.1 System Architecture Overview

An LSEG Real-Time Distribution System network typically hosts the following components:

- Core Infrastructure: LSEG Real-Time Advanced Distribution Server (ADS), LSEG Real-Time Advanced Distribution Hub (ADH), etc.
- Consumer applications that typically request and receive information from the network
- Provider applications that typically write information to the network. Provider applications fall into one of two categories:
 - Interactive provider applications which receive and interpret request messages and reply back with any needed information.
 - Non-interactive provider applications which publish data, regardless of user requests or which applications consume the data.
- Permissioning infrastructure: Data Access Control System
- Devices that interact with the markets: Data Feed Direct and LSEG Real-Time Edge Device

The following figure illustrates a typical deployment of an LSEG Real-Time Distribution System network and some of its components. Components that use the Enterprise Transport API can alternatively choose to leverage RFA, depending on user needs and required access levels. The next sections describe the components shown in the diagram and explain how the Enterprise Transport API integrates with each.



Legend:

ADH: LSEG Real-Time Advanced Distribution Hub
 ADS: LSEG Real-Time Advanced Distribution Server
 DACS: Data Access Control System
 LDF-D: LSEG Data Feed Direct
 DP: Delivery Platform

Figure 19. Typical LSEG Real-Time Distribution System Components

4.2 LSEG Real-Time Advanced Distribution Server

The LSEG Real-Time Advanced Distribution Server provides a consolidated distribution solution for LSEG, value-added, and third-party data for trading-room systems. It distributes information using the same Open Message Model and Rssl Wire Format protocols exposed by the Enterprise Transport API.

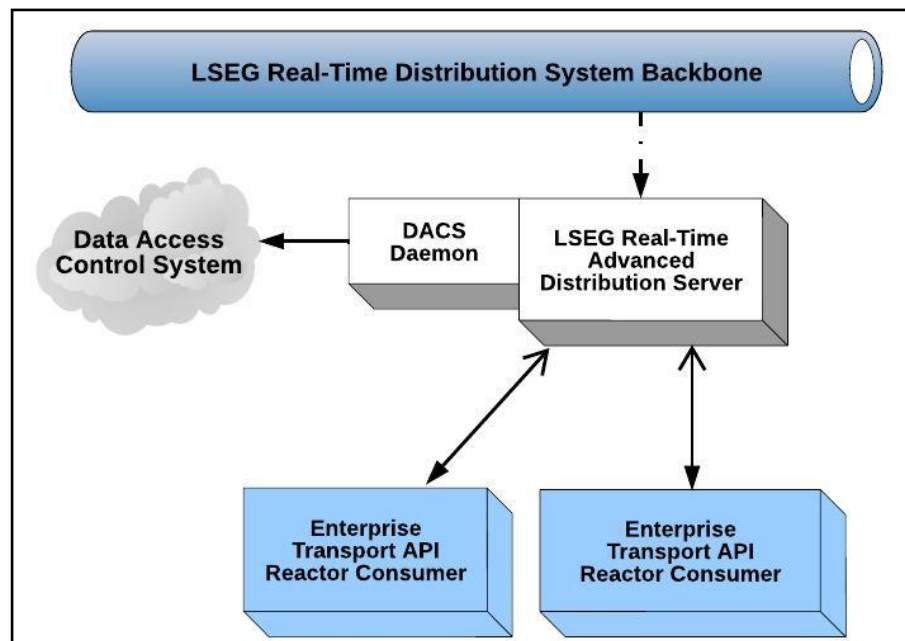


Figure 20. Enterprise Transport API and LSEG Real-Time Advanced Distribution Server

As a distribution device for market data, the LSEG Real-Time Advanced Distribution Server delivers data from the LSEG Real-Time Advanced Distribution Hub. Because the LSEG Real-Time Advanced Distribution Server leverages multiple threads, it can offload the encoding, fan out, and writing of client data. By distributing its tasks in this fashion, LSEG Real-Time Advanced Distribution Server can support a large number of client applications.

The LSEG Real-Time Advanced Distribution Server communicates with its API clients via point-to-point communication.

4.3 LSEG Real-Time Advanced Distribution Hub

The **LSEG Real-Time Advanced Distribution Hub** is a networked, data distribution server that runs in the LSEG Real-Time Distribution System. It consumes data from a variety of content providers and reliably fans this data out to multiple LSEG Real-Time Advanced Distribution Servers over a multicast backbone. Enterprise Transport API-based non-interactive or interactive provider applications can publish content directly into an LSEG Real-Time Advanced Distribution Hub, thus distributing data more widely across the network. Non-interactive provider applications can publish content to an LSEG Real-Time Advanced Distribution Hub via TCP or multicast connection types.

The LSEG Real-Time Advanced Distribution Hub leverages multiple threads, both for inbound traffic processing and outbound data fanout. By leveraging multiple threads, the LSEG Real-Time Advanced Distribution Hub can offload the overhead associated with request and response processing, caching, data conflation, and fault tolerance management. By offloading overhead in such a fashion, the LSEG Real-Time Advanced Distribution Hub can support high throughputs.

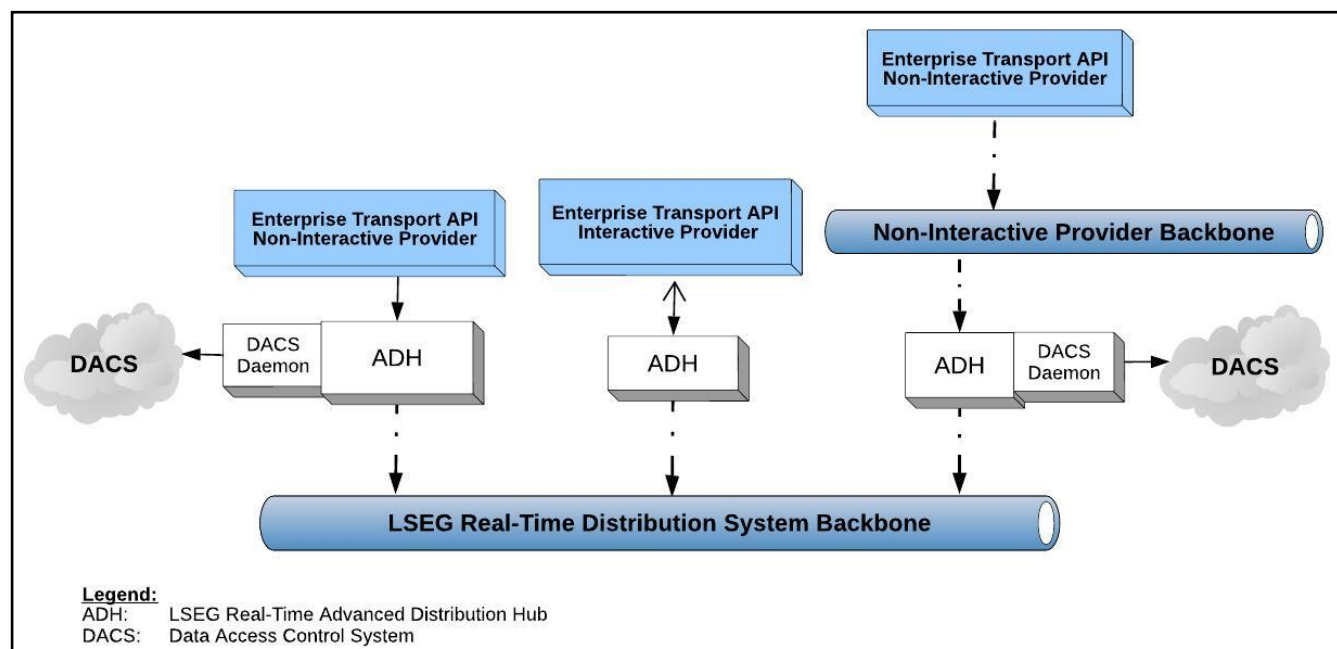


Figure 21. Enterprise Transport API and the LSEG Real-Time Advanced Distribution Hub

4.4 Elektron

The **Delivery Platform** is an open, global, ultra-high-speed network and hosting environment, which allows users to access and share a variety of content including Real-Time data. The Delivery Platform allows access to information from a wide network of content providers, including exchanges, where all exchange data is normalized using the Open Message Model.

Real-Time content, one of the content sets available via the Delivery Platform, can be obtained by consuming applications written to any Real-Time API or by connecting to on-prem LSEG Real-Time Distribution Systems (i.e., cascaded LSEG Real-Time Advanced Distribution Hub and LSEG Real-Time Advanced Distribution Server). Consumer applications authenticate and can discover endpoints via the Delivery Platform and use that information to connect to Real-Time -- Optimized (LSEG's cloud offering) which ultimately sources data from LSEG Real-Time infrastructure.

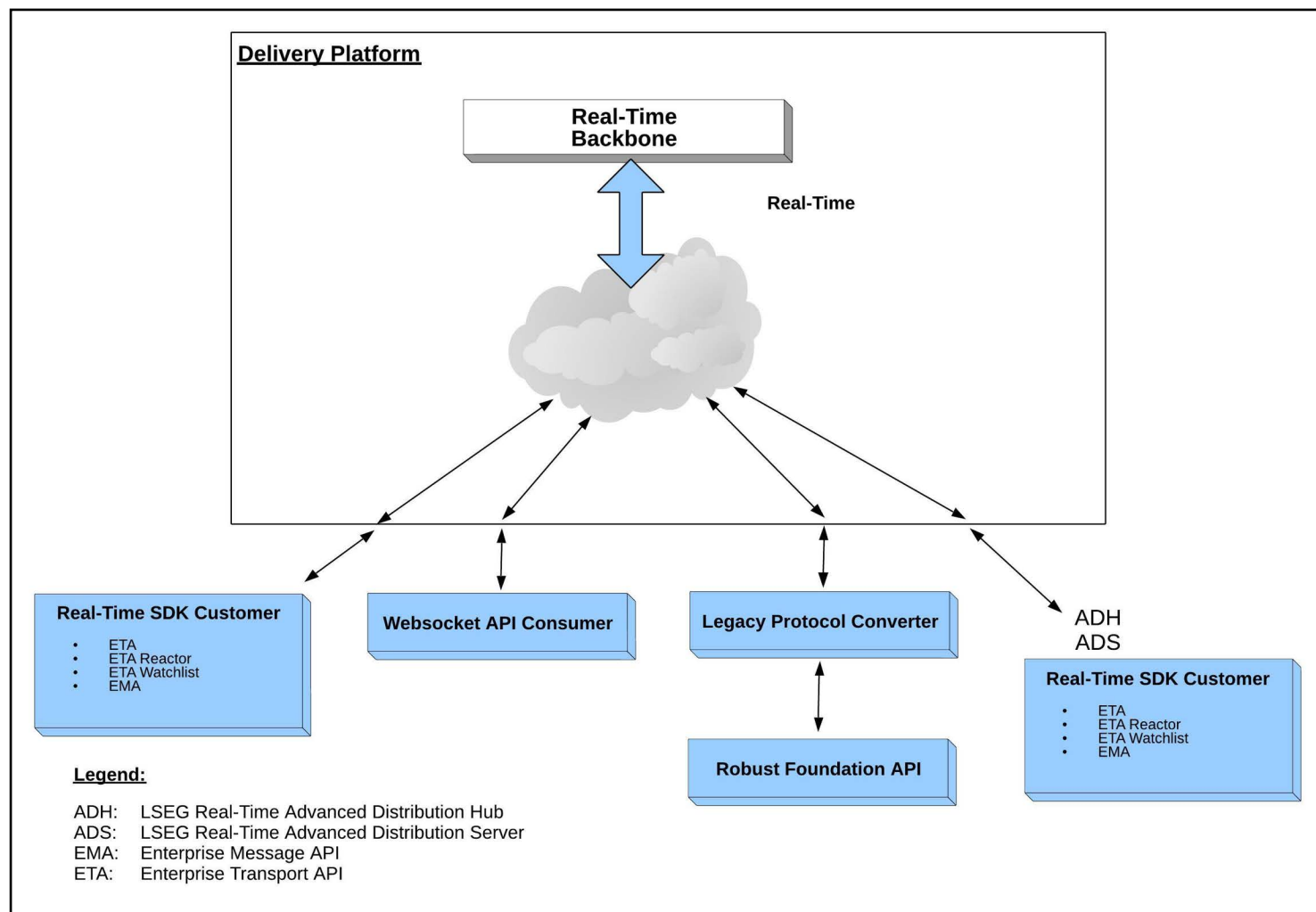


Figure 22. LSEG Real-Time APIs and Delivery Platform

4.5 Data Feed Direct

Data Feed Direct is a fully managed LSEG exchange feed providing an ultra-low-latency solution for consuming data from specific exchanges. The Data Feed Direct normalizes all exchange data using the Open Message Model.

To access this content, an Enterprise Transport API consumer application can connect directly to the Data Feed Direct or via a cascaded LSEG Real-Time Distribution System architecture.

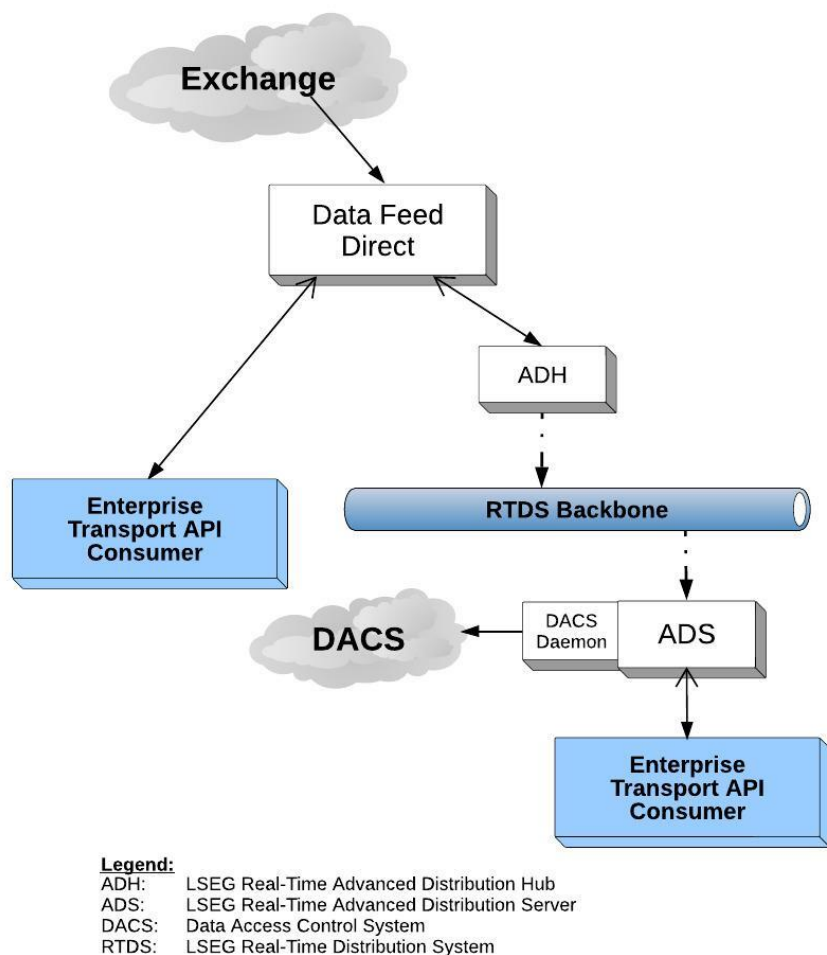


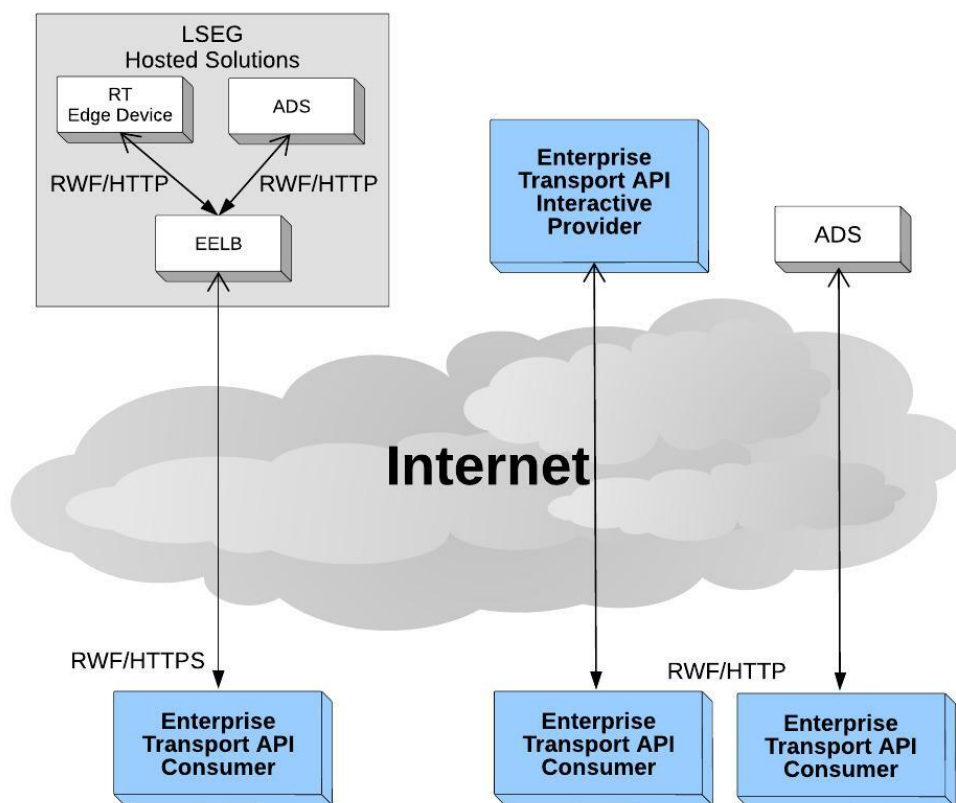
Figure 23. Enterprise Transport API and Data Feed Direct

4.6 Internet Connectivity via HTTP and HTTPS

Consumer and provider applications can use the Enterprise Transport API to establish encrypted connections or HTTPS (on certain platforms) over the public Internet.

- Consumer and non-interactive provider applications can establish connections via HTTP tunneling, socket and websocket connections.
- LSEG Real-Time Advanced Distribution Servers and OMM interactive provider applications can accept incoming Enterprise Transport API connections tunneled via HTTP (such functionality is available across all supported platforms).
- Consumer applications can leverage HTTPS to establish an encrypted tunnel to certain LSEG hosted solutions, performing key and certificate exchange.

For further details, refer to Section 10.15.



Legend:

ADS: LSEG Real-Time Advanced Distribution Server
 EELB: Encryption Endpoint and Load Balancer
 RT: Real-Time
 RWF: Wire Format
 SSL: Sink Source Library

Figure 24. Enterprise Transport API and Internet Connectivity

4.7 Direct Connect

The Enterprise Transport API allows OMM interactive provider applications and consumer applications to directly connect to one another. This includes Open Message Model applications written to RFA. The following diagram illustrates various direct connect combinations.

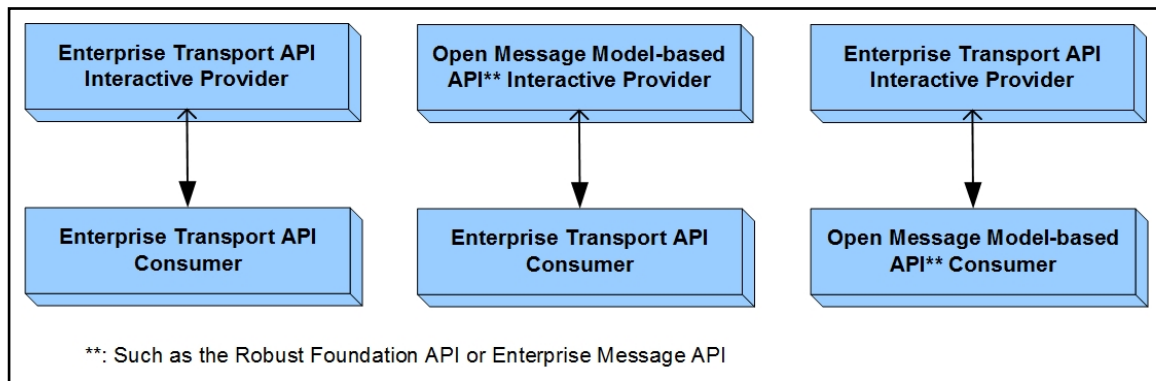


Figure 25. Transport API and Direct Connections

5 Model and Package Overviews

5.1 Enterprise Transport API Models

5.1.1 Open Message Model

The **Open Message Model** is a collection of message header and data constructs. Some Open Message Model message header constructs (such as the Update message) have implicit market logic associated with them, while others (such as the Generic message) allow for free-flowing bi-directional messaging. You can combine Open Message Model data constructs in various ways to model data ranging from simple (i.e., flat) primitive types to complex multi-level hierarchal data.

The layout and interpretation of any specific Open Message Model (also referred to as a domain model) is described within that model's definition and is not coupled with the API. The Open Message Model is a flexible and simple tool that provides the building blocks to design and produce domain models to meet the needs of the system and its users. The Enterprise Transport API provides structural representations of Open Message Model constructs and manages the Rssl Wire Format binary-encoded representation of the Open Message Model. Users can leverage LSEG-provided Open Message Model constructs to consume or provide Open Message Model data throughout the LSEG Real-Time Distribution System.

5.1.2 Rssl Wire Format

Rssl Wire Format is the encoded representation of the Open Message Model; a highly-optimized, binary format designed to reduce the cost of data distribution compared to previous wire formats. Binary encoding represents data in the machine's native manner, enabling further use in calculations or data manipulations. Rssl Wire Format allows for serializing Open Message Model message and data constructs in an efficient manner while still allowing you to model rich content types. You can use Rssl Wire Format to distribute field identifier-value pair data (similar to Marketfeed), self-describing data (similar to Qform), as well as more complex, nested hierarchal content.

5.1.3 Domain Message Model

A Domain Message Model describes a specific arrangement of Open Message Model message and data constructs. A Domain Message Model defines any:

- Specialized behavior associated with the domain
- Specific meanings or semantics associated with the message data

Unless a Domain Message Model specifies otherwise, any implicit market logic associated with a message still applies (e.g., an Update message indicates that previously received data is being modified by corresponding data from the Update message).

5.1.3.1 Domain Model

A **Domain Model** is a domain message model typically provided or consumed by an LSEG product (i.e., LSEG Real-Time Distribution System, Data Feed Direct, or LSEG Real-Time). Some currently-defined Domain Models allow for authenticating to a provider (e.g., Login), exchanging field or enumeration dictionaries (e.g., Dictionary), and providing or consuming various types of market data (e.g., Market Price, Market by Order, Market by Price). LSEG's defined models have a domain value of less than 128. For extended definitions of the currently-defined Domain Models, refer to the *Transport API Domain Model Usage Guide*.

5.1.3.2 User-Defined Domain Model

A **User-Defined Domain Model** is a Domain Message Model defined by a third party. These might be defined to solve a need specific to a user or system in a particular deployment and which is not resolved through the use of a Domain Model. Any user-defined model must use a domain value between 128 and 255.

Customers can have their domain model designer work with LSEG to define their model as a standard Domain Model. Working directly with LSEG can help ensure interoperability with future Domain Model definitions and with other LSEG products.

5.2 Packages

The Enterprise Transport API consists of several packages, each serving a different purpose within an application. While some packages are interdependent, others can be used alone or with other packages. Each package serves a distinct purpose as described in the following sections.

As needs evolve, additional packages can be added to the Enterprise Transport API.

5.2.1 Transport Package

The **Transport Package** provides a mechanism to efficiently distribute messages across a variety of communication protocols. This package provides a receiver-transparent way for senders to combine or pack multiple messages into one outbound packet, and it will internally fragment and reassemble messages which exceed the size of an outbound packet. This package exposes structural representations to manage connection properties and information. The Transport Package includes interface functions that assist with establishing connections and the sending or receiving of data. This package utilizes some header files from the Data Package, but has no other dependencies other than system libraries.

To access all transport functionality, an application must import from the **com.refinitiv.eta.transport** package.

The Transport Package is described in more detail in Chapter 10.

5.2.2 Codec Package

The **Codec Package** defines object-oriented representations for everything you need to encode and decode Open Message Model content. This includes definitions that:

- Expose data types (primitive and container types) and manage their Rssl Wire Format binary representation. These data types in turn make up components of Open Message Model data.
 - Primitive types are simple, atomically updating constructs, usually provided by the operating system (e.g., Integer, Date).
 - Container types can model more complex data and be modified more granularly than a primitive type (e.g., field identifier-value pairs, key-value pairs, self-describing name-value pairs).
- Expose message classes and manage their Rssl Wire Format binary-encoded representation. The Codec defines message header elements that flow between various applications in LSEG Real-Time Distribution System (e.g., update messages). Some header elements are standard to the market data environment (such as conflation information, state information, permission information, and item key elements used for stream identification). Message headers contain generic attributes in which usage and meaning are defined within specific DMMs (e.g., Market Price, Market By Order). All messages can carry payload information of varying format and layouts.

To access codec package functionality, an application must import from the **com.refinitiv.eta.codec** package.

The codec package is described with more detail in Chapter 11 and Chapter 12.

6 Building an Open Message Model Consumer

6.1 Overview

This chapter provides an overview of how to create a consumer application. A consumer application can establish a connection to other interactive provider applications, including the LSEG Real-Time Distribution System, Data Feed Direct, and Delivery Platform. After connecting successfully, a consumer can then consume (i.e., send data requests and receive responses) and publish data (i.e., post data) or forward data (i.e., Round Trip Time messages).

The following steps summarize the general process:

1. Establish network communication.
2. Log in.
3. Obtain source directory information.
4. Load or download all necessary dictionary information.
5. Issue requests, process responses, forward generic messages, and/or post information.
6. Log out and shut down.

The **Consumer** example application included with the Enterprise Transport API products provides an example implementation of a consumer application. The application is written with simplicity in mind and demonstrates the uses of the Enterprise Transport API. Portions of functionality have been abstracted and can easily be reused, though you might need to modify it to achieve your own unique performance and functionality goals.

6.2 Establish Network Communication

The first step of any Enterprise Transport API consumer application is to establish a network connection with its peer component (i.e., another application with which to interact). A consumer typically creates an outbound connection to the well-known hostname and port of an Interactive Provider. The consumer uses the **Transport.connect** function to initiate the connection and then performs any additional connection initialization processes as described in this document.

After the consumer's connection is active, ping messages might need to be exchanged. The negotiated ping timeout is available via the **Channel**. The connection can be terminated if ping heartbeats are not sent or received within the expected time frame. LSEG recommends sending ping messages at intervals one third the size of the ping timeout.

Detailed information and use case examples for using various transports provided by the Enterprise Transport API are specified in Chapter 10, Transport Package Detailed View.

6.3 Perform Login Process

Applications authenticate with one another using the Login domain model. A consumer must register with the system using a Login request prior to issuing any other requests or opening any other streams.

After receiving a Login request, an interactive provider determines whether a user is permissioned to access the system. The interactive provider sends back a Login response, indicating to the consumer whether access is granted.

- If the application is denied, the Login stream is closed, and the consumer application cannot send additional requests.
- If the application is granted access, the Login response contains information about available features, such as Posting, Pause and Resume, and the use of Dynamic Views. The consumer application can use this information to tailor its interaction with the provider.

Content is encoded and decoded using the Message Package (described in Chapter 12, Message Package Detailed View) and the Data Package (described in Chapter 11, Data Package Detailed View). Further information about Login domain usage and messaging is available in the *Transport API LSEG Domain Model Usage Guide*.

6.4 Obtain Source Directory Information

The Source Directory domain model conveys information about all available services in the system. A consumer typically requests a Source Directory to retrieve information about available services and their capabilities. This includes information about supported domain types, the service's state, the quality of service, and any item group information associated with the service. At minimum, LSEG recommends that the application requests the Info, State, and Group filters for the Source Directory.

- The Source Directory Info filter contains the service name and **serviceId** information for all available services. When the consumer discovers an appropriate service, it uses the service's **serviceId** on all subsequent requests to that service.
- The Source Directory State filter contains status information for service, which informs the consumer whether the service is Up and available, or Down and unavailable.
- The Source Directory Group filter conveys item group status information, including information about group states, as well as the merging of groups. Additional information on item groups is available in Section 13.4.

Content is encoded and decoded using the Enterprise Transport API's Message Package (as described in Chapter 12, Message Package Detailed View) and Data Package (as described in Chapter 11, Data Package Detailed View). Information about the Source Directory domain and its associated filter entry content is available in the *Enterprise Transport API LSEG Domain Model Usage Guide*.

6.5 Load or Download Necessary Dictionary Information

Some data requires the use of a dictionary for encoding or decoding. This dictionary typically defines type and formatting information and directs the application as to how to encode or decode specific pieces of information. Content that uses the **FieldList** type requires the use of a field dictionary (usually the **RDMFieldDictionary**, though it could also be a user-defined or modified field dictionary).

After a dictionary has been loaded from local files or downloaded through the Dictionary Domain, the API maintains a cached copy of the dictionary. Subsequent application access to dictionary information uses this cached copy and does not result in an additional dictionary request being sent to the provider.

A local dictionary is loaded from dictionary files available on the client system. A remote dictionary is obtained from a provider using Dictionary Domain requests. Once successfully loaded or downloaded, both local and remote dictionaries are maintained in cache and accessed in the same manner by the application.

A source directory message should provide information about:

- Any dictionaries required to decode the content provided on a service.
- Which dictionaries are available for download.

A consumer application can determine whether to load necessary dictionary information from a local file or download the information from the provider if available.

- If loading from a file, the Enterprise Transport API offers several utility functions to load and manage a properly-formatted field dictionary.
- If downloading information, the application issues a request using the Dictionary domain model. The provider application should respond with a dictionary response. Because a dictionary response often contains a large amount of content, it is typically broken into a multi-part message. the Enterprise Transport API offers several utility functions for encoding and decoding of the Dictionary domain content.

For information on the utility functions used in both instances and for information about the Dictionary domain and its expected content formats, refer to the *Transport API LSEG Domain Model Usage Guide*.

Content is encoded and decoded using the Enterprise Transport API Message Package (as described in 12, Message Package Detailed View) and the Enterprise Transport API Data Package (as described in 11, Data Package Detailed View).

6.6 Issue Requests, Forward Generic Messages, and/or Post Information

After the consumer application successfully logs in and obtains Source Directory and Dictionary information, it can request additional content. When issuing the request, the consuming application can use the **serviceId** of the desired service, along with the stream's identifying information. Requests can be sent for any domain using the formats defined in that domain model specification. Domains provided by LSEG are defined in the *Enterprise Transport API LSEG Domain Model Usage Guide*.

At this point, a consumer application can also post information to capable provider applications. For more information, refer to Section 13.9.

Content is encoded and decoded using the Enterprise Transport API Message Package (as described in Chapter 12, Message Package Detailed View) and Data Package (as described in Chapter 11, Data Package Detailed View).

6.7 Log Out and Shut Down

When the consumer application is done retrieving, forwarding, or posting content, it should close all open streams and shut down the network connection. Issuing an **CloseMsg** for the **streamId** associated with the Login closes all streams opened by the consumer.

- For more information on closing streams, refer to Section 12.2.5.
- For information on the Message Package, refer to Chapter 12, Message Package Detailed View.

When shutting down the consumer, the application should release any unwritten pool buffers obtained from **Channel1.getBuffer**. Calling **Channel1.close** terminates the connection to the provider application. Detailed information and transport code examples are provided in Chapter 10, Transport Package Detailed View.

6.8 Additional Consumer Details

The following locations provide specific details about using consumers and the Enterprise Transport API:

- The **consumer** application demonstrates one way of implementing of a consumer application. The application's source code contain additional information about specific implementation and behaviors.
- For reviewing high-level encoding and decoding concepts, refer to Chapter 9, Encoding and Decoding Conventions.
- For a detailed look at the Transport Package, used for the application's network communication, refer to Chapter 10, Transport Package Detailed View.
- For a detailed look at the Data Package, typically used for encoding and decoding payload content, refer to Chapter 11, Data Package Detailed View.
- For a detailed look at the Message Package, used for all message encoding and decoding, refer to Chapter 12, Message Package Detailed View.
- For specific information about the Domain Message Models required by this application type, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

7 Building an Open Message Model Interactive Provider

7.1 Overview

This chapter provides a high-level description of how to create an OMM interactive provider application. An OMM interactive provider application opens a listening socket on a well-known port allowing consumer applications to connect. After connecting, consumers can request data from the interactive provider.

The following steps summarize this process:

- Establish network communication
- Accept incoming connections
- Handle login requests
- Provide source directory information
- Provide or download necessary dictionaries
- Handle requests and post messages
- Dispatch Round Trip Time messages
- Sends out messages for round trip latency monitoring.
- Disconnect consumers and shut down

The **Provider** example application included with the Enterprise Transport API package provides one way of implementing an OMM interactive provider. The application is written with simplicity in mind and demonstrates the use of the Enterprise Transport API. Portions of the functionality are abstracted for easy reuse, though you might need to customize it to achieve your own unique performance and functionality goals.

7.2 Establish Network Communication

The first step of any Enterprise Transport API Interactive Provider application is to establish a listening socket, usually on a well-known port so that consumer applications can easily connect. The provider uses the **Transport.bind** function to open the port and listen for incoming connection attempts.

Whenever a consumer application attempts to connect, the provider uses the **Server.accept** function to begin the connection initialization process.

Once the connection is active, the consumer and provider applications might need to exchange ping messages. A negotiated ping timeout is available via **Channel** corresponding to each connection (this value might differ on a per-connection basis). The provider may choose to terminate a connection if ping heartbeats are not sent or received within the expected time frame. LSEG recommends sending ping messages at intervals one-third the size of the ping timeout.

For detailed information and use cases for the various transports provided by the Enterprise Transport API, refer to Chapter 10, Transport Package Detailed View.

7.3 Perform Login Process

Applications authenticate with one another using the Login domain model. An OMM interactive provider must handle the consumer's Login request messages and supply appropriate responses.

After receiving a Login request, the interactive provider can perform any necessary authentication and permissioning.

- If the Interactive Provider grants access, it should send a **RefreshMsg** to convey that the user successfully connected. This message should indicate the feature set supported by the provider application.

- If the Interactive Provider denies access, it should send a **StatusMsg**, closing the connection and informing the user of the reason for denial.

Content is encoded and decoded using the Transport API Message Package (as described in 12, Message Package Detailed View) and the Transport API Data Package (as described in 11, Data Package Detailed View). For further information on Login domain usage and messaging, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

7.4 Provide Source Directory Information

The Source Directory domain model conveys information about all available services in the system. A consumer typically requests a Source Directory to retrieve information about available services and their capabilities. This includes information about supported domain types, the service's state, the Quality of Service, and any item group information associated with the service. LSEG recommends that at a minimum, an interactive provider supply the Info, State, and Group filters for the Source Directory.

- The Source Directory Info filter contains the name and **serviceId** for each available service. The interactive provider should populate the filter with information specific to the services it provides.
- The Source Directory State filter contains status information for the service informing the consumer whether the service is Up (available) or Down (unavailable).
- The Source Directory Group filter conveys item group status information, including information about group states, as well as the merging of groups. If a provider determines that a group of items is no longer available, it can convey this information by sending either individual item status messages (for each affected stream) or a Directory message containing the item group status information. Additional information about item groups is available in Section 13.4.

Content is encoded and decoded using the Enterprise Transport API's Message Package (as described in Chapter 12, Message Package Detailed View) and Data Package (as described in Chapter 11, Data Package Detailed View). For details on the Source Directory domain and all of its associated filter entry content, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

7.5 Provide or Download Necessary Dictionaries

Some data requires the use of a dictionary for encoding or decoding. The dictionary typically defines type and formatting information, and tells the application how to encode or decode information. Content that uses the **FieldList** type requires the use of a field dictionary (usually the **RDMFieldDictionary**, though it can instead be user-defined or a modified field dictionary).

The Source Directory message should notify the consumer about dictionaries needed to decode content sent by the provider. If the consumer needs a dictionary to decode content, it is ideal that the interactive provider application also make this dictionary available to consumers for download. The provider can inform the consumer whether the dictionary is available via the Source Directory.

If connected to a supporting LSEG Real-Time Advanced Distribution Hub, a provider application may also download the RWFFld and RWFEnum dictionaries to retrieve the appropriate dictionary information for providing field list content. A provider can use this feature to ensure it has the appropriate version of the dictionary or to encode data. The LSEG Real-Time Advanced Distribution Hub that supports the Provider Dictionary Download feature sends a Login request message containing the **SupportProviderDictionaryDownload** login element. The dictionary request is sent using the Dictionary domain model.¹

The Enterprise Transport API offers several utility functions for loading, downloading, and managing a properly-formatted field dictionary. There are also utility functions provided to help the provider encode into an appropriate format for downloading or decoding downloaded dictionary. For available Dictionary utility methods, refer to the *Transport API Java Edition LSEG Domain Model Usage Guide*.

Content is encoded and decoded using the Enterprise Transport API Message Package (as described in Chapter 12, Message Package Detailed View) and the Transport API Data Package (as described in Chapter 11, Data Package Detailed View).

Information about the Login and Dictionary domains, their expected content and formatting, and dictionary utility functions, is available in the *Enterprise Transport API LSEG Domain Model Usage Guide*.

1. Because this is instantiated by the provider, the application should use a **streamId** with a negative value. Additional details are provided in subsequent chapters.

7.6 Handle Requests and Post Messages

A provider can receive a request for any domain, though this should typically be limited to the domain capabilities indicated in the Source Directory. When a request is received, the provider application must determine if it can satisfy the request by:

- Comparing **msgKey** identification information received against the content available from the provider.
- Determining whether it can provide the requested Quality of Service.
- Ensuring that the consumer does not already have a stream open for the requested information.

If a provider can service a request, it should send appropriate responses. However, if the provider cannot satisfy the request, the provider should send a **StatusMsg** to indicate the reason and close the stream. All requests and responses should follow specific formatting as defined in the domain model specification. For details on all domains provided by LSEG, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

If a provider application receives a Post message, the provider should determine the correct handling for the post. This depends on the application's role in the system and might involve storing the post in its cache or passing it farther up into the system. If the provider is the destination for the Post, the provider should send any requested acknowledgments, following the guidelines described in Section 13.9.

Content is typically encoded and decoded using the Enterprise Transport API's Message Package (as described in Chapter 12, Message Package Detailed View) and Data Package (as described in Chapter 11, Data Package Detailed View).

7.7 Dispatch Round Trip Time Messages

Optionally, a provider might send a Round Trip Time (RTT) message to gather statistical information regarding message latency. While the Enterprise Transport API does not regulate rules for implementing RTT, the Enterprise Transport API provides a several examples of applying this feature in the provider applications. In general, if a provider wants to support the RTT feature, the provider must support methods for sending generic RTT messages to consumers as well as extend relevant callback methods for RTT calculation.

For further information, refer to RTT details in the Enterprise Transport API Java Edition *RDM Usage Guide*.

7.8 Disconnect Consumers and Shut Down

When shutting down, the provider application should close the listening socket by calling **Server.close**. Closing the listening socket prevents new connection attempts. The provider application can either leave consumer connections intact or shut them down.

If the provider decides to close consumer connections, the provider should send a **StatusMsg** on each connection's login stream, thus closing the stream. At this point, the consumer should assume that its other open streams are also closed. The provider should then release any unwritten pool buffers it has obtained from **Channel.getBuffer** and call **Channel.close** for each connected client.

For detailed information and use case examples for the transport, refer to Chapter 10, Transport Package Detailed View10, Transport Package Detailed View.

7.9 Additional Interactive Provider Details

For specific details about OMM interactive providers and the Enterprise Transport API use, refer to the following locations:

- The **Provider** application demonstrates one implementation of an OMM interactive provider application. The application's source code has additional information about specific implementation and behaviors.
- To review high-level encoding and decoding concepts, refer to Chapter 9, Encoding and Decoding Conventions.
- For a detailed look at the Transport Package, used for the application's network communication, refer to Chapter 10, Transport Package Detailed View.

- For a detailed look at the Data Package, typically used for encoding and decoding payload content, refer to Chapter 11, Data Package Detailed View.
- For a detailed look at the Message Package, used for all message encoding and decoding, refer to Chapter 12, Message Package Detailed View.
- For specific information about Domain Message Models required by this application type, refer to the *Enterprise Transport API Java Edition LSEG Domain Model Usage Guide*.

8 Building an Open Message Model Non-Interactive Provider

8.1 Overview

This chapter provides an outline of how to create an OMM non-interactive provider application which can establish a connection to an LSEG Real-Time Advanced Distribution Hub server. Once connected, a non-interactive provider can publish information into the LSEG Real-Time Advanced Distribution Hub cache without needing to handle requests for the information. The LSEG Real-Time Advanced Distribution Hub can cache the information and along with other LSEG Real-Time Distribution System components, provide the information to any consumer applications that indicate interest.

The general process can be summarized by the following steps:

- Establish network communication
- Perform Login process
- Perform Dictionary Download
- Provide Source Directory information
- Provide content
- Log out and shut down

Included with the Enterprise Transport API package, the **NIPProvider** example application provides an implementation of a non-interactive provider written with simplicity in mind and demonstrates the use of the Enterprise Transport API. Portions of the functionality are abstracted for easy reuse, though you might need to modify it to achieve your own performance and functionality goals.

Content is encoded and decoded using the Transport API Message Package (as described in Chapter 12, Message Package Detailed View) and the Transport API Data Package (as described in Chapter 11, Data Package Detailed View).

8.2 Establish Network Communication

The first step of any non-interactive provider application is to establish network communication with an LSEG Real-Time Advanced Data Hub server. To do so, the OMM non-interactive provider typically creates an outbound connection to the well-known hostname and port of an LSEG Real-Time Advanced Distribution Hub. The non-interactive provider application uses the **Transport.connect** method to initiate the connection process and then performs connection initialization processes as described in this document.

After establishing a connection, ping messages might need to be exchanged. The negotiated ping timeout is available via the **Channel1**. If ping heartbeats are not sent or received within the expected time frame, the connection can be terminated. LSEG recommends sending ping messages at intervals one-third the size of the ping timeout.

For detailed information on the various transports provided by the Enterprise Transport API and associated use case examples, refer to Chapter 10, Transport Package Detailed View.

8.3 Perform Login Process

Applications authenticate with one another using the Login domain model. An OMM non-interactive provider must register with the system using a Login request¹ prior to providing any content.

After receiving a Login request, the LSEG Real-Time Advanced Distribution Hub determines whether the non-interactive provider is permitted to access the system. The LSEG Real-Time Advanced Distribution Hub sends a Login response to the non-interactive provider which indicates whether the LSEG Real-Time Advanced Distribution Hub has granted it access. If the application is denied, the LSEG Real-Time Advanced Distribution Hub closes the Login stream and the non-interactive provider application cannot perform any additional

1. Because this is done in an interactive manner, the non-interactive provider should assign a **streamId** with a positive value (which the LSEG Real-Time Advanced Distribution Hub will reference) when sending its response.

communication. If the application gains access to the LSEG Real-Time Advanced Distribution Hub, the Login response informs the application of this. The provider must now provide a Source Directory and/or download dictionary.

For details on using the Login domain and expected message content, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

8.4 Perform Dictionary Download

If connected to an LSEG Real-Time Advanced Distribution Hub that support dictionary downloads, an OMM non-interactive provider can download the RWFFld and RWFEnum dictionaries to retrieve appropriate information when providing field list content. A Non-Interactive Provider can use this feature to ensure they are using the correct version of the dictionary or to encode data. An LSEG Real-Time Advanced Distribution Hub that supports the Provider Dictionary Download feature sends a Login response message containing the

SupportProviderDictionaryDownload login element. The dictionary request is send using the Dictionary domain model².

The Transport API offers several utility functions you can use to download and manage a properly-formatted field dictionary. The API also includes other utility functions that help the provider encode into an appropriate format for downloading or decoding a downloaded dictionary.

For details on using the Login domain, expected message content, and dictionary utility functions, refer to the *Enterprise Transport API LSEG Data Models Usage Guide*.

8.5 Provide Source Directory Information

The Source Directory domain model conveys information about all available services in the system. After completing the Login process, an OMM non-interactive provider must provide a Source Directory refresh³ indicating:

- Service, service state, Quality of Service, and capability information associated with the non-interactive provider.
- Supported domain types and any item group information associated with the service.

At a minimum, LSEG recommends that the non-interactive provider send the Info, State, and Group filters for the Source Directory.

- The Source Directory Info filter contains service name and **serviceId** information for all available services, though non-interactive providers typically provide data on only one service.
- The Source Directory State filter contains status information for service. This informs the LSEG Real-Time Advanced Distribution Hub whether the service is Up and available or Down and unavailable.
- The Source Directory Group filter conveys item group status information, including information about group states as well as the merging of groups. For additional information about item groups, refer to Section 13.4.

For details on the Source Directory domain and all of its associated filter entry content, refer to the *Enterprise Transport API LSEG Domain Model Usage Guide*.

8.6 Provide Content

After providing a Source Directory, the non-interactive provider application can begin pushing content to the LSEG Real-Time Advanced Distribution Hub. Each unique information stream should begin with a **RefreshMsg**, conveying all necessary identification information for the content⁴. The initial identifying refresh can be followed by other status or update messages. Some LSEG Real-Time Advanced Distribution Hub functionality, such as cache rebuilding, may require that non-interactive provider applications publish the message key on all **RefreshMsgs**. For more information, refer to component-specific documentation.

2. Because this is instantiated by the provider, the application should use a **streamId** with a negative value.

3. Because this is instantiated by the provider, the non-interactive provider should use a **streamId** with a negative value.

4. Because the provider instantiates these information streams, a negative value **streamId** should be used for each stream. Additional details are provided in subsequent chapters.

NOTE: Some components, depending on their specific functionality and configuration, require that non-interactive provider applications publish the **msgKey** in **UpdateMsgs**. To avoid component or transport migration issues, non-interactive provider applications can choose to always include this information, however this incurs additional bandwidth use and overhead. When designing your application, read the documentation for your other components to ensure that you take into account any other requirements.

Content is typically encoded and decoded using the Transport API Message Package (as described in Chapter 12, Message Package Detailed View) and the Transport API Data Package (as described in Chapter 11, Data Package Detailed View).

8.7 Log Out and Shut Down

After publishing content to the system, the non-interactive provider application should close all open streams and shut down the network connection.

- For more information about closing streams, refer to Section 12.2.5.
- For information about the Message Package, refer to Chapter 12, Message Package Detailed View.

When shutting down the provider, the application should release all unwritten pool buffers obtained from **Channel.getBuffer**. Calling **Channel.getBuffer** terminates the connection to the LSEG Real-Time Advanced Distribution Hub. Detailed information for transport and associated use cases are provided in Chapter 10, Transport Package Detailed View.

8.8 Additional Non-Interactive Provider Details

For specific details about OMM non-interactive providers and Enterprise Transport API use, refer to the following locations:

- The **NIPProvider** application demonstrates one implementation of an OMM non-interactive provider application. The application's source code has additional information about specific implementation and behaviors.
- For reviewing high-level encoding and decoding concepts, refer to Chapter 9, Encoding and Decoding Conventions.
- For a detailed look at the Transport Package, used for the application's network communication, refer to Chapter 10, Transport Package Detailed View.
- For a detailed look at the Data Package, typically used for encoding and decoding payload content, refer to Chapter 11, Data Package Detailed View.
- For a detailed look at the Message Package, used for all message encoding and decoding, refer to Chapter 12, Message Package Detailed View.
- For specific information about the Domain Message Models required by the application, refer to the *Enterprise Transport API Java Edition LSEG Domain Model Usage Guide*.

9 Encoding and Decoding Conventions

9.1 Concepts

The Enterprise Transport API Codec package allows the user to encode and decode constructs and various content. The Codec Package defines a single encode iterator type and a single decode iterator type. The Enterprise Transport API supports single-iterator encoding and decoding such that a single instance can encode or decode the full depth and breadth of a user's content. The application controls the depth of decoding, so you can skip content of no interest. Less efficiently, you can continue to leverage the Enterprise Transport API to use separate iterator instances and hence allow the user to separate portions of content across iterators when encoding or decoding.

The Codec package does not provide inherent threading or locking capability. Separate iterator and type instances do not cause contention and do not share resources between instances. Any needed threading, locking, or thread-model implementation is at the discretion of the application. Different application threads can encode or decode different messages without requiring a lock; thus each thread must use its own iterator instance and each message should be encoded or decoded using unique and independent buffers. Though possible, LSEG recommends that you do not encode or decode related messages (ones that flow on the same stream) on different threads as this can scramble the delivery order.

You can use the Codec package with the Transport package and user-defined transports:

- To use the Codec package with the Transport package, obtain **TransportBuffers** from the Transport package and encode and/or decode using the Codec package.
- To use the Codec package with user-defined transports, obtain **Buffers** from the Codec package, and encode and/or decode using the Codec package.

The rest of this chapter refers to **TransportBuffers**, unless **Buffers** are explicitly specified. However, **Buffers** can be used wherever **TransportBuffers** are specified.

9.1.1 Data Types

The Enterprise Transport API offers a wide variety of data types categorized into two groups:

- **Primitive Types:** A primitive type represents simple, atomically updating information such as values like integers, dates, and ASCII string buffers (refer to Section 11.2).
- **Container Types:** A container type can model data representations more intricately and manage content at a more granular level than primitive types. Container types represent complex information such as field identifier-value, name-value, or key-value pairs (refer to Section 11.3). The Enterprise Transport API offers several uniform, homogeneous container types (i.e., all entries house the same type of data). Additionally, there are several non-uniform, heterogeneous container types in which different entries can hold different types of data.

9.1.2 Composite Pattern and Nesting

The following diagram illustrates the use of Enterprise Transport API data types to resemble a composite pattern.

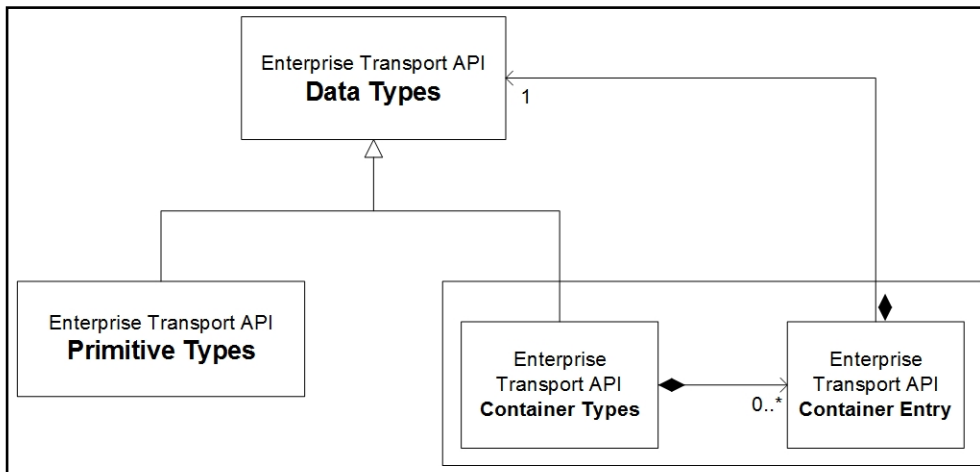


Figure 26. Enterprise Transport API and the Composite Pattern

The diagram highlights the following:

- Being made up of both primitive and container types, Enterprise Transport API data type values mirror the composite pattern's component.
- Primitive types mimic the composite pattern's leaf, conveying concrete information for the user.
- The container type and its entries are similar to the composite pattern's composite. This allows for housing other container types and, in some cases such as field and element lists, housing primitive types.

The housing of other types is also referred to as *nesting*. Nesting allows:

- Messages to house other messages or container types
- Container types to house other messages, container, or primitive types

This provides the flexibility for domain model definitions and applications to arrange and nest data types in whatever way best achieves their goals.

9.2 Encoding Semantics

Because the Enterprise Transport API supports several styles of encoding, the user can choose whichever method best fits their needs.

9.2.1 Init and Complete Suffixes

Encoding functions that have a suffix of **Init** or **Complete** (e.g. **FieldEntry.encodeInit** and **FieldEntry.encodeComplete**) allow the user to encode the type part-by-part, serializing each portion of data with each called function.

Functions without a suffix of **Init** or **Complete** (e.g. **FieldEntry.encode**, **Int.encode**, or **Msg.encode**) perform encoding within a single call, typically used for encoding simple types like Integer or incorporating previously encoded data (referred to as *pre-encoded* data).

9.2.2 The Encode Iterator: EncodeIterator

To encode content you must use an **EncodeIterator** and can use a single encode iterator to manage the entire encoding process¹ (including state and position information).

For example, if you want to encode a message that contains an **FieldList** composed of various primitive types, you can use the same **EncodeIterator** to encode all contents. In this case, initialize the iterator before encoding the message, and then pass the iterator as a parameter when encoding each portion. You do not need to perform additional initialization or clearing. When encoding finishes, you can determine the total encoded length and clear the iterator, reusing it for another encoding. If needed, you can use individual iterators for each level of encoding or for pre-encoding portions of data. However, when using separate iterators, you must initialize each iterator before starting the associated encoding process.

Initialization of an **EncodeIterator** consists of several steps. Create an **EncodeIterator** by calling the **CodecFactory.createEncodeIterator** method. After creating the iterator, clear it using the **EncodeIterator.clear** method. Each **EncodeIterator** requires a **TransportBuffer** (into which it encodes) and the Rssl Wire Format version information (to ensure that the proper version of the wire format is encoded). Use the **EncodeIterator.setBufferAndRWFVersion** method to set the **TransportBuffer** and Rssl Wire Format version (refer to Section 9.5.1).

1. A single **EncodeIterator** can support up to sixteen levels of nesting, allowing for sixteen **Init** calls without a single **Complete** call. Because the most complex Domain Model currently requires only five levels, sixteen is believed to be sufficient. Should an encoding require more than sixteen levels of nesting, multiple iterators can be used.

9.2.2.1 EncodeIterator Functions

NOTE: More encoding examples are provided throughout this manual and in the Enterprise Transport API package's example applications.

The following table describes methods that you can use with **EncodeIterator**. The methods listed below that take **TransportBuffers** also support **Buffers** (from the Codec package). Whereas **TransportBuffers** are used with the Transport package, **Buffers** (from the Codec package) are available for use with user-defined transports.

METHOD	DESCRIPTION
<code>EncodeIterator.clear</code>	Clears members necessary for encoding and readies the iterator for reuse. You must clear the EncodeIterator prior to starting any encoding process. For performance purposes, only those members necessary for proper functionality are cleared.
<code>EncodeIterator.setBufferAndRWFVersion</code>	Associates an EncodeIterator with the TransportBuffer into which it encodes and Rssl Wire Format versioning information. TransportBuffer.data should refer to sufficient space for encoding. Rssl Wire Format Versioning information ensures that the Enterprise Transport API uses the appropriate wire format version while encoding. Rssl Wire Format information is typically available on the connection between applications. Refer to Section 9.5.1.
<code>EncodeIterator.realignBuffer</code>	If an encoding process exceeds the space allocated in the current TransportBuffer , this method dynamically associates a new, larger buffer with the encoding process, allowing encoding to continue.

Table 4: EncodeIterator Utility Methods

9.2.2.2 EncodeIterator: Basic Use Example

The following example illustrates how to initialize **EncodeIterator** in a typical fashion.

```
/* create and clear iterator to prepare for encoding */
EncodeIterator encodeIter = CodecFactory.createEncodeIterator();
encodeIter.clear();

/* associate buffer and iterator (code assumes buffer has sufficient memory) and set proper protocol
   version information on iterator (typically obtained from Channel associated with the connection
   once it becomes active)
   */
if (encodeIter.setBufferAndRWFVersion(buffer, chnl.majorVersion(), chnl.minorVersion()) <
    CodecReturnCodes.SUCCESS)
{
    System.out.printf("Error (%d) (errno: %d) encountered with setBufferAndRWFVersion. Error Text:
        %s\n", error.errorId(), error.sysError(), error.text());
}

/* Perform all content encoding now that iterator is prepared. */
```

Code Example 1: EncodeIterator Usage Example

9.2.3 Content Roll Back with Example

Every **Complete** method has a **success** parameter, which allows you to discard unsuccessfully encoded content and roll back to the last successfully encoded portion.

For example, you begin encoding a list that contains multiple entries, but the tenth entry in the list fails to encode. To salvage the successful portion of the encoding, pass the **success** parameter as **false** when calling the failed entry's **Complete** method. This rolls back encoding to the end of the last successful entry. The remaining **Complete** methods should be called, after which the application can use the encoded content. You can begin a new encoding for the remaining entries.

The following example demonstrates the use of the roll back procedure. This example encodes an **Map** with two entries. The first entry succeeds; so **success** is passed in as **true**. However, encoding the second entry's contents fails, so the second map entry is rolled back, and the map is completed. To highlight the rollback feature, only those portions relevant to the example are included.

```
/* example shows encoding a map with two entries, where second entry content fails so it is
   rolled back */
retCode = map.encodeInit(encIter, 0, 0);

/* Encode the first map entry - this one succeeds */
retCode = mapEntry.encodeInit(encIter, entryKey, 0);
/* encode contents - assume this succeeds */
/* Passing true for the success parameter completes encoding of this entry */
retCode = mapEntry.encodeComplete(encIter, true);

/* Encode the second map entry - this one fails */
retCode = mapEntry.encodeInit(encIter, entryKey, 0);
/* encode contents - assume this fails */
/* Passing false for the success parameter rolls back the encoding to the end of the previous
   entry */
retCode = mapEntry.encodeComplete(encIter, false);

/* Now complete encoding of the map - this results in only one entry being contained in the map
   */
retCode = map.encodeComplete(encIter, true);
```

Code Example 2: Encoding Rollback Example

9.3 Decoding Semantics

Using the Enterprise Transport API, applications can decode the full depth of the content or skip over portions in which the application is not interested. Each container type provided by the Enterprise Transport API includes functionality for decoding the container header and decoding each entry in the container. If an application wishes to decode information present in a container entry, it can invoke the specific decode function associated with the nested type. When nested content is completely decoded, the next container entry can be decoded. If an application wishes to skip decoding data nested in a container entry, it can simply call the container entry decode method again without invoking the decoder for nested content. A decoding application will typically loop on decode until **CodecReturnCodes.END_OF_CONTAINER** is returned.

9.3.1 The Decode Iterator: DecodeIterator

All decoding requires the use of a **DecodeIterator**. You can use a single decode iterator to manage the full decoding process, internally managing various state and position information while decoding.

For example, when decoding a message that contains a **FieldList** composed of various primitive types, you can use the same **DecodeIterator** to decode all contents, including primitive types. In this case, you want to initialize the iterator before decoding the message and then pass the iterator as a parameter when decoding other portions (without additional initialization or clearing). After you completely decode all needed content, you can clear the iterator and reuse it for another decoding. If needed, you can use individual iterators for each level of decoding. However, if you use separate iterators, you must initialize each iterator before the decoding process that it manages.

Initialization of a **DecodeIterator** consists of several steps. Create a **DecodeIterator** by calling the **CodecFactory.createDecodeIterator** method. After the iterator is created, use **DecodeIterator.clear** to clear **DecodeIterator**. Each **DecodeIterator** requires a **TransportBuffer** from which it decodes and Rssl Wire Format version information, which ensures decoding of the appropriate version of the wire format (refer to Section 9.5.1).

NOTE: Additional concrete decoding examples are provided throughout this manual as well as in the example applications provided with the Enterprise Transport API package.

9.3.2 Functions for Use with DecodeIterator

The following table describes the methods that you can use with **DecodeIterator**. The methods listed below that take **TransportBuffers** also support **Buffers** (from the Codec package). **TransportBuffers** are used with the Transport package. **Buffers** (from the Codec package) are available for use with user-defined transports.

METHOD	DESCRIPTION
DecodeIterator.clear	Clears members necessary for decoding and readies the iterator for reuse. You must clear DecodeIterator before decoding content. For performance purposes, only those members required for proper functionality are cleared.
DecodeIterator.setBufferAndRWFVersion	Associates the DecodeIterator with the TransportBuffer from which to decode and Rssl Wire Format version information. Set TransportBuffer.data to refer to the content to be decoded. Rssl Wire Format Version information ensures that the Enterprise Transport API uses the appropriate wire format version when encoding. Rssl Wire Format information is typically available on the connection between applications. Refer to Section 9.5.1Section 9.5.1.
DecodeIterator.finishDecodeEntries	The decoding process typically runs until the end of each container, indicated by CodecReturnCodes.END_OF_CONTAINER . This method will skip past remaining entries in the container and perform necessary synchronization between the content and iterator so that decoding can continue.

Table 5: DecodeIterator Utility Methods

9.3.3 DecodeIterator: Basic Use Example

The following example demonstrates a typical **DecodeIterator** initialization process.

```
/* create and clear iterator to prepare for decoding */
DecodeIterator decodeIter = CodecFactory.createDecodeIterator();
decodeIter.clear();

/* associate buffer and iterator (code assumes buffer has sufficient memory) and set proper
   protocol version information on iterator (typically obtained from Channel associated with
   the connection once it becomes active)
*/
if (decodeIter.setBufferAndRWFVersion(buffer, chnl.majorVersion(), chnl.minorVersion()) <
    CodecReturnCodes.SUCCESS)
{
    System.out.printf("Error (%d) (errno: %d) encountered with setBufferAndRWFVersion. Error
        Text: %s\n", error.errorId(), error.sysError(), error.text());
}

/* Perform all content decoding now that iterator is prepared. */
```

Code Example 3: DecodeIterator Usage Example

9.4 Return Code Values

Codec functionality returns codes indicating success or failure.

- On failure conditions, these codes inform the user of the error.
- On success conditions, these codes provide the application additional direction regarding the next encoding steps.

When using the Codec package, return codes greater than or equal to `CodecReturnCodes.SUCCESS` indicate some type of specific success code, while codes less than `CodecReturnCodes.SUCCESS` indicate some type of specific failure.

NOTE: The Transport Layer has special semantics associated with its return codes. It does not follow the same semantics as the Codec package. For detailed handling instructions and return code information, refer to Chapter 10, Transport Package Detailed View.

9.4.1 Success Codes

The following table describes success values of `CodecReturnCodes` return codes associated with the Codec.

CODEC RETURN CODE	DESCRIPTION
<code>CodecReturnCodes.SUCCESS</code>	Indicates operational success. Does not indicate next steps, though additional encoding or decoding might be required.
<code>CodecReturnCodes.ENCODE_MSG_KEY_ATTRIB</code>	Indicates that initial message encoding was successful and now the application should encode <code>msgKey</code> attributes. This return occurs if the application indicates that the message should include <code>msgKey</code> attributes when calling <code>Msg.encodeInit (MsgKeyFlags.HAS_ATTRIB)</code> without populating pre-encoded data into <code>msgKey.encAttrib</code> . For further details, refer to Section 12.1.2 and Code Example 42.
<code>CodecReturnCodes.ENCODE_EXTENDED_HEADER</code>	Indicates that initial message encoding (and <code>msgKey</code> attribute encoding) was successful, and the application should now encode <code>extendedHeader</code> content. This return occurs if an application indicates that the message should include <code>extendedHeader</code> content when calling <code>Msg.encodeInit</code> without populating pre-encoded data into the <code>extendedHeader</code> . For further details on message encoding information, refer to Chapter 12, Message Package Detailed View.
<code>CodecReturnCodes.ENCODE_CONTAINER</code>	Indicates that initial encoding succeeded and that the application should now encode the specified <code>containerType</code> . • For details on container types, refer to Section 11.3. • For details on encoding messages, refer to Chapter 12, Message Package Detailed View.
<code>CodecReturnCodes.SET_COMPLETE</code>	Indicates that <code>FieldList</code> or <code>ElementList</code> encoding is complete. Additionally encoded entries are encoded in the standard way with no additional data optimizations. For further information, refer to Section 11.6.
<code>CodecReturnCodes.DICT_PART_ENCODED</code>	Indicates that the dictionary encoding utility method successfully encoded part of a dictionary message (because dictionary messages tend to be large, they might be segmented into a multi-part message). • For specific information about the Dictionary domain and the utility functions provided by the Transport API, refer to the <i>Enterprise Transport API Java Edition LSEG Domain Model Usage Guide</i>. • For more details on message fragmentation, refer to Section 13.1.

Table 6: Codec Package Success CodecReturnCodes

CODEC RETURN CODE	DESCRIPTION
CodecReturnCodes.BLANK_DATA	Indicates that the decoded primitiveType is a blank value. The contents of the primitive type should be ignored; any display or calculation should treat the value as blank. For further details on primitive types, refer to Section 11.2.
CodecReturnCodes.NO_DATA	Indicates that the containerType being decoded contains no data and was decoded from an empty payload. Informs the application not to continue to decode container entries (as none exist).
CodecReturnCodes.END_OF_CONTAINER	Indicates that the decoding process has reached the end of the current container. If decoding nested content, additional decoding might still be needed. The application can move back up the nesting stack and continue decoding the next container entry by calling the container's specific entry decode method. For example, if decoding an FieldList contained in an MapEntry , when this code is returned, it signifies that the contained field list decoding is complete. For details on container types, refer to Section 11.3.
CodecReturnCodes.SET_SKIPPED	Indicates that FieldList or ElementList decoding skipped over contained, set-defined data because a set definition database was not provided. Any standard encoded entries will still be decoded. For further information on set definitions, refer to Section 11.6.
CodecReturnCodes.SET_DEF_DB_EMPTY	Indicates that decoding of a set definition database completed successfully, but the database was empty. For further information, refer to Section 11.6.

Table 6: Codec Package Success CodecReturnCodes (Continued)

9.4.2 Failure Codes

RETURN CODE	DESCRIPTION
CodecReturnCodes.FAILURE	Indicates a general failure, used when no specific details are available.
CodecReturnCodes.BUFFER_TOO_SMALL	Indicates that the TransportBuffer on the EncodeIterator lacks sufficient space for encoding.
CodecReturnCodes.INVALID_ARGUMENT	Indicates an invalid argument was provided to an encoding or decoding method.
CodecReturnCodes.ENCODING_UNAVAILABLE	Indicates that the invoked method does not contain encoding functionality for the specified type. There might be other ways to encode content or the type might be invalid in the combination being performed.
CodecReturnCodes.UNSUPPORTED_DATA_TYPE	Indicates that the type is not supported for the operation being performed. This might indicate a primitiveType is used where a containerType is expected or the opposite.
CodecReturnCodes.UNEXPECTED_ENCODER_CALL	Indicates that encoding functionality was used in an unexpected sequence or the called method is not expected in this encoding.
CodecReturnCodes.INCOMPLETE_DATA	Indicates that the TransportBuffer on the DecodeIterator does not have enough data for proper decoding.
CodecReturnCodes.INVALID_DATA	Indicates that invalid data was provided to the invoked method.

Table 7: Codec Package Failure Return Codes

RETURN CODE	DESCRIPTION
CodecReturnCodes.ITERATOR_OVERRUN	Indicates that the application is attempting to nest more levels of content than is supported by a single EncodeIterator ^a . If this occurs, you should use multiple iterators for encoding.
CodecReturnCodes.VALUE_OUT_OF_RANGE	Indicates that a value being encoded using a set definition exceeds the allowable range for the type as specified in the definition. For further information on set definitions, refer to Section 11.6.
CodecReturnCodes.SET_DEF_NOT_PROVIDED	Indicates that FieldList or ElementList encoding requires a set definition database which was not provided. For more information, refer to Section 11.6.
CodecReturnCodes.TOO_MANY_LOCAL_SET_DEFS	Indicates that encoding exceeds the maximum number of allowed local set definitions. Currently 15 local set definitions are allowed per database. For more information, refer to Section 11.6.
CodecReturnCodes.DUPLICATE_LOCAL_SET_DEFS	Indicates that content includes a duplicate set definition that collides with a definition already stored in the database. For more information, refer to Section 11.6.
CodecReturnCodes.ILLEGAL_LOCAL_SET_DEF	Indicates that the setId associated with a contained definition exceeds the allowable value. Currently setId values up to 15 are allowed. For more information, refer to Section 11.6.

Table 7: Codec Package Failure Return Codes (Continued)

a. A single **EncodeIterator** can support up to sixteen levels of nesting (this allows for sixteen **Init** calls without a single **Complete** call). Currently, the most complex Domain Model requires five levels, so sixteen is sufficient. If an encoding requires more than sixteen levels of nesting, multiple iterators can be employed.

9.4.3 CodecReturnCodes Methods

CodecReturnCodes contains the following methods:

METHOD	DESCRIPTION
CodecReturnCodes.toString	Returns a Java String representation for a CodecReturnCodes value (e.g. "INCOMPLETE_DATA" for CodecReturnCodes.INCOMPLETE_DATA).
CodecReturnCodes.Info	Returns a Java String representation of the meaning associated with a CodecReturnCodes value (e.g. "Failure: Not enough data was provided." for CodecReturnCodes.INCOMPLETE_DATA).

Table 8: CodecReturnCodes Methods

9.5 Versioning

The Transport API supports two types of versioning:

- Protocol Versioning: Allows for the exchange of protocol type and version information across a connection established with the Transport Package. Protocol and version information can be provided to the **EncodeIterator** and **DecodeIterator** to ensure the proper handling and use of the appropriate wire format version.

NOTE: LSEG strongly recommends that you write all Enterprise Transport API applications to leverage wire format versioning.

- Library Versioning: Allows for applications to programmatically query library version information. Library versioning ensures that expected libraries are used and that all versions match in the application.

9.5.1 Protocol Versioning

Consumer and provider applications using the Transport can provide protocol type and version information. This data is supplied as part of **ConnectOptions** or **BindOptions** and populated via the **protocolType**, **majorVersion**, and **minorVersion** methods. When establishing a connection, data is exchanged and negotiated between client and server: **protocolType**.

- If the client's specified **protocolType** does not match the server's specified **protocolType**, the connection is refused.
- If the **protocolType** information matches, version information is compared and a compatible version determined.

After a connection becomes active, negotiated version information is available via the **Channel** from both client and server and can be used for encoding and decoding:

- To populate version information on a **EncodeIterator**, call the **EncodeIterator.setBufferAndRWFVersion** method.
- To populate version information on a **DecodeIterator**, call the **DecodeIterator.setBufferAndRWFVersion** method.

The Transport layer is data neutral and does not change or depend on data distribution. Versioning information is provided only to help client and server applications manage the data they communicate. For further details on the Transport, refer to Chapter 10, Transport Package Detailed View.

NOTE: Properly using Enterprise Transport API's versioning functionality helps minimize future impacts associated with underlying format modifications and enhancements, ensuring compatibility with other Enterprise Transport API-enabled components.

Typically, an increase in the major version is associated with the introduction of an incompatible change. An increase in the minor version tends to signify the introduction of a compatible change or extension. **minorVersion**

9.5.1.1 Protocol Versioning Methods

The Codec Package contains several defined values that you can access via **Codec** methods and use with protocol versioning:

METHOD	DESCRIPTION
protocolType	Defines the protocolType value associated with Rssl Wire Format. Define other protocols using different protocolType values.
majorVersion	Sets the value associated with the current major version. If incompatible changes are introduced, this value is incremented.
minorVersion	Sets the value associated with the current minor version. If extensions or compatible changes are introduced, this value is incremented.

Table 9: Codec Methods

9.5.1.2 Codec Package Protocol Values

The Codec package also provides protocol values associated with currently supported protocols:

VALUE NAME	DESCRIPTION
Codec.RWF_PROTOCOL_TYPE	Defines the protocolType value associated with Rssl Wire Format.
Codec.JSON_PROTOCOL_TYPE	Defines the protocolType value associated with JSON protocol.

Table 10: Codec Package Protocol Values

9.5.2 Library Versioning

The Enterprise Transport API library version information is contained in the **MANIFEST.MF** of the **eta.jar** file. The **MANIFEST.MF** contains the Enterprise Transport API version data, the internal LSEG build version data, and the product date.

There are several ways in which you can obtain this data. From a console, you can use the following **jar** command to extract the **MANIFEST.MF** then examine the contents, which provides the Enterprise Transport API package version data and internal version (which provides the internal LSEG build version data). Any issues raised to support should include this version data.

```
jar xf eta.jar META-INF/MANIFEST.MF
type META-INF/MANIFEST.MF
```

Code Example 4: Extract Package information from MANIFEST.MF

Additionally, each Enterprise Transport API library includes a utility method, defined in Table 11, to programmatically extract library version information. Each method populates a **LibraryVersionInfo** object, as defined in Table 12.

METHOD	DESCRIPTION
Codec.queryVersion	Retrieves version data associated with the Codec Package library.
Transport.queryVersion	Retrieves version data associated with the Transport Package library.

Table 11: Library Version Utility Methods

METHOD	DESCRIPTION
productVersion	Returns the Package version as specified by the Specification-Version in the MANIFEST.MF .
internalVersion	Returns the internal LSEG build data as specified by the Implementation-Version in the MANIFEST.MF .
productDate	Returns the build date for the product release as specified by the Implementation-Vendor in the MANIFEST.MF .

Table 12: LibraryVersionInfo Methods

10 Transport Package Detailed View

10.1 Concepts

The Enterprise Transport API offers a Transport Package capable of communicating with other Open Message Model-based components, including but not limited to LSEG Real-Time Distribution System, LSEG Real-Time, EDF Direct, and other LSEG Real-Time Distribution System API Open Message Model-based applications. The Transport Package efficiently sends and receives data across TCP/IP-based networks, connection types, and presents a message-based interface to applications for ease of reading and writing data.

The package exposes a feature set that includes a receiver-transparent way for senders to combine or pack multiple messages into one outbound packet, as well as transparent fragmentation and reassembly of messages which exceed the size of an outbound packet. Class representations are provided for managing connections (referred to as channels).

The transport layer offers multiple degrees of thread safety, all programmatically configurable by the application. This ranges from a fully thread-safe option¹ to the ability for an application to turn off all protective locking². Threading implementation and thread-model selection is managed by the application. The transport provides different locking options to provide maximum flexibility to the user. For more information, refer to Section 10.2.4.

The transport supports both non-blocking and blocking I/O models, however use of blocking I/O is not recommended. When a blocking operation is occurring, control will not be returned to the application until the operation has fully completed (e.g. all information is written). This prevents the application from performing additional tasks, including heartbeat sending and monitoring, while the transport operation may be waiting for the operating system. By employing an I/O notification mechanism (e.g. select, poll), an application can leverage a non-blocking I/O model, using the I/O notification to alert the application when data is available to read or when output space is available for writing to. The following sections are written with an emphasis on non-blocking I/O use, though blocking behavior is also described. All examples are written from a non-blocking I/O perspective.

10.1.1 Transport Types

The transport supports configuration of multiple connection types for different systems, while providing a single interface for a look and feel that is similar among all connections and components. Developers should ensure that the components to which they intend to connect are configured to support the appropriate transport type.

10.1.1.1 Socket Transport

The Enterprise Transport API provides a transport for efficiently distributing messages across a TCP/IP-based reliable network (**SOCKET**). This transport is capable of connecting to various Open Message Model-based components, including but not limited to LSEG Real-Time Distribution System, LSEG Real-Time, LDF Direct, and other Enterprise Transport API or RFA Open Message Model-based applications. On specific platforms, applications can also leverage tunneling through HTTP (**HTTP**) or TLS Encrypted (**ENCRYPTED**) connection types for Internet connectivity.

The socket transport allows for both establishing outbound connections and for creating listening sockets to accept inbound connections. Once a connection is established, both connected components can send and receive information. Outbound connections are typically created by OMM consumer applications to connect to an LSEG Real-Time Advanced Distribution Server or OMM interactive provider, or by OMM non-interactive provider applications to connect to an LSEG Real-Time Advanced Distribution Hub. Listening sockets are typically created by OMM interactive provider applications to allow OMM consumer applications or LSEG Real-Time Advanced Distribution Servers to instantiate connections to it and request data.

10.1.1.2 WebSocket Transport

The Enterprise Transport API provides a WebSocket transport (**WEBSOCKET**) which is functionally the same as the socket transport but with an additional WebSocket-based protocol, which optionally leverages the **ENCRYPTED** connection type. In addition to a sub-protocol that delivers Rssl Wire Format-encrypted data, the WebSocket transport provides a JSON sub-protocol compatible with components which also support the Simplified Streaming protocol.

1. When this option is enabled, Transport can function correctly during simultaneous execution by multiple application threads.

2. When this option is enabled, all locking is disabled for additional performance. If required, the application must provide any necessary thread safety.

10.1.1.3 Reliable Multicast Transport

The Enterprise Transport API provides an efficient transport for exchanging messages over a UDP multicast-based network (**RELIABLE_MCAST**). This transport leverages the same technology used on the LSEG Real-Time Distribution System Backbone to improve reliability of message delivery and automatically re-sequence out-of-order messages.

OMM non-interactive provider applications may create multicast connections for publishing to an LSEG Real-Time Advanced Distribution Hub.

10.1.1.4 Sequenced Multicast Transport

The Enterprise Transport API provides an efficient transport for reading messages over the UDP Multicast-based network (**SEQ_MCAST**). The Sequenced Multicast protocol is a special, unreliable UDP multicast with built-in sequence numbers that allow the user to ensure order and identify gaps in their applications.

10.1.2 Channel Object

The **Channel** object represents a connection that can send or receive information across a network, regardless of whether the connection is outbound or accepted by a listening socket. The Transport Package internally manages any memory associated with a **Channel** object, and the application does not need to create nor free memory (associated with the channel). **Channel** is typically used to perform any action on the connection that it represents (e.g. reading, writing, disconnecting, etc). See the subsequent sections for more information about **Channel** use within the transport.

For information on how to set the Channel connection types using the **ConnectOptions** parameters, refer to Section 10.1.2.3.

The following table describes the methods of the **Channel** object.

METHOD	DESCRIPTION
blocking	A boolean representing the blocking mode of the Channel .
bufferUsage	Gets the total number of used buffers for this Channel .
close	Close the Channel .
connectionType	A ConnectionType that indicates the type of underlying connection being used. For more information, refer to Section 10.1.2.2.
flush	Flush this Channel . For more details, refer to Section 10.10.2.
getBuffer	Retrieves a TransportBuffer for use. For more details, refer to Section 10.8.
hostname	Provides the name of the host to which a consumer or Non-Interactive Provider application connects.
info	Gets information about this Channel . For more details, refer to Section Section 10.14.2.
init	Continues Channel initialization for non-blocking channels. For more details, refer to Section 10.5.
ioctl	Set or get some I/O values programmatically. For more details, refer to Section 10.14.6.
majorVersion	When a Channel becomes active for a client or server, this is populated with the negotiated major version number that is associated with the content being sent on this connection. Typically, a major version increase is associated with the introduction of incompatible change. The transport layer is data neutral and does not change nor depend on any information in content being distributed. This information is provided to help client and server applications manage the information they are communicating. For more details, refer to Section 9.5.1.

Table 13: Channel Methods

METHOD	DESCRIPTION
minorVersion	When a Channel becomes active for a client or server, this is populated with the negotiated minor version number that is associated with the content being sent on this connection. Typically, a minor version increase is associated with a fully backward compatible change or extension. The transport layer is data neutral and does not change nor depend on any information in content being distributed. This information is provided to help client and server applications manage the information they are communicating. For more details, refer to Section 9.5.1.
oldSelectableChannel	It is possible for a SelectableChannel to change over time, typically due to some kind of connection keep-alive mechanism. If this occurs, this is typically communicated via a return code of TransportReturnCodes.READ_FD_CHANGE (for further information, refer to Section 10.6). The previous SelectableChannel is stored in oldSelectableChannel so the application can properly unregister and then register the new SelectableChannel with their I/O notification mechanism.
packBuffer	For more details, refer to Section 10.11.
ping	Send a ping (i.e. heart beat) message to the far end of the connection.
pingTimeout	When a Channel becomes active for a client or server, this is populated with the negotiated ping timeout value. This is the number of seconds after which no communication can result in a connection being terminated. Both client and server applications should send heartbeat information within this interval. The typically used rule of thumb is to send a heartbeat every pingTimeout /3 seconds. For more information, refer to Section 10.12.
port	Provides the server port number to which the consumer or Non-Interactive Provider application is connected.
protocolType	When a Channel becomes active for a client or server, this is populated with the protocolType associated with the content being sent on this connection. If the protocolType indicated by a server does not match the protocolType that a client specifies, the connection will be rejected. The transport layer is data-neutral and allows the flow of any type of content. ProtocolType is provided to help client and server applications manage the information they communicate. For more details, refer to Section 9.5.1.
read	Read on this Channel . For more details, refer to Section 10.6.
reconnectClient	Used for tunneling solution to reconnect and bridge connections. This only applies to http and encrypted connections, where it might be needed to keep connections alive through proxy servers.
releaseBuffer	Releases a TransportBuffer . Should only be used if the buffer could not be successfully written.
SelectableChannel	Returns the java.nio.channels.SelectableChannel object that can be used in an I/O notification mechanism (e.g. to register a Selector). This is the SelectableChannel associated with this end of the network connection.
state	The state associated with the Channel . Until the channel has completed its initialization handshake and has transitioned to an active state, no reading or writing can be performed. Section 10.1.2.1 describes channel state values.
userSpecObject	A reference that can be set by the user of the Channel . This value can be set directly or via the connection options and is not modified by the transport. This information can be useful for coupling this Channel with other user created information, such as a watch list associated with this connection.
write	Write on this Channel . For more details, refer to Section 10.9.

Table 13: Channel Methods (Continued)

10.1.2.1 Channel State Values

ENUMERATED NAME	DESCRIPTION
ACTIVE	Indicates that a Channel is active. This channel can perform any connection-related actions, such as reading or writing.
CLOSED	Indicates that a Channel has been closed. This typically occurs as a result of an error inside of a transport method call and is often related to a socket being closed or becoming unavailable. Appropriate error value return codes and Error information should be available for the user.
INACTIVE	Indicates that a Channel is inactive. This channel cannot be used. This state typically occurs after a channel is closed by the user.
INITIALIZING	Indicates that a Channel requires additional initialization. This initialization is typically additional connection handshake messages that need to be exchanged. When using blocking I/O, an Channel is typically active when it is returned and no additional initialization is required by the user.

Table 14: Channel State Values

10.1.2.2 ConnectionType Values

Connection types are used in several areas of the transport. When creating a connection, an application can specify which connection type to use (refer to Section 10.3). Additionally, after a connection is established, the **Channel.ConnectionType** will indicate the connection type being used.

CONNECTIONTYPE	DESCRIPTION
SOCKET	Indicates that the Channel uses a standard, TCP-based socket connection. This type can be used to connect between any Transport-based applications.
HTTP	Indicates that the Channel tunnels using HTTP. This type can be used to connect between any Transport-based applications. For more information, refer to Section 4.6.
ENCRYPTED	Indicates that the Channel tunnels using encryption. The encryption use is transparent to the client application. For a server to accept encrypted connection types the use of an external encryption/decryption device is required (encryption / decryption is not performed by the server). Because data will already be decrypted when it arrives at the server, a Channel may indicate that a connection type is HTTP or SOCKET, even if the connection was established by specifying ENCRYPTED . For more information, refer to Section 4.6.
RELIABLE_MCAST	Indicates that the Channel uses a UDP-based, reliable multicast connection type. This connection type is available only to applications using the Transport.connect function to establish their connection. The reliable multicast connection type ensures proper ordering of content across the network and, through the use of an acknowledgment and retransmission mechanism, backfills recent packet gaps. In situations where a packet gap cannot be filled, the application is notified of the gap situation. The default behavior for this connection type is to stay connected to the multicast, even in a gap situation. This allows the application to attempt recovery in a manner that might minimize any affect on the network. You can control this behavior via the disconnectOnGaps option described in Table 29.

Table 15: ConnectionType Values

CONNECTIONTYPE	DESCRIPTION
SEQUENCED_MCAST	Indicates that the Channel uses a UDP-based, sequenced multicast connection type. This connection type is available only to applications using the Transport.connect function to establish their connection. Though this connection type uses sequence numbers which enables gap detection, it only ensures the proper ordering of content across the network; it does not acknowledge or retransmit packets to fill a gap. The default behavior for this connection type is to stay connected to the multicast, even in a gap situation. This allows for the application to attempt recovery in a manner that might minimize any affect on the network. You can control this behavior via the disconnectOnGaps option described in Table 29.
UNIDIR_SHMEM	Indicates that the Channel is using a shared memory connection type. This connection type offers a one-way data flow from a single server to multiple clients using a shared memory segment for content delivery. However, the server and clients must run on the same machine. For compatibility purposes, this connection type provides a Channel.SelectableChannel to the application. This SelectableChannel will always indicate that something is available to read, even when there is not. This ensures that the application is reading content with as little latency as possible. If needed, the application can implement alternate approaches that would allow for a less CPU intensive read algorithm.  WARNING! Enterprise Transport API applications using this connection type require appropriate run-time permissions to create and lock memory on the system (e.g. mlock()). See operating system-specific information for details on ensuring applications have proper system access rights.
WEBSOCKET	Indicates that the Channel uses a TCP-based connection type that uses the WebSocket protocol (https://tools.ietf.org/html/rfc6455). A connection using the WebSocket protocol provides a bidirectional, full-duplex communication channel operating over HTTP as the initial transport mechanism.

Table 15: ConnectionType Values (Continued)

10.1.2.3 Channel Connection Types

The following table summarizes possible Channel connection types and the **ConnectOptions** parameter values that you can use to set them.

CHANNEL CONNECTION TYPE	CONNECTOPTIONS.CONNECTIONTYPE		SUBPROTOCOL LIST POSSIBILITIES
Unencrypted Websocket	ConnectionTypes.WEBSOCKET	Not used	The following possibilities apply: - If the <pre>ConnectOptions .wSocketOpts .protocols</pre> parameter is explicitly set to empty or mismatched with server support, websocket connections are denied. - Default <pre>ConnectOptions .wSocketOpts .protocols value=""</pre> - Possible values: <pre>ConnectOptions .wSocketOpts .protocols value= "tr_json2, rssl.rwf, rssl.json.v2"</pre>
Encrypted Websocket	ConnectionTypes.ENCRYPTED	<pre>ConnectOptions .encryptionOptions .connectionType = ConnectionTypes .WEBSOCKET</pre>	
Unencrypted Socket	ConnectionTypes.SOCKET	Not used	Not used; RWF is implied.
Encrypted Socket	ConnectionTypes.ENCRYPTED	<pre>ConnectOptions .encryptionOpts .connectionType = ConnectionTypes.SOCKET</pre>	

Table 16: Channel Settings for Socket and Websocket Connection Types

10.1.3 Server Object

The **server** object is used to represent a server that is listening for incoming connection requests. Any memory associated with a **Server** structure is internally managed by the Transport Package, and the application does not need to create nor destroy this type. The **Server** is typically used to accept or reject incoming connection attempts. See the subsequent sections for more information about **Server** use within the transport.

For information on how to set Server connection types using the **BindOptions** parameters, refer to Section 10.1.3.1.

The following table describes **Server** methods.

METHOD	DESCRIPTION
accept	Accepts an incoming connection. For more details, refer to Section 10.4.2.
bufferUsage	Returns the total number of used buffers for the Server .
close	Closes a Server . Active Channels accepted from this Server will not be closed.
info	Gets information about the Server . For more details, refer to section Section 10.14.5 (check link)
ioctl	Allows change some I/O values programmatically for a Server . For more details, refer to Section 10.14.5.
portNumber	The port number that this Server is bound to and listening for incoming connections on.
SelectableChannel	Represents a <code>java.nio.channels.SelectableChannel</code> that can be used in some kind of I/O notification mechanism (e.g. Selector). This is the SelectableChannel associated with listening socket. When triggered, this typically indicates that there is an incoming connection and Server.accept should be called.
state	The ChannelState associated with the Server . A server is typically returned as active unless an error occurred during the Transport.bind call or the Close method was called. Table 6 describes possible state values.
userSpecObject	A reference that can be set by the user of the Server . This value can be set directly or via the bind options and is not modified by the transport. This information can be useful for coupling this Server with other user created information, such as a list of associated Channel objects.

Table 17: Server Methods

10.1.3.1 Server Connection Types

The following table summarizes possible Server connection types and the **BindOptions** parameter values that you can use to set them.

SERVER CONNECTION TYPE	BINDOPTIONS.CONNECTIONTYPE	SUBPROTOCOL LIST POSSIBILITIES
Unencrypted Socket	ConnectionTypes.SOCKET	The following possibilities apply: - By default, socket incoming connections are accepted. - To accept websocket connections, set the BindOptions.wSocketOpts.protocols parameter. - Default for websocket: BindOptions.wSocketOpts.protocols value="" - Possible values: BindOptions.wSocketOpts.protocols value="tr_json2, rssl.rwf, rssl.json.v2" - If the BindOptions.wSocketOpts.protocols parameter is empty or mismatched with incoming connections, websocket connections are denied. - Socket connection logic ignores the BindOptions.wSocketOpts.protocols configuration and supports only RWF.
Unencrypted Websocket		
Encrypted Socket	ConnectionTypes.WEBSOCKET	
Encrypted Websocket		

Table 18: Server Settings for Socket and Websocket Connection Types

10.1.4 Transport Error Handling

Many Transport Package methods take a parameter for returning detailed error information. This **Error** object populated only in the event of an error condition and should only be inspected when a specific failure code is returned from the method itself.

In several cases (e.g. Transport.connect), positive return values are reserved or have special meaning, for example bytes remaining to write to the network. As a result, some negative return codes might be used to indicate success. Any specific transport-related success or failure error handling is described along with the method that requires it.

Error methods are described in the following table.

METHOD	DESCRIPTION
channel	A reference to the Channel object on which the error occurred.
errorId	A Transport API-specific return code that specifies what error occurred. Refer to the following sections for specific error conditions that might arise.
sysError	Populated with the system errno or error number associated with the failure. This information is only available when the failure occurs as a result of a system function, and will be populated by 0 otherwise.
text	Detailed text describing the error. This can include Enterprise Transport API-specific error information, underlying library-specific error information, or a combination of both. Error text information is limited to 1,200 bytes in length.
clear	Clears the Error object.

Table 19: Error Methods

10.1.5 General Transport Return Codes

Application should monitor return values from all Transport API methods that provide return codes. The table below lists general error codes. Specific error codes are detailed in the following sections. For Transport return codes specific to a method, refer to that method's section:

- **Channel.init** return codes: Section 10.5.4.
- **Channel.read** return codes: Section 10.6.3.
- **Channel.write** return codes: Section 10.9.5.
- **Channel.flush** return codes: Section 10.10.3.

TRANSPORT RETURN CODE	DESCRIPTION
TransportReturnCodes.SUCCESS	Indicates successful completion of the operation.
TransportReturnCodes.FAILURE	Indicates that initialization has failed and cannot progress. The Channel.state should be CLOSED . See the Error content for more information.
TransportReturnCodes.INIT_NOT_INITIALIZE D	Indicates that the Transport has not been initialized. See the Error content for more details. For details on initializing, refer to Section 10.2.

Table 20: General Transport Return Codes

10.1.6 Application Lifecycle

The figure below illustrates the typical lifecycle and method calls of a client or server application using the Transport API.

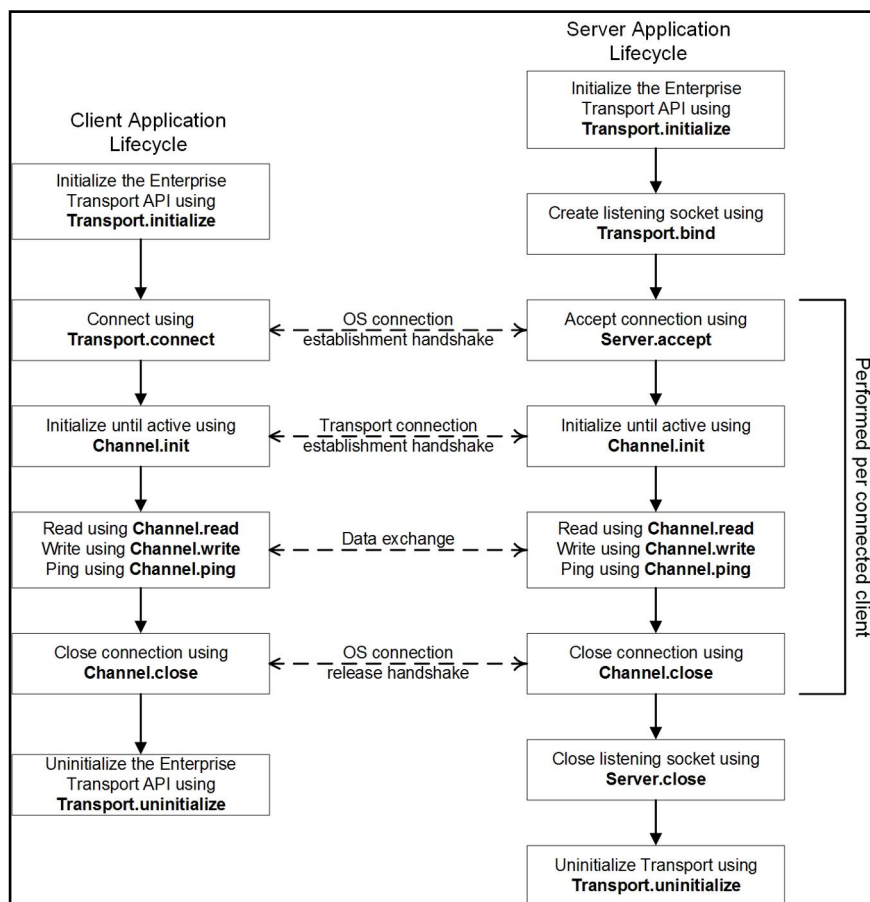


Figure 27. Transport Application Lifecycle

10.2 Initializing and Uninitializing the Transport

Every application using the transport, client or server, must first initialize it. This initialization process allows the Transport to pre-allocate internal memory associated with buffering and channel management. During this process, the transport also performs any necessary boot strapping associated with its underlying dependencies, such as WinSock or WinINET if on a Windows platform.

Similarly, when an application has completed its usage of the Transport, it must uninitialize it. The uninitialization process will release internal resources. These resources will eventually be garbage collected.

Additionally, you can use a few initialization arguments (**InitArgs**).

10.2.1 Initialization and Uninitialization Methods

The following table provides additional information about the Transport methods used for initializing and uninitializing.

METHOD	DESCRIPTION
initialize	The first Transport function that an application should normally call. This creates and initializes internal memory (i.e., objects). The initialize method also allows the user to specify the locking model they want applied to the Transport. For more information, refer to Section 10.2.4.
uninitialize	The last Transport method that an application should call. This uninitializes internal resources. These resources will eventually be garbage collected.

Table 21: Initialization and Uninitialization Methods

10.2.2 Initialization Arguments Methods

The following table provides information about the arguments used with initialization.

METHOD	DESCRIPTION
globalLocking	Allows the user to specify the locking model they want applied to the Transport. The argument defaults to false . For more information, refer to Section 10.2.4. After global locking is chosen, it cannot be changed without uninitializing and reinitializing the Transport.
socketProtocolPoolLimit	Allows the user to set the initial number of Socket Protocol objects allocated in the Transport. The Transport pools and reuses TCP/Websocket Channel and Server objects in the Transport. Any additional objects will be garbage collected when the Channel or Server close. If the limit is set, Transport pools only limited number of Socket Protocol objects. The argument defaults to -1 , which means unlimited. After a limit is chosen, it cannot be changed without uninitializing and reinitializing the Transport.
clear	Clears this object, so that it can be reused. All initialization arguments will be reset to default values.

Table 22: Initialization Arguments InitArgs Methods

10.2.3 Initialization Reference Counting with Example

Both the **initialize** and **uninitialize** methods use reference counting. This allows only the first call to **initialize** to perform any memory allocation / object creation and only the last necessary call to **uninitialize** to undo the work of initialize. Only a single **initialize** call need be made within an application, however this call must be the first Transport method call performed.

The following example demonstrates the use of **initialize** and **uninitialize**.

```
Error error = TransportFactory.createError();
InitArgs initArgs = TransportFactory.createInitArgs();
initArgs.globalLocking(true);
/* Starting Transport use, must call initialize first */
if (Transport.initialize(initArgs, error) != TransportReturnCodes.SUCCESS)
{
    System.out.println("Initialize(): failed <" + error.text() + ">");
    /* End application */
    return 0;
}

/* Any transport use occurs here - see following sections for all other functionality */
/* All Transport use is complete, must uninitialize */
Transport.uninitialize();

/* End application */
return 0;
```

Code Example 5: Transport Initialization and Uninitialization

10.2.4 Transport Locking Models

The Transport offers the choice of several locking models. These locking models are designed to offer maximum flexibility and allow the transport to be used in the manner that best fits the application's design. There are three types of locking that occur in the transport. **Global locking** is used to protect any resources that are shared across connections or channels, such as connection pools. **Read and Write Channel locking** is used to protect any resources that are shared within a single connection or channel, such as a channel's buffer pool. **Shared pool locking** is used to protect a server's shared buffer pool, which is used to share one pool of buffers across multiple connections.

All three types of locking can be enabled or disabled, depending on the needs of the application:

- Global locking is controlled by **InitArgs.globalLocking(boolean)**, with **InitArgs** as a parameter to the **Transport.initialize()** method. After global locking is chosen, it cannot be changed without uninitializing and reinitializing the transport. This behavior ensures that a locking change is not pushed onto pre-established connections.
- For client connections, Channel locking is controlled on a per channel basis via **ConnectOptions.channelReadLocking(Boolean)** and **ConnectOptions.channelWriteLocking(Boolean)**, with **ConnectOptions** as a parameter to the **Transport.connect** method. Once channel locking is chosen, it cannot be changed without closing and reconnecting the connection.
- For server connections, Channel locking is controlled on a per channel basis via **AcceptOptions.channelReadLocking(Boolean)** and **AcceptOptions.channelWriteLocking(Boolean)**, with **AcceptOptions** as a parameter to the **Server.accept** method. Once channel locking is chosen, it cannot be changed without closing and re-accepting a connection.
- Shared pool locking is controlled on a per-server basis via **BindOptions.sharedPoolLock(boolean)**, with **BindOptions** as a parameter to the **Transport.bind()** method (for more information, refer to Section 10.4.1.1).

The following table describes the locking models and when to use each one.

LOCK MODEL	DESCRIPTION
None	The “no locking” model can be used for single-threaded applications to avoid any locking overhead as there is no risk of multiple thread access. It is additionally useful for multi-threaded applications that utilize the Transport from within a single thread, when the locking is managed by the application. An application can read a Channel from one thread and write to the same Channel using a different thread. This requires synchronization while creating and destroying connections so the use of Global lock is preferable.
Global, Channel (and Shared if using a Server)	Both global locking and channel locking will be enabled. This, in addition to enabling shared pool locking, will provide full thread safety. This setting allows for accessing the same channel from multiple threads. Note that writing messages from multiple threads can result in ordering issues and it is not recommended to write related messages across different threads. Reading across multiple threads can also introduce ordering issues associated with information received, which may or may not impact ordering of related messages.
Global	Global locking is enabled and channel locking is disabled. This allows for any globally shared resources (accessed through Transport methods) to be protected, but any channel related resources are not thread safe. This model allows for each channel to be handled by its own dedicated thread, but channel creation and destruction can occur across threads.
Channel	Global locking is disabled and Channel locking is enabled. This allows for accessing the same channel from multiple threads for reading and writing, but globally shared resources to be unprotected.
Shared	Global locking is disabled, Channel locking is disabled, and Shared locking is enabled. This allows for sharing of the shared pool buffers.

Table 23: Locking Types

10.3 Creating the Connection

The Transport Package allows for outbound connections to be established and managed. An outbound connection allows an application to connect to a listening socket or multicast network, often to some type of Provider running on a well known port number or multicast group address and port.

10.3.1 Network Topologies

The Enterprise Transport API supports two types of network topologies:

- **unified**: A **unified** network topology is one where the **Channel** uses the same connection information (**address:port**) to send and receive all content.
- **segmented**: A **segmented** network topology is one where the **Channel** uses different connection information for sending and receiving. In the case of a **segmented** network, this allows for sent content and received content to be on different underlying **address:port** combinations.

On TCP-based networks, the Enterprise Transport API supports only a **unified** topology (**ConnectionTypes.SOCKET**, **ConnectionTypes.WEBSOCKET**, **ConnectionTypes.HTTP**, and **ConnectionTypes.ENCRYPTED**), but on multicast-based networks, the Enterprise Transport API supports both **unified** and **segmented** topologies (**ConnectionTypes.RELIABLE_MCAST** and **ConnectionTypes.SEQUENCED_MCAST**).

For configuration information on network topologies, refer to Table 26.

10.3.1.1 TCP-based Networks

If an application needs to communicate with multiple devices using a **ConnectionTypes.SOCKET**, **ConnectionTypes.HTTP**, or **ConnectionTypes.ENCRYPTED** connection type, a unique (point-to-point) connection is required for each device. Any content that needs to go to all devices must be written (or “fanned out”) on all connections, which is the application’s responsibility. The following diagram illustrates this scenario:

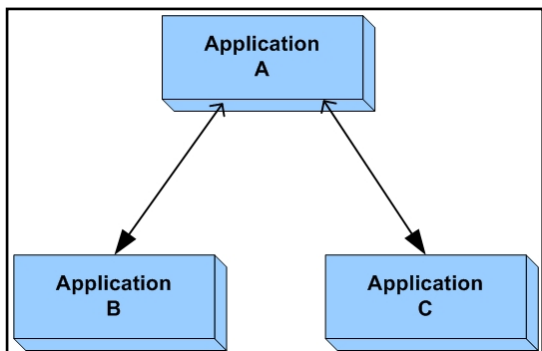


Figure 28. Unified TCP Network

In Figure 28, Application A has a unique, point-to-point connection with each of the applications B and C. If Application A wants to send the same content to both applications B and C, Application A must send the same content over each connection. In this scenario, if content is sent over only one connection, only the application on the corresponding end of that connection receives the content.

For TCP connections, consumer and non-interactive provider applications connect as shown in the following diagram. The arrows used in the figure depict the directions in which connections are established. Consumers typically connect to a well known port number associated with some kind of Interactive Provider (e.g., the LSEG Real-Time Advanced Distribution Server or LSEG Real-Time), while non-interactive providers typically connect to a well known port on the LSEG Real-Time Advanced Distribution Hub.

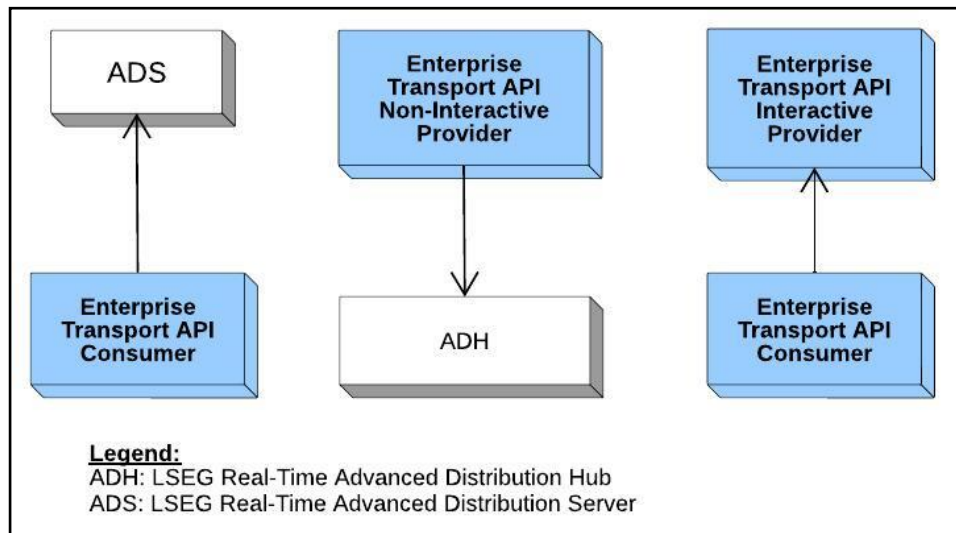


Figure 29. TCP Connection Creation

10.3.1.2 Multicast-based Networks: Unified

If an application wishes, it can communicate with multiple devices using a single connection to a multicast network (presuming the other devices access the same multicast network). In this case, a single transmission is sufficient to send data to all connected devices.

In the following diagram (Figure 30), all applications send and receive content on the same multicast network. Because the same network is used for sending and receiving traffic, all traffic is seen by all applications. Anything sent by one application will be received by all other applications on the network.

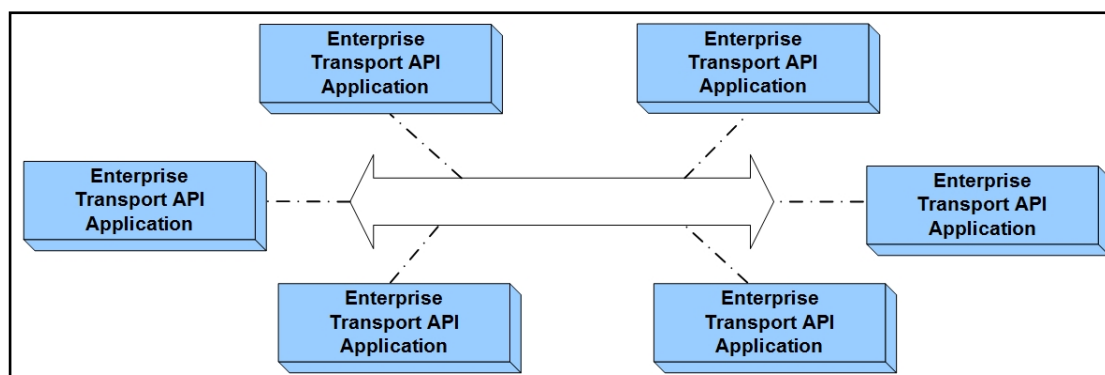


Figure 30. Unified Multicast Network

10.3.1.3 Multicast-based Networks: Segmented

In segmented multicast networks, applications transmit and receive data over different networks allowing users to separate applications based on the content they need to send or receive.

In Figure 31:

- Applications A - C only send content on Network 1; they do not receive content from Network 1 (i.e., Application A does not see content sent by applications B or C). Applications A - C receive only the content sent on Network 2 (by applications D - F).
- Applications D - F only send content on Network 2; they do not receive content from Network 2 (i.e., Application D does not see content sent by applications E or F). Applications D - F receive only the content sent on Network 1 (by applications A - C).

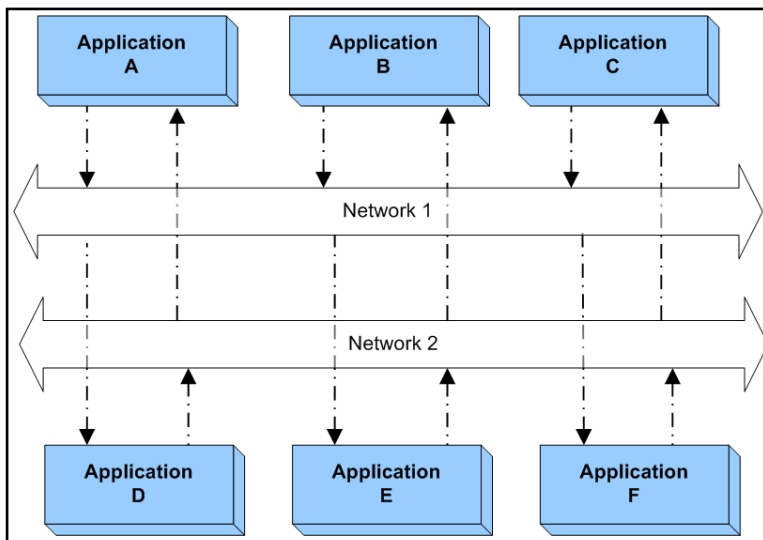


Figure 31. Segmented Multicast Network

Figure 32 illustrates non-interactive provider applications using outbound multicast connections leveraging a segmented connection type. This allows the LSEG Real-Time Advanced Distribution Hub to receive only content published by non-interactive provider applications (via the **NiProv Send Network**).

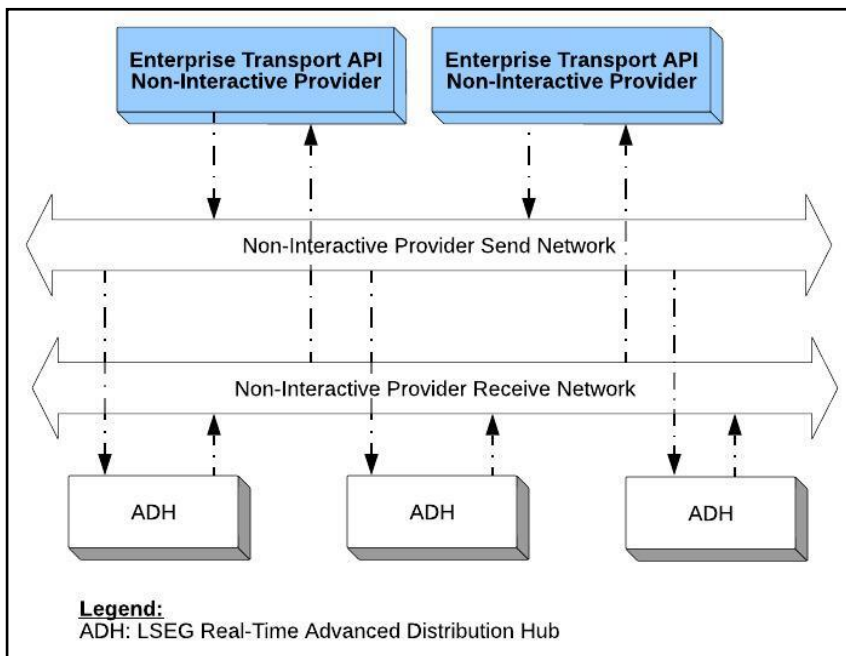


Figure 32. Multicast Connection Creation

10.3.2 Creating the Outbound Connection: `Transport.connect` Method

An application can create an outbound connection by using the `Transport.connect` method.

METHOD	DESCRIPTION
<code>Transport.connect</code>	Establishes an outbound connection, which can leverage standard sockets, HTTP, or HTTPS. Returns an Channel that represents the connection to the user. In the event of an error, <code>NULL</code> is returned and additional information can be found in the Error structure. Connection options are passed in via a ConnectOptions object described in Table 25. Once a connection is established and transitions to the ConnectionTypes.ACTIVE state, this Channel can be used for other transport operations. For more information about channel initialization, refer to Section 10.5.

Table 24: `Transport.connect` Method

10.3.2.1 ConnectOptions Methods

METHOD	DESCRIPTION
<code>blocking</code>	If set to true, blocking I/O will be used for this Channel. When I/O is used in a blocking manner on a Channel, any reading or writing will complete before control is returned to the application. In addition, the <code>Transport.connect</code> method will complete any initialization on the Channel prior to returning it. Blocking I/O prevents the application from performing any operations until the I/O operation is completed. Blocking I/O is typically not recommended. An application can leverage an I/O notification mechanism to allow efficient reading and writing, while using other cycles to perform other necessary work in the application. An I/O notification mechanism enables the application to read when data is available, and write when output space is available.
<code>channelReadLocking</code>	Accessor method, used to set or check if the connection will use locking on reading.
<code>channelWriteLocking</code>	Accessor method, used to set or check if the connection will use locking on writing.
<code>clear</code>	Clears this object, so that it can be reused.
<code>componentVersion</code>	An optional, user-defined component version string appended behind the standard Enterprise Transport API component version information. If the combined component version length exceeds the maximum supported by the Enterprise Transport API, the user-defined information will be truncated.
<code>compressionType</code>	The type of compression the client would like performed for this connection. Compression is negotiated between the client and server and may not be performed if only the client has it enabled. For more information about supported compression types and compression negotiation, refer to Section 10.4.3.
<code>connectionType</code>	Type of connection to establish. Creation of encrypted, TCP-based socket, HTTP, and UDP-based multicast connection types are available across all supported platforms. connectionTypes are described in more detail in Section 10.1.2.2.
<code>credentialsInfo</code>	credentialsInfo object representing Proxy credentials. For more information, refer to Section 10.15.2.3.
<code>guaranteedOutputBuffers</code>	A guaranteed number of buffers made available for this Channel to use while writing data. Guaranteed output buffers are allocated at initialization time. For more information, refer to Section 10.8.

Table 25: `ConnectOptions` Methods

METHOD	DESCRIPTION
majorVersion	The major version of the protocol that the client intends to exchange over the connection. This value is negotiated with the server at connection time. The outcome of the negotiation is provided via the majorVersion information on the Channel. Typically, a major version increase is associated with the introduction of incompatible change. The transport layer is data neutral and does not change nor depend on any information in content being distributed. This information is provided to help client and server applications manage the information they are communicating. For more details, refer to Section 9.5.1.
minorVersion	The minor version of the protocol that the client intends to exchange over the connection. This value is negotiated with the server at connection time. The outcome of the negotiation is provided via the minorVersion information on the Channel. Typically, a minor version increase is associated with a fully backward compatible change or extension. The transport layer is data neutral and does not change nor depend on any information in content being distributed. This information is provided to help client and server applications manage the information they are communicating. For more details, refer to Section 9.5.1.
multicastOpts	A substructure containing multicast-based connection type-specific options. These settings are used for ConnectionTypes.RELIABLE_MCAST. For information about specific options, refer to Section 10.3.2.5.
numInputBuffers	The number of sequential input buffers to allocate for reading data into. This controls the maximum number of bytes that can be handled with a single network read operation. Input buffers are allocated at initialization time.
pingTimeout	The clients desired ping timeout value. This may change through the negotiation process between the client and the server. After the connection becomes active, the actual negotiated value becomes available through the pingTimeout value on the Channel. When determining the desired ping timeout, the typically used rule of thumb is to send a heartbeat every pingTimeout/3 seconds. For more information, refer to Section 10.12.
protocolType	The protocol type that the client intends to exchange over the connection. If the protocolType indicated by a server does not match the protocolType that a client specifies, the connection will be rejected. When a Channel becomes active for a client or server, this information becomes available via the protocolType on the Channel. The transport layer is data neutral and does not change nor depend on any information in content being distributed. This information is provided to help client and server applications manage the information they are communicating. For more details, refer to Section 9.5.1.
segmentedNetworkInfo	Connection parameters when sending and receiving on different networks. This is typically used with multicast networks that have different groups of senders and receivers (e.g., NIPProvider can send on one network and receive on another). segmentedNetworkInfo is described in more detail in Section 10.3.2.5.
seqMCastOpts	A substructure containing multicast-based, connection type-specific options. These settings are used for ConnectionTypes.SEQUENCED_MCAST. For information about specific options, refer to Section 10.3.2.7.
shmemOpts	A substructure containing shared memory-based connection type-specific options. These settings are used for ConnectionTypes.UNIDIR_SHMEM. For information about specific options, refer to Section 10.3.2.6.

Table 25: ConnectOptions Methods (Continued)

METHOD	DESCRIPTION
sysRecvBufSize	Accessor method, used to set or get the system's receive buffer size used for this connection. Not setting or setting of 0 indicates to use the default size of 64 KB. This can also be set or changed via Channel.ioctl for values less than or equal to 64 KB. For values larger than 64KB, you must use this method to set sysRecvBufSize prior to the connect system call.
sysSendBufSize	Accessor method, used to set or get the system's send buffer size used for this connection. No setting or a setting of 0 indicates to use the default size of 64KB.
tcpOpts	TcpOpts object representing TCP-based connection type-specific options. These settings are used for ConnectionTypes of SOCKET , HTTP , and ENCRYPTED . For information about specific options, refer to Section 10.3.2.4.
tunnelingInfo	tunnelingInfo object representing tunneling connection specific options. This is only valid for HTTP and Encrypted connection types. - For more information on ConnectionTypes, refer to Section 10.1.2.2. - For more information on TunnelingInfo, refer to Section 10.15.1.1.
unifiedNetworkInfo	unifiedNetworkInfo object representing connection parameters used when sending and receiving on same network. This is typically used with ConnectionTypes.SOCKET , ConnectionTypes.HTTP , ConnectionTypes.ENCRYPTED , and fully connected/mesh multicast networks. unifiedNetworkInfo is described in more detail in Section 10.3.2.2.
userSpecObject	A reference that can be set by the application. This value is not modified by the transport, but will be preserved and stored in the userSpecObject of the Channel returned from the Transport.connect method. This information can be useful for coupling this Channel with other user created information, such as a watch list associated with this connection.
wSocketOpts	Specifies the WSocketOpts object, which contains options for use with WebSocket connections (i.e., the connection type ConnectionTypes.WEBSOCKET). For further information on WSocketOpts , refer to Section 10.3.2.8.

Table 25: ConnectOptions Methods (Continued)

10.3.2.2 UnifiedNetworkInfo Method Options

METHOD	DESCRIPTION
address	Configures the address or hostname to use in a unified network configuration. All content will be sent and received on this address:serviceName pair.
serviceName	Configures the numeric port number or service name (as defined in etc/services file) to use in a unified network configuration. All content will be sent and received on this address:serviceName pair.
interfaceName	A character representation of an IP address or hostname associated with the local network interface to use for sending and receiving content. This value is intended for use in systems which have multiple network interface cards, and if not specified the default network interface will be used.
unicastServiceName	Configures the numeric port number or service name (as defined in the etc/services file) to use for all unicast UDP traffic in a unified network configuration. This parameter is only required for multicast connection types (ConnectionTypes.RELIABLE_MCAST and ConnectionTypes.SEQUENCED_MCAST). If multiple connections or applications are running on the same host, this must be unique for each connection.

Table 26: UnifiedNetworkInfo Method Options

10.3.2.3 SegmentedNetworkInfo Method Options

METHOD	DESCRIPTION
recvAddress	Configures the receive address or hostname to use in a segmented network configuration. All content is received on this recvAddress:recvServiceName pair.
recvServiceName	Configures the receive network's numeric port number or service name (as defined in the etc/services file) to use in a segmented network configuration. All content is received on this recvAddress:recvServiceName pair.
sendAddress	Configures the send address or hostname to use in a segmented network configuration. All content is sent on this sendAddress:sendServiceName pair.
sendServiceName	Configures the send network's numeric port number or service name (as defined in the etc/services file) to use in a segmented network configuration. All content is sent on this sendAddress:sendServiceName pair.
interfaceName	A character representation of an IP address or hostname associated with the local network interface to use for sending and receiving content. This value is intended for use in systems which have multiple network interface cards, and if not specified the default network interface is used.
unicastServiceName	Configures the numeric port number or service name (as defined in the etc/services file) to use for all unicast UDP traffic in a unified network configuration. This parameter is only required for multicast connection types (ConnectionTypes.RELIABLE_MCAST and ConnectionTypes.SEQUENCED_MCAST). If multiple connections or applications are running on the same host, this must be unique for each connection.

Table 27: SegmentedNetworkInfo Method Options

10.3.2.4 TcpOpts Method Option

METHOD	DESCRIPTION
tcpNodelay	If set to true , this disables Nagle's Algorithm for all accepted connections. Nagle's Algorithm allows more efficient use of TCP by delaying and combining small packets to reduce repeated overhead of TCP headers. Disabling Nagle's Algorithm can lead to lower latency by removing this delay, but can add increased bandwidth use as a result of the additional TCP header used with each small packet.

Table 28: TcpOpts Method Option

10.3.2.5 MCastOpts Method Options

METHOD	DESCRIPTION
disconnectOnGaps	Defaults to false, so if any multicast gap situation occur the underlying connection will not be closed. This allows the application to perform any item level recovery it may be able to do in order to reduce unnecessary bandwidth of full recovery on the multicast network. If set to true, the underlying connection will be closed when any multicast gap situation occurs. A multicast gap situation is reported as a return value of PACKET_GAP_DETECTED , SLOW_READER , or CONGESTION_DETECTED from Channel1.read .
packetTTL	Controls the maximum number of components (network switches, etc.) a multicast datagram can traverse before it is removed from the network. Setting this to 0 , prevents packets from leaving the sending machine. When set to 255, the packet is not limited in the number of components it can traverse and is not removed from the network.
tcpControlPort	Specifies the port number that rrdump (a monitoring tool available in the LSEG Real-Time Distribution System Infrastructure Tools package) should use. If set to or left as NULL, tcpControlPort uses the same port number as the unicastServiceName setting. If set to -1 , a control port will not be opened.
portRoamRange	Specifies the number of port numbers on which to attempt binding if the unicastServiceName fails to bind. The unicastServiceName is used as the starting point and will increment by 1 until it reaches the number specified in portRoamRange or successfully binds. If set to 0 , port roaming is disabled and the connection will attempt to bind only to the unicastServiceName .

Table 29: MCastOpts Method Options

10.3.2.6 ShmemOpts Method

METHOD	DESCRIPTION
maxReaderLag	Maximum number of messages that the client can have waiting to be read. If the client “lags” the server by more than this amount, the client will be disconnected on its next attempt to read. The default is equal to 75% of the number of buffers in the shared memory segment.

Table 30: ShmemOpts Method Option

10.3.2.7 SeqMCastOpts Method

METHOD	DESCRIPTION
maxMsgSize	Sets the maximum amount of data (in bytes) that can be sent and received on any packet over a ConnectionType.SEQUENCED_MCAST connection. Defaults to 3000 bytes.
instanceId	The originating IP address and port and the instanceId identify the sequenced multicast channel. When multiple applications run on the same host, unique instanceId values allow them to operate independently.

Table 31: SeqMCastOpts Method Option

10.3.2.8 WSocketOpts Structure

OPTION	DESCRIPTION
httpCallback	Sets a callback to provide an HTTP message which includes header and cookie information during the WebSocket handshake.
protocols	Specifies a whitespace- or comma-delineated list of supported protocols, in order of preference, starting with the most preferred. Clients and servers negotiate which protocol to use based on this list, working from most-preferred to least-preferred. When using with the BindOptions option, if protocols is not defined, the Channel does not accept incoming WebSocket connection requests. Supported protocols include rss1.json.v2, rss1.rwf, and tr_json2. While tr_json2 is used for simplified streaming connections, RTSDK product development plans to deprecate tr_json2 in the near future. rss1.json.v2 functionally replaces tr_json2. By default setting, protocols is undefined.
maxMsgSize	Sets the maximum size of a message that the WebSocket transport will read/write on the client side.

Table 32: WSocketOpts Options

10.3.2.9 RssiHttpMessage Structure of httpcallback

METHOD	DESCRIPTION
Cookies	Returns HTTP cookie information in httpCallback during the WebSocket handshake.
Data	Returns HTTP body information in httpCallback during the WebSocket handshake.
HttpHeaders	Returns HTTP headers list in httpCallback during the WebSocket handshake.
HttpRequestConnectionInfo	Returns HTTP request connection information in httpCallback during the WebSocket handshake. Request information is provided for the server. This connection information provides HTTP URL that was requested by the client, as ConnectionUri . It also includes HTTP request parameters, such as “/ WebSocket?UUID=ID-128256512”.
HttpResponseConnectionInfo	Returns HTTP request connection information in httpCallback during the WebSocket handshake. Request information is provided for the server.
userSpecObject	A reference that can be set by the application. This value is not modified by the Transport. See BindOptions.userSpecObject , AcceptOptions.userSpecObject , and ConnectOptions.userSpecObject .

Table 33: HttpMessage Methods

10.3.2.10 HttpRequestConnectionInfo Structure of httpcallback

METHOD	DESCRIPTION
ConnectionLine	Return HTTP connection line of the HTTP request during the WebSocket handshake.
ConnectionUri	Returns requested URL during the WebSocket handshake. For WebSocket connection, it is expected always to be “/WebSocket”. This part also includes HTTP request parameters, such as “/ WebSocket?UUID=ID-128256512”.

Table 34: HttpRequestConnectionInfo Methods

METHOD	DESCRIPTION
ContentLength	Returns the value of HTTP Content-Length header of the HTTP request during the WebSocket handshake.
RequestMethod	Returns HTTP method of the HTTP request during the WebSocket handshake. For WebSocket connection, it is expected always to be GET.

Table 34: HttpRequestConnectionInfo Methods (Continued)

10.3.3 Transport.connect Outbound Connection Creation Example

The following example demonstrates basic **Transport.connect** use in a non-blocking manner. The application first populates the **ConnectOptions** object and then attempts to connect. If the connection succeeds, the application then registers the **Channel1.SelectableChannel** with the I/O notification mechanism and continues with connection initialization (as described in Section 10.5).

```
Channel channel;
Selector selector;
Error error = TransportFactory.createError();
ConnectOptions cOpts = TransportFactory.createConnectOptions();

/* populate connect options, then pass to Transport.connect method - Transport should already be
   initialized */

cOpts.connectionType(ConnectionTypes.SOCKET); /* use standard socket connection */
cOpts.unifiedNetworkInfo().address("localhost"); /* connect to server running on same machine */
cOpts.unifiedNetworkInfo().serviceName("14002"); /* server is running on port number 14002 */
cOpts.pingTimeout(30); /* clients desired ping timeout is 30 seconds, pings should be sent every 10 */
cOpts.blocking(false); /* perform non-blocking I/O */
cOpts.compressionType(CompressionTypes.NONE); /* client does not desire compression for this
   connection */

/* populate version and protocol with RWF information */
cOpts.protocolType(Codec.protocolType());
cOpts.majorVersion(Codec.majorVersion());
cOpts.minorVersion(Codec.minorVersion());

if ((channel = Transport.connect(cOpts, error)) == null)
{
    System.out.println("Connection failure: " + error.text() + ", errorId=" + error.errorId()
        + " sysError=" + error.sysError());

    /* End application, uninitialize to clean up first */
    Transport.uninitialize();
    return;
}

/* Connection was successful, add SelectableChannel to I/O notification mechanism and initialize
   connection */
try
{

```

```

/* register for read and write select */
selector = Selector.open();
channel.SelectableChannel().register(selector, SelectionKey.OP_READ | SelectionKey.OP_WRITE,
    channel);
}
Catch (Exception e)
{
    /* Selector.open() and SelectableChannel.register() can throw numerous exceptions. */
    /* For this example catch all as Exception. */

    // handle exception and abort.
    return;
}

/* Continue on with connection initialization process, refer to Section 10.5 for more details. */

```

Code Example 6: Creating a Connection Using Transport.connect

10.3.4 Tunneling Connection Keep Alive

A client connection that is leveraging a connection type of **ConnectionTypes.HTTP** or **ConnectionTypes.ENCRYPTED** may be connecting through proxy devices as it tunnels through the Internet. Some proxy devices will force-close connections after certain elapsed time or time of day requirements are met. If one of these proxy devices is in a tunneling connections path, it can result in periodic connection loss. The Transport provides the **Channel.reconnectClient** method which allows a tunneling client application to pro-actively create another connection and bridge data flow from the existing connection, which will be closed, to the new connection. An application can use this, along with knowledge of the proxy device's time requirements, to keep an applications connection alive beyond the time limits enforced by the proxy which helps to avoid data recovery scenarios. This method is not used to perform any kind of connection or data recovery after a connection is closed or disconnected or for any non-tunneled connection types.

10.4 Server Creation and Accepting Connections

10.4.1 Creating a Listening Socket

The Transport Package allows you to establish and manage listening sockets, typically associated with a server. Listening sockets can be leveraged to create an application that accepts connections created through the use of the **Transport.connect** method. Listening sockets are used mainly by OMM interactive provider applications and are typically established on a well-known port number (known by other connecting applications).

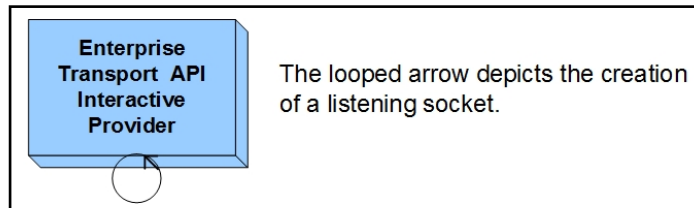


Figure 33. Transport API Server Creation

An application can create a listening socket connection by using the **Transport.bind** method, described in the following table.

METHOD	DESCRIPTION
Transport.bind	Establishes a listening socket connection, which supports connections from standard socket and HTTP Transport.connect users. Returns a server that represents the listening socket connection to the user. In the event of an error, NULL is returned and additional information can be found in the Error object. Options are passed in via a BindOptions object described in Section 10.4.1.1. Once a listening socket is established, this server can begin accepting connections. For more information, refer to Section 10.4.2.

Table 35: Transport.bind Method

10.4.1.1 BindOptions Methods

METHOD	DESCRIPTION
channelsBlocking	If set to true , blocking I/O will be used for all connected Channels . When I/O is used in a blocking manner on a Channel , any reading or writing will complete before control is returned to the application. Blocking I/O prevents the application from performing any operations until the I/O operation is completed. Blocking I/O is typically not recommended. An application can leverage an I/O notification mechanism to allow efficient reading and writing, while using other cycles to perform other necessary work in the application. An I/O notification mechanism enables the application to read when data is available, and write when output space is available.
clear	Clears this object so that it can be reused.
clientToServerPings	If set to true , heartbeat messages are required to flow from the client to the server. If set to false , the client is not required to send heartbeats. LSEG Real-Time Distribution System and other LSEG components typically require this to be set to true .
componentVersion	An optional, user-defined component version string appended behind the standard ETA component version information. If the combined component version length exceeds the maximum supported by the Enterprise Transport API, the user-defined information will be truncated.

Table 36: BindOptions Methods

METHOD	DESCRIPTION
compressionType	The type of compression the server wants to apply for this connection. Compression is negotiated between the client and server and may not be performed if only the server has this enabled. The server can force compression, regardless of client settings, by using the forceCompression option. For more information about supported compression types and compression negotiation, refer to Section 10.4.3.
compressionLevel	Sets the level of compression to apply. Allowable values are 0 to 9. - A compressionLevel of 1 results in the fastest compression. - A compressionLevel of 9 results in the best compression. - A compressionLevel of 6 is a compromise between speed and compression. - A compressionLevel of 0 will copy the data with no compression applied. For more information on supported compression levels, refer to Section 10.4.3.
connectionType	Specifies the type of connection to establish. ConnectionTypes are described in more detail in Section 10.1.2.2.
encryptionOpts	If the connectionType is set to ENCRYPTED , encryptionOpts sets the TLS encryption options. For information, refer to Section 10.4.1.2.
forceCompression	If set to true , this forcibly enables compression, regardless of client preference. When enabled, compression will use the compressionType and compressionLevel specified by the server. If set to false , compression is negotiated between the client and server. For more information about supported compression types and compression negotiation, refer to Section 10.4.3.
guaranteedOutputBuffers	A guaranteed number of buffers made available for each Channel to use while writing data. Each buffer is created to contain maxFragmentSize bytes. Guaranteed output buffers are allocated at initialization time. For more information, refer to Section 10.8. NOTE: For ConnectionTypes.UNIDIR_SHMEM, this parameter determines the number of buffers in the shared memory segment. The size of the shared memory segment will approximate guaranteedOutputBuffers * maxFragmentSize.
interfaceName	A character representation of an IP address or hostname for the local network interface to which to bind. The Transport will establish connections on the specified interface. This value is intended for use in systems which have multiple network interface cards. If not populated, a connection can be accepted on all interfaces (INADDR_LOOPBACK is used). If the loopback address (127.0.0.1) is specified, connections can be accepted only when instantiating from the local machine (INADDR_LOOPBACK is used).
majorVersion	Specifies the major version of the protocol supported by the server. The actual major version used is negotiated with the client at connection time. The outcome of the negotiation is provided via majorVersion on the Channel. Typically, the major version increases with the introduction of an significant (i.e., incompatible) change. The transport layer is data-neutral and allows the flow of any type of content. majorVersion is provided to help client and server applications manage the information they communicate. For more details, refer to Section 9.5.1.
maxFragmentSize	The maximum size buffer that will be written to the network. If a larger buffer is required, the Transport will internally fragment the larger buffer into smaller maxFragmentSize buffers. This is different from application level message fragmentation done via the Message Package (as discussed in Section 13.1). Any guaranteed, shared, or input buffers created will use this size. This value is passed to all connected client applications and enforces a common message size between components. For more information about Transport buffer fragmentation, refer to Section 10.9.
maxOutputBuffers	The maximum number of output buffers allowed for use by each Channel . (maxOutputBuffers - guaranteedOutputBuffers) is equal to the number of shared pool buffers that each Channel is allowed to use. Shared pool buffers are only used if all guaranteedOutputBuffers are unavailable. If equal to the guaranteedOutputBuffers value, no shared pool buffers are available.

Table 36: BindOptions Methods (Continued)

METHOD	DESCRIPTION
minorVersion	The minor version of the protocol supported by the server. The actual minor version used is negotiated with the client at connection time. The outcome of the negotiation is provided via minorVersion on the Channel. Typically, the minor version increases with the introduction of a fully backward-compatible change or extension. The transport layer is data-neutral and allows the flow of any type of content. minorVersion is provided to help client and server applications manage the information they communicate. For more details, refer to Section 9.5.1.
minPingTimeout	The server's lowest allowable ping timeout value. This is the lowest possible value allowed in the negotiation between client and servers pingTimeout values. After the connection becomes active, the actual negotiated value becomes available through the pingTimeout value on the Channel. When determining the desired ping timeout, the rule of thumb is to send a heartbeat every pingTimeout/3 seconds. For more information, refer to Section 10.12.
numInputBuffers	The number of sequential input buffers used by each Channel for data reading. This controls the maximum number of bytes that can be handled with a single network read operation on each channel. Each input buffer will be created to contain maxFragmentSize bytes. Input buffers are allocated at initialization time.
pingTimeout	The server's maximum allowable ping timeout value. This is the largest possible value allowed in the negotiation between the client and the server's pingTimeout value. After the connection becomes active, the actual negotiated value becomes available through the pingTimeout value on the Channel. When determining the desired ping timeout, the rule of thumb is to send a heartbeat every pingTimeout/3 seconds. For more information, refer to Section 10.12.
protocolType	Sets the protocol type that the server uses on its connections. The server rejects connections from clients that do not use the specified protocolType. Server must bind using RSSL_RWF_PROTOCOL_TYPE. When a Channel becomes active for a client or server, this information becomes available via the protocolType on the Channel. The transport layer is data-neutral and allows the flow of any type of content. protocolType is provided to help client and server applications manage the information they communicate. For more details, refer to Section 9.5.1.
serverBlocking	If set to true, blocking I/O will be used for this server. When I/O is used in a blocking manner on a server, the Server.accept method will complete any initialization on the Channel prior to returning it. Blocking I/O prevents the application from performing any operations until the I/O operation is completed. Blocking I/O is typically not recommended. An application can leverage an I/O notification mechanism to allow efficient use, while using other cycles to perform other necessary work in the application.
serverSharedSocket	If set to true, the server permits sharing of the socket. - On Linux, this feature is available only with certain patch levels on Linux 6. - On Windows, the serverSharedSocket method sets the SO_REUSEADDR option instead of SO_EXCLUSIVEADDRUSE on the server socket.
	NOTE: Setting option serverSharedSocket to RSSL_TRUE would trigger an error in case option SO_REUSEPORT is not supported by Linux.
	 WARNING! If a hacker can use the SO_REUSEADDR socket option to hijack a port in a server application, the application is not secure.
serverToClientPings	If set to true, heartbeat messages are required to flow from the server to the client. If set to false, the server is not required to send heartbeats. LSEG Real-Time Distribution System and other LSEG components typically require this to be set to true.

Table 36: BindOptions Methods (Continued)

METHOD	DESCRIPTION
serviceName	A character representation of a numeric port number or service name (as defined in the etc/services file) on which to bind and open a listening socket.
sharedPoolSize	The maximum number of buffers to make available as part of the shared buffer pool. The shared buffer pool can be drawn upon by any connected Channel, where each channel is allowed to use up to (maxOutputBuffers - guaranteedOutputBuffers) number of buffers. Each shared pool buffer will be created to contain maxFragmentSize bytes. If set to 0, a default of 1,048,567 shared pool buffers will be allowed. The shared pool is not fully allocated at bind time. As needed, shared pool buffers are added and reused until the server is shut down. For more information, refer to Section 10.8. NOTE: It is considered an invalid configuration to allow more shared pool buffers (maxOutputBuffers - guaranteedOutputBuffers) than the sharedPoolSize. If this happens, an error is returned from the Transport.bind method.
sharedPoolLock	If set to true , the shared buffer pool will have its own locking performed. This setting is independent of any other locking mode options. Enabling a shared pool lock allows shared pool use to remain thread safe while still disabling channel locking. For more information, refer to Section 10.2.4.
tcpOpts	Specifies the TcpOpts class, which contains the tcpNoDelay option for use with TCP-based connection types. For further information about tcpNoDelay , refer to Section 10.3.2.4.
userSpecObject	A reference that can be set by the application. This value is not modified by the transport, but is preserved and stored in the userSpecObject of the server returned from the Transport.bind method if a userSpecObject was not specified in the AcceptOptions . This information can be useful for coupling this server with other user-created information, such as a list of connected Channels .
WSocketOpts	Specifies the WSocketOpts object, which contains options for use with WebSocket connections (i.e., the connection type ConnectionTypes.WEBSOCKET). For further information about WSocketOpts, refer to Section 10.1.4.2.

Table 36: BindOptions Methods (Continued)

10.4.1.2 Transport.bind Listening Socket Connection Creation Example

The following example demonstrates basic **Transport.bind** use in a non-blocking manner. The application first populates the **BindOptions** and then attempts to create a listening socket. If the bind succeeds, the application then registers the **Server.SelectableChannel** with the I/O notification mechanism and waits to be alerted of incoming connection attempts. For more details on accepting or rejecting incoming connection attempts, refer to Section 10.4.2.

```
Server srvr = null;
BindOptions bOpts = TransportFactory.createBindOptions();
Selector selector = null;

/* populate bind options, then pass to bind method - ETA should already be initialized */

bOpts.serviceName("14002"); /* server is running on port number 14002 */
bOpts.pingTimeout(45); /* servers desired ping timeout is 45 seconds, pings should be sent every 15 */
bOpts.minPingTimeout(30); /* min acceptable ping timeout is 30 seconds, pings should be sent every 10 */

/* set up buffering, configure for shared and guaranteed pools */
bOpts.guaranteedOutputBuffers(1000);
bOpts.maxOutputBuffers(2000);
bOpts.sharedPoolSize(50000);
bOpts.sharedPoolLock(true);

bOpts.serverBlocking(false); /* perform non-blocking I/O */
bOpts.channelsBlocking(false); /* perform non-blocking I/O */
bOpts.compressionType(CompressionTypes.NONE); /* server does not desire compression for this
connection */

/* populate version and protocol with RWF information or protocol specific info */
bOpts.protocolType(Codec.protocolType());
bOpts.majorVersion(Codec.majorVersion());
bOpts.minorVersion(Codec.minorVersion());

if ((srvr = Transport.bind(bOpts, error)) == null)
{
    System.out.printf("Error (%d) (errno: %d) encountered with bind. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());

    /* End application, uninitialized to clean up first */
    Transport.uninitialize();
    return null;
}

/* Bind was successful, register Selector and wait for connections */
try
{
    selector = Selector.open();
}
catch (Exception e)
{
    System.out.println("Open Selector Exception: " + e.getMessage());
}
```

```
}  
try  
{  
    srvr.SelectableChannel().register(selector, SelectionKey.OP_ACCEPT, srvr);  
}  
catch (Exception e)  
{  
    System.out.println("Register Selector Exception: " + e.getMessage());  
}  
  
/* Use accept for incoming connections, read and write data to established connections, etc */
```

Code Example 7: Creating a Listening Socket Using `Transport.bind`

10.4.2 Accepting Connection Requests

After establishing a listening socket, the `server.SelectableChannel` can be registered with an I/O notification mechanism. An alert from the I/O notification mechanism on the server's `SelectableChannel` indicates that a connection request has been detected. An application can begin the process of accepting or rejecting the connection by using the `Server.accept` method.

METHOD	DESCRIPTION
Server.accept	Uses the <code>server</code> that represents the listening socket connection and begins the process of accepting the incoming connection request. Returns a <code>Channel</code> that represents the client connection. In the event of an error, <code>NULL</code> is returned and additional information can be found in the <code>Error</code> object. The <code>Transport.accept</code> method can also begin the rejection process for a connection through the use of the <code>AcceptOptions</code> object as described in Section 10.4.2.1. Once a connection is established and transitions to <code>ChannelState.ACTIVE</code>, this <code>Channel</code> can be used for other transport operations. For more information about channel initialization, refer to Section 10.5.

Table 37: Server.accept Method

10.4.2.1 AcceptOptions Methods

METHOD	DESCRIPTION
nakMount	Indicates that the server wants to reject the incoming connection. This may be due to some kind of connection limit being reached. For non-blocking connections to successfully complete rejection, the initialization process must still be completed. For more information about channel initialization, refer to Section 10.5.
userSpecObject	A reference that can be set by the application. This value is not modified by the transport, but will be preserved and stored in the <code>userSpecObject</code> of the <code>Channel</code> returned from the <code>Server.accept</code> method. If this value is not set, the <code>Channel.userSpecObject</code> will be set to the <code>userSpecObject</code> associated with the <code>Server</code> that is accepting this connection.
channelReadLocking	Sets or checks whether the connection will use locking on reading.
channelWriteLocking	Sets or checks whether the connection will use locking on writing.
sysSendBufSize	Sets or checks the system's send buffer size used for this connection. No setting, or a setting of 0 indicates to use the default (64K). <code>sysRecvBufSize</code> is set via the <code>BindOptions</code> (for details, refer to Section 10.4.1.1).
clear	Clears the object for reuse.

Table 38: AcceptOptions Methods

10.4.2.2 Server.accept Accepting Connection Example

The following example demonstrates basic **Server.accept** use. The application first populates **AcceptOptions** and then attempts to accept the incoming connection request. If the accept succeeds, the application registers the new **Channel.SelectableChannel** with the I/O notification mechanism and continues with connection initialization, described in Section 10.5.

```
/* Accept is typically called when servers socketId indicates activity */
Channel chnl = null;
AcceptOptions aOpts = TransportFactory.createAcceptOptions();

/* populate accept options, then pass to accept method - ETA should already be initialized */
aOpts.nakMount(false); /* allow the connection */

if ((chnl = srvr.accept(aOpts, error)) == null)
{
    System.out.printf("Error (%d) (errno: %d) encountered with accept. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());

    /* End application, uninitialize to clean up first */
    Transport.uninitialize();
    return null;
}

/* Accept was successful, register Selector and wait for connections */
Selector selector = null;
try
{
    selector = Selector.open();
}
catch (Exception e)
{
    System.out.println("Open Selector Exception: " + e.getMessage());
}
try
{
    chnl.SelectableChannel().register(selector, SelectionKey.OP_READ | SelectionKey.OP_WRITE,
        chnl);
}
catch (Exception e)
{
    System.out.println("Register Selector Exception: " + e.getMessage());
}

/* Continue the connection initialization process, for details, refer to Section 10.5 */
```

Code Example 8: Accepting Connection Attempts using Server.accept

10.4.3 Compression Support

As mentioned, the Transport supports the use of data compression. The Enterprise Transport API supports the following **CompressionTypes**:

COMPRESSION TYPE	DESCRIPTION
NONE	Do not compress data on the connection.
ZLIB	Use zlib compression on the connection. Zlib, an open source utility, employs a variation of the LZ77 algorithm while compressing and decompressing data.
LZ4	Use LZ4 compression on the connection. LZ4 is a lossless data compression algorithm that is focused on speed of compression and decompression. It belongs to the LZ77 family of byte-oriented compression schemes. NOTE: The ConnectionTypes.WEBSOCKET connection type with the JSON protocol does not support LZ4 compression.

Table 39: CompressionTypes Values

The use of compression is negotiated during the connection establishment process. If the client's configured **ConnectOptions.compressionType** and the server's **BindOptions.compressionType** match, compression will be leveraged for the connection. The server's specified **compressionLevel** will determine the quality of the compression, where a lower value favours less time consumed when compressing and a higher number compresses data into a smaller size. It is possible for a server to force the use of compression, regardless of the client's configuration. This can be achieved by setting the **BindOptions.forceCompression** parameter, in which case both the server's **compressionType** and **compressionLevel** will be used. However, this does not apply to the **ConnectionTypes.WEBSOCKET** connection type.

Though compression may be enabled on a connection, it is still possible for individual buffers to be uncompressed. For efficiency, the Transport uses a default compression threshold of thirty bytes. Any message larger than the threshold will be compressed. This threshold can be changed via the **Channel.ioctl** method (refer to Section 10.14). If a message is larger than the compression threshold, it is still possible to be uncompressed by calling **Channel.write** with **WriteArgs.flags** set to **WriteArgs.flagWriteFlags.DO_NOT_COMPRESS**. For more information, refer to Section 10.9.

10.5 Channel Initialization

After a **Channel** is returned from the client's **Transport.connect** or server's **Server.accept** call, the channel may need to continue the initialization process.

NOTE: For both client and server channels, to complete the channel initialization process, more than one call to **Channel.init** might be required.

Additional initialization is required as long as the **Channel.state** is **ChannelState.INITIALIZING**.

- If using a non-blocking I/O, this is the typical state from which a **Channel** starts and multiple initialization calls might be needed to transition to active.
- If using a blocking I/O, when successful, **Transport.connect** and **Server.accept** return a completely initialized channel in an active state.

Internally, the initialization process involves several actions. The initialization includes any necessary Enterprise Transport API connection handshake exchanges, including any HTTP or HTTPS negotiation. Compression, ping timeout, and versioning related negotiations also take place during the initialization process. This process involves exchanging several messages across the connection, and once all message exchanges have completed the **Channel.state** will transition.

- If the connection is accepted (i.e., all negotiations were successful), the **Channel.state** will become **ChannelState.ACTIVE**.
- If the connection is rejected (i.e., due to either failed negotiation or a **server** rejection of the connection by setting **AcceptOptions.nakMount** to **true**), the **Channel.state** will become **ChannelState.CLOSED**, and the application should close the channel to clean up any associated resources.

10.5.1 Channel.init Method

METHOD	DESCRIPTION
Channel.init	Continues initialization of a Channel. This channel could originate from Transport.connect or Server.accept. This method exchanges various messages to perform necessary Enterprise Transport API negotiations and handshakes to complete channel initialization. If using blocking I/O, this method is typically not used because Transport.connect and Server.accept return active channels. Requires the use of the InProgInfo object, refer to Section 10.5.2. The Channel can be used for all additional transport functionality (e.g. reading, writing) after the state transitions to ChannelState.ACTIVE. If a connection is rejected or initialization fails, the state transitions to ChannelState.CLOSED, and the application should close the channel to clean up any associated resources. The return values are described in Section 10.5.4.

Table 40: Channel.init Method

10.5.2 InProgInfo Object

Use the **InProgInfo** object with the **Channel.init** method to initialize a channel.

In certain circumstances, the initialization process might need to create new or additional underlying connections. If this occurs, the application must unregister the previous **SelectableChannel** and register the new **SelectableChannel** with the I/O notification mechanism in use with associated information being conveyed by **InProgInfo** and **InProgFlags**.

METHOD	DESCRIPTION
flags	Combination of bit values to indicate special behaviors and presence of optional InProgInfo content. flags uses the following enumeration values: NONE: Indicates that channel initialization is still in progress and subsequent calls to Channel.init are needed for completion. The call did not change the SelectableChannel. SCKT_CHNL_CHANGE: Indicates that the call changed the SelectableChannel. The previous SelectableChannel is now stored in InProgInfo.oldSelectableChannel so it can be unregistered with the I/O notification mechanism. The new SelectableChannel is stored in InProgInfo.newSelectableChannel so it can be registered with the I/O notification mechanism. However, channel initialization is still in progress and subsequent calls to Channel.init are needed to complete it.
oldSelectableChannel	Populated if flags indicate that Channel.init needs to perform a socket change. If this occurs, the oldSelectableChannel contains the java.nio.channels.SelectableChannel associated with the previous connection so the application can unregister this with the I/O notification mechanism.
newSelectableChannel	Populated if flags indicate that Channel.init needs to perform a socket change. If this occurs, the newSelectableChannel contains the java.nio.channels.SelectableChannel associated with the new connection so the application can register this with the I/O notification mechanism.

Table 41: InProgInfo Methods

10.5.3 Calling Channel.init

Typically, calls to **Channel.init** are driven by I/O on the connection, however this can also be accomplished by using a timer to periodically call the method or looping on a call until the channel transitions to active or a failure occurs. Other than any overhead associated with the method call, there is no harm in calling **Channel.init** more frequently than required. If work is not required, the method returns, indicating that the connection is still in progress.

If using I/O, a client application should register the **Channel** with a **Selector** by calling **Channel.SelectableChannel.register** method with a **Selector** and **SelectionKey** of **OP_READ**, **OP_WRITE**, and **OP_CONNECT**. After the user calls the **Selector.select** method and the **Channel** is ready for writing (or ready to complete its connection sequence), the **Channel.init** method is called (this sends the initial connection handshake message). When the **Channel** has data to read, **init** is called - this typically reads the next portion of the handshake. This process continues until the connection is active.

A server application would typically register the **Server** with a **Selector** by calling the **Server.SelectableChannel.register** method with a **Selector** and **SelectionKey** of **OP_ACCEPT**. After the user calls the **Selector.select** method, and the **Server** is ready to accept a new connection, and the **Server.accept** method should be called. **accept** returns a **Channel**. Register the **Channel** with a **Selector** and **SelectionKey** of **OP_READ**. After the user calls the **Selector.select** method and the **Channel** has data to read, the **Channel.init** method is called (this typically reads the initial portion of the handshake and will send out any necessary response). This process continues until the connection is active.

10.5.4 Channel.init Return Codes

The following table defines the return codes that can occur when using **Channel.init**.

RETURN CODE	DESCRIPTION
TransportReturnCodes.SUCCESS	Indicates the initialization process completed successfully. The Channel.state should be ChannelState.ACTIVE .
TransportReturnCodes.FAILURE	Indicates that initialization has failed and cannot progress. The Channel.state should be ChannelState.CLOSED , and the application should close the channel to clean up associated resources. For more details, refer to the Error content.
TransportReturnCodes.CHAN_INIT_IN_PROGRESS	Indicates that initialization is still in progress. Check InProgInfo.flags to determine whether the SelectableChannel changed. The Channel.state should be ChannelState.INITIALIZING .
TransportReturnCodes.CHAN_INIT_REJECTED	Indicates the connection was rejected. For more details, refer to the Error content.
TransportReturnCodes.INIT_NOT_INITIALIZED	Indicates that the Transport is not initialized. For more details, refer to the Error content. For information on initializing, refer to Section 10.2.

Table 42: **Channel.init** Return Codes

10.5.5 Channel.init Example

The example below shows general use of **Channel.init**. Use of I/O notification is assumed, and the example assumes that the code is being executed due to some I/O notification.

```
/* Channel.init() is typically called based on activity on the selector, though a timer or looping
   can be used - the Channel.init() method should continue to be called until the connection becomes
   active, at which point reading and writing can begin. */
InProgInfo inProgInfo = TransportFactory.createInProgInfo();
if (chnl.state() == ChannelState.INITIALIZING)
{
    if ((retCode = chnl.init(inProgInfo, error)) < TransportReturnCodes.SUCCESS)
    {
        System.out.printf("Error (%d) (errno: %d) encountered with init. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
        /* The application should close the channel to clean up any associated resources. */
    }
    else
```

```

{
    /* Handle return code appropriately */
    switch (retCode)
    {
        case TransportReturnCodes.CHAN_INIT_IN_PROGRESS:
            /* Initialization is still in progress, check the InProgInfo for additional information */
            if (inProgInfo.flags() == InProgFlags.SCKT_CHNL_CHANGE)
            {
                System.out.println("\nSession In Progress - New Channel: " + chnl.SelectableChannel() +
                    " Old Channel: " + inProgInfo.SelectableChannel());
                /* cancel old channel read select */
                try
                {
                    SelectionKey key = inProgInfo.SelectableChannel().keyFor(selector);
                    key.cancel();
                }
                catch (Exception e) {} // old channel may be null so ignore
                /* add new channel read select */
                try
                {
                    chnl.SelectableChannel().register(selector, SelectionKey.OP_READ |
                        SelectionKey.OP_WRITE, chnl);
                }
                catch (Exception e)
                {
                    System.out.println("register select Exception: " + e.getMessage());
                }
            }
        else
        {
            System.out.println("\nChannel " + chnl.SelectableChannel() + " In Progress...");
        }
        break;
        case TransportReturnCodes.SUCCESS:
            System.out.printf("Channel on port %d is now active - reading and writing can begin.\n",
                chnl.SelectableChannel().socket().getLocalPort());
            break;
        default:
            System.out.println("\nBad return value portno=" +
                chnl.SelectableChannel().socket().getLocalPort() + "<" + error.text() + ">");
            /* Likely unrecoverable, connection should be closed */
            break;
    }
}
}

```

Code Example 9: Channel Initialization Process Using Channel.init

10.6 Reading Data

When a client or server **Channel.state** is **ChannelState.ACTIVE**, an application can receive data from the connection. The arrival of this data is often announced by the I/O notification mechanism with which the **Channel.SelectableChannel** is registered. The Transport reads data from the network as a byte stream, after which it determines **TransportBuffer** boundaries and returns each buffer one by one. The **numInputBuffers** connect or bind option controls the maximum length of the byte stream that the transport can internally process with each network read.

NOTE: When a **TransportBuffer** is returned from **Channel.read**, the contents are only valid until the next call to **Channel.read**.

To reduce potentially unnecessary copies, returned information simply points into the internal **Channel** input buffer. If the application requires the contents of the buffer beyond the next **Channel.read** call, the application can copy the contents of the buffer and allow the user to control the duration of the life cycle of the memory.

If the connection uses compression, the **Channel.read** method will perform any necessary decompression prior to returning information to the application. For available compression types, refer to Section 10.4.3.

It is possible for **Channel.read** to succeed and return a NULL buffer. When this occurs, it indicates that a portion of a fragmented buffer has been received. The Transport Package internally reassembles all parts of the fragmented buffer and after processing the last fragment, returns the entire buffer to the user through **Channel.read**.

If a packed buffer is received, each call to **Channel.read** returns an individual message (i.e., portion of contents) from the packed buffer. Every subsequent call to **Channel.read** continues to return portions of the packed buffer until the buffer is emptied. Message packing is transparent to the application that receives a packed buffer. For more information about packing, refer to Section 10.11.

If a packed buffer is received and the application leverages WebSocket transport with JSON then **Channel.read** will return only one message with all the sending packed messages as items of JSON array.

10.6.1 Channel.read Method

METHOD	DESCRIPTION
Channel.read	Provides the user with data received from the connection. This method expects the Channel to be in the active state. When data is available, a TransportBuffer referring to the information is returned, which is valid until the next call to Channel.read. If a blocking I/O is used, the Channel.read method will not return until there is information to return or an error has occurred. A ReadArgs parameter passed into the function is used to convey return code information as well as communicate whether there is additional information to read. An I/O notification mechanism may not inform the user of this additional information as it has already been read from the socket and is contained in the Channel input buffer. ReadArgs also conveys the number of bytes and uncompressed bytes read. The ReadArgs.readRetVal method is used to get the return code. An Error parameter passed into the method is used to convey error information if the ReadArgs.readRetVal value indicates an error. Return values are described in Section 10.6.3.

Table 43: Channel.read Method

10.6.2 ReadFlags Values

FLAG VALUE	DESCRIPTION
READ_NO_FLAGS	Channel data does not have associated read flags.
READ_NODE_ID	Channel data includes a valid node ID.
READ_SEQNUM	Channel data includes a sequence number.
READ_INSTANCE_ID	The message includes an instance ID.
READ_RETRANSMIT	Channel data is a retransmission of previous content.

Table 44: ReadFlags Values

10.6.3 Channel.read Return Codes

The following table defines return codes that can occur when using `Channel.read`.

RETURN CODE	BUFFER CONTENTS	DESCRIPTION
TransportReturnCodes.SUCCESS	Populated if the full buffer is available, NULL otherwise. The buffer's length indicates the number of bytes to which the data refers.	Indicates that the <code>Channel.read</code> call was successful and there are no remaining bytes in the input buffer. The I/O notification mechanism will notify the user when additional information arrives. The ping timer should be updated, refer to Section 10.12.
Any positive value > 0	Populated if full buffer is available, NULL otherwise. The buffer's length indicates the number of bytes to which the data refers.	Indicates that the <code>Channel.read</code> call was successful and there are remaining bytes in the input buffer. The I/O notification mechanism will not notify the user of these bytes. The <code>Channel.read</code> method should be called again to ensure that the remaining bytes are processed. The ping timer should be updated (for details, refer to Section 10.12). NOTE: If there are additional bytes to process, you should call <code>Channel.read</code> again. Because the bytes are already contained in the transport input buffer, an I/O notification mechanism will not alert the user of their presence.
TransportReturnCodes.READ_WOULD_BLOCK	NULL	Indicates that the <code>Channel.read</code> call has nothing to return to the user.
TransportReturnCodes.READ_PING	NULL	Indicates that a heartbeat message was received. The ping timer should be updated (for details, refer to Section 10.12).
TransportReturnCodes.FAILURE	NULL	Indicates a failure condition, often that the connection is no longer available. The <code>Channel</code> should be closed (for details, refer to Section 10.13). For more details, refer to Error content.

Table 45: Channel.read Return Codes

RETURN CODE	BUFFER CONTENTS	DESCRIPTION
TransportReturnCodes.PACKET_GAP_DETECTED	NULL	Indicates that a packet gap was detected in the inbound transport content. This may be recoverable above the transport layer, so the Channel is left in a connected state. If needed, an application can configure the transport to disconnect whenever this occurs by using the disconnectOnGaps option. For details on this option, refer to Section 10.3.2.5.
TransportReturnCodes.SLOW_READER	NULL	Indicates that the reader is not keeping up with the data rate and a packet gap was detected in the inbound transport content. This may be recoverable above the transport layer, so the Channel is left in a connected state. If needed, an application can configure the transport to disconnect whenever this occurs by using the disconnectOnGaps option. For details on this option, refer to Section 10.3.2.5.
TransportReturnCodes.CONGESTION_DETECTED	NULL	Indicates network congestion and that a gap was detected in the inbound transport content. This may be recoverable above the transport layer, so the Channel is left in a connected state. If needed, an application can configure the transport to disconnect whenever this occurs by using the disconnectOnGaps option. For details on this option, refer to Section 10.3.2.5.
TransportReturnCodes.READ_FD_CHANGE	NULL	Indicates that the connections SelectableChannel has changed. This can occur as a result of internal connection keep-alive mechanisms. The previous SelectableChannel is stored in the Channel.oldSelectableChannel so it can be removed from the I/O notification mechanism. The Channel.newSelectableChannel contains the new file descriptor, which should be registered with the I/O notification mechanism.
TransportReturnCodes.READ_IN_PROGRESS	NULL	Indicates that a Channel.read call on the Channel is already in progress. This can be due to another thread performing the same operation.
TransportReturnCodes.INIT_NOT_INITIALIZED	NULL	Indicates that the Transport has not been initialized. See the Error content for more details. For information on initializing, refer to Section 10.2.

Table 45: Channel.read Return Codes (Continued)

10.6.4 Channel.read Example

The following example shows typical use of **Channel.read** and assumes use of an I/O notification mechanism. This code would be similar for client or server based **Channel** structures.

```

/* Channel.read() use, be sure to keep track of the return values from read so data is not
   stranded in the input buffer */
ReadArgs readArgs = TransportFactory.createReadArgs();
TransportBuffer buffer = null;

if ((buffer = chnl.read(readArgs, error)) != null)
{
    /* if a buffer is returned, we have data to process and code is success */
    /* Process data and update ping monitor (Section 9.8) since data was received */

    /* Process data and update ping monitor (Section 9.8) since data was received */
    if (readArgs.readRetVal() > TransportReturnCodes.SUCCESS)
    {
        /* There is more data to read and process and I/O notification may not trigger for it */
        /* Either schedule another call to read or loop on read until */
        /* retCode == TransportReturnCodes.SUCCESS and there is no data left in internal input buffer */
    }
}
else
{
    /* Handle return codes appropriately, not all return values are failure conditions */
    int retCode = readArgs.readRetVal();
    switch (retCode)
    {
        case TransportReturnCodes.READ_PING:
            /* Update ping monitor (for details, refer to Section 10.12) */
            break;
        case TransportReturnCodes.READ_FD_CHANGE:
        {
            System.out.println("\nRead() Channel Change - Old Channel: " + chnl.oldSelectableChannel()
+
                " New Channel: " + chnl.SelectableChannel());
            /* File descriptor changed, typically due to tunneling keep-alive */
            /* Unregister old socketId and register new socketId */
            try
            {
                SelectionKey key = chnl.SelectableChannel().keyFor(selector);
                key.cancel();
            }
            catch (Exception e) {} // old channel may be null so ignore
            /* Up to application whether to register with write set - depends on need for write
            notification */
            try
            {
                chnl.SelectableChannel().register(selector, SelectionKey.OP_READ |
                    SelectionKey.OP_WRITE, chnl);
            }
        }
    }
}

```

```

    }
    catch (Exception e)
    {
        System.out.println("\nregister select Exception: " + e.getMessage());
    }
}
break;
case TransportReturnCodes.READ_WOULD_BLOCK: /* Nothing to read */
case TransportReturnCodes.READ_IN_PROGRESS: /* Reading from multiple threads: this is
        dangerous*/
    /* Handle as application sees fit, output warning, etc */
    break;
case TransportReturnCodes.INIT_NOT_INITIALIZED:
case TransportReturnCodes.FAILURE:
    System.out.printf("Error (%d) (errno: %d) encountered with read. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
    /* Connection should be closed */
    break;
default:
    System.out.printf("Unexpected return code (%d) encountered!", retCode);
    /* Likely unrecoverable, connection should be closed */
}
}
}

```

Code Example 10: Receiving Data Using Channel.read

10.7 Writing Data: Overview

When a client or server `Channel.state` is `ChannelState.ACTIVE`, it is possible for an application to write data to the connection. Writing involves a multi-step process. Because the Transport provides efficient buffer management, the user must obtain a `TransportBuffer` from the Transport buffer pool (refer to Section 10.8).

After a buffer is acquired, the user can populate the `TransportBuffer` directly or use the Enterprise Transport API to encode.

At this point, the user can choose to pack additional information into the same buffer (refer to Section 10.11) or add the buffer to the transports outbound queue (refer to Section 10.9). If queued information cannot be passed to the network, a function is provided to allow the application to continue attempts to flush data to the connection (refer to Section 10.10.2). An I/O notification mechanism can be used to help with determining when the network is able to accept additional bytes for writing. The Transport can continue to queue data, even if the network is unable to write. The following figure depicts this process and the following sections describe the functionality used to write information to the connection.

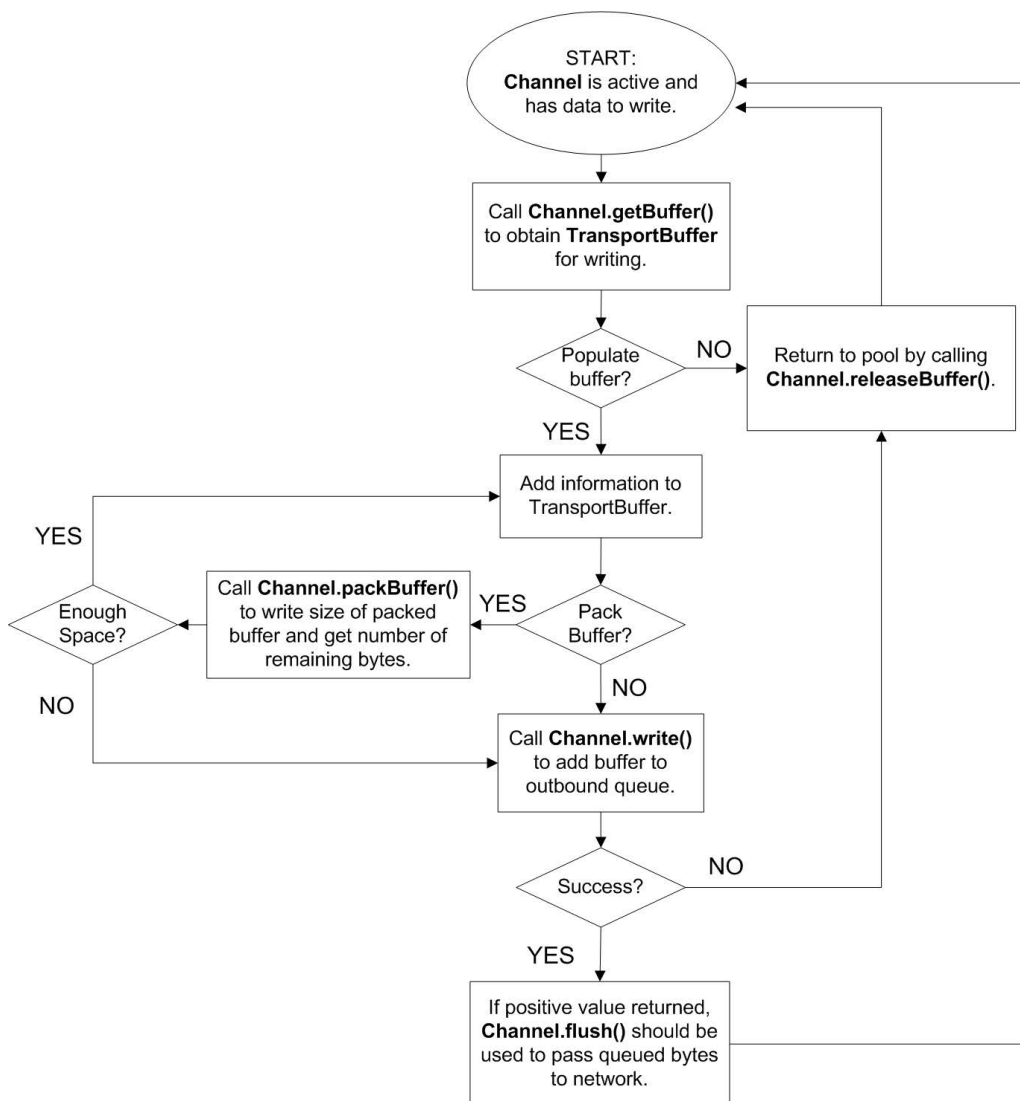


Figure 34. Enterprise Transport API Writing Flow Chart

10.8 Writing Data: Obtaining a Buffer

To write information, the user must obtain a **TransportBuffer** from the Transport buffer pool. This buffer can originate from the guaranteed output buffer pool associated with the **Channel** or the shared buffer pool associated with the **server**. A **TransportBuffer** is backed by a Java **ByteBuffer**. After acquiring a buffer, the user can populate the **TransportBuffer** directly by using the **ByteBuffer** reference from the **TransportBuffer.data** method, or by using the Enterprise Transport API to encode data (refer to 9, Encoding and Decoding Conventions). If the buffer is not used or the **Channel.write** method call fails, the buffer must be released back into the pool, using **Channel.releaseBuffer**, to ensure proper reuse and cleanup. If the buffer is successfully passed to **Channel.write**, when flushed to the network the buffer will be returned to the correct pool by the transport.

The number of buffers made available to an **Channel** is configurable through **ConnectOptions** or **BindOptions**. When connecting, the **guaranteedOutputBuffers** setting controls the number of available buffers. When connections are accepted by a **server**, the **maxOutputBuffers** parameter controls the number of available buffers per connection. This value is the sum of the number of **guaranteedOutputBuffers** and any available shared pool buffers. For more information about available **Transport.connect** and **Transport.bind** methods, refer to Table 25 and Table 36.

10.8.1 Transport Buffer Management Channel Methods

METHOD NAME	DESCRIPTION
Channel.getBuffer	Obtains a TransportBuffer of the requested size from the guaranteed or shared buffer pool. After populating the buffer, the length method can be used to get the number of bytes encoded. If the requested size is larger than the maxFragmentSize, the transport will create and return the buffer to the user. When written, this buffer will be fragmented by the Channel.write method (refer to Section 10.9). Because of some additional book keeping required when packing, the application must specify whether a buffer should be 'packable' when calling Channel.getBuffer. For more information on packing, refer to Section 10.11. For performance purposes, an application is not permitted to request a buffer larger than maxFragmentSize and have the buffer be 'packable.' If the buffer is not used or the Channel.write call fails, the buffer must be returned to the pool using Channel.releaseBuffer. If the Channel.write call is successful, the buffer will be returned to the correct pool by the transport. An Error parameter passed into the method conveys error information if a buffer request cannot be satisfied. Return values are described in Table 48. NOTE: For shared memory connection types (ConnectionTypes.UNIDIR_SHMEM) only one buffer can be obtained at a time. The application must release or write the buffer it has before the application can obtain another buffer.
Channel.releaseBuffer	Releases a TransportBuffer back to the correct pool. This should only be called with buffers that originate from Channel.getBuffer and are not successfully passed to Channel.write .
Channel.bufferUsage	Returns the number of buffers currently in use by the Channel , this includes buffers that the application holds and buffers internally queued and waiting to be flushed to the connection.

Table 46: Buffer Management Channel Methods

10.8.2 Transport Buffer Management Server Method

METHOD	DESCRIPTION
bufferUsage	Returns the number of shared pool buffers currently in use by all channels connected to the Server , this includes shared pool buffers that the application holds and shared pool buffers internally queued and waiting to be flushed.

Table 47: Buffer Management Server Methods

10.8.3 Channel.getBuffer Return Values

The following table defines **TransportReturnCodes** and **Error.errorId** values that can occur while using **Channel.getBuffer**.

RETURN CODE	DESCRIPTION
Valid buffer returned Success Case: TransportReturnCodes.SUCCESS	A TransportBuffer is returned to the user. Transport Buffer.data refers to the underlying ByteBuffer.
NULL buffer returned Error Code: TransportReturnCodes.NO_BUFFERS	NULL is returned to the user. This value indicates that there are no buffers available to the user. See Error content for more details. This typically occurs because all available buffers are queued and pending flushing to the connection. The application can use Channel.flush to attempt releasing buffers back to the pool (refer to Section 10.10.2). Additionally, the Channel.ioctl method can be used to increase the number of guaranteedOutputBuffers (refer to Section 10.14).
NULL buffer returned Error Code: TransportReturnCodes.FAILURE	NULL is returned to the user. This value indicates that some type of general failure has occurred. The Channel should be closed, refer to Section 10.13. See Error content for more details.
NULL buffer returned Error Code: TransportReturnCodes.INIT_NOT_INITIALIZED	Indicates that the Transport has not been initialized. See the Error content for more details. For information on initializing, refer to Section 10.2.

Table 48: Channel.getBuffer Return Codes

10.9 Writing Data to a Buffer

After a **TransportBuffer** is obtained from **Channel.getBuffer** and populated with the user's data, the buffer can be passed to the **Channel.write** method. Though the name seems to imply it, this method may not write the contents of the buffer to the connection. By queuing, the Transport can attempt to use the network layer more efficiently by combining multiple buffers into a single socket write operation. Additionally, queuing allows the application to continue to 'write' data, even while the network has no available space in the output buffer. If **Channel.write** does not pass all data to the socket, unwritten data will remain in the outbound queue for future writing. If an error occurs, any **TransportBuffer** that has not been successfully passed to **Channel.write** should be released to the pool using **Channel.releaseBuffer**. The following table describes the **Channel.write** method as well as some additional parameters associated with it.

The example in Section 10.9.6 demonstrates the use of **Channel.getBuffer** and **Channel.releaseBuffer**.

10.9.1 Channel.write Method

METHOD	DESCRIPTION
write	Performs any writing or queuing of data. This method expects the Channel to be in the active state and the buffer to be properly populated, where length reflects the actual number of bytes used. If blocking I/O is used, the write method will not return until data was written to the connection or an error has occurred. The WriteArgs parameter passed into the method specifies the WriteFlags, WritePriorities, and conveys the number of bytes written and also uncompressed bytes written. WriteArgs allows for several modifications to be specified for this call. For more information, refer to Section 10.9.2. The Transport supports writing data at different priority levels (for more details, refer to Section 10.10.1). - The WriteArgs.uncompressedBytesWritten method returns the number of bytes to be written, including any transport header overhead but not taking into account any compression. - The WriteArgs.bytesWritten method returns the number of bytes to be written, including any transport header overhead and taking into account any compression. - The WriteArgs.seqNum method returns the message's sequence number. If compression is disabled, uncompressedBytesWritten and bytesWritten should match. The number of bytes saved through the compression process can be calculated by (bytesWritten - uncompressedBytesWritten). Return values are described in Section 10.9.5. NOTE: Before passing a buffer to Channel.write, it is required that the application set length to the number of bytes actually used. This ensures that only the required bytes are written to the network.

Table 49: Channel.write Function

10.9.2 WriteFlags Values

WRITEFLAG	DESCRIPTION
NO_FLAGS	No modification will be performed to this Channel.write operation.
DO_NOT_COMPRESS	Though the connection might have compression enabled, this flag value indicates that this message will not be compressed. This flag value applies only to the contents of the TransportBuffer passed in with this Channel.write call.
DIRECT_SOCKET_WRITE	When set, the Channel.write method will attempt to pass the contents of the TransportBuffer directly to the socket write operation, bypassing any internal transport queuing. If any information is currently queued, this buffer will also be queued and the Channel.flush method will be invoked to ensure proper ordering of outbound data. Use of this modification will result in a higher CPU writing cost however it might decrease latency when internal queues are empty. This can be useful for writing at low data rates or when the return codes from Channel.write and Channel.flush indicate that data is not queued.
WRITE_SEQNUM	Indicates that the writer wants to attach a sequence number to this message
WRITE_RETRANSMIT	Indicates that this message is a retransmission of previous content and requires a user-supplied sequence number to indicate which packet is being retransmitted.

Table 50: writeFlags

10.9.3 Compression

The **Channel.write** method performs all necessary compression associated with the connection. Because of information order changes, compression can only be applied to a single priority level. If writing data using different priorities, the first priority level used will leverage compression and all other priority levels will be sent uncompressed. For available compression types, refer to Section 10.4.3.

10.9.4 Fragmentation

In addition to compression, the **Channel.write** method performs any necessary fragmentation of large buffers. This fragmentation process subdivides one large buffer into smaller **maxFragmentSize** portions, where each part is placed into a buffer acquired from the pool associated with the **Channel**. If the fragmentation cannot fully complete, often due to a shortage of pool buffers, this is indicated by the **TransportReturnCodes.WRITE_CALL_AGAIN** code. In this situation, the application should use **Channel.flush** to write queued buffers to the connection - this will release buffers back to the pool. When additional pool buffers are available, the application can call **Channel.write** with the same buffer to continue the fragmentation process from where it left off. The Transport keeps track of necessary information to identify and track individual fragmented messages. This allows an application to write unrelated messages between portions of a fragmented buffer as well as writing multiple fragmented messages that may be interleaved.

Currently, shared memory (**ConnectionTypes.UNIDIR_SHMEM**) connections do not support fragmentation.

NOTE: In the event that the connection is unable to accept additional bytes to write, the Transport queues on the user's behalf. The application can attempt to pass queued data to the network by using the **Channel.write** method.

10.9.5 Channel.write Return Codes

The following table lists all **TransportReturnCodes** that can occur when using the **Channel.write** method.

TRANSPORT RETURN CODE	DESCRIPTION
TransportReturnCodes.SUCCESS	Indicates that the Channel.write method was successful and additional bytes have not been internally queued. The Channel.flush method does not need to be called. The application should not release the TransportBuffer ; the Enterprise Transport API will release it.
Any positive value > 0	Indicates that the Channel.write method has succeeded and there is information internally queued by the transport. To pass internally queued information to the connection, the Channel.flush method must be called. This information can be queued because there is not sufficient space in the connections output buffer. An I/O notification mechanism can be used to indicate when the Channel has write availability. The application should not release the TransportBuffer ; the Enterprise Transport API will release it.
TransportReturnCodes.WRITE_FLUSH_FAILED	Indicates that the Channel.write method has succeeded, however an internal attempt to flush data to the socket has failed - the channel's state should be inspected. This might not be a failure condition and can occur if there is no available socket output buffer space. If the flush failure is unrecoverable, the Channel.state will transition to ChannelState.CLOSED . If the connection closes, Error information will be populated. The application should not release the TransportBuffer ; the Enterprise Transport API will release it.

Table 51: Channel.write TransportReturnCodes

TRANSPORT RETURN CODE	DESCRIPTION
TransportReturnCodes.WRITE_CALL_AGA IN	Indicates that a large buffer could not be fully fragmented with this Channel.write call. This is typically due to all pool buffers being unavailable. An application can use Channel.flush to free up pool buffers or use Channel.ioctl to increase the number of available pool buffers. After pool buffers become available again, the same buffer should be used to call Channel.write an additional time (the same priority level must be used to ensure fragments are ordered properly). This will continue the fragmentation process from where it left off. If the application does not subsequently pass the TransportBuffer to Channel.write, the buffer should be released by calling Channel.releaseBuffer.
TransportReturnCodes.FAILURE	Indicates that a general write failure has occurred. The Channel should be closed (refer to Section 10.13). For more details, refer to any Error content. The application should release the TransportBuffer by calling Channel.releaseBuffer.
TransportReturnCodes.BUFFER_TOO_S M ALL	Indicates that either the buffer has been corrupted, possibly by exceeding the allowable length, or it is not a valid pool buffer. For more details, refer to any Error content. If this TransportBuffer was obtained from Channel.getBuffer, the application should release it by calling Channel.releaseBuffer.
TransportReturnCodes.INIT_NOT_INITI AL ZED	Indicates that the Transport has not been initialized. - For more details, refer to any Error content. - For information on initializing, refer to Section 10.2. The application's attempt to call Channel.getBuffer should have failed for the same reason, so a TransportBuffer should not be present.

Table 51: Channel.write TransportReturnCodes (Continued)

10.9.6 Channel.getBuffer and Channel.write Example

The following example shows typical use of **Channel.getBuffer** and **Channel.write**. This code would be similar for client or server based **Channels**.

```
/* Channel.getBuffer() and Channel.write() use, be sure to keep track of the return values from write so
   data is not stranded in the output buffer - Channel.flush() may be required to continue attempting to
   pass data to the connection */
TransportBuffer buffer = null;
EncodeIterator encIter = CodecFactory.createEncodeIterator();
RequestMsg msg = (RequestMsg)CodecFactory.createMsg();
WriteArgs writeArgs = TransportFactory.createWriteArgs();

/* Ask for a 500 byte non-packable buffer to write into */
if ((buffer = chnl.getBuffer(500, false, error)) != null)
{
    /* if a buffer is returned, we can populate and write, encode a Msg into the buffer */
    /* set the buffer and version on an EncodeIterator */
    encIter.clear();
    encIter.setBufferAndRWFVersion(buffer, chnl.majorVersion(), chnl.minorVersion());
    /* populate message and encode it - for message encoding information, refer to Section 12.2.9.1 */
    retCode = msg.encode(encIter);

    /* Now write the data - keep track of return code */
    /* this example writes buffer as high priority and no write modification flags */
    writeArgs.priority(WritePriorities.HIGH);
    retCode = chnl.write(buffer, writeArgs, error);

    if (retCode > TransportReturnCodes.SUCCESS)
    {
        /* The write was successful and there is more data queued in the Transport. The flush method
           (discussed in Section 10.10.2) should be used to continue attempting to flush data to the
           connection. ETA will release buffer.*/
    }
    else
    {
        /* Handle return codes appropriately, not all return values are failure conditions */
        switch(retCode)
        {
            case TransportReturnCodes.SUCCESS:
                /* Successful write and all data has been passed to the connection */
                /* Continue with next operations. ETA will release buffer.*/
                break;
            case TransportReturnCodes.WRITE_CALL_AGAIN:
                /* Large buffer is being split by transport, but out of output buffers. Schedule a */
                /* call to flush (refer to Section 10.10.2) and then call the write method again with */
                /* this same exact buffer to continue the fragmentation process. Only release the */
                /* buffer if not passing it to write again. */
                break;
            case TransportReturnCodes.WRITE_FLUSH_FAILED:
```

```

    /* The write was successful, but an attempt to flush failed. ETA will release the */
    /* buffer. Must check channel state to determine if this is unrecoverable or not */
    if (chnl.state() == ChannelState.CLOSED)
    {
        System.out.printf("Error (%d) (errno: %d) encountered with write. Error Text:
            %s\n", error.errorId(), error.sysError(), error.text());
        /* Connection should be closed, return failure */
    }
    else
    {
        /* Successful write call, data is queued. The flush method (refer to */
        /* Section 10.10.2) should be used to continue attempting to flush data to the */
        /* connection. */
    }
    break;
case TransportReturnCodes.INIT_NOT_INITIALIZED:
case TransportReturnCodes.FAILURE:
    System.out.printf("Error (%d) (errno: %d) encountered with write. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
    /* Buffer must be released - return code from releaseBuffer can be checked */
    chnl.releaseBuffer(buffer, error);
    /* Connection should be closed, return failure */
    break;
default:
    System.out.printf("Unexpected return code (%d) encountered!", retCode);
    /* Likely unrecoverable, connection should be closed */
}
}
}
else
{
    /* The flush method (refer to Section 10.10.2) should be used to attempt to free buffers back to the
    */
    /* pool */
}
}

```

Code Example 11: Writing Data Using `Channel.write`, `Channel.getBuffer`, and `Channel.releaseBuffer`

10.10 Managing Outbound Queues

Because it may not be possible for the `Channel.write` method to pass all data to the underlying socket, some data may be queued by the Transport. Applications can use the `Channel.flush` method to continue attempting to pass queued data to the connection.

10.10.1 Ordering Queued Data: WritePriorities

Using the `Channel.write` method, an application can associate a priority with each `TransportBuffer`. Priority information is used to determine outbound ordering of data, and can allow for higher priority information to be written to the connection before lower priority data, even if the lower priority data was passed to `Channel.write` first. Only queued data will incur any ordering changes due to priority, and data directly written to the socket by `Channel.write` will not be impacted.

Priority ordering occurs as part of the `Channel.flush` call (refer to Section 10.10.2), where the `priorityFlushStrategy` determines how to handle each priority level. The default `priorityFlushStrategy` writes buffers in the order: High, Medium, High, Low, High, Medium. This provides a slight advantage to the medium priority level and a greater advantage to high priority data. Data order is preserved within each priority level (thus, if all buffers are written with the same priority, data is not reordered). If a particular priority level being flushed does not have content, `Channel.flush` will move to the next priority in the `priorityFlushStrategy`. The `priorityFlushStrategy` can be changed for each `Channel` by using the `Channel.ioctl` method (refer to Section 10.14).

10.10.1.1 Priority Ordering

The following figure presents an example of a possible priority write ordering. On the left, there are three queues and each queue is associated with one of the available `Channel.write` priority values. As the user calls `Channel.write` and assigns priorities to their buffers, they will be queued at the appropriate priority level. As the `Channel.flush` method is called, buffers are removed from the queues in a manner that follows the `priorityFlushStrategy`.

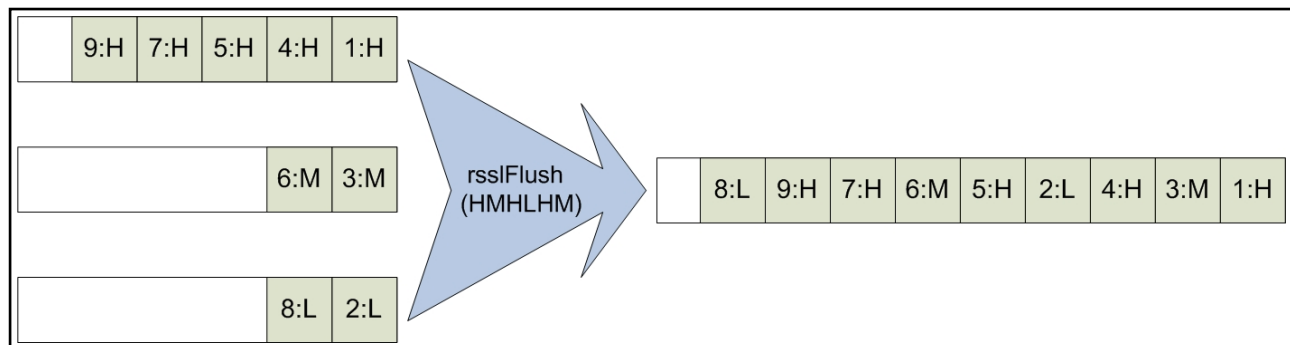


Figure 35. Channel.write Priority Scenario

On the left side of the figure there are three outbound queues, one for each priority value. As buffers enter the queues (as a result of an `Channel.write` call), they are marked with a number and the priority value associated with their queue. The number indicates the order the buffers were passed to `Channel.write`, so the buffer marked **1** was the first buffer into `Channel.write`, the buffer marked **5** was the 5th buffer into `Channel.write`. Buffers are marked **H** if they are in the high priority queue, **M** if they are in the medium priority queue, or **L** if they are in the low priority queue. Buffers leave the queue (as a result of a `Channel.flush` call) in the order specified by the `priorityFlushStrategy`, which by default is **HMHLM**. In Figure 35, the queue on the right side represents the order in which buffers are written to the network and the order that they will be returned when `Channel.read` is called. The buffers will still be marked with their **number:priority** information so it is easy to see how data is reordered by any priority writing.

Notice that though data was reordered between various priorities, individual priority levels are not reordered. Thus, all buffers in the high priority are written in the order they are queued, even though some medium and low buffers are sent as well.

10.10.1.2 WritePriorities Values

PRIORITYVALUE	MEANING
HIGH	If not directly written to the socket, this TransportBuffer will be flushed at the high priority.
MEDIUM	If not directly written to the socket, this TransportBuffer will be flushed at the medium priority.
LOW	If not directly written to the socket, this TransportBuffer will be flushed at the low priority.

Table 52: WritePriorities Values

10.10.2 Channel.flush Method

If all available output space is used for a connection, data might be queued as a result. An I/O notification mechanism can be used to alert the application when output space becomes available on a connection.

NOTE: The return value from **Channel.flush** indicates whether there are any queued bytes left to pass to the connection. If this is a positive value (typical when operating system output buffers lack space), the application should continue to call **Channel.flush** until all bytes have been written.

METHOD NAME	DESCRIPTION
Channel.flush	Writes queued data to the connection. This method expects the Channel to be in the active state. If data is not queued, the Channel.flush method is not required and should return immediately. This method performs any buffer reordering that might occur due to priorities passed in on the Channel.write method. For more information about priority writing, refer to Section 10.10.1. Return values are described in Table 54.

Table 53: Channel.flush Method

10.10.3 Channel.flush Return Codes

The following table defines the return **TransportReturnCodes** that can occur when using **Channel.flush**.

RETURN CODE	DESCRIPTION
TransportReturnCodes.SUCCESS	Indicates that the Channel.flush method has succeeded and additional bytes are not internally queued. The Channel.flush method need not be called.
Any positive value > 0	Indicates that the Channel.flush method has succeeded, however data is still internally queued by the transport. The Channel.flush method must be called again. Data might still be queued because the connections output buffer does not have sufficient space. An I/O notification mechanism can indicate when the SelectableChannel has write availability.
TransportReturnCodes.FAILURE	Indicates that a general failure has occurred, often because the underlying connection is unavailable or closed. The Channel should be closed (refer to Section 10.13). For more details, refer to the Error content.
TransportReturnCodes.INIT_NOT_INITIALIZED	Indicates that the Transport is not initialized. For more details, refer to the Error content. For information on initializing, refer to Section 10.2.

Table 54: Channel.flush TransportReturnCodes

10.10.4 Channel.flush Example

The following example shows typical use of **Channel.flush**. This example assumes the use of an I/O notification mechanism. This code would be similar for client- or server-based **Channels**.

```
/* Channel.flush() use, be sure to keep track of the return values from flush so data is not stranded in
   the output buffer - flush may need to be called again to continue attempting to pass data to the
   connection */

/* Assuming this section of code was called because of a write selector notification */
if ((retCode = chnl.flush(error)) > TransportReturnCodes.SUCCESS)
{
    /* There is still data left to flush, leave our write notification enabled so we get called again,
       If everything wasn't flushed, it usually indicates that the TCP output buffer cannot accept more
       yet */
}
else
{
    switch (retCode)
    {
        case TransportReturnCodes.SUCCESS:
            /* Everything has been flushed, no data is left to send - unset write notification */
            SelectionKey key = chnl.SelectableChannel().keyFor(selector);
            try
            {
                chnl.SelectableChannel().register(selector, key.interestOps() - SelectionKey.OP_WRITE,
                    chnl);
            }
            catch (Exception e)
            {
                System.out.println("\nregister select Exception: " + e.getMessage());
            }
            break;
        case TransportReturnCodes.INIT_NOT_INITIALIZED:
        case TransportReturnCodes.FAILURE:
            System.out.printf("Error (%d) (errno: %d) encountered with flush. Error Text: %s\n",
                error.errorId(), error.sysError(), error.text());
            /* Connection should be closed, return failure */
            break;
        default:
            System.out.printf("Unexpected return code (%d) encountered!", retCode);
            /* Likely unrecoverable, connection should be closed */
    }
}
```

Code Example 12: Channel.flush Use

10.11 Packing Additional Data into a Buffer

If an application is writing many small buffers, it might be advantageous to combine the small buffers into one larger buffer. This can increase the efficiency of the transport layer by reducing overhead associated with each write operation, though it might increase latency associated with each smaller buffer.

It is up to the writing application to determine when to stop packing, and the mechanism used can vary greatly. One simple algorithm is to pack a fixed number of messages each time. A slightly more complex technique could use the returned length from **Channel.packBuffer** to determine the amount of remaining space and pack until the buffer is nearly full. Both of these mechanisms can introduce a variable amount of latency as they both depend on the rate at which data arrives (i.e., the packed buffer will not be written until enough data arrives to fill it). One method that can balance this is to use a timer to limit the amount of time a packed buffer is held. If the buffer is full prior to the timer expiring, the data is written, otherwise whenever the timer expires, whatever is in the buffer will be written (regardless of the amount of data in the buffer). This limits latency to a maximum, acceptable amount as set by the duration of the timer.

The **Channel.packBuffer** method packs multiple messages into one **TransportBuffer**.

METHOD	DESCRIPTION
Channel.packBuffer	Packs the contents of a passed-in TransportBuffer and returns the number of bytes remaining in the TransportBuffer. An application can use the length returned to determine the amount of space available to continue packing buffers. For a buffer to allow packing, it must be requested from Channel.getBuffer as 'packable' and cannot exceed the maxFragmentSize. Packing is not supported for shared memory (ConnectionTypes.UNIDIR_SHMEM) connections.

Table 55: Channel.packBuffer Method

10.11.1 Channel.packBuffer Return Values

The following table defines return and error code values that can occur when using **Channel.packBuffer**.

RETURN CODE	DESCRIPTION
0 or greater Success Case	Indicates the amount of available bytes remaining in the buffer for packing. Zero means that bytes are not available for packing.
Less than 0 Failure Case	This value indicates that some type of general failure has occurred. The Channel should be closed (refer to Section 10.13). For more details, refer to Error content.

Table 56: Channel.packBuffer Return Values

10.11.2 Example: Channel.getBuffer, Channel.packBuffer, and Channel.write

The following example shows typical use of **Channel.getBuffer**, **Channel.getBuffer**, and **Channel.write**. This code is similar for client- or server-based **Channel** structures.

```
/* Channel.getBuffer(), Channel.packBuffer() and Channel.write() use, be sure to keep track of the
   return values from write so data is not stranded in the output buffer - flush may be required to
   continue attempting to pass data to the connection */
TransportBuffer buffer = null;
EncodeIterator encIter = CodecFactory.createEncodeIterator();
RequestMsg msg = (RequestMsg)CodecFactory.createMsg();
WriteArgs writeArgs = TransportFactory.createWriteArgs();
```

```

/* Ask for a 6000 byte packable buffer to write multiple messages into */
if ((buffer = chnl.getBuffer(6000, true, error)) != null)
{
    /* if a buffer is returned, we can populate and write, encode a Msg into the buffer */

    /* set the buffer and version on an EncodeIterator */
    encIter.clear();
    encIter.setBufferAndRWFVersion(buffer, chnl.majorVersion(), chnl.minorVersion());
    /* populate message and encode it. For more details on message encoding, refer to Section 12.2.9.1 */
    retCode = msg.encode(encIter);

    /* Instead of writing, lets continue packing messages into the buffer */
    /* This will take the existing buffer and return how many bytes remain to continue encoding into */
    if ((retCode = chnl.packBuffer(buffer, error)) < TransportReturnCodes.SUCCESS)
    {
        System.out.printf("Error (%d) (errno: %d) encountered with packBuffer. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
        /* Buffer must be released - return code from releaseBuffer can be checked */
        chnl.releaseBuffer(buffer, error);
        /* Connection should be closed, return failure */
    }

    /* check retCode, if there is enough bytes remaining, continue to pack additional messages */

    /* encode an additional message */
    /* set the buffer and version on an EncodeIterator */
    encIter.setBufferAndRWFVersion(buffer, chnl.majorVersion(), chnl.minorVersion());
    /* populate message and encode it - for more details on message encoding, refer to Section 12.2.9.1 */
    retCode = msg.encode(encIter);

    /* Instead of writing, let's continue packing messages into the buffer */
    /* This will take the existing buffer and return the number of bytes available for encoding */
    if ((retCode = chnl.packBuffer(buffer, error)) < TransportReturnCodes.SUCCESS)
    {
        System.out.printf("Error (%d) (errno: %d) encountered with packBuffer. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
        /* Buffer must be released - return code from releaseBuffer can be checked */
        chnl.releaseBuffer(buffer, error);
        /* Connection should be closed, return failure */
    }

    /* Packing can continue like this until the application determines its time to stop - this can be due
       to the buffer not containing enough space for an additional message, a timer alerting that enough
       pack time has elapsed, etc */

    /* After packing is complete, write the buffer as normal */
    writeArgs.priority(WritePriorities.HIGH);
    retCode = chnl.write(buffer, writeArgs, error);
}

```

```

    /* For a full, write error-handling example, refer to the Example in Section Section 10.9.6. */
}
else
{
    /* Use the flush method (Section 10.10.2) to free buffers back to the pool */
}

```

Code Example 13: Message Packing Using `Channel.packBuffer`

10.12 Ping Management

Ping or heartbeat messages indicate the continued presence of an application. These are typically required only when no other data is exchanged. For example, there may be long periods of time that elapse between requests made from a consumer application. In this situation, the consumer sends periodic heartbeat messages to inform the providing application that it is still connected. Because the provider application is likely sending data more frequently (providing updates on any streams the consumer has requested), the provider might not need to send heartbeats (as the other data sufficiently announces its continued presence). The application is responsible for managing the sending and receiving of heartbeat messages on each connection.

10.12.1 Ping Timeout

Applications are able to configure their desired **pingTimeout** values, where the *ping timeout* is the point at which a connection is terminated due to inactivity. Heartbeat messages are typically sent every one-third of the **pingTimeout**, ensuring that heartbeats are exchanged prior to a ping timeout. This can be useful for detecting a connection loss prior to any kind of network or operating system notification.

pingTimeout values are negotiated between a connecting client application and the server application, where the server can specify a minimum allowable ping timeout (via the **minPingTimeout** option) and the direction in which heartbeats flow (via **serverToClientPings** and **clientToServerPings**). For more information on specifying these options, refer to Section 10.3.2.1 and Section 10.4.1.1. During negotiation, the lowest **pingTimeout** value is selected. Because **minPingTimeout** sets the lowest possible value, if a client's specified **pingTimeout** value is less than **minPingTimeout**, the connection uses the **minPingTimeout** as its **pingTimeout** value. After a connection transitions to the active state, the negotiated **pingTimeout** is available through the **Channel.pingTimeout**.

The Transport uses the following formula to determine the negotiated **pingTimeout** value:

```

/* Determine lesser of client or servers pingTimeout */
if (client.pingTimeout < server.pingTimeout)
    connection.pingTimeout = client.PingTimeout;
else
    connection.pingTimeout = server.pingTimeout;
/* Determine whether timeout is less than minimum allowable timeout */
if (connection.pingTimeout < server.minPingTimeout)
    connection.pingTimeout = server.minPingTimeout;

```

Code Example 14: Ping Negotiation Calculation

10.12.2 Channel.ping Function

An application typically monitors both messages and heartbeats. If bytes are flushed to the network, this is considered sufficient as a heartbeat so any timer mechanism associated with sending heartbeats can be reset. When bytes are received or **Channel.read** returns **TransportReturnCodes.READ_PING** (refer to Section 10.6), this is comparable to receiving a heartbeat so any timer mechanism associated with receiving heartbeats can be reset. If either the sending or receiving heartbeat timer mechanism reaches or surpasses the **Channel.pingTimeout** value, the connection should be closed.

The following table describes the **Channel.ping** method, used to send heartbeat messages.

METHOD	DESCRIPTION
Channel.ping	Attempts to write a heartbeat message on the connection. This method expects an active Channel. If an application calls the Channel.ping method while other bytes are queued for output, the Transport layer suppresses the heartbeat message and attempts to flush bytes to the network on the user's behalf. When using a shared memory (UNIDIR_SHMEM) connection type, pings can only be sent from server to client. Return values are described in Table 58.

Table 57: Channel.ping method

10.12.3 Channel.ping Return Values

The following table defines the **TransportReturnCodes** that can occur when using **Channel.ping**.

TRANSPORT RETURN CODE	DESCRIPTION
SUCCESS	Indicates that the Channel.ping method succeeded and additional bytes are not internally queued.
Any positive value > 0	Indicates that queued data was sent as a heartbeat but data is still internally queued by the transport. The Channel.flush method must be called to continue passing queued bytes to the connection. Data might still be queued because the connections output buffer does not have sufficient space. An I/O notification mechanism indicate when the SelectableChannel has write availability.
FAILURE	This value indicates that some type of general failure has occurred. The Channel should be closed (refer to Section 10.13). For more details, refer to the Error content.

Table 58: Channel.ping TransportReturnCodes

10.12.4 Channel.ping Example

The following example shows typical use of **Channel.ping**. This example assumes the use of some kind of timer mechanism to execute when necessary. This code would be similar for client or server based **Channels**.

```
/* Channel.ping() use - this demonstrates sending of heartbeats */
/* Additionally, an application should determine if data or pings have been received, if not application
   should determine if pingTimeout has elapsed, and if so connection should be closed */

/* First, send our ping, if there is other data queued, that will be flushed instead */
if ((retCode = chnl.ping(error)) > TransportReturnCodes.SUCCESS)
{
    /* There is still data left to flush, leave our write notification enabled so we get called again,
```

```

    If everything wasn't flushed, it usually indicates that the TCP output buffer cannot accept more yet
    */
}
else
{
    switch (retCode)
    {
        case TransportReturnCodes.SUCCESS:
            /* Ping message has been sent successfully */
            break;
        case TransportReturnCodes.INIT_NOT_INITIALIZED:
        case TransportReturnCodes.FAILURE:
            System.out.printf("Error (%d) (errno: %d) encountered with ping. Error Text: %s\n",
                error.errorId(), error.sysError(), error.text());
            /* Connection should be closed, return failure */
            break;
        default:
            System.out.printf("Unexpected return code (%d) encountered!", retCode);
            /* Likely unrecoverable, connection should be closed */
    }
}
}

```

Code Example 15: Channel.ping Use

10.13 Closing Connections

10.13.1 Functions for Closing Connections

When an error occurs on a connection or a **Channel** is being disconnected, the **Channel.close** method should be called to perform any necessary cleanup and to shutdown the underlying socket. This will release any pool-based resources back to their respective pools. If the application is holding any buffers obtained from **Channel.getBuffer**, they should be released using **Channel.releaseBuffer** prior to closing the channel.

If a server is being shut down, use the **Server.close** method to close the listening socket and perform any necessary cleanup. All currently connected **Channels** will remain open. This allows applications to continue sending and receiving data, while preventing new applications from connecting. The server has the option of calling **Channel.close** to shut down any currently connected applications.

METHOD	DESCRIPTION
Channel.close	Closes a client- or server-based Channel . This releases any pool-based resources back to their respective pools, closes the connection, and performs any additional necessary cleanup.
	NOTE: If an application is multi-threaded, all other threads that depend on the closed channel should complete their use prior to calling Channel.close .
Server.close	Closes a listening socket associated with a server . The Server.close releases any pool-based resources back to their respective pools, closes the listening socket, and performs any additional necessary cleanup. Established connections remain open, allowing for continued exchange of data. If needed, the server can use Channel.close to shutdown any remaining connections.

Table 59: Connection Closing Functionality

10.13.2 Close Connections Example

The following example shows typical use of **Channel.close** and **Server.close**.

```
/* Channel.close() */
if (chnl.close(error) < TransportReturnCodes.SUCCESS)
{
    System.out.printf("Error (%d) (errno: %d) encountered with channel close. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}

/* Server.close() */
if (srvr.close(error) < TransportReturnCodes.SUCCESS)
{
    System.out.printf("Error (%d) (errno: %d) encountered with server close. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
```

Code Example 16: Closing a Connection Using Channel.close and Server.close

10.14 Utility Methods

The Transport layer provides several additional utility methods. These methods can be used to query more detailed information for a specific connection or change certain **Channel** or **server** parameters during run-time. These methods are described in the following tables.

10.14.1 General Transport Utility Methods

METHOD NAME	DESCRIPTION
Channel.info	Allows the application to query Channel negotiated parameters and settings and retrieve all current settings. This includes maxFragmentSize and negotiated compression information as well as many other values. This populates a ChannelInfo object. For a full list of available settings, refer to Section 10.14.2.
Server.info	Allows the application to query Server related values, such as current and peak shared pool buffer usage statistics. This populates a ServerInfo object, defined in Section 10.14.5.
Channel.ioctl	Allows the application to change various settings associated with the Channel . Available IoctlCodes are defined in Section 10.14.6.
Server.ioctl	Allows the application to change various settings associated with the Server . Available IoctlCodes are defined in Section 10.14.7.
Transport.hostByName	Takes a Java String populated with a hostname, looks up and returns a Java InetSocketAddress .
Transport.userName	Queries the username associated with the owner of the current process, and returns a String .

Table 60: Transport Utility Methods

10.14.2 ChannelInfo Methods

The following table describes the values available to the user through using the **Channel.info** method. This information is returned as part of the **ChannelInfo** object.

METHOD	DESCRIPTION
clientToServerPings	Gets whether the client is expected to send heartbeat messages: - If set to true, heartbeat messages must flow from client to server. - If set to false, the client is not required to send heartbeats. LSEG Real-Time Distribution System and other LSEG components typically require this value to be set to true .
clientHostname	The host name of the connecting client. Valid only for Channels that were accepted (i.e., returned from Transport.accept) and whose ChannelState is ACTIVE .
clientIP	The IP address of the connecting client. Valid only for Channels that were accepted (i.e., returned from Transport.accept) and whose ChannelState is ACTIVE .
componentInfo	Retrieves a Java List of ComponentInfos . One ComponentInfo object will be present for each connected device that supports connected component versioning. For more detailed information on the ComponentInfo structure, refer to Section 10.14.4.
compressionThreshold	Gets the compression threshold. Messages smaller than the threshold are not compressed; messages larger than the threshold are compressed.
compressionType	Sets the type of compression to use on this connection. Refer to Section 10.4.3 for more information about supported compression types.
guaranteedOutputBuffers	The guaranteed number of buffers which this Channel can use while writing data. Each buffer can contain maxFragmentSize bytes. Guaranteed output buffers are allocated at initialization time. For more details on obtaining a buffer, refer to Section 10.8. You can configure guaranteedOutputBuffers using Channel.ioctl , as described in Section 10.14.6.
maxFragmentSize	The maximum allowed buffer size which can be written to the network. If a larger buffer is required, the Transport will internally fragment the larger buffer into smaller buffers whose size is set to maxFragmentSize . This is the largest size a user can request while still being 'packable.'
maxOutputBuffers	The maximum number of output buffers which this Channel can use. (maxOutputBuffers - guaranteedOutputBuffers) is equal to the number of shared pool buffers that this Channel can use. Shared pool buffers are only used if all guaranteedOutputBuffers are unavailable. If maxOutputBuffers is equal to the guaranteedOutputBuffers value, shared pool buffers are unavailable. You can configure maxOutputBuffers using Channel.ioctl , as described in Section 10.14.6.
multicastStats	If using a connection type of ConnectionTypes.RELIABLE_MCAST , this substructure reports information about sent and received packets, including any gap or retransmission information. For details on options used with multicastStats , refer to Section 10.14.3.
numInputBuffers	Gets the number of sequential input buffers into which the Channel reads data. This controls the maximum number of bytes that can be handled with a single network read operation on each channel. Each input buffer can contain maxFragmentSize bytes. Input buffers are allocated at initialization time.
pingTimeout	Gets the negotiated ping timeout value. Typically, the rule of thumb in handling heartbeats is to send a heartbeat every pingTimeout/3 seconds. For more details on pingTimeout , refer to Section 10.12.1.

Table 61: ChannelInfo Methods

METHOD	DESCRIPTION
port	Gets the server port number to which the consumer or non-interactive provider application connects.
priorityFlushStrategy	The current priority level order used when flushing buffers to the connection, where H = High priority, M = Medium priority, and L = Low priority. Allows for up to 32 one-byte characters to be represented. When passed to Channel.write, each buffer is associated with the priority level at which it should be written. The default priorityFlushStrategy writes buffers in the order: High, Medium, High, Low, High, Medium. This provides a slight advantage to the medium-priority level and a greater advantage to high-priority data. Data order is preserved within each priority level and if all buffers are written with the same priority, the order of data does not change. You can configure priorityFlushStrategy using Channel.ioctl, as described in Section 10.14.6.
securityProtocol	Gets the security protocol of the connection.
securityProtocolVersion	Gets the security protocol version of the connection.
serverToClientPings	Gets whether server is expected to send heartbeat messages: - If set to true, heartbeat messages must flow from server to client. - If set to false, the server is not required to send heartbeats. LSEG Real-Time Distribution System and other LSEG components typically require this value to be set to true.
sysRecvBufSize	Gets the size of the receive or input buffer associated with the underlying transport. The Transport has an additional input buffer controlled by numInputBuffers. For some connection types, you can configure sysRecvBufSize using Channel.ioctl, as described in Section 10.14.6.
sysSendBufSize	Gets the size of the send or output buffer associated with the underlying transport. The Transport has additional output buffers, controlled by maxOutputBuffers and guaranteedOutputBuffers. For some connection types, you can configure sysSendBufSize using Channel.ioctl, as described in Section 10.14.6.

Table 61: ChannelInfo Methods (Continued)

10.14.3 multicastStats Methods

METHOD	DESCRIPTION
gapsDetected	Returns a count of the number of detected packet gaps detected and reported to the application. This is a result of packet loss on the network and may indicate a more serious network problem.
mcastRcvd	The number of multicast packets received by this Channel .
mcastSent	The number of multicast packets sent by this Channel .
retransPktsRcvd	The number of retransmitted packets received by this Channel . This is populated only for reliable multicast type connections.
retransPktsSent	The number of retransmitted packets sent by this Channel . Packets are retransmitted in response to retransmission requests. If a packet cannot be retransmitted, this results in a gap occurring and indicates a network problem, which applications are notified of via Channel.read . This is populated only for reliable multicast type connections.

Table 62: multicastStats Methods

METHOD	DESCRIPTION
retransReqRcvd	This is the number of retransmission requests received by this Channel . Retransmission requests are received if another component on the network missed a packet sent by this channel and may indicate a network problem if gaps are also being detected. This is populated only for reliable multicast type connections.
retransReqSent	The number of retransmission requests sent by this Channel . Retransmission requests are sent in an attempt to recover a missed packet and may indicate a network problem if gaps are also detected. This is populated only for reliable multicast type connections.
unicastRcvd	The number of unicast UDP packets received by this Channel .
unicastSent	The number of unicast UDP packets sent by this Channel .

Table 62: **multicastStats** Methods (Continued)

10.14.4 componentInfo Method

METHOD	DESCRIPTION
componentVersion	A TransportBuffer containing an ASCII string that indicates the product version of the connected component.

Table 63: **componentInfo** Options

10.14.5 ServerInfo Methods

The following table describes values available to the user through the use of the **Server.info** method. This information is returned as part of the **ServerInfo** object.

METHOD	DESCRIPTION
Clear	Clears this object, so that it can be reused.
currentBufferUsage	The number of currently used shared pool buffers across all users connected to the server .
peakBufferUsage	The maximum achieved number of used shared pool buffers across all users connected to the server . This value can be reset through the use of Server.ioctl , as described in Section 10.14.7.

Table 64: **ServerInfo** Methods

10.14.6 Channel.ioctl ioctlCodes

The following table provides a description of the **ioctlCodes** available for use with the **Channel.ioctl** method.

OPTION ENUMERATION	DESCRIPTION
COMPRESSION_THRESHOLD	Allows a Channel to change the size (in bytes) at which buffer compression occurs, must be greater than 30 bytes. Value is an int .
HIGH_WATER_MARK	Allows a Channel to change the internal Enterprise Transport API output queue depth water mark, which has a default value of 6,144 bytes. When the Enterprise Transport API output queue exceeds this number of bytes, the Channel.write method internally attempts to flush content to the network. Value is an int .
MAX_NUM_BUFFERS	Allows a Channel to change its maxOutputBuffers setting. Value is an int .

Table 65: **Channel.ioctl** **ioctlCodes**/**OctlCodes**

OPTION ENUMERATION	DESCRIPTION
NUM_GUARANTEED_BUFFERS	Allows a Channel to change its guaranteedOutputBuffers setting. Value is an int .
PRIORITY_FLUSH_ORDER	Allows a Channel to change its priorityFlushStrategy. Value is a String, where each character is either: • H for high priority • M for medium priority • L for low priority The String should not exceed 32 entries. At least one H and one M must be present, however no L is required. If low priority flushing is not specified, the low priority queue is flushed only when other data is not available for output.
SYSTEM_READ_BUFFERS	Allows a Channel to change the TCP receive buffer size associated with the connection. Value is an int, which defaults to 64 (KB). If the value is larger than 64KB, the value needs to be specified before the socket connects to the remote peer. • For servers and SYSTEM_READ_BUFFERS larger than 64 KB, use BindOptions.sysRecvBufferSize to set the receive buffer size prior to calling Transport.bind. • For clients and SYSTEM_READ_BUFFERS larger than 64 KB, use ConnectOptions.sysRecvBufferSize to set the receive buffer size prior to calling Transport.connect.
SYSTEM_WRITE_BUFFERS	Allows a Channel to change the TCP send buffer size associated with the connection. Value is an int .

Table 65: **Channel.ioctl** **ioctlCodes** **IOCtlCodes** (Continued)

10.14.7 Server.ioctl **ioctlCodes**

The following table provides a description of the **ioctlCodes** available for use with the **Server.ioctl** method.

IOCTL CODE	DESCRIPTION
SERVER_NUM_POOL_BUFFERS	Allows a server to change its sharedPoolSize setting. Value is an int .
SERVER_PEAK_BUF_RESET	Allows a server to reset the peakBufferUsage statistic. Value is not required.

Table 66: **Server.ioctl** **ioctlCodes** **IOCtlCodes**

10.15 Encrypted and Proxy Connections

Consumer applications can establish Internet connections via HTTP proxies and can establish TLS-encrypted connections with supported protocols. Provider applications can support encrypted HTTP, TCP, and Websocket protocols. Encrypted and proxy connections are supported across all platforms.

10.15.1 Configuring HTTPS, HTTP, and Proxy Connections

An HTTP tunneling connection uses a connection type of `ConnectionTypes.HTTP`, while an HTTPS tunneling connection uses a connection type of `ConnectionTypes.ENCRYPTED`. Additional configuration is required on an HTTPS tunneling connection, which can be specified using the `TunnelingInfo` method.



WARNING! If you use an encrypted tunneling connection type, you might encounter trust issues with DigiCert certificates. JRE8 Update 91 and higher support DigiCert certificates. If you encounter problems with DigiCert certificates, upgrade to JRE8 Update 91 or higher.

A consumer application needs to configure additional parameters, in addition to `tunnelingType`. By setting the `HTTPproxy` configuration parameter to `true`, the application will be connected via a proxy. The configuration parameters `HTTPproxyHostname` and `HTTPproxyPort` specify the proxy host name and port. A client connection that leverages a connection type of `ConnectionTypes.HTTP`, `ConnectionTypes.WEBSOCKET`, `ConnectionTypes.SOCKET`, or `ConnectionTypes.ENCRYPTED` might be connecting through proxy devices as it tunnels through the Internet.

If a consumer application uses the connection configured for connection type `ConnectionTypes.ENCRYPTED`, additional configuration parameters apply. The parameters specify security settings, such as: `KeystoreType`, `KeystoreFile`, `KeystorePasswd`, `SecurityProvider`, `KeyManagerAlgorithm`, `TrustManagerAlgorithm`. The Enterprise Transport API uses the JDK `java.security` package. If the parameters `KeystoreType`, `SecurityProvider`, `KeyManagerAlgorithm`, and `TrustManagerAlgorithm` are not specified, the JDK `java.security` package provides default settings.

10.15.1.1 TunnelingInfo Methods

METHOD	DESCRIPTION
<code>tunnelingType</code>	The Tunneling type. Possible values are None , http , or encrypted . For HTTP Tunneling, <code>tunnelingType</code> has to be set to http or encrypted . For other encrypted connection types, this must be set to None . For additional configuration details, refer to Section 10.15.3 (for connection types) and Section 10.15.4 (for encrypted servers).
<code>HTTPproxy</code>	Sets whether the tunneling application goes through an HTTP proxy server.
<code>HTTPproxyHostName</code>	Configures the address or hostname of the HTTP proxy server to which the application connects. HTTPproxy has to be true .
<code>HTTPproxyPort</code>	Configures the Port Number of the HTTP proxy server to which the consumer application connects. If you configure this method, HTTPproxy must also be set to true .
<code>objectName</code>	Configures the object name for load balancing to the various ADSs as part of a hosted solution.
<code>KeyManagerAlgorithm</code>	Specifies the Java Key Management algorithm. Defaults to the property ssl.KeyManagerFactory algorithm in the JDK security properties file (java.security). By default, Oracle JDK uses SunJX509 .
<code>KeystoreType</code>	Configures the type of keystore for the certificate file. Defaults to the property keystore.type in the JDK security properties file (java.security). By default, RTSDK uses JKS .
<code>KeystoreFile</code>	Configures the keystore file that contains your own private keys and any public key certificates you received from a third party. The JDK utility keytool creates this file.
<code>KeystorePasswd</code>	Configures the password for the specified keystore file.

Table 67: TunnelingInfo Methods

METHOD	DESCRIPTION
SecurityProtocol	Specifies the cryptographic protocol to use. By default, Oracle JDK uses TLS .
SecurityProtocolVersions	Specifies the version of the configured cryptographic protocol to use. By default, assumes TLS protocol and sets "1.2" and "1.3".
SecurityProvider	Specifies the Java Cryptography Package that the provider uses. By default, Oracle JDK uses SunJSSE . RTSDK library also supports Conscrypt security provider. To choose this provider, specify "Conscrypt" as the argument to this method.
TrustManagerAlgorithm	Specifies the Java Trust Management algorithm, which defaults to the property ssl.TrustManagerFactory.algorithm in the JDK security properties file (java.security). By default, Oracle JDK uses PKIX .

Table 67: TunnelingInfo Methods (Continued)

10.15.1.2 Configuration Example

The following procedure describes how to provide the required authentication credentials to the Enterprise Transport API. The following procedure illustrates how to modify the **Consumer** example.

1. Open **Consumer.java** located in **Examples/com/refinitiv/eta/examples/consumer**.
2. For a connection type of **ConnectionTypes.ENCRYPTED**, edit the following code in the **setEncryptedConfiguration** method with the proxy server hostname and port to which you will connect:

```
options.tunnelingInfo().HTTPproxyHostName("myProxy");
options.tunnelingInfo().HTTPproxyPort(8443);
```

3. In the **setEncryptedConfiguration** method, edit the following code for the Keystore file and its password:

```
options.tunnelingInfo().KeystoreFile("myKeystore.jks");
options.tunnelingInfo().KeystorePasswd("myKeystorePasswd");
```

4. For a connection type of **ConnectionTypes.HTTP**, edit the following code in the **setHTTPconfiguration** method with the proxy server hostname and port to which you will connect.

```
options.tunnelingInfo().HTTPproxyHostName("myProxy");
options.tunnelingInfo().HTTPproxyPort(8080);
```

10.15.2 Proxy Authentication

You can configure some proxy servers to authenticate client applications before they pass through the proxy to their destination. The Enterprise Transport API supports Negotiate(Kerberos), Kerberos, NTLM, and Basic authentication schemes.

NOTE: A consumer application needing NTLM authentication should add the Apache jar files (in the load's **Libs/ApacheClient** directory) to the **CLASSPATH**.

Authentication schemes:

- Establish the type of credentials an application must provide to the proxy server.
- Define how to encode the credentials required for authentication.
- Determine the “handshake” process during which messages are exchanged between the proxy and the application during the authentication process.

This section:

- Provides an overview of the proxy authentication process.
- Details how to programmatically supply consumer applications with the user credentials required to authenticate with a proxy.
- Provides tips for troubleshooting proxy authentication failures.

10.15.2.1 The Proxy Authentication Process

If a Enterprise Transport API consumer's connection is configured to connect to a provider via a proxy server (using HTTP or Encrypted tunneling) which requires authentication, the Enterprise Transport API will automatically participate in the authentication process. The application must supply Enterprise Transport API-valid credentials (described in Section 10.15.2.2). Specifically, the Enterprise Transport API automatically parses the list of supported authentication schemes (sent by the proxy), selects the most appropriate scheme, re-connects to the proxy (if necessary), and exchanges the messages required by the selected authentication scheme.

A typical proxy authentication adheres to the following process:

- The consumer application uses either HTTP or HTTPS protocol to connect to a provider (e.g., an ADS, or a Enterprise Transport API C provider application, not shown) via a proxy server.
- Because authentication is enabled on the proxy server, the proxy server sends an HTTP response to the application indicating authentication is required. This response contains the HTTP error code# 407, and includes a list of authentication schemes enabled on the proxy.

The initial response sent from the proxy server may also indicate that the connection between it and the consumer application will be closed.
- If the Enterprise Transport API supports at least one of the authentication schemes specified in the list sent by the proxy server, it reconnects to the proxy (if necessary) and sends a message containing:
 - The name of the authentication scheme it will use.
 - The user credentials (e.g. a username and a password) encoded in the format prescribed by the authentication scheme.
- The proxy server attempts to authenticate the user credentials provided by the application. Depending on the configuration of the proxy server and the authentication scheme, the proxy server may attempt to authenticate the credentials against an LDAP server, a Microsoft ActiveDirectory™ server, or its own credentials datastore.
 - If the authentication scheme requires only that the application send a single message containing the user credentials (e.g., the BASIC authentication scheme), and the proxy server was able to successfully authenticate these credentials, then the proxy sends a response to the Enterprise Transport API with the HTTP “OK” error code# 200, indicating a successful authentication.
 - If the authentication scheme is NTLM (illustrated in Section 10.15.2.5), then the authentication process requires a negotiation (i.e., multiple messages sent back and forth). After the initial message, the proxy server again sends a message to the Enterprise Transport API containing HTTP error code #407, and (typically) additional handshaking details to be processed by the application. The Enterprise Transport API uses this information to send a message back to the proxy server to continue the authentication process until authentication ultimately succeeds (or fails). When successful, the proxy sends a response to the Enterprise Transport API with HTTP “OK” error code# 200.
 - If the authentication scheme is Negotiate/Kerberos, (illustrated in Section 10.15.2.6) then the authentication process requires additional handshaking with a Domain Controller. After the proxy server sends a message to the Enterprise Transport API containing HTTP error code #407, the Enterprise Transport API does all the necessary Kerberos handshaking with the Domain Controller (which for Kerberos is the Key Distribution Center or KDC) to obtain the needed Kerberos service ticket. The Enterprise Transport API uses this ticket to authenticate. When successful, the proxy sends a response to the Enterprise Transport API with HTTP “OK” error code #200.
- If authentication fails, the proxy server sends a response with an HTTP error code and text/HTML indicating the failure.

10.15.2.2 Supplying the Enterprise Transport API with Credentials for Proxy Authentication

When the Enterprise Transport API connects to a proxy server that requires authentication, the proxy server sends a response to the Enterprise Transport API containing HTTP error code# 407, indicating that authentication is required and includes a list of authentication schemes enabled on the proxy server. Authentication schemes are listed in order from 'most secure' (i.e., Negotiate/Kerberos) to 'least secure' (i.e., Basic). The Enterprise Transport API attempts to use the first provided authentication scheme (i.e., Negotiate) and if that fails, the Enterprise Transport API attempts to use the next authentication scheme (and so on) in the order provided. So in Code Example 17, the order of attempted authentication schemes is: Negotiate(Kerberos) -> Kerberos -> NTLM -> Basic.

All authentication schemes require a username and a password during the authentication process. Negotiate, Kerberos, and NTLM also include additional details while authenticating (described in Section 10.15.2.3).

The following sample "407" response includes a highlighted list of authentication schemes enabled on the proxy:

```
HTTP/1.1 407 Proxy Authentication Required (Forefront TMG requires authorization to fulfill the
    request. Access to the Web Proxy filter is denied.)
Via: 1.1 OAKLPC101
Proxy-Authenticate: Negotiate
Proxy-Authenticate: Kerberos
Proxy-Authenticate: NTLM
Proxy-Authenticate: Basic realm="hostname.ntdomain.company.com"
Connection: close
Proxy-Connection: close
Pragma: no-cache
Cache-Control: no-cache
Content-Type: text/html
Content-Length: 726
```

Code Example 17: Sample 407 Proxy Response Listing the Authentication Schemes Enabled on the Proxy

10.15.2.3 CredentialsInfo Methods

METHOD	DESCRIPTION
HTTPproxyUsername	The username to authenticate. Needed for all authentication protocols.
HTTPproxyPasswd	The password to authenticate. Needed for all authentication protocols.
HTTPproxyDomain	The domain of the user to authenticate. Needed for NTLM, Negotiate/Kerberos, or Kerberos authentication protocols. For Negotiate/Kerberos or Kerberos authentication protocols, HTTPproxyDomain should be the same as the domain in the 'realms' and 'domain_realm' sections of the Kerberos configuration file, see the HTTPproxyKRB5configFile .
HTTPproxyLocalHostname	The local hostname of the client. Needed for NTLM authentication protocol only.
HTTPproxyKRB5ConfigFile	The complete path of the Kerberos5 configuration file (krb5.ini , krb5.conf , or some other custom file). Needed for Negotiate/Kerberos and Kerberos authentications.

Table 68: CredentialsInfo Methods

10.15.2.4 Providing Credentials and Modifying the Consumer Example

The following procedure describes how to provide the required authentication credentials to the Enterprise Transport API and how to modify the **Consumer** example.

Open **Consumer.java** located in **Examples/com/refinitiv/eta/examples/consumer**.

For a connection type of **ConnectionTypes.ENCRIPTED**, edit in method **setCredentials** the following code with the username, password, and domain you will use:

```
options.credentialsInfo().HTTPproxyUsername("firstName.lastName");  
options.credentialsInfo().HTTPproxyPasswd("myPasswd");  
options.credentialsInfo().HTTPproxyDomain("myDomain");
```

Also in method **setCredentials** you may need to change the Kerberos configuration file location:

```
options.credentialsInfo().HTTPproxyKRB5configFile("C:\\WINDOWS\\krb5.ini");
```

10.15.2.5 Proxy Authentication using NTLM

The following diagram illustrates proxy authentication between a Enterprise Transport API consumer and a proxy server using Windows Authentication (i.e., NTLM).

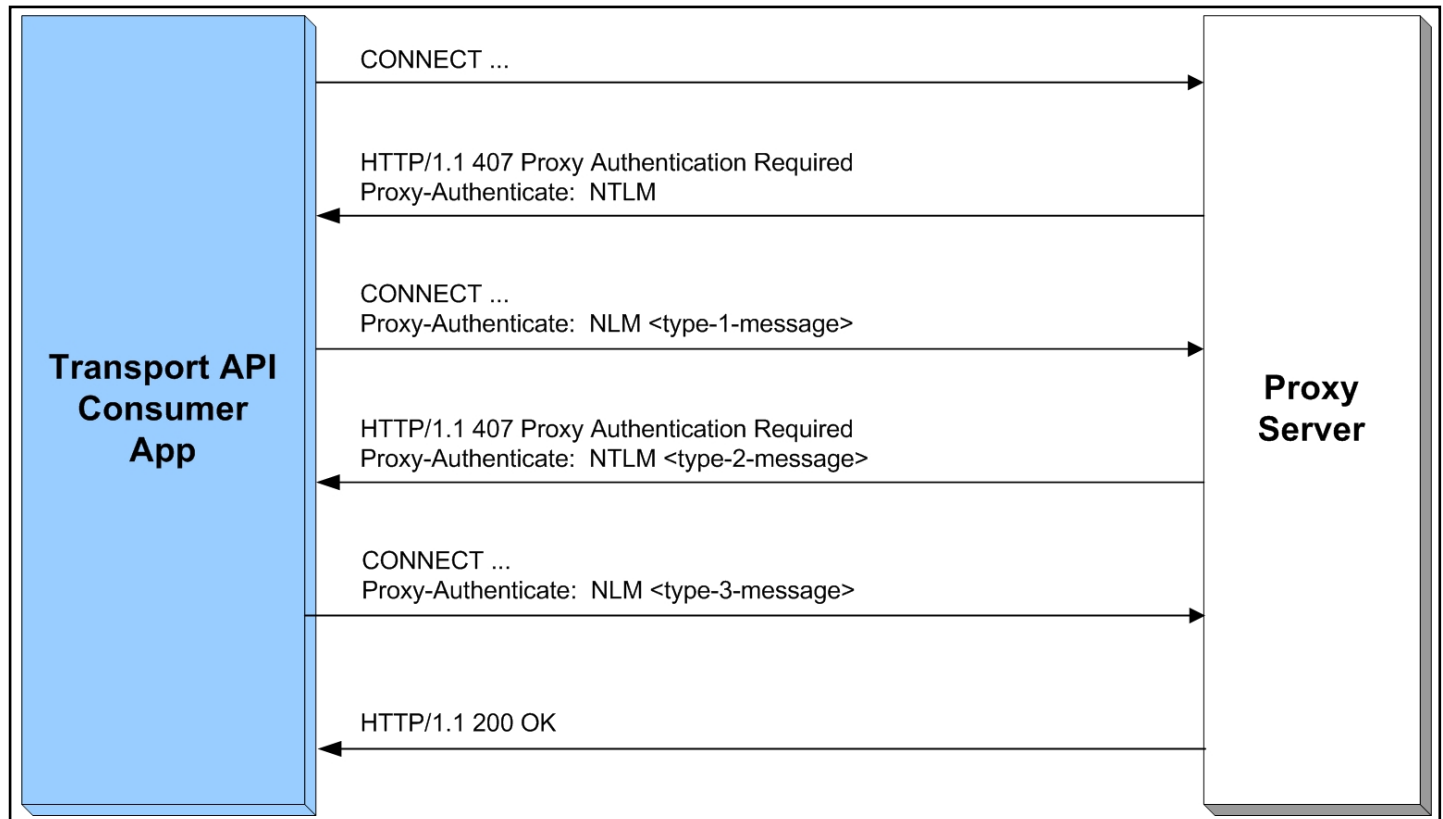


Figure 36. Enterprise Transport API Consumer Application authenticating with a Proxy Server using NTLM

10.15.2.6 Proxy Authentication using Negotiate/Kerberos

In the following diagram, from the perspective of the consumer application, the only additional “work” required to support proxy authentication is to programmatically supply the Enterprise Transport API with the credentials required for authentication.

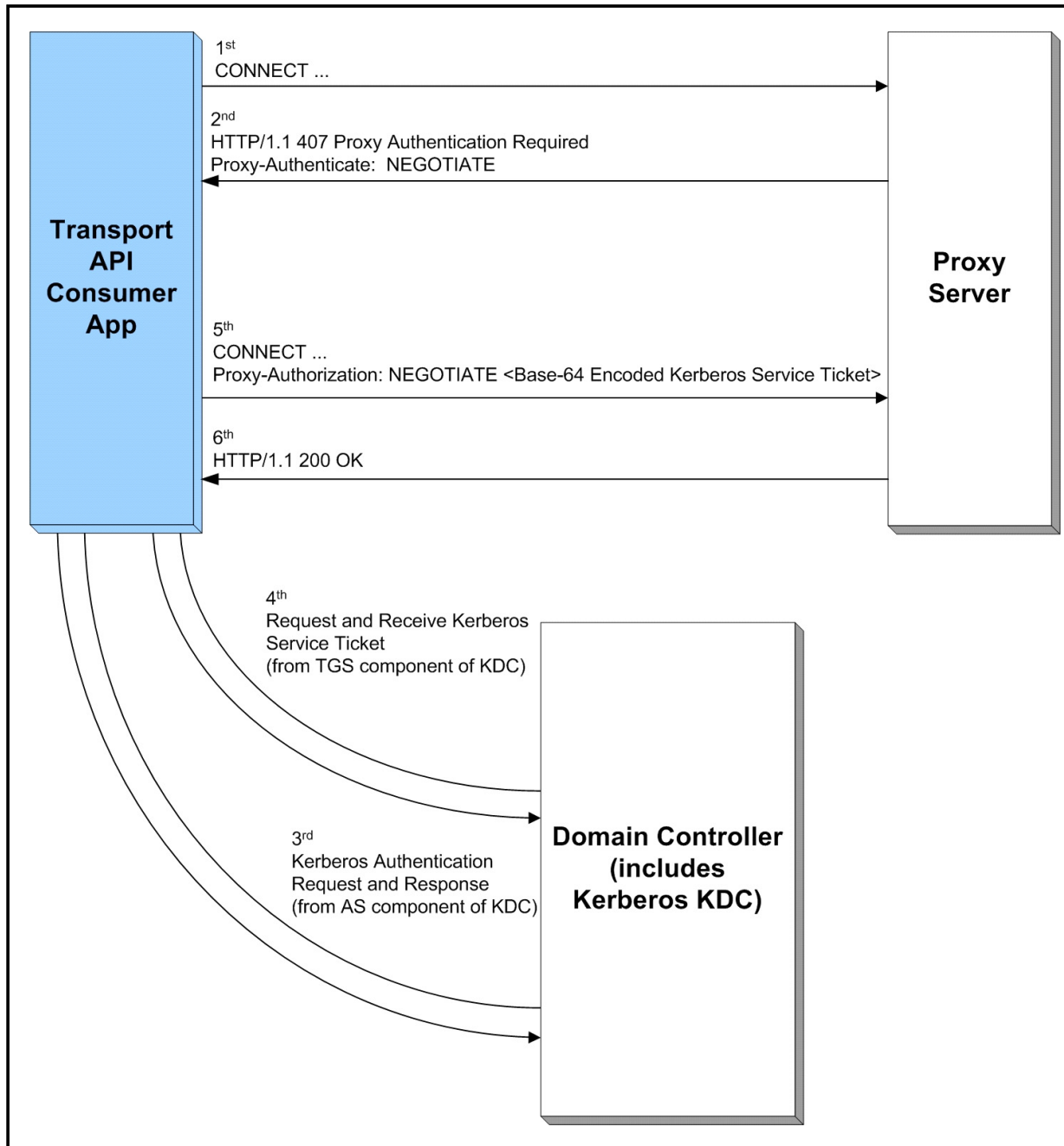


Figure 37. Enterprise Transport API Consumer Application Authenticating with a Proxy Server using Negotiate/Kerberos

10.15.3 Encrypted SOCKET and WEBSOCKET Connections

You can create encrypted connection types using either `ConnectionTypes.SOCKET` or `ConnectionTypes.WEBSOCKET`. To specify these connections, `TunnelingInfo.tunnelingType` must be set to `None`, with further configuration provided in `ConnectOptions.encryptionOptions`. As above, these parameters specify the security settings, and can be configured with the following methods:

METHOD	DESCRIPTION
connectionType	Required. Specifies the type of connection. Acceptable values are: <code>ConnectionTypes.SOCKET</code> <code>ConnectionTypes.WEBSOCKET</code> <code>ConnectionTypes.HTTP</code>
KeyManagerAlgorithm	Specifies the Java Key Management algorithm. Defaults to the property <code>ssl.KeyManagerFactory</code> algorithm in the JDK security properties file (<code>java.security</code>). The Oracle JDK is <code>SunJX509</code> .
KeystoreType	Configures the type of keystore for the certificate file. Defaults to the property <code>keystore.type</code> in the JDK security properties file (<code>java.security</code>). By default, RTSDK uses <code>JKS</code> .
KeystoreFile	Configures the keystore file that contains your private keys and any public key certificates you received from a third party. The JDK utility <code>keytool</code> creates this file. If not specified, the JVM uses its default keystore.
KeystorePasswd	Configures the password for the specified keystore file.
SecurityProtocol	Specifies which cryptographic protocol to use (by default, Oracle JDK uses <code>TLS</code>).
SecurityProtocolVersions	Specifies the version of the configured cryptographic protocol to use. By default, assumes TLS protocol and sets "1.2" and "1.3".
SecurityProvider	Specifies which Java Cryptography Package provider to use. By default, Oracle JDK uses <code>SunJSSE</code> . RTSDK library also supports Conscrypt security provider. To choose this provider, specify "Conscrypt" as the argument to this method.
TrustManagerAlgorithm	Specifies which Java Trust Management algorithm to use. By default, the connection uses the property <code>ssl.TrustManagerFactory.algorithm</code> in the JDK security properties file (<code>java.security</code>). By default, Oracle JDK uses <code>PKIX</code> .

Table 69: Encrypted SOCKET and WEBSOCKET Connection Methods

10.15.4 Encrypted Server

To configure an encrypted server, `BindOptions.ConnectionType` must be set to `ConnectionTypes.ENCRYPTED`. Further configuration is in `BindOptions.EncryptionOptions` as follows:

METHOD	DESCRIPTION
connectionType	Required. Specifies the type of connection. Acceptable values are: <code>ConnectionTypes.SOCKET</code> <code>ConnectionTypes.WEBSOCKET</code> <code>ConnectionTypes.HTTP</code>
KeyManagerAlgorithm	Specifies the Java Key Management algorithm. Defaults to the property <code>ssl.KeyManagerFactory</code> algorithm in the JDK security properties file (<code>java.security</code>). The Oracle JDK is <code>SunJX509</code> .

Table 70: Encrypted Server Methods

METHOD	DESCRIPTION
KeystoreType	Configures the type of keystore for the certificate file. Defaults to the property keystore.type in the JDK security properties file (java.security). By default, RTSDK uses JKS .
KeystoreFile	Configures the keystore file that contains the server private key and server certificate. The JDK utility keytool creates this file. If not specified, the default behavior is to load JVM's default keystore set with javax.net.ssl.keystore .
KeystorePasswd	Configures the password for the specified keystore file. If not specified, the default behavior is to load the JVM's default keystore set with javax.net.ssl.keystorepassword .
SecurityProtocol	Specifies which cryptographic protocol to use (by default, Oracle JDK uses TLS).
SecurityProtocolVersions	Specifies the version of the configured cryptographic protocol to use. By default, assumes TLS protocol and sets "1.2" and "1.3".
SecurityProvider	Specifies which Java Cryptography Package provider to use. By default, Oracle JDK uses SunJSSE . RTSDK library also supports Conscrypt security provider. To choose this provider, specify "Conscrypt" as the argument to this method.
TrustManagerAlgorithm	Specifies which Java Trust Management algorithm to use. By default, the connection uses the property ssl.TrustManagerFactory.algorithm in the JDK security properties file (java.security). By default, Oracle JDK uses PKIX .

Table 70: Encrypted Server Methods (Continued)

10.15.5 Debugging a Tunnel Connection

To debug a tunneling consumer connection, add the following JVM argument when running the consumer:

```
-Djavax.net.debug=all
```

Debugging a tunneling connection with this argument provides SSL/TLS details that can be useful if the SSL/TLS handshake fails.

11 Data Package Detailed View

11.1 Concepts

The Codec Package exposes a collection of data types that can combine in a variety of ways to assist with modeling user's data. These types are split into two categories:

- A **Primitive Type** represents simple, atomically updating information. Primitive types represent values like integers, dates, and ASCII string buffers (refer to Section 11.2).
- A **Container Type** models more intricate data representations than Enterprise Transport API primitive types and can manage dynamic content at a more granular level. Container types represent complex types like field identifier-value, name-value, or key-value pairs (refer to Section 11.3). The Enterprise Transport API offers several uniform (i.e., homogeneous) container types whose entries house the same type of data. Additionally, there are several non-uniform (i.e., heterogeneous) container types in which different entries can hold different types of data.

Primitive and Container types are also presented as a part of the **DataTypes** constants in the ranges:

- 0 to 127 are Primitive Types as described in Section 11.2.
- 128 to 255 are Container Types as described in Section 11.3.

Each type represented with a constant has a corresponding class definition used when encoding or decoding that type.

11.2 Primitive Types

A primitive type represents some type of base, system information (such as integers, dates, or array values). If contained in a set of updating information, primitive types update atomically (incoming data replaces any previously held values). Primitive types support ranges from simple primitive types (e.g., an integer) to more complex primitive types (e.g., an array).

The **DataTypes** includes constant values that define the type of a primitive:

- Values between 0 and 63 are **base primitive types**. Base primitive types support the full range of values allowed by the primitive type and are discussed in Table 71.

When contained in a **FieldEntry** or **ElementEntry**, base primitive types can also represent a **blank value**. A blank value indicates that no value is currently present and any previously stored or displayed primitive value should be cleared. When decoding any base primitive value, the interface method (See Table 71) returns **CodecReturnCodes.BLANK_DATA**. To encode blank data into a **FieldEntry** or **ElementEntry**, refer to Section 11.3.1 and Section 11.3.2.
- Values between 64 and 127 are **set-defined primitive types**, which define fixed-length encodings for many of the base primitive types (e.g., **DataTypes.INT_1** is a one byte fixed-length encoding of **DataTypes.INT_1**). These types can be leveraged only within a Set Definition and encoded or decoded as part of a **FieldList** or **ElementList**. Only certain set-defined primitive types can represent blank values. For more details about set-defined primitive types, refer to Section 11.6.

The following table provides a brief description of each base primitive type, along with interface methods used for encoding and decoding. Several primitive types have a more detailed description following the table.

ENUM TYPE	PRIMITIVE TYPE	TYPE DESCRIPTION
DataTypes.UNKNOWN	None	Indicates that the type is unknown. DataTypes.UNKNOWN is valid only when decoding a Field List type and a dictionary look-up is required to determine the type. This type cannot be passed into encoding or decoding functions. Encode Interface: None Decode Interface: None
DataTypes.INT	Int ^a	A signed integer type. Can currently represent a value of up to 63 bits along with a one bit sign (positive or negative). Encode Interface: Int.encode Decode Interface: Int.decode
DataTypes.UINT	UInt ^b	An unsigned integer type. Can currently represent an unsigned value with precision of up to 64 bits. Encode Interface: UInt.encode Decode Interface: UInt.decode
DataTypes.FLOAT	Float	A four-byte, floating point type. Can represent the same range of values allowed with the Java Float type. Follows IEEE 754 specification. Encode Interface: Float.encode Decode Interface: Float.decode
DataTypes.DOUBLE	Double	An eight-byte, floating point type. Can represent the same range of values allowed with the Java Double type. Follows IEEE 754 specification. Encode Interface: Double.encode Decode Interface: Double.decode
DataTypes.REAL	Real ^c	An optimized RssI Wire Format representation of a decimal or fractional value which typically requires less bytes on the wire than Float or Double types. The user specifies a value with a hint for converting to decimal or fractional representation. For more details on this type, refer to Section 11.2.2. Encode Interface: Real.encode Decode Interface: Real.decode
DataTypes.DATE	Date	Defines a date with month, day, and year values. For more details on this type, refer to Section 11.2.3. Encode Interface: Date.encode Decode Interface: Date.decode
DataTypes.TIME	Time	Defines a time with hour, minute, second, millisecond, microsecond, and nanosecond values. For more details on this type, refer to Section 11.2.4. Encode Interface: Time.encode Decode Interface: Time.decode
DataTypes.DATETIME	DateTime	Combined representation of date and time. Contains all members of DataTypes.DATE and DataTypes.TIME . For more details on this type, refer to Section 11.2.5. Encode Interface: DateTime.encode Decode Interface: DateTime.decode

Table 71: Enterprise Transport API Primitive Types

ENUM TYPE	PRIMITIVE TYPE	TYPE DESCRIPTION
DataTypes.QOS	Qos	Defines Quality of Service information such as data timeliness (e.g., real time) and rate (e.g., tick-by-tick). Allows a user to send Quality of Service information as part of the data payload. Similar information can also be conveyed using multiple message headers. For more details on this type, refer to Section 11.2.6. Encode Interface: Qos.encode Decode Interface: Qos.decode
DataTypes.STATE	State	Represents data and stream state information. Allows a user to send state information as part of data payload. Similar information can also be conveyed in several message headers. For more details on this type, refer to Section 11.2.7. Encode Interface: State.encode Decode Interface: State.decode
DataTypes.ENUM	Enum ^d	Represents an enumeration type, defined as an unsigned, two-byte value. Many times, this enumeration value is cross-referenced with an enumeration dictionary (e.g., enumtype.def) or a well-known, constant definition (e.g., those contained in the com.refinitiv.eta.rdm package). Encode Interface: Enum.encode Decode Interface: Enum.decode
DataTypes.ARRAY	Array	The array type allows users to represent a simple base primitive type list (all primitive types except Array). The user can specify the base primitive type that an array carries and whether each is of a variable or fixed-length. Because the array is a primitive type, if any primitive value in the array updates, the entire array must be resent. For more details on this type, refer to Section 11.2.8. Encode Interface: Refer to Section 11.2.8.2. Decode Interface: Refer to Section 11.2.8.5.
DataTypes.BUFFER	Buffer ^e	Represents a raw byte buffer type. Any semantics associated with the data in this buffer is provided from outside of the Enterprise Transport API, either via a field dictionary (e.g., RDMFieldDictionary) or a Domain Model Message definition. For more details on this type, refer to Section 11.2.9. Encode Interface: Buffer.encode Decode Interface: Buffer.decode
DataTypes.ASCII_STRING	Buffer ^e	Represents an ASCII string which should contain only characters that are valid in ASCII specification. Because this might be NULL terminated, use the provided length when accessing content. The Enterprise Transport API does not enforce or validate encoding standards: this is the user's responsibility. For more details on this type, refer to Section 11.2.9. Encode Interface: Buffer.encode Decode Interface: Buffer.decode

Table 71: Enterprise Transport API Primitive Types (Continued)

ENUM TYPE	PRIMITIVE TYPE	TYPE DESCRIPTION
DataTypes.UTF8_STRING	Buffer ^e	Represents a UTF8 string which should follow the UTF8 encoding standard and contain only characters valid within that set. Because this might be NULL terminated, use the provided length when accessing content. The Enterprise Transport API does not enforce or validate encoding standards: this is the user's responsibility. For more details on this type, refer to Section 11.2.9. Encode Interface: Buffer.encode Decode Interface: Buffer.decode
DataTypes.RMTES_STRING	Buffer ^e	Represents an RMTES string which should follow the RMTES encoding standard and contain only characters valid within that set. The Enterprise Transport API does not enforce or validate encoding standards: this is the user's responsibility. For more details on this type, refer to Section 11.2.9. Encode Interface: Buffer.encode Decode Interface: Buffer.decode

Table 71: Enterprise Transport API Primitive Types (Continued)

- a. This type allows a value ranging from (-2^{63}) to $(2^{63} - 1)$.
- b. This type allows a value ranging from 0 up to $(2^{64} - 1)$.
- c. This type allows a value ranging from (-2^{63}) to $(2^{63} - 1)$. This can be combined with hint values to add or remove up to seven trailing zeros, fourteen decimal places, or fractional denominators up to 256.
- d. This type allows a value ranging from 0 to 65,535.
- e. The Enterprise Transport API handles this type as opaque data, simply passing the length specified by the user and that number of bytes, no additional encoding or processing is done to any information contained in this type. Any specific encoding or decoding required for the information contained in this type is done outside of the scope of the Enterprise Transport API, before encoding or after decoding this type. This type allows for a length of up to 65,535 bytes.

11.2.1 DataTypes Methods

DataTypes contains the following methods.

METHOD	DESCRIPTION
primitiveTypeSize	Returns the maximum encoded size for base and set-defined primitive types. If the type allows for content of varying length (e.g. Array , Buffer , etc.), a value of 255 is returned (though the maximum encoded length may exceed 255).
isPrimitiveType	- If the dataType represents a primitive type, returns true. - If the dataType represents a container type, returns false.
isContainerType	- If the dataType represents a container type, returns true. - If the dataType represents a primitive type, returns false.
toString	Returns a Java String representation for a DataTypes value.

Table 72: DataTypes Methods

11.2.2 Real

Real is a object that represents decimals or fractional values in a bandwidth-optimized format.

The **Real** preserves the precision of encoded numeric values by separating the numeric value from any decimal point or fractional denominator. Developers should note that in some conversion cases, there may be a loss of precision; this is an example of a narrowing precision conversion. Because the IEEE 754 specification (used for **float** and **double** types) cannot represent some values exactly, rounding (per the IEEE 754 specification) may occur when converting between **Real** representation and **float** or **double** representations, either using the provided helper methods or manually (using the conversion formulas provided). In cases where precision may be lost, converting to a string or using the provided string conversion helper as an intermediate point can help avoid the rounding precision loss.

11.2.2.1 Methods

Real contains the following methods:

METHOD	DESCRIPTION
isBlank	Returns a Boolean value. Indicates whether the data is considered blank. - If true, the value and hint should be ignored - If false, value and hint determine the resultant value. This allows State to be represented as blank when used either as a primitive type or a set-defined primitive type.
hint	Returns a RealHints value (hint) which defines how to interpret the value contained in State . Hint values can add or remove up to seven trailing zeros, 14 decimal places, or fractional denominators up to 256. For more information about hint values, refer to Section 74.
toLong	The raw value represented by the State (omitting any decimal or denominator). Typically requires the application of hint before interpreting or performing any calculations. This member can currently represent up to 63 bits and a one-bit sign (positive or negative). Its value can range from (-2^{63}) to $(2^{63} - 1)$.
toDouble	Uses the formulas described in Section 11.2.2.3 to convert a State to a Java Double type.
toString	Converts a State type to a numeric String representation. Blank is output as an empty zero length String .
value(long, hint)	Sets the raw long value and hint .
value(double, hint)	Uses the formulas described in Section 11.2.2.4 to convert a float and hint to a State type.
value(float, hint)	Uses the formulas described in Section 11.2.2.4 to convert a float and hint to a State type.
value(String)	Converts a numeric String with denominator or decimal information to a State type. Interprets a String of +0 as a blank State .
encode	Encodes a State into a buffer.
decode	Decodes a State from a buffer.
equals	Compares one State to another specified State . Returns true if equal, false otherwise.
copy	Performs a deep copy of one State into another specified State .
blank	Clears the object and sets isBlank to true .
clear	Clears the object, so that you can reuse it. isBlank is set to false .

Table 73: Real Methods

11.2.2.2 hint Values

The following table defines the available **RealHints** values for use with **Real**. The conversion routines described in Section 11.2.2.3 use **Real**'s **hint** and **toLong** value.

ENUM	DESCRIPTION
RealHints.EXPONENT_14	Negative exponent operation, equivalent to 10^{-14} . Shifts decimal by 14 positions.
RealHints.EXPONENT_13	Negative exponent operation, equivalent to 10^{-13} . Shifts decimal by 13 positions.
RealHints.EXPONENT_12	Negative exponent operation, equivalent to 10^{-12} . Shifts decimal by 12 positions.
RealHints.EXPONENT_11	Negative exponent operation, equivalent to 10^{-11} . Shifts decimal by 11 positions.
RealHints.EXPONENT_10	Negative exponent operation, equivalent to 10^{-10} . Shifts decimal by ten positions.
RealHints.EXPONENT_9	Negative exponent operation, equivalent to 10^{-9} . Shifts decimal by nine positions.
RealHints.EXPONENT_8	Negative exponent operation, equivalent to 10^{-8} . Shifts decimal by eight positions.
RealHints.EXPONENT_7	Negative exponent operation, equivalent to 10^{-7} . Shifts decimal by seven positions.
RealHints.EXPONENT_6	Negative exponent operation, equivalent to 10^{-6} . Shifts decimal by six positions.
RealHints.EXPONENT_5	Negative exponent operation, equivalent to 10^{-5} . Shifts decimal by five positions.
RealHints.EXPONENT_4	Negative exponent operation, equivalent to 10^{-4} . Shifts decimal by four positions.
RealHints.EXPONENT_3	Negative exponent operation, equivalent to 10^{-3} . Shifts decimal by three positions.
RealHints.EXPONENT_2	Negative exponent operation, equivalent to 10^{-2} . Shifts decimal by two positions.
RealHints.EXPONENT_1	Negative exponent operation, equivalent to 10^{-1} . Shifts decimal by one position.
RealHints.EXPONENT0	Exponent operation, equivalent to 10^0 . value does not change.
RealHints.EXPONENT1	Positive exponent operation, equivalent to 10^1 . Depending on the type of conversion, this adds or removes one trailing zero.
RealHints.EXPONENT2	Positive exponent operation, equivalent to 10^2 . Depending on the type of conversion, this adds or removes two trailing zeros.
RealHints.EXPONENT3	Positive exponent operation, equivalent to 10^3 . Depending on the type of conversion, this adds or removes three trailing zeros.
RealHints.EXPONENT4	Positive exponent operation, equivalent to 10^4 . Depending on the type of conversion, this adds or removes four trailing zeros.
RealHints.EXPONENT5	Positive exponent operation, equivalent to 10^5 . Depending on the type of conversion, this adds or removes five trailing zeros.
RealHints.EXPONENT6	Positive exponent operation, equivalent to 10^6 . Depending on the type of conversion, this adds or removes six trailing zeros.
RealHints.EXPONENT7	Positive exponent operation, equivalent to 10^7 . Depending on the type of conversion, this adds or removes seven trailing zeros.
RealHints.FRACTION_1	Fractional denominator operation, equivalent to 1/1. Value does not change.
RealHints.FRACTION_2	Fractional denominator operation, equivalent to 1/2. Depending on the type of conversion, this adds or removes a denominator of two.

Table 74: RealHints Enumeration Values

ENUM	DESCRIPTION
RealHints.FRACTION_4	Fractional denominator operation, equivalent to 1/4. Depending on the type of conversion, this adds or removes a denominator of four.
RealHints.FRACTION_8	Fractional denominator operation, equivalent to 1/8. Depending on the type of conversion, this adds or removes a denominator of eight.
RealHints.FRACTION_16	Fractional denominator operation, equivalent to 1/16. Depending on the type of conversion, this adds or removes a denominator of 16.
RealHints.FRACTION_32	Fractional denominator operation, equivalent to 1/32. Depending on the type of conversion, this adds or removes a denominator of 32.
RealHints.FRACTION_64	Fractional denominator operation, equivalent to 1/64. Depending on the type of conversion, this adds or removes a denominator of 64.
RealHints.FRACTION_128	Fractional denominator operation, equivalent to 1/128. Depending on the type of conversion, this adds or removes a denominator of 128.
RealHints.FRACTION_256	Fractional denominator operation, equivalent to 1/256. Depending on the type of conversion, this adds or removes a denominator of 256.
RealHints.INFINITY	Value should be interpreted as infinity (Inf).
RealHints.NEG_INFINITY	Value should be interpreted as negative infinity (-Inf).
RealHints.NOT_A_NUMBER	Value should be interpreted as not a number (NaN).

Table 74: RealHints Enumeration Values (Continued)

11.2.2.3 Hint Use Case: Converting an Real to a Float or a Double

An application can convert between a **Real** and a Java **float** or **double** as needed. Converting a **Real** to a **double** or **float** is typically done to perform calculations or display data after receiving it.

The conversion process adds or removes decimal or denominator information from the value to optimize transmission sizes. In a **Real** type, the decimal or denominator information is indicated by the **Real.hint**, and the **Real.toLong** indicates the value (less any decimal or denominator). If the **Real.isBlank** member is **true**, this is handled as blank regardless of information contained in the **Real.hint** and **Real.toLong** methods.

For this conversion, both the hint and its value are stored in the **Real** object. You can use the following example to perform this conversion, where **outputValue** is a system **float** or **double** to store output:

```
/* perform calculation and assign output to outputValue - may require appropriate float or double
   casts depending on type of outputValue */
outputValue = real.toDouble();
```

Code Example 18: Real Conversion to Double/Float

11.2.2.4 Hint Use Case: Converting Double or Float to an Real

To convert a **double** or **float** type to a **Real** type (typically done to prepare for transmission), the user must determine which hint value to use based on the type of value used:

- When converting a decimal value, the chosen hint value must be less than **RealHints.FRACTION_1**.
- When converting a fractional value, the chosen hint value must be greater than or equal to **RealHints.FRACTION_1**.

You can use the following example to perform the conversion, where **inputValue** is the unmodified input **float** or **double** value and **inputHint** is the hint chosen by the user:

```
/* Perform calculation and store output in Real object - may require appropriate float or double casts
   depending on type of inputValue */
real.value(inputValue, inputHint);
```

Code Example 19: Real Conversion from Double/Float

11.2.3 Date

11.2.3.1 Date Methods

Date represents the date (i.e., **day**, **month**, and **year**) in a bandwidth-optimized fashion.

If **day**, **month**, and **year** are all set to **0** the **Date** is blank. If any individual member is represented as a blank value (**0**), only that member is blank. This is useful for representing dates which specify **month** and **year**, but not **day**. The **Date** type can be represented as blank when used as a primitive type and a set-defined primitive type.

METHOD	DESCRIPTION
day	Sets or gets the day . Represents the day of the month, where 0 indicates a blank entry. day allows a range of 0 to 255 , though the value typically does not exceed 31 .
month	Sets or gets the month . Represents the month of the year, where 0 indicates a blank entry. month allows a range of 0 to 255 , though the value typically does not exceed 12 .
year	Sets or gets the year . Represents the year, where 0 indicates a blank entry. You can use this member to specify a two- or four-digit year (where specific usage is indicated outside of the Enterprise Transport API). year allows a range of 0 to 65,535 .
blank	Sets all members in Date to 0 . Because 0 represents a blank date value, this performs the same functionality as the Date.clear method.
isBlank	Returns true if Date is blank, otherwise false .
isValid	Verifies the contents of the Date object. Determines whether the specified day is valid within the specified month (e.g., a day greater than 31 is considered invalid for any month). This method uses the year value to determine leap year validity of day numbers for February. If Date is blank or valid, true is returned; false otherwise.
format	Sets or gets the format of Date as a string representation. For available DateTimeStringFormatTypes , refer to Section 11.2.3.2.
toString	Converts the Date to a Java String according to the specified format : - If format is STR_DATETIME_RSSL, the string will be "DD MM YYYY" (e.g., 30 JAN 2018). - If format is STR_DATETIME_ISO8601, the string will be "YYYY-MM-DD" (e.g., 2018-01-30).
value	Converts a Java String date to Date from one of the following formats: "DD MMM YYYY" (e.g., 30 JAN 2018) "MM/DD/YYYY" (e.g., 01/30/2018) - ISO8601 format "YYYY-MM-DD" (e.g., 2018-01-30)
equals	Compares the Date to another specified Date . Returns true if equal, false otherwise.
copy	Performs a deep copy of the Date to another specified Date .
encode	Encodes a Date into a buffer.
decode	Decodes a Date from a buffer.
clear	Clears the object for reuse. Because 0 represents a blank date value, this performs the same functionality as the Date.blank method.

Table 75: Date Methods

11.2.3.2 DateTimeStringFormatTypes

DateTimeStringFormatTypes represents the **Date**-to-string conversion format types.

FORMAT	DESCRIPTION
STR_DATETIME_ISO8601	Converts the Date structure to a string in ISO8601's dateTime format: “ YYYY-MM-DD ” (e.g., 2018-01-20).
STR_DATETIME_RSSL	Converts the Date structure to a string in the format: “ DD MM YYYY ” (e.g., 30 JAN 2018).

Table 76: DateTimeStringFormatTypes

11.2.4 Time

Time represents time (hour, minute, second, millisecond, microsecond, and nanosecond) in a bandwidth-optimized fashion. This type is represented as Greenwich Mean Time (GMT) unless noted otherwise¹.

11.2.4.1 Time Methods

If all methods are set to their respective blank values, **Time** is blank. If any individual member is set to a blank value, only that member is blank. This is useful for representing times without **second**, **millisecond**, **microsecond**, or **nanosecond** values. The **Time** type can be represented as blank when it is used as a primitive type and a set-defined primitive type.

METHOD	DESCRIPTION
hour	Sets or gets the hour of the day (hour). Represents the hour of the day using a range of 0 to 255 (255 represents a blank hour value), though the value does not typically exceed 23 .
minute	Sets or gets the minute of the hour (minute). Represents the minute of the hour using a range of 0 to 255 (255 represents a blank minute value), though the value does not typically exceed 59 .
second	Sets or gets the second of the minute (second). Represents the second of the minute using a range of 0 to 255 (255 represents a blank second value), though the value does not typically exceed 59 .
millisecond	Sets or gets the millisecond of the second (millisecond). Represents the millisecond of the second using a range of 0 - 65,535 (65535 represents a blank millisecond value), though the value does not typically exceed 999 .
microsecond	Sets or gets the microsecond of the millisecond (microsecond). Represents the microsecond of the millisecond using a range of 0 - 2047 (2047 represents a blank microsecond value), though the value does not typically exceed 999 .
nanosecond	Sets or gets the nanosecond of the microsecond (nanosecond). Represents the nanosecond of the microsecond using a range of 0 - 2047 (where 2047 represents a blank nanosecond value), though the value does not typically exceed 999 .
blank	Sets all members in Time to their blank values.
isBlank	Returns true if all members in Time are set to their blank values.
isValid	Verifies the contents of a populated Time structure. Validates the ranges of the hour , minute , second , millisecond , microsecond , and nanosecond members. If Time is blank or valid, true is returned; false otherwise.
format	Sets or gets the format of Time as a string representation. For available DateTimeStringFormatTypes , refer to Section 11.2.4.2.

Table 77: Time Methods

1. The provider's documentation should indicate whether the providing application provides times in another representation.

METHOD	DESCRIPTION
toString	Converts Time to a Java String according to the specified format : - If format is STR_DATETIME_RSSL, the string will be “<i>hour:minute:second:milli:micro:nano</i>” (e.g., 15:24:54:627:843:143). - If format is STR_DATETIME_ISO8601, the string will be “<i>hour:minute:second.nnnnnnnnn</i>” (e.g., 15:24:54.627843143), with trailing zeros trimmed, where <i>nnnnnnnnn</i> is <i>millisecond microsecond nanosecond</i>.
value	Converts a Java String time to Time from one of the following formats: “<i>HH:MM</i>” (e.g., 13:01) “<i>HH:MM:SS</i>” (e.g., 15:23:54) - ISO8601 format
equals	Compares Time to another specified Time . If equal, returns true ; false otherwise.
copy	Performs a deep copy of Time to another specified Time .
encode	Encodes a Time into a buffer.
decode	Decodes a Time from a buffer.
clear	Clears the object for reuse.

Table 77: Time Methods (Continued)

11.2.4.2 DateTimeStringFormatTypes

DateTimeStringFormatTypes represents the **Time**-to-string conversion format types.

FORMAT	DESCRIPTION
STR_DATETIME_ISO8601	Converts the Time structure to a string in ISO8601's dateTime format: “ <i>hour:minute:second.nnnnnnnnn</i> ” (e.g., 15:24:54.627843143), with trailing zeros trimmed, where <i>nnnnnnnnn</i> is <i>millisecond microsecond nanosecond</i> .
STR_DATETIME_RSSL	Converts the Time structure to a string in the format: “ <i>hour:minute:second:milli:micro:nano</i> ” (e.g., 15:24:54:627:843:143).

Table 78: DateTimeStringFormatTypes

11.2.5 DateTime

DateTime represents the date (**date**) and time (**time**) in a bandwidth-optimized fashion. This time value is represented as Greenwich Mean Time (GMT) unless noted otherwise².

11.2.5.1 DateTime Methods

DateTime provides convenient methods to set or get **Date** and **Time** values directly, or **Date** and **Time** can be retrieved and used independently.

If **date** and **time** values are set to their respective blank values, **DateTime** is blank. If any individual member is set to a blank value, only that member is blank. The **DateTime** type can be represented as blank when it is used as a primitive type and a set-defined primitive type.

DateTime contains the following methods:

METHOD	DESCRIPTION
date	Returns the Date portion of the DateTime and conforms to the behaviors described in Section 11.2.3.
time	Returns the Time portion of the DateTime and conforms to the behaviors described in Section 11.2.4.
day	Sets or gets the day of the month. The valid range is 0 to 255 , where 0 indicates a blank entry (though the value does not typically exceed 31).
month	Sets or gets the month of the year. The valid range is 0 to 255 , where 0 indicates a blank entry (though the value does not typically exceed 12).
year	Sets or gets the year. You can use this member to specify a two- or four-digit year (where specific usage is indicated outside of the Enterprise Transport API). The valid range is 0 to 65,535 , where 0 indicates a blank entry.
hour	Sets or gets the hour of the day. The valid range is 0 to 255 , where 255 represents a blank hour value (though the value does not typically exceed 23).
minute	Sets or gets the minute of the hour. The valid range is 0 to 255 , where 255 represents a blank minute value (though the value does not typically exceed 59).
second	Sets or gets the second of the minute. The valid range is 0 to 255 , where 255 represents a blank second value (though the value does not typically exceed 59).
millisecond	Sets or gets the millisecond of the second. The valid range is 0 to 65535 , where 65535 represents a blank millisecond value (though the value does not typically exceed 999).
microsecond	Sets or gets the microsecond of the millisecond. The valid range is 0 to 2047 , where 2047 represents a blank microsecond value (though the value does not typically exceed 999).
nanosecond	Sets or gets the nanosecond of the microsecond. The valid range is 0 to 2047 , where 2047 represents a blank nanosecond value (though the value does not typically exceed 999).
gmtTime	Sets the date time to the present time in GMT zone.
localTime	Sets the date time to the present time in the local time zone.
blank	Sets all members in DateTime to their respective blank values.
isBlank	Returns true if all members in Date and Time are set to the values used to signify blank.

Table 79: DateTime Methods

2. The provider's documentation should indicate whether the providing application provides times in another representation.

METHOD	DESCRIPTION
isValid	Determines whether day is valid for the specified month (e.g., a day greater than 31 is considered invalid for any month) as determined by the specified year (to calculate whether it is a leap year). Also validates the range of hour , minute , second , millisecond , microsecond , and nanosecond members. If DateTime is blank or valid, true is returned; false otherwise.
millisSinceEpoch	Returns the date-time value as milliseconds since the January 1, 1970 (midnight UTC/GMT) epoch.
format	Sets or gets the format of Date as a string representation. For available DateTimeStringFormatTypes , refer to Section 11.2.5.2.
toString	Converts DateTime to a Java String according to the specified format : - If format is STR_DATETIME_RSSL, the string will be “%d %b %Yhour:minute:second:milli:micro:nano” (e.g., 30 JAN 2018 15:24:54:627:843:143). - If format is STR_DATETIME_ISO8601, the string will be “YYYY-MM-DDThour:minute:second.nnnnnnnnn” (e.g., 2018-01-30T15:24:54.627843143), with trailing zeros trimmed, where nnnnnnnnn is <i>millisecond microsecond nanosecond</i>.
value(String)	Converts a Java String representation of a date and time to a DateTime . This method supports: Date values conforming to “%d %b %Y” format (e.g., 30 NOV 2010) or “%m/%d/%Y” format (e.g., 11/30/2010). Time values conforming to “%H:%M” format (e.g., 15:24), “%H:%M:%S” format (e.g., 15:24:54), or “hour:minute:second:milli:micro:nano” format (e.g., 15:24:54:627:843:143). - ISO8601's DateTime format
value(long)	Sets date-time using a number equal to milliseconds since the January 1, 1970 (midnight UTC/GMT) epoch.
equals	Compares two DateTime structures. Returns true if equal; false otherwise.
copy	Performs a deep copy of DateTime to another specified DateTime .
encode	Encodes a date and time into a buffer.
decode	Decodes a date and time from a buffer.
clear	Clears this object, so that you can reuse it. Sets all members to 0.

Table 79: DateTime Methods (Continued)

11.2.5.2 DateTimeStringFormatTypes

DateTimeStringFormatTypes represents the **DateTime**-to-string conversion format types.

FORMAT	DESCRIPTION
STR_DATETIME_ISO8601	Converts the DateTime structure to a string in ISO8601's dateTime format: “YYYY-MM-DDThour:minute:second.nnnnnnnnn” (e.g., 2018-01-30T15:24:54.627843143), with trailing zeros trimmed, where nnnnnnnnn is <i>millisecond microsecond nanosecond</i> .
STR_DATETIME_RSSL	Converts the DateTime structure to a string in the format “%d %b %Yhour:minute:second:milli:micro:nano” (e.g., 30 JAN 2018 15:24:54:627:843:143).

Table 80: DateTimeStringFormatTypes

11.2.6 Qos

Qos classifies data into two attributes:

- **Timeliness**: Conveys the age of data.
- **Rate**: Conveys the rate at which data changes.

Some timeliness or rate values allow you to provide additional time or rate data, for more details refer to Section 11.2.6.1, Section 11.2.6.2, and Section 11.2.6.3.

If present in a data payload, specific handling and interpretation associated with Quality of Service information is provided from outside of the Enterprise Transport API, possibly via the specific Domain Message Model definition.

Several Enterprise Transport API message headers also contain Quality of Service data. When present, this data is typically used to request or convey the Quality of Service associated with a particular stream. For more information about Quality of Service use within a message, refer to Section 12.2.1 and Section 12.2.2. When conflated data is sent, additional conflation data might be included with update messages. For further details on conflation, refer to Section 12.2.3.

11.2.6.1 Methods

Qos contains the following methods:

METHOD	DESCRIPTION
timeliness	Sets or gets the timeliness . Describes the age of the data (e.g., real time). Timeliness values are described in Section 11.2.6.2.
rate	Sets or gets the rate . Describes the rate at which the data changes (e.g., tick-by-tick). Rate values are described in Section 11.2.6.3.
dynamic	Describes the changeability of the Quality of Service within the requested range, typically over the life of a data stream. • If set to false, the Quality of Service should not change following the initial establishment. • If set to true, the Quality of Service can change over time to other values within the requested range. Quality of Service can change due to permissioning information, stream availability, network congestion, or other reasons. Specific information about dynamically changing Quality of Service should be described in documentation for components that support this behavior.
isDynamic	Returns true if the Quality of Service is dynamic. Describes the changeability of the quality of service, typically over the life of a data stream.
timeInfo	Sets or gets the timeInfo . Conveys detailed information about data timeliness , typically the amount of time delay. timeInfo allows for a range of 0 to 65,535. This information is present only when timeliness is set to QosTimeliness.DELAYED .
rateInfo	Sets or gets the rateInfo . Conveys detailed information about rate , typically the interval of time during which data are conflated. Conflation combines multiple information updates into a single update, usually reducing network traffic. rateInfo allows for a range of 0 to 65,535. This information is present only when rate is set to QosRates.TIME_CONFLATED .
equals	Compares this Qos with a specified Qos . • Returns true if the values contained in the structure are identical. • Returns false if the values contained in the structure differ.
isBetter	Compares this Qos with a specified Qos to determine which has better overall quality. • Returns true if this Qos is better. • Returns false if this Qos is not better.

Table 81: Qos Methods

METHOD	DESCRIPTION
isInRange	Determines whether this Qos lies within a range from best Qos to worst Qos . - Returns true if this Qos falls between best and worst Qos - Returns false if this Qos falls outside of the best or worst Qos range.
blank	Clears this object and sets it to blank.
isBlank	Returns true if Qos is blank, otherwise false .
toString	Returns a Java String representation for this Qos .
copy	Performs a deep copy of the Qos to another specified Qos .
encode	Encodes Qos into a buffer.
decode	Decodes Qos from a buffer.
clear	Clears this object, so that you can reuse it. Sets all members in Qos to an initial value of 0 . This includes setting rate and timeliness to their unspecified values (not intended to be encoded or decoded).

Table 81: Qos Methods (Continued)

11.2.6.2 Qos Timeliness Values

QOS TIMELINESS	DESCRIPTION
QosTimeliness.UNSPECIFIED	timeliness is unspecified. Typically used by Quality of Service initialization methods and not intended to be encoded or decoded.
QosTimeliness.REALTIME	timeliness is real time: data is updated as soon as new data is available. This is the highest-quality timeliness value. In conjunction with a rate of QosRates.TICK_BY_TICK , real time is the best overall Quality of Service.
QosTimeliness.DELAYED_UNKNOWN	timeliness is delayed, though the amount of delay is unknown. This is a lower quality than QosTimeliness.REALTIME and might be worse than QosTimeliness.DELAYED (in which case the delay is known).
QosTimeliness.DELAYED	timeliness is delayed and the amount of delay is provided in Qos.timeInfo . This is lower quality than QosTimeliness.REALTIME and might be better than QosTimeliness.DELAYED_UNKNOWN .

Table 82: QosTimeliness Values

11.2.6.3 QosRates Values

QOS RATE	DESCRIPTION
QosRates.UNSPECIFIED	rate is unspecified. Typically used by Quality of Service initialization methods and not intended to be encoded or decoded.
QosRates.TICK_BY_TICK	rate is tick-by-tick (i.e., data is sent for every update). This is the highest quality rate value. The best overall Quality of Service is a tick-by-tick rate with a timeliness of QosTimeliness.REALTIME .
QosRates.JIT_CONFLATED	rate is Just-In-Time (JIT) Conflated, meaning that quality is typically tick-by-tick, but if a data burst occurs (or if a component cannot keep up with tick-by-tick delivery), multiple updates are combined into a single update to reduce traffic. This value is usually considered a lower quality than QosRates.TICK_BY_TICK. Because JIT conflation is triggered by an application's inability to keep up with data rates, the effective rate depends on whether the application can sustain full data rates. Use of this value typically results in a rate similar to QosRates.TICK_BY_TICK. However, when the application cannot keep up with data rates, it results in a rate similar to QosRates.TIME_CONFLATED, where rateInfo is determined by the provider. Specific information about conflationTime or conflationCount might be present in an UpdateMsg. For further details, refer to Section 12.2.3.
QosRates.TIME_CONFLATED	rate is time-conflated. The interval of time (usually in milliseconds) over which data are conflated is provided in Qos.rateInfo. This is lower quality than QosRates.TICK_BY_TICK and at times even lower than QosRates.JIT_CONFLATED. Specific information about the conflationTime or conflationCount might be present in the UpdateMsg. For more details, refer to Section 12.2.3.

Table 83: QosRates Values

11.2.7 State

State conveys data and stream health information. When present in a header, **State** applies to the state of the stream and data. When present in a data payload, the meaning of **State** should be defined by the Domain Message Model.

Several Enterprise Transport API message headers also contain **State** data. When present in a message header, **State** typically conveys the overall data and stream health of messages flowing over a particular stream. For more information on using **State** in a message, refer to Section 12.2.1, Section 12.2.2, and Section 12.2.4. A decision table that provides example behaviors for various state combinations is available in Appendix A, Item and Group State Decision Table.

11.2.7.1 Methods

State contains the following methods:

METHOD	DESCRIPTION
streamState	Sets or gets the streamState , which conveys data about the stream's health. StreamState values are described in Section 11.2.7.2.
dataState	Sets or gets the dataState , which conveys data about the health of data flowing within a stream. dataState values are described in Section 11.2.7.4.
code	Sets or gets the code , which is a value that conveys additional information about the current state. Typically indicates more specific information (e.g., pertaining to a condition occurring upstream causing current data and stream states). code is typically used for informational purposes. StateCode values are described in Section 11.2.7.6. NOTE: An application should not trigger specific behavior based on this content.
text	Sets or gets the text , which is a Buffer containing specific text regarding the current data and stream state. Typically used for informational purposes. Encoded text has a maximum allowed length of 32,767 bytes. NOTE: An application should not trigger specific behavior based on this content.
equals	Compares the State with another specified State . - Returns true if the values contained in the structure are identical. - Returns false if the values contained in the structure differ.
isBlank	Returns true if State is blank, otherwise false .
isFinal	- Returns true if the State represents a final state for a stream (i.e., stream is Closed, Closed Recover, Redirected, or NonStreaming). - Returns false if the State is not final.
toString	Returns a Java String representing this State , including streamState , dataState , code and text .
copy	Perform a deep copy of the State to another specified State .
encode	Encodes State into a buffer.
decode	Decodes State into a buffer.
clear	Clears this object for reuse. Sets all members in State to an initial value. This includes setting streamState to its unspecified value (not intended to be encoded or decoded).

Table 84: State Methods

11.2.7.2 StreamStates Values

STREAM STATE	DESCRIPTION
StreamStates.UNSPECIFIED	streamState is unspecified. Typically used as a structure initialization value and is not intended to be encoded or decoded.
StreamStates.OPEN	streamState is open. This typically means that data is streaming: as data changes, they are sent on the stream.
StreamStates.NON_STREAMING	streamState is non-streaming. After receiving a final RefreshMsg or StatusMsg , the stream is closed and updated data is not delivered without a subsequent re-request. Update messages might still be received between the first and final part of a multi-part refresh. For further details, refer to Section 13.1.
StreamStates.CLOSED_RECOVER	streamState is closed, however data can be recovered on this service and connection at a later time. This state can occur via either a RefreshMsg or a StatusMsg . Single Open behavior can modify this state (continuing to indicate a stream state of StreamStates.OPEN) and attempt to recover data on the user's behalf. For further details on Single Open behavior, refer to Section 13.5.
StreamStates.CLOSED	streamState is closed. Data is not available on this service and connection and is not likely to become available, though the data might be available on another service or connection. This state can result from either an RefreshMsg or an StatusMsg .
StreamStates.REDIRECTED	streamState is redirected. The current stream is closed and has new identifying information. The user can issue a new request for the data using the new message key data from the redirect message. This state can result from either a RefreshMsg or a StatusMsg . For further details, refer to Section 12.1.3.2.

Table 85: StreamStates Values

11.2.7.3 StreamStates Methods

METHOD	DESCRIPTION
info	Returns a Java String representation of any information associated with a StreamStates value (e.g. "Closed, Recoverable" for StreamStates.CLOSED_RECOVER).
toString	Returns a Java String representation for a StreamStates value (e.g. "CLOSED_RECOVER" for StreamStates.CLOSED_RECOVER).

Table 86: StreamStates Methods

11.2.7.4 DataStates Values

DATA STATE	DESCRIPTION
DataStates.NO_CHANGE	Indicates there is no change in the current state of the data. When available, it is preferable to send more concrete state information (such as OK or SUSPECT) instead of NO_CHANGE . This typically conveys code or text information associated with an item group, but no change to the group's previous data and stream state has occurred.
DataStates.OK	dataState is OK . All data associated with the stream is healthy and current.
DataStates.SUSPECT	dataState is SUSPECT (also known as a stale-data state). A suspect data state means some or all of the data on a stream is out-of-date (or that it cannot be confirmed as current, e.g., the service is down). If an application does not allow suspect data, a stream might change from open to closed or closed recover as a result. For further details, refer to Section 13.5.

Table 87: DataStates Values

11.2.7.5 DataStates Methods

METHOD	DESCRIPTION
info	Returns a Java String representation of any information associated with a DataStates value (e.g. "No Change" for DataStates.NO_CHANGE).
toString	Returns a Java String representation for a DataStates value (e.g. "NO_CHANGE" for DataStates.NO_CHANGE).

Table 88: DataStates Methods

11.2.7.6 StateCodes Values

STATE CODE	DESCRIPTION
StateCodes.ALREADY_OPEN	Indicates that a stream is already open on the connection for the requested data.
StateCodes.APP_AUTHORIZATION_FAILED	Indicates that application authorization using the secure token has failed.
StateCodes.DACS_DOWN	Indicates that the connection to the Data Access Control System is down and users are not allowed to connect.
StateCodes.DACS_MAX_LOGINS_REACHED	Indicates that the maximum number of logins has been reached.
StateCodes.DACS_USER_ACCESS_TO_APP_DENIED	Indicates that the application is denied access to the system.
StateCodes.ERROR	Indicates an internal error from the sender.
StateCodes.EXCEEDED_MAX_MOUNTS_PER_USER	Indicates that the login was rejected because the user exceeded their maximum number of allowed mounts.
StateCodes.FAILOVER_COMPLETED	Indicates that recovery from a failover condition has finished.

Table 89: StateCodes Values

STATE CODE	DESCRIPTION
StateCodes.FAILOVER_STARTED	Indicates that a component is recovering due to a failover condition. User is notified when recovery finishes via StateCodes.FAILOVER_COMPLETED .
StateCodes.FULL_VIEW_PROVIDED	Indicates that the full view (e.g., all available fields) is being provided, even though only a specific view was requested. Section 13.8 discusses views in more detail.
StateCodes.GAP_DETECTED	Indicates that a gap was detected between messages. A gap might be detected via an external reliability mechanism (e.g., transport) or using the seqNum present in Enterprise Transport API messages.
StateCodes.GAP_FILL	Indicates that the received content is meant to fill a recognized gap.
StateCodes.INVALID_ARGUMENT	Indicates that the request includes an invalid or unrecognized parameter. Specific information should be contained in the text .
StateCodes.INVALID_VIEW	Indicates that the requested view is invalid, possibly due to bad formatting. Additional information should be available in the text . Section 13.8 discusses views in more detail.
StateCodes.JIT_CONFLATION_STARTED	Indicates that JIT conflation has started on the stream. User is notified when JIT Conflation ends via StateCodes.REALTIME_RESUMED .
StateCodes.NO_BATCH_VIEW_SUPPORT_IN_REQ	Indicates that the provider does not support batch and/or view functionality.
StateCodes.NO_RESOURCES	Indicates that no resources are available to accommodate the stream.
StateCodes.NON_UPDATING_ITEM	Indicates that a streaming request was made for non-updating data.
StateCodes.NONE	Indicates that additional state code information is not required, nor present.
StateCodes.NOT_ENTITLED	Indicates that the request was denied due to permissioning. Typically indicates that the requesting user does not have permission to request on the service, to receive requested data, or to receive data at the requested Quality of Service.
StateCodes.NOT_FOUND	Indicates that requested information was not found, though it might be available at a later time or through changing some parameters used in the request.
StateCodes.NOT_OPEN	Indicates that the stream was not opened. Additional information should be available in the text .
StateCodes.PREEMPTED	Indicates the stream was preempted, possibly by a caching device. Typically indicates the user has exceeded an item limit, whether specific to the user or a component in the system. Relevant information should be contained in the text .
StateCodes.REALTIME_RESUMED	Indicates that JIT conflation on the stream has finished.
StateCodes.SOURCE_UNKNOWN	Indicates that the requested service is not known, though the service might be available at a later point in time.
StateCodes.TIMEOUT	Indicates that a timeout occurred somewhere in the system while processing requested data.

Table 89: StateCodes Values(Continued)

STATE CODE	DESCRIPTION
StateCodes.TOO_MANY_ITEMS	Indicates that a request cannot be processed because too many other streams are already open.
StateCodes.UNABLE_TO_REQUEST_AS_BATCH	Indicates that a batch request cannot be used for this request. The user can instead split the batched items into individual requests. Section 13.7 discusses batch requesting in more detail.
StateCodes.UNSUPPORTED_VIEW_TYPE	Indicates that the domain on which a request is made does not support the requested viewType . Section 13.8 discusses views in more detail.
StateCodes.USAGE_ERROR	Indicates invalid usage within the system. Specific information should be contained in the text .
StateCodes.USER_UNKNOWN_TO_PERM_SYS	Indicates that the user is unknown to the permissioning system and is not allowed to connect.

Table 89: StateCodes Values(Continued)

11.2.7.7 StateCodes Methods

METHOD	DESCRIPTION
info	Returns a Java String representation of any information associated with a StateCodes value (e.g. “Non-updating item” for StateCodes.NON_UPDATING_ITEM).
toString	Returns a Java String representation for a StateCodes value (e.g. “NON_UPDATING_ITEM” for StateCodes.NON_UPDATING_ITEM).

Table 90: StateCodes Methods

11.2.8 Array

The **Array** is a uniform primitive type that can contain multiple simple primitive entries. An **Array** can contain zero to N primitive type entries³, where zero entries indicates an empty **Array**.

Each **ArrayEntry** can house only simple primitive types such as **Int**, **Real**, or **Date**. An **ArrayEntry** cannot house any container types or other **Array** types. This is a uniform type, where the **Array.primitiveType** method indicates the single, simple primitive type of each entry. **Array** uses simple replacement rules for change management. When new entries are added, or any array entry requires a modification, all entries must be sent with the **Array**. This new **Array** entirely replaces any previously stored or displayed data.

An **ArrayEntry** can be encoded from pre-encoded data or by encoding individual pieces of data as provided. When encoding, the application passes the primitive type (when data is not encoded) or a **Buffer** (containing the pre-encoded primitive).

When decoding, the encoded content of the **ArrayEntry** is available as a **Buffer** by calling the **ArrayEntry.encodedData** method. Further decoding of the entry's content can be skipped by invoking the entry decoder to move to the next **ArrayEntry** or the contents can be further decoded by invoking the specifically contained type's primitive decode function (refer to Section 11.2).

NOTE: Although it can house other primitive types, **Array** is itself considered a primitive type and can be represented as a blank value.

11.2.8.1 Array Methods

METHOD	DESCRIPTION
primitiveType	Using a DataTypes value, primitiveType describes the base primitive type of each entry. Array can only contain simple primitive types and cannot house container types or other Arrays .
itemLength ^a	Sets the expected length of all array entries. - If set to 0, entries are variable length and each encoded entry can have a different length. - If set to a non-zero value, each entry must be the specified length (e.g. sending primitiveType of DataTypes.ASCII_STRING with itemLength set to 3 indicates that each array entry will be a fixed-length three-byte string). When using a fixed length, the application still passes in the base primitive type when encoding (e.g., if encoding fixed length DataTypes.INT types, an Int is passed in regardless of itemLength). When encoding buffer types as fixed length: - Any content that exceeds itemLength will be truncated - Any content that is shorter than itemLength will be padded with the \0 (NULL) character
encodedData	Returns a TransportBuffer that contains all encoded primitive types in the contents (if any). This refers to encoded Array payload and length information. The length information is available via the Buffer.length method.
encodeInit	Begins encoding an Array . This method expects that the Array.primitiveType and Array.itemLength methods have been properly populated. The EncodeIterator specifies the TransportBuffer into which it encodes data. Entries can be encoded after this method returns.

Table 91: Array Structure Members

3. An **Array** currently has a maximum entry count of 65,535. This type has an approximate maximum encoded length of 5 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of an **Array** entry is bound by the maximum encoded length of the primitive types being contained. These limitations can change in subsequent releases.

METHOD	DESCRIPTION
encodeComplete	Completes encoding of an Array. This method expects the same EncodeIterator used with encodeInit and ArrayEntry.encode methods. Set the boolean parameter to: • True if the array encoded successfully and to finish encoding. • False if encoding of any entry failed and to roll back the encoding process to the last successfully-encoded point in the contents. All entries should be encoded before calling encodeComplete.
decode	Begins decoding an Array. This method decodes from the TransportBuffer specified to the DecodeIterator.
isBlank	Returns true if State is blank, otherwise false.
clear	Clears this object, so that you can reuse it. Sets all members to an initial value.  TIP: When decoding, the Array object can be reused without using clear.

Table 91: Array Structure Members (Continued)

a. Only specific types are allowed as fixed-length encodings. **DataTypes.INT** and **DataTypes.UINT** can support one-, two-, four-, or eight-byte fixed lengths. **DataTypes.TIME** supports three- or five-byte fixed lengths. **DataTypes.DATETIME** supports seven- or nine-byte fixed lengths. **DataTypes.ENUM** supports one- or two-byte fixed lengths. **DataTypes.BUFFER**, **DataTypes.ASCII_STRING**, **DataTypes.UTF8_STRING**, and **DataTypes.RMTES_STRING** support any legal length value; see those types for allowable lengths.

11.2.8.2 ArrayEntry Methods

METHOD	DESCRIPTION
encodedData	- When encoding, this method specifies pre-encoded data for an ArrayEntry. Populate a Buffer with pre-encoded data, then call this method with the Buffer. - When decoding, the decode method will populate a Buffer with the encoded primitive type (if any). Call this method without a parameter to return the Buffer containing the encoded primitive type.
encodeBlank	Encodes a blank entry.
encode	Encodes an ArrayEntry. This method expects the same EncodeIterator used with Array.encodeInit. - If encoding from pre-encoded data, specify the Buffer populated with pre-encoded data. - If encoding from a primitive type, specify the primitive type. (e.g. UInt). This method should be called for each entry being encoded. The specified type must match the Array.primitiveType.
decode	Decodes an ArrayEntry and populates an internal Buffer (available via encodedData method) with encoded entry contents. This method expects the same DecodeIterator used with Array.decode. Any contained primitive type's decode method can be called based on Array.primitiveType (e.g. UInt.decode) (refer to Section 11.2). Calling ArrayEntry.decode again will decode and provide the next entry in the Array until no more entries are available.
clear	Clears the object so that you can reuse it. Sets all members to an initial value.  TIP: When decoding, you can reuse the Array object without using clear.

Table 92: ArrayEntry Methods

11.2.8.3 Encoding: Example 1

The following code samples demonstrate how to encode an **Array**. In the first example, the array is set to encode unsigned integer entries, where the entries have a fixed length of two bytes each. The example encodes two array entries. The first entry is encoded from a primitive **UInt** type; the second entry is encoded from a **Buffer** containing a pre-encoded **UInt** type. The example includes error handling for the initial encode method only, and omits additional error handling to simplify the sample code.

```

/* EXAMPLE 1 - Array of fixed length unsigned integer values */
/* populate array structure prior call to Array.encodeInit() */
/* encode unsigned integers in the array */
Array array = CodecFactory.createArray();
ArrayEntry arrayEntry = CodecFactory.createArrayEntry();
array.primitiveType(DataType.UINT);
/* send fixed length values where each uint is 2 bytes */
array.itemLength(2);

/* begin encoding of array - assumes that encIter is already populated with buffer and version
   information, store return value to determine success or failure */
if ((retCode = array.encodeInit(encIter)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Array.encodeInit. Error Text:
        %s\n", error.errorId(), error.sysError(), error.text());
}
else
{
    UInt uInt = CodecFactory.createUInt();
    uInt.value(23456);
    /* array encoding was successful */

    /* encode first entry from a UInt from a primitive type */
    retCode = arrayEntry.encode(encIter, uInt);

    /* encode second entry from a pre-encoded UInt contained in a buffer */
    arrayEntry.encodedData(encUInt);
    retCode = arrayEntry.encode(encIter);
}

/* complete array encoding. If success parameter is true, this will finalize encoding. If
   success parameter is false, this will roll back encoding prior to encodeInit */
retCode = array.encodeComplete(encIter, success);

```

Code Example 20: Array Encoding Example #1

11.2.8.4 Encoding: Example 2

This example demonstrates encoding an **Array** containing ASCII string values. The example includes error handling for the initial encode method only, and omits additional error handling to simplify the sample code.

```

/* EXAMPLE 2 - Array of variable length ASCII string values */
/* populate array structure prior to call to Array.encodeInit() */
/* encode ASCII Strings in the array */
Buffer stringBuffer = CodecFactory.createBuffer();
array.primitiveType(DataTypes.ASCII_STRING);
/* itemLength 0 indicates variable length entries */
array.itemLength(0);

/* begin encoding of array - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
if ((retCode = array.encodeInit(encIter)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Array.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
    stringBuffer.data("ENTRY 1");
    /* array encoding was successful */

    /* encode first entry from a buffer containing an ASCII_STRING primitive type */
    retCode = arrayEntry.encode(encIter, stringBuffer);
}

/* complete array encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = array.encodeComplete(encIter, success);

```

Code Example 21: Array Encoding Example #2

11.2.8.5 Decoding: Example

The following example decodes an **Array** and each of its entries to the primitive value. This sample code assumes the contained primitive type is a **UInt**. Typically an application invokes the specific primitive decoder for the contained type or uses a switch statement to allow for a more generic array entry decoder. This example uses the same **DecodeIterator** when calling the primitive decoder method. An application could optionally use a new **DecodeIterator** by setting the encoded entry buffer on a new iterator. To simplify the example, some error handling is omitted.

```
/* decode into the array structure header */
if ((retCode = array.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* decode each array entry */
    while ((retCode = arrayEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with ArrayEntry.decode. Error Text: %s\n",
                error.errorId(), error.sysError(), error.text());
        }
        else
        {
            /* Decode array entry into primitive type. We can use the same decode iterator, or set
               the encoded entry buffer onto a new iterator */
            retCode = uInt.decode(decIter);
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with Array.decode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
```

Code Example 22: Array Decoding Example

11.2.9 Buffer

Buffer represents some type of user-provided content along with the content's length. **Buffer** can:

- Represent various buffer and string types, such as ASCII, RMTEs, or UTF8 strings.
- Contain encoded data on both container and message header structures.

No validation or enforcement checks are performed on the contents of a **Buffer**. Any desired validation can be performed by the user depending on the specific type of content represented by **Buffer**. Null termination is not required with this type.

Though **Buffers** are typically backed by Java **ByteBuffers**, they can also be backed by Java **Strings**.

- When decoding, the backing **ByteBuffer** is available via the **Buffer.data** method. When accessing backing data, use **Buffer.position** method for the position and **Buffer.length** method for the length, not the position and limit of the backing **ByteBuffer** returned from **Buffer.data**.
- When encoding, it is the user's responsibility to provide a **ByteBuffer** of suitable length to the **Buffer.data** method. LSEG recommends that users pool their **ByteBuffers** for reuse, otherwise it will be garbage collected whenever the reference is lost.

NOTE: If data is backed by a Java **String** and the **Buffer.data** method is called, garbage is created to return a **ByteBuffer**.

Blank buffers are conveyed as a zero-length **Buffer**.

- When decoding, the **Buffer.isBlank** method will return **true** when the **Buffer.length** is **0**.
- When encoding, to specify blank, back the **Buffer** with a zero-length **ByteBuffer** (**ByteBuffer**'s position and limit equal) or call **Buffer.data** method with length of **0**.

11.2.9.1 Methods

Buffer contains the following methods:

STRUCTURE MEMBER	DESCRIPTION
length	The length, in bytes, of the content pointed to by data. After encoding, the length method can be used to get the number of bytes encoded. NOTE: The backing ByteBuffer is initially set along with initial position and length. This method returns the initial length if there was no operation on the backing ByteBuffer that would change the position (such as get or put). If the backing ByteBuffer position has been changed by reading or writing to the buffer, this method returns the change in position (i.e. difference between current position and initial position).
position	Returns the position of the buffer.
isBlank	Returns true if the length is 0 , otherwise false .
data	Returns a Java ByteBuffer that contains some type of content, where the specific type description of the content is provided outside of the Enterprise Transport API via an external source (domain model definition, field dictionary, etc.). • Do not use the position and limit from the ByteBuffer. • Use position and length from the Buffer. NOTE: If data is backed by a Java String, garbage is created to return a ByteBuffer.
data(ByteBuffer)	Sets the Buffer data to the ByteBuffer. Position and length are derived from the ByteBuffer's position and limit.
data(ByteBuffer, position, limit)	Sets the Buffer data to the ByteBuffer. Position and length will be set to the specified position and length.

Table 93: Buffer Methods

STRUCTURE MEMBER	DESCRIPTION
data(String)	Sets the Buffer data to the contents of the String . This Buffer 's position will be set to zero and length will be set to the specified String 's length.
equals	Tests whether the Buffer is equal to another specified Buffer . The two objects are equal if they have the same length and the two sequence of elements are equal. If one buffer is backed by a String and the other buffer is backed by a ByteBuffer , the String will be compared as 8-bit ASCII.
copy	Utility to copy this Buffer's data, starting at this Buffer's position, for this Buffer's length, to a destination. The destination can be another Buffer , a ByteBuffer , or a byte[] (with or without a destination offset). The destination must have adequate space.
encode	Encodes a Buffer .
decode	Decodes a Buffer .
toString	Converts the underlying buffer into a Java String . This should only be called when the Buffer is known to contain ASCII data.  WARNING! Unless the underlying buffer is a String , this method creates garbage.
toHexString	Converts the underlying buffer into a formatted hexadecimal String .  WARNING! This method creates garbage.
clear	Clears this object, so that you can reuse it. Sets the backing ByteBuffer/String to null and position and length to 0.

Table 93: Buffer Methods (Continued)

11.2.9.2 Example

For performance purposes contents are not copied while decoding **Buffer**. This may result in the **Buffer.data** exposing additional encoded contents beyond the **Buffer.position** and **Buffer.length** to be exposed. The user can determine appropriate handling to suit their needs. One option is to display the **Buffer** as an ASCII string using **Buffer.toString** method as illustrated in the following example:

```
/* display only the specified length of Buffer contents */
System.out.println(buffer.toString());
```

Code Example 23: Displaying Contents of an Buffer

11.2.10 RMTEs Decoding

Use special consideration when handling and converting **RmtesBuffers** that contain RMTEs data. This allows for the application of partial content updates, used to efficiently change already received RMTEs content by sending only those portions that need to be changed. For a more detailed description of RMTEs, refer to the *Multilingual Text Encoding Standard Specification*.

The typical process for handling RMTEs content contained in an **RmtesBuffer** involves storing content, applying partial updates, and converting to the desired character set. The Transport API provides several structures and functions to help with this storage and conversion as described in the following sections.

 **WARNING!** RMTEs processing is an expensive procedure that incurs multiple content copies. To avoid unnecessary processing, users should confirm that content providers are actually sending RMTEs prior to using this function. If the content type is not RMTEs, do not use this function^a.

- a. Although the type specified in the field dictionary may indicate RMTEs, the actual content might not be encoded as such. Unless content uses RMTEs encoding, this functionality is not necessary.

11.2.10.1 RmtesCacheBuffer: Structure

The **RmtesCacheBuffer** is a simple structure used to store initial RMTEs content and when applying partial updates. Any character set conversions should be performed on the content stored in the **RmtesCacheBuffer**.

RmtesCacheBuffer includes the following methods:

STRUCTURE MEMBER	DESCRIPTION
length()	Returns the length integer of the content pointed to by data ; it represents the number of bytes used in the cache. For example, if data refers to 100 bytes and nothing is cached, length should be set to 0. If data refers to 100 bytes, and 50 bytes are currently in cache, length should be set to 50.
length(Integer)	Sets the length (in bytes) of the actual data that is cached.
data()	Returns the RMTEs content as a ByteBuffer . The length member should be set to the number of bytes of data in the buffer.
byteData(ByteBuffer)	Sets the ByteBuffer data to that of the input. The length must be set separately for the RmtesCacheBuffer using length(Integer) .
allocatedLength()	Returns the length (in bytes) allocated when creating data . This is typically larger than length to allow for the growth of data when applying future partial updates.
allocatedLength(Integer)	Sets the allocatedLength of the data (in bytes).

Table 94: RmtesCacheBuffer Methods

11.2.10.2 RmtesBuffer: Structure Members

The **RmtesBuffer** is a simple structure used to store RMTEs content. Any character set conversions should be performed on the content stored in the **RmtesBuffer**.

RmtesBuffer includes the following methods:

STRUCTURE MEMBER	DESCRIPTION
allocatedLength()	Returns the length (in bytes) allocated when creating data . This is typically larger than length to allow for the growth of data when applying future partial updates.
allocatedLength(Integer)	Sets the allocatedLength of the data (in bytes).
byteData(ByteBuffer)	Sets the ByteBuffer data to that of the input. The length must be set separately for the RmtesBuffer using length(Integer) .
data()	Returns the RMTEs content as a ByteBuffer . The length member should be set to the number of bytes of data in the buffer.
length()	Returns the length integer of the content pointed to by data ; it represents the number of bytes of RMTEs content.
length(Integer)	Sets the length (in bytes) of the actual data.
toString()	Converts the underlying buffer into a Java String . This should only be called when the RmtesBuffer is known to contain UTF-16 data.
	 WARNING! Unless the underlying buffer is a String , this method creates garbage.

Table 95: RmtesBuffer Methods

11.2.10.3 RmtesDecoder

The **RmtesDecoder** tool manages caching and decoding of data. Its inputs are the **RmtesBuffer** and **RmtesCacheBuffer** to be decoded.

DECODE INTERFACE	DESCRIPTION
RMTEsApplyToCache(Buffer, RmtesCacheBuffer)	Applies the buffer's partial update data to the RmtesCacheBuffer . NOTE: The RmtesCacheBuffer must refer to enough memory for storing and modifying the RMTEs content.
hasPartialRMTEsUpdate(Buffer)	Returns a boolean for whether the buffer contains a partial update command: true: RMTEs content in the buffer contains a partial update command. false: RMTEs content in the buffer does not contain a partial update command.
RMTEsToUCS2(RmtesBuffer, RmtesCacheBuffer)	Converts the given RmtesCacheBuffer into UCS2 Unicode and stores the data into the RmtesBuffer .

Table 96: RmtesDecoder Decode Methods

11.2.10.4 Example: Converting RMTES to UCS-2

The following example illustrates storing and converting RMTES content. This example converts from RMTES to UCS-2 and assumes that:

- The input buffer is populated with RMTES content.
- The allocated size of 100 bytes is sufficient for conversion and storage.

To simplify the example, some error handling is omitted.

```

/* create cache buffer for storing RMTES and applying partial updates */
RmtesCacheBuffer rmtesCache = CodecFactory.createRmtesCacheBuffer(100);
/* create RmtesBuffer to convert into */
RmtesBuffer rmtesBuffer = CodecFactory.createRmtesBuffer(100);
/* create RmtesDecoder used for the decoding process */
RmtesDecoder decoder = CodecFactory.createRmtesDecoder();
/*Our Buffer of data we are converting */
Buffer data = CodecFactory.createBuffer();

/* apply RMTES content to cache, if successful convert to UCS-2 */
if ((retVal = decoder.RMTESApplyToCache(data, rmtesCache)) < CodecReturnCodes.SUCCESS)
{
    /* error while applying to cache */
    System.out.println("Error encountered while applying buffer to RMTES cache. Error code: "
        + CodecReturnCodes.toString(retVal));
}
else if ((retVal = decoder.RMTESToUCS2(rmtesBuffer, rmtesCache)) < CodecReturnCodes.SUCCESS)
{
    /* error when converting */
    System.out.println("Error encountered while converting from RMTES to UCS-2. Error code: "
        + CodecReturnCodes.toString(retVal));
}
else
{
    /* SUCCESS: Conversion was successful - application can now use converted content stored in
        rmtesBuffer */
}

```

Code Example 24: Converting RMTES to UCS-2 Example

11.3 Container Types

Container Types can model more complex data representations and have their contents modified at a more granular level than primitive types. Some container types leverage simple entry replacement when changes occur, while other container types offer entry-specific actions to handle changes to individual entries. The Enterprise Transport API offers several uniform (i.e., homogeneous) container types, meaning that all entries house the same type of data. Additionally, there are several non-uniform (i.e., heterogeneous) container types in which different entries can hold varying types of data.

The **DataTypes** enumeration exposes values that define the type of a container. For example, when a **containerType** is housed in an **Msg**, the message would indicate the **containerType**'s enumerated value. Values ranging from 128 to 224 represent container types. Enterprise Transport API messages and container types can house other Enterprise Transport API container types. Only the **FieldList** and **ElementList** container types can house both primitive types and other container types.

The following table provides a brief description of each container type and its housed entries.

ENUM TYPE NAME	DESCRIPTION	ENTRY TYPE INFORMATION
DataTypes.FIELD_LIST	Container Type: FieldList A highly optimized, non-uniform type, that contains field identifier-value paired entries. fieldId refers to specific name and type information as defined in an external field dictionary (such as RDMFieldDictionary). You can further optimize this type by using set-defined data as described in Section 11.6. For more details on this container, refer to Section 11.3.1.	Entry type is FieldEntry, which can house any DataType, including set-defined data (Section 11.6), base primitive types (Section 11.2), and container types. - If the information and entry being updated contains a primitive type, previously stored or displayed data is replaced. - If the entry contains another container type, action values associated with that type specify how to update the information.
DataTypes.ELEMENT_LIST	Container Type: ElementList A self-describing, non-uniform type, with each entry containing name, dataType, and a value. This type is equivalent to FieldList, but without the optimizations provided through fieldId use. Use of set-defined data allows for further optimization, as discussed in Section 11.6. For more details on this container, refer to Section 11.3.2.	Entry type is ElementEntry, which can house any DataType, including set-defined data (Section 11.6), base primitive types (Section 11.2), and container types. - If the updating information and entry contain a primitive type, any previously stored or displayed data is replaced. - If the entry contains another container type, action values associated with that type specify how to update the information.
DataTypes.MAP	Container Type: Map A container of key-value paired entries. Map is a uniform type, where the base primitive type of each entry's key and the containerType of each entry's payload are specified on the Map. - For more information on base primitive types, refer to Section 11.2. - For more details on this container, refer to Section 11.3.3.	Entry type is MapEntry, which can include only container types, as specified on the Map. Each entry's key is a base primitive type, as specified on the Map. Each entry has an associated action, which informs the user of how to apply the information stored in the entry.

Table 97: Enterprise Transport API Container Types

ENUM TYPE NAME	DESCRIPTION	ENTRY TYPE INFORMATION
DataTypes.SERIES	Container Type: Series A uniform type, where the containerType of each entry is specified on the Series . This container is often used to represent table-based information, where no explicit indexing is present or required. As entries are received, the user should append them to any previously-received entries. For more details on this container, refer to Section 11.3.4.	Entry type is SeriesEntry , which can include only container types, as specified on the Series . SeriesEntry types do not contain explicit actions; though as entries are received, the user should append them to any previously received entries.
DataTypes.VECTOR	Container Type: Vector A container of position index-value paired entries. This container is a uniform type, where the containerType of each entry's payload is specified on the Vector . Each entry's index is represented by an unsigned integer. For more details on this container, refer to Section 11.3.5.	Entry type is VectorEntry , which can house only container types, as specified on the Vector . Each entry's index is an unsigned integer. Each entry has an associated action, which informs the user on how to apply the information stored in the entry.
DataTypes.FILTER_LIST	Container Type: FilterList A non-uniform container of filterId -value paired entries. A filterId corresponds to one of 32 possible bit-value identifiers, typically defined by a domain model specification. FilterId 's can be used to indicate interest or presence of specific entries through the inclusion of the filterId in the message key's filter member. - For more information about the message key, refer to Section 12.1.1.3. - For more details on this container, refer to Section 11.3.6.	Entry type is FilterEntry , which can house only container types. Though the FilterList can specify a containerType , each entry can override this specification to house a different type. Each entry has an associated action, which informs the user of how to apply the information stored in the entry.
DataTypes.MSG	Container Type: Msg Indicates that the contents are another message. This allows the application to house a message within a message or a message within another container's entries. This type is typically used with posting (described in Section 13.9). For more details on message encoding and decoding, refer to 12, Message Package Detailed View.	None
DataTypes.NO_DATA	Container Type: None Indicates there are no contents. - When DataTypes.NO_DATA is housed in a message, the message has no payload. - If DataTypes.NO_DATA is housed in a container type, each container entry has no payload.^a	None
DataTypes.ANSI_PAGE	Container Type: None Indicates that contents are ANSI Page format. Though the Enterprise Transport API does not natively support encoding and decoding for the ANSI Page format, the Enterprise Transport API supports the use of a separate ANSI Page encoder/decoder. For further details, refer to the <i>Enterprise Transport API ANSI Library Manual</i> . For more details on housing non-LSEG Rssl Wire Format types inside of container types, refer to Section 11.3.7.	None

Table 97: Enterprise Transport API Container Types (Continued)

ENUM TYPE NAME	DESCRIPTION	ENTRY TYPE INFORMATION
DataTypes.XML	Container Type: None Indicates that contents are XML-formatted data. Though the Enterprise Transport API does not natively support encoding and decoding XML, the Enterprise Transport API supports the use of a separate XML encoder/decoder. For more details on housing non-LSEG Rssl Wire Format types inside of container types, refer to Section 11.3.7.	None
DataTypes.OPAQUE	Container Type: None Indicates that the contents are opaque and additional details are not provided through the Enterprise Transport API. Any specific information about the concrete type housed in the opaque payload should be defined in the specific domain model associated with the message. For more details on housing non-LSEG Rssl Wire Format types inside of container types, refer to Section 11.3.7.	None

Table 97: Enterprise Transport API Container Types (Continued)

a. A **FilterList** can indicate a type of **DataTypes.NO_DATA**, however an individual **FilterEntry** can override using the entry-specific **containerType**.

11.3.1 FieldList

The **FieldList** is a container of entries (known as **FieldEntry**s) paired by the values of their field identifiers. A **field identifier** (known as a **fieldId**), is a signed, two-byte value that refers to specific name and type information defined by an external field dictionary (e.g., **RDMFieldDictionary**). A field list can contain zero to N^4 entries, where zero indicates an empty field list.

11.3.1.1 Structure Members

FieldList includes the following methods:

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (lags) that indicate the presence of optional field list content. For more information about FieldListFlags values, refer to Section 11.3.1.2. - You can use the following convenient methods to set specific FieldListFlags: applyHasInfo, applyHasSetData, applyHasSetId, applyHasStandardData. - You can use the following convenient methods to check whether specific FieldListFlags are set: checkHasInfo, checkHasSetData, checkHasSetId, checkHasStandardData.
dictionaryId	Sets or gets a two-byte, signed integer (dictionaryId) that refers to the external dictionary family for use when interpreting content in this FieldEntry. The field dictionary contains specific name and type information which correlates to fieldId values present in each FieldEntry. An example of this would be the RDMFieldDictionary, which has a dictionaryId value of 1. If not present, a value of 1 should be assumed. If using the default dictionary (RDMFieldDictionary), dictionaryId is not required and is assumed have an id value of 1. A dictionaryId should be provided as part of the initial refresh message on a stream or on the first refresh message after issuing a CLEAR_CACHE command. A dictionaryId can be changed in two ways. - If a dictionaryId is provided on a refresh message (solicited or unsolicited), the specified dictionary is used across all messages on the stream until a new dictionaryId is provided in a subsequent refresh. This new dictionary is now used for all messages on the stream until another dictionaryId is provided. - If a FieldEntry contains a fieldId of 0, this reserved value indicates a temporary dictionary change. In this situation, this entry's value is the new dictionaryId (encoded / decoded as an Int). When a dictionaryId is changed in this manner, the change is only in effect on the remaining entries in the field list or until another fieldId of 0 is encountered. Any containerTypes housed inside the remaining entries also adopt this temporary dictionary. When the end of the field list is reached, the dictionaryId from the refresh takes precedence once again. dictionaryId values have an allowed range of -16,384 to 16,383.
fieldListNum	Sets or gets the fieldListNum, which is a two-byte, signed integer referring to an external fieldlist template, also known as a record template. The record template contains information about all possible fields in a stream and is typically used by caching implementations to pre-allocate storage. fieldListNum values have an allowed range of -32,768 to 32,767.
setId	Sets or gets a two-byte, unsigned integer (setId) corresponding to the set definition used for encoding or decoding the set-defined data in this FieldList. - When encoding, this is the set definition used to encode any set-defined content. - When decoding, this is the set definition used for decoding any set-defined content. If a setId value is not present on a message containing set-defined data, a setId of 0 is implied. setId values have an allowed range of 0 to 32,767. Currently, only values 0 to 15 are used. These indicate locally-defined set definition use. Refer to Section 11.6 for more information.

Table 98: FieldList Methods

4. A field list currently has a maximum entry count of 65,535, where the first 255 entries may contain set-defined types. This type has an approximate maximum encoded length of 5 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of each field entry has a maximum encoded length of 65,535 bytes. These limitations could be changed in subsequent releases.

METHOD	DESCRIPTION
encodedSetData	Sets or gets encodedSetData , which is a Buffer (with position and length) that contains the encoded set-defined data, if any, contained in the message. If populated, contents are described by the set definition associated with the setId member. If this is populated while encoding, this is assumed to be pre-encoded set data. If this is populated while decoding, this represents encoded set data. For more information, refer to Section 11.6.
encodedEntries	Returns the encodedEntries , which is a Buffer (with position and length) that contains the encoded fieldId -value pair encoded data, if any, contained in the message. This would refer to encoded FieldList payload and length information.
encodeInit	Begins encoding a FieldList. The Enterprise Transport API will encode all content into the Buffer to which the passed in EncodeIterator refers. Entries can be encoded after this method returns. - If you are encoding set-defined data, pass in the set definition database to this method. The Enterprise Transport API will use the specified definition to validate and optimize content while encoding. - To reserve space for encoding, pass in a maximum length hint value (associated with the expected maximum encoded length of set-defined content in this FieldList). If the approximate encoded set data length is not known, you can pass in a value of 0. For more details on local set definitions, refer to Section 11.6.
encodeComplete	Completes the encoding of a FieldList. This method expects the same EncodeIterator that was used with encodeInit and all entries. - If encoding succeeds, a boolean success parameter setting of true finishes the encoding. - If encoding any entry fails, a boolean success parameter setting of false rolls back encoding to the last successfully encoded point in the contents. Encode all field entries prior to this call.
decode	Begins decoding a FieldList from the Buffer referenced in the DecodeIterator. This method allows the user to pass in local set definitions. If the FieldList contains set-defined data (e.g., if the FieldListFlags.HAS_SET_DATA flag is present), the Transport API decodes the set-defined entries when definitions are present. Otherwise, set-defined entries are skipped while decoding entries.
clear	Clears the object so that you can reuse it. When decoding, you can reuse FieldList without needing to call clear.

Table 98: FieldList Methods (Continued)

11.3.1.2 FieldListFlag Values

FIELD LIST FLAG	MEANING
NONE	Indicates that optional flags are not set.
FieldListFlags.HAS_FIELD_LIST_INFO	Indicates that dictionaryId and fieldListNum members are present, which should be provided as part of the initial refresh message on a stream or on the first refresh message after issuance of a CLEAR_CACHE command.
FieldListFlags.HAS_STANDARD_DATA	Indicates that the FieldList contains standard fieldId -value pair encoded data. This value can be set in addition to FieldListFlags.HAS_SET_DATA if both standard and set-defined data are present in this FieldList . If no entries are present in the FieldList , this flag value should not be set.

Table 99: FieldListFlag Values

FIELD LIST FLAG	MEANING
FieldListFlags.HAS_SET_DATA	Indicates that the FieldList contains set-defined data. This value can be set in addition to FieldListFlags.HAS_STANDARD_DATA if both standard and set-defined data are present in this FieldList . If no entries are present in the FieldList , this flag value should not be set. For more information, refer to Section 11.6.
FieldListFlags.HAS_SET_ID	Indicates the presence of a setId , used to determine the set definition used for encoding or decoding the set data on this FieldList . For more information, refer to Section 11.6.

Table 99: FieldListFlag Values (Continued)

11.3.1.3 FieldEntry Methods

Each **FieldEntry** can house any **DataTypes**. This includes primitive types (as described in Section 11.2), set-defined types (as described in Section 11.6), or container types. If updating information, when the **FieldEntry** contains a primitive type, it replaces any previously stored or displayed data associated with the same **fieldId**. If the **FieldEntry** contains another container type, action values associated with that type indicate how to modify the information.

METHOD	DESCRIPTION
fieldId	Sets or gets the signed two-byte value (fieldId) that refers to specific name and type information defined by an external field dictionary, such as the RDMFieldDictionary. Negative fieldId values typically refer to user-defined values while positive fieldId values typically refer to LSEG-defined values. fieldId has an allowable range of -32,768 to 32,767 where LSEG defines positive values and the user defines negative values. A fieldId value of 0 is reserved to indicate dictionaryId changes, where the type of fieldId 0 is an Int.
dataType	Sets or gets the DataTypes of this FieldEntry's contents. - While encoding, dataType must be set to the enumerated value of the type being encoded. - While decoding, if dataType is DataTypes.UNKNOWN, the user must determine the type of contained information from the associated field dictionary. If set-defined data is used, dataType will indicate specific DataTypes information as indicated by the set definition.
encodedData	Sets or gets encodedData, which is a Buffer (with position and length) containing the encoded content of this FieldEntry. - If populated on encode methods, this indicates that data is pre-encoded and encodedData will be copied while encoding. - If populated on decoding functions, this refers to the encoded FieldEntry's payload and length information.
encode(w/primitiveType)	Encodes a FieldEntry with a primitive data type (e.g. UInt). This method expects the same EncodeIterator used with FieldList.encodeInit. You must properly populate FieldEntry.fieldId and FieldEntry.dataType. Call this method for each primitiveType entry being encoded.
encode	Encodes a FieldEntry with pre-encoded data. This method expects the same EncodeIterator used with FieldList.encodeInit. You must properly populate FieldEntry.fieldId and FieldEntry.dataType. Set encodedData with pre-encoded data before calling this method. Call this method for each pre-encoded entry being encoded.
encodeBlank	Encodes a blank FieldEntry. This method expects the same EncodeIterator used with FieldList.encodeInit. Call this method for each blank entry being encoded.

Table 100: FieldEntry Methods

METHOD	DESCRIPTION
encodeInit	Encodes a FieldEntry from a complex type, such as a container type or an array. This method expects the same EncodeIterator used with FieldList.encodeInit. You must properly populate FieldEntry.fieldId and FieldEntry.dataType. To reserve space needed for encoding, you can pass in a maximum-length hint value, associated with the expected maximum-encoded length of this field. If the approximate encoded length is not known, you can pass in a value of 0 which allows the maximum content length. Typical use (e.g. encode an element list as a field entry): 1. Call FieldEntry.encodeInit. 2. Call one or more encoding methods for the complex type using the same buffer. 3. Call FieldEntry.encodeComplete.
encodeComplete	Completes encoding of a FieldEntry for a complex type, such as a container type or an array. This method expects the same EncodeIterator used with FieldList.encodeInit, FieldEntry.encodeInit, and all other entry encoding. • If encoding succeeds, set the boolean success to true to finish entry encoding. • If encoding the entry fails, set the boolean success parameter to false to roll back the encoding of this particular FieldEntry.
decode	Decodes a FieldEntry, expecting the same DecodeIterator used with FieldList.decode and populates encodedData with the entry's encoded contents. • If decoding set-defined entries, the FieldEntry.dataType populates with the type from the set definition. • If decoding standard fieldId-value data, FieldEntry.dataType is set to DataTypes.UNKNOWN, indicating that the user must determine the type from a field dictionary. After determining the type, the specific decode method can be called if needed. Calling FieldEntry.decode again will begin decoding the next entry in the FieldList until no more entries are available.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, FieldEntry can be reused without using clear.

Table 100: FieldEntry Methods (Continued)

11.3.1.4 Rippling

The **FieldList** container supports rippling fields. When *rippling*, newly received content associated with a **fieldId** replaces previously received content associated with the same **fieldId**. The previously-received content is moved to a new **fieldId** (typically indicated in a field dictionary⁵). Rippling is typically used as a way to reduce bandwidth consumption. Normally, if previously-received data were still relevant, it would need to be sent with subsequent updates even though the value was not changing. Rippling allows this data to be removed from subsequent updates; however the consumer must use the ripple information from a field dictionary to correctly propagate previously received content. Rippling is the responsibility of the consumer application, and the Enterprise Transport API does not perform entry rippling.

5. In the Domain Model Field Dictionary, the '**RIPPLES TO**' column defines the **fieldId** information to use when rippling.

11.3.1.5 Encoding Example

The following example illustrates how to encode a **FieldList**. The example encodes four **FieldEntry** values:

- The first encodes an entry from a primitive **Date** type
- The second from a pre-encoded buffer containing an encoded **UInt**
- The third as a blank **Real** value
- The fourth as an **Array** complex type. The pattern followed while encoding the fourth entry can be used for encoding of any container type into a **FieldEntry**.

This example demonstrates error handling for the initial encode method. To simplify the example, additional error handling is omitted (though it should be performed). This example shows encoding of standard **fieldId**-value data.

```
/* populate field list structure prior to call to FieldList.encodeInit()
   NOTE: the fieldId, dictionaryId and fieldListNum values used for this example do not correspond
   to actual id values */

/* indicate that standard data will be encoded and that dictionaryId and fieldListNum are included */
fieldList.applyHasStandardData();
fieldList.applyHasInfo();
/* populate dictionaryId and fieldListNum with info needed to cross-reference fieldIds and cache */
fieldList.dictionaryId(2);
fieldList.fieldListNum(5);

/* begin encoding of field list - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
if ((retCode = fieldList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with FieldList.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
    /* fieldListInit encoding was successful */
    /* create a single FieldEntry and reuse for each entry */
    FieldEntry fieldEntry = CodecFactory.createFieldEntry();
    /* stack allocate a date and populate {day, month, year} */
    com.refinitiv.eta.codec.Date date = CodecFactory.createDate();
    date.month(3);
    date.day(18);
    date.year(2013);

    /* FIRST Field Entry: encode entry from the Date primitive type. Populate and encode field entry
       with fieldId and dataType information for this field */
    fieldEntry.fieldId(16);
    fieldEntry.dataType(DataTypes.DATE);
    retCode = fieldEntry.encode(encIter, date);
}
```

```

/* SECOND Field Entry: encode entry from preencoded buffer containing an encoded UInt type */
/* populate and encode field entry with fieldId and dataType information for this field */
/* because we are re-populating all values on FieldEntry, there is no need to clear it */
fieldEntry.fieldId(1080);
fieldEntry.dataType(DataTypes.UINT);
/* assuming encUInt is a Buffer with length and data properly populated */
fieldEntry.encodedData(encUInt);
/* no data parameter is passed in because pre-encoded data is set on FieldEntry itself */
retCode = fieldEntry.encode(encIter);

/* THIRD Field Entry: encode entry as a blank Real primitive type */
/* populate and encode field entry with fieldId and dataType information for this field */
fieldEntry.fieldId(22);
fieldEntry.dataType(DataTypes.REAL);
retCode = fieldEntry.encodeBlank(encIter);

/* FOURTH Field Entry: encode entry as a complex type, Array primitive */
/* populate and encode field entry with fieldId and dataType information for this field */
/* need to ensure that FieldEntry is appropriately cleared - clearing will ensure that encData
   is properly emptied */
fieldEntry.clear();
fieldEntry.fieldId(1021);
fieldEntry.dataType(DataTypes.ARRAY);
/* begin complex field entry encoding, we are not sure of the approximate max encoding length */
retCode = fieldEntry.encodeInit(encIter, 0);
{
    /* now encode nested container using its own specific encode methods */
    /* encode Real values into the array */
    array.primitiveType(DataTypes.REAL);
    /* values are variable length */
    array.itemLength(0);
    /* begin encoding of array - using same encIterator as field list */
    if ((retCode = array.encodeInit(encIter)) < CodecReturnCodes.SUCCESS)

        /*----- Continue encoding array entries. See example in Section 11.2.8 ----- */

        /* Complete nested container encoding */
        retCode = array.encodeComplete(encIter, success);
}
/* complete encoding of complex field entry. If any array encoding failed, success is false */
retCode = fieldEntry.encodeComplete(encIter, success);
}
/* complete fieldList encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = fieldList.encodeComplete(encIter, success);

```

Code Example 25: FieldList Encoding Example

11.3.1.6 Decoding Example

The following example demonstrates how to decode a **FieldList** and is structured to decode each entry to the contained value. This example uses a switch statement to invoke the specific decoder for the contained type, however to simplify the example, necessary cases and some error handling are omitted. This example uses the same **DecodeIterator** when calling the primitive decoder method. An application could optionally use a new **DecodeIterator** by setting the **encodedData** on a new iterator.

```
/* decode into the field list structure */
if ((retCode = fieldList.decode(decIter, localSetDefs)) >= CodecReturnCodes.SUCCESS)
{
    /* decode each field entry */
    while ((retCode = fieldEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with FieldEntry.decode. Error Text: %s\n",
                error.errorId(), error.sysError(), error.text());
        }
        else
        {
            /* look up type in field dictionary and call correct primitive decode method */
            DictionaryEntry dictionaryEntry = dictionary.entry(fieldEntry.fieldId());
            switch (dictionaryEntry.rwfType())
            {
                case DataTypes.REAL:
                    retCode = real.decode(decIter);
                    break;
                case DataTypes.DATE:
                    retCode = date.decode(decIter);
                    break;
                /* full switch statement omitted to shorten sample code */
            }
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with FieldList.decode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
```

Code Example 26: FieldList Decoding Example

11.3.2 ElementList

ElementList is a self-describing container type. Each entry, known as an **ElementEntry**, contains an element **name**, **dataType** enumeration, and value. An element list is equivalent to **FieldList**, where name and type information is present in each element entry instead of optimized via a field dictionary. An element list can contain zero to N^6 entries, where zero indicates an empty element list.

11.3.2.1 Structure Members

METHOD	DESCRIPTION
flags	Sets or gets flags, which is a combination of bit values that indicate whether optional, element-list content is present. For more information about ElementListFlags values, refer to Section 11.3.2.2. - You can use the following convenient methods to set specific ElementListFlags: applyHasInfo, applyHasSetData, applyHasSetId, applyHasStandardData. - You can use the following convenient methods to check whether specific ElementListFlags are set: checkHasInfo, checkHasSetData, checkHasSetId, checkHasStandardData.
elementListNum	Sets or gets a two-byte signed integer (elementListNum) that refers to an external element-list template, also known as a record template. A record template contains information about all possible entries contained in the stream and is typically used by caching mechanisms to pre-allocate storage. elementListNum values have a range of -32,768 to 32,767.
setId	Sets or gets a two-byte unsigned integer (setId) that corresponds to the set definition used for encoding or decoding the set-defined data in this ElementList. - When encoding, this is the set definition used to encode any set-defined content. - When decoding, this is the set definition used for decoding any set-defined content. setId values have an allowed range of 0 to 32,767. Currently, only values 0 to 15 are used. These indicate locally-defined set definition use. If a setId value is not present on a message containing set-defined data, a setId of 0 is implied. For more information, refer to Section 11.6.
encodedSetData	Sets or gets the encoded set-defined data (encodedSetData), which is a Buffer (with position and length) containing the encoded set-defined data, if any, contained in the message. If populated, contents are described by the set definition associated with the setId member. - If this is populated while encoding, this is assumed to be pre-encoded set data. - If this is populated while decoding, this represents encoded set data. For more information, refer to Section 11.6.
encodedEntries	Returns encodedEntries, which is a Buffer (with position and length) that contains all encoded element name, dataType, value encoded data, if any, contained in the message. This would refer to encoded ElementList payload and length information.

Table 101: ElementList Methods

6. An element list currently has a maximum entry count of 65,535, where the first 255 entries may contain set-defined types. This type has an approximate maximum encoded length of 5 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of element entry has a maximum encoded length of 65,535 bytes. These limitations can change in subsequent releases.

METHOD	DESCRIPTION
encodeInit	Starts encoding an ElementList. The Transport API encodes data into the TransportBuffer referred to by the EncodeIterator. Entries can be encoded after this method returns. - If encoding set-defined data, pass the set definition database into this method. The Transport API uses the specified definition to validate and optimize content while encoding. - You can reserve space for encoding by passing in a maximum-length hint value (associated with the expected maximum-encoded length of set-defined content in this ElementList). If the approximate length of encoded set data is not known, you can pass in a value of 0. For more details on local set definitions, refer to Section 11.6.
encodeComplete	Completes the encoding of an ElementList. This method expects the same EncodeIterator used with ElementList.encodeInit and all entries. - If all entries were encoded successfully, a boolean success parameter setting of true finishes encoding. - If encoding of any entry failed, a boolean success parameter setting of false rolls back the encoding process to the last successfully encoded point in the contents. Encode any element entries prior to this call.
decode	Starts decoding an ElementList. This method will decode from the Buffer referred to by the passed-in DecodeIterator and allows the user to pass in local set definitions. If the ElementList contains set-defined data (e.g., ElementListFlags.HAS_SET_DATA is present), the Transport API will decode set-defined entries when their definitions are present. Otherwise, the Transport API skips set-defined entries when decoding entries.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse ElementList without needing to call clear.

Table 101: ElementList Methods (Continued)

11.3.2.2 ElementListFlags Values

ELEMENTLISTFLAG VALUES	MEANING
NONE	Indicates that optional flags are not set.
ElementListFlags.HAS_ELEMENT_LIST_INFO	Indicates the presence of the elementListNum member. This member is provided as part of the initial refresh message on a stream or on the first refresh message after a CLEAR_CACHE command.
ElementListFlags.HAS_STANDARD_DATA	Indicates that the ElementList contains standard element name , dataType , value-encoded data. You can set this value in addition to ElementListFlags.HAS_SET_DATA if both standard and set-defined data are present in this ElementList . If the ElementList does not have entries, do not set this flag value.

Table 102: ElementListFlags Flags Values

ELEMENTLISTFLAG VALUES	MEANING
ElementListFlags.HAS_SET_DATA	Indicates that ElementList contains set-defined data. - If both standard and set-defined data are present in this ElementList, this value can be set in addition to ElementListFlags.HAS_STANDARD_DATA. - If the ElementList does not have entries, do not set this flag value. For more information, refer to Section 11.6.
ElementListFlags.HAS_SET_ID	Indicates the presence of a setId and determines the set definition to use when encoding or decoding set data on this ElementList. For more information, refer to Section 11.6.

Table 102: ElementListFlags Flags Values (Continued)

11.3.2.3 ElementEntry Methods

Each **ElementList** can contain multiple **ElementEntry**s and each **ElementEntry** can house any **DataTypes**, including primitive types (refer to Section 11.2), set-defined types (refer to Section 11.6), or container types. If an **ElementEntry** is a part of updating information and contains a primitive type, any previously stored or displayed data is replaced. If an **ElementEntry** contains another container type, action values associated with that type indicate how to modify data.

METHOD	DESCRIPTION
name	Sets or gets a Buffer containing the name associated with this ElementEntry. Element names are defined outside of the Enterprise Transport API, typically as part of a domain model specification or dictionary. A name can be empty; however this provides no identifying information for the element. The name buffer allows for content length ranging from 0 bytes to 32,767 bytes.
dataType	Sets or gets the dataType, which defines the DataTypes of this ElementEntry's contents. - While encoding, set this to the enumerated value of the target type. - While decoding, dataType describes the type of contained data so that the correct decoder can be used. If set-defined data is used, dataType will indicate any specific DataTypes information as defined in the set definition.
encodedData	Sets or gets the encoded content (encodedData) of this ElementEntry. If populated on encode methods, this indicates that data is pre-encoded and encodedData copies while encoding. While decoding, this refers to the encoded ElementEntry's payload and length data.
encode(w/ primitiveType)	Encodes an ElementEntry with a primitive data type (e.g. UInt). This method expects the same EncodeIterator used with ElementList.encodeInit. ElementEntry.name and ElementEntry.dataType must be properly populated. Call this method for each primitiveType entry that you want to encode.
encode	Encodes an ElementEntry with pre-encoded data. This method expects the same EncodeIterator used with ElementList.encodeInit. You must properly populate ElementEntry.name and ElementEntry.dataType and also set encodedData with pre-encoded data before calling this method. Call this method for each pre-encoded entry that you want to encode.
encodeBlank	Encodes a blank ElementEntry. This method expects the same EncodeIterator used with ElementList.encodeInit. You must properly populate ElementEntry.name and ElementEntry.dataType. Call this method for each blank entry that you want to encode.

Table 103: ElementEntry Methods

METHOD	DESCRIPTION
encodeInit	Encodes an ElementEntry from a complex type, such as a container type or an array. This method expects the same EncodeIterator used with ElementList.encodeInit. You must properly populate ElementEntry.name and ElementEntry.dataType. To reserve the appropriate amount of space while encoding, you can pass in a max-length hint value (associated with the expected maximum-encoded length of this element) to this method. If the approximate encoded length is not known, you can pass in a value of 0. Typical use (e.g. encode an element list as a field entry): 1. Call ElementEntry.encodeInit. 2. Call one or more encoding methods for the complex type using the same buffer. 3. Call ElementEntry.encodeComplete.
encodeComplete	Completes the encoding of an ElementEntry. This method expects the same EncodeIterator used with ElementList.encodeInit, ElementEntry.encodeInit, and all other entry encoding. • If this specific entry is encoded successfully, a boolean success parameter setting of true finishes entry encoding. • If this specific entry fails to encode, a boolean success parameter setting of false rolls back the encoding of only this ElementEntry.
decode	Decodes an ElementEntry. This method expects the same DecodeIterator used with ElementList.decode and populates encodedData with encoded entry contents. After this method returns, you can use the ElementEntry.dataType to invoke the correct contained type's decode methods. Calling ElementEntry.decode again starts decoding the next entry in the ElementList until no more entries are available.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse ElementEntry without using clear.

Table 103: ElementEntry Methods (Continued)

11.3.2.4 ElementList Encoding Example

The following example demonstrates how to encode an **ElementList** and encodes four **ElementEntry** values:

- The first encodes an entry from a primitive **Time** type
- The second encodes from a pre-encoded buffer containing an encoded **UInt**
- The third encodes as a blank **Real** value
- The fourth encodes as a **FieldList** container type

The pattern used to encode the fourth entry can be used to encode any container type into an **ElementEntry**. This example demonstrates error handling for the initial encode method. However, additional error handling is omitted to simplify the example. This example shows the encoding of standard **name**, **dataType**, and **value** data.

```
/* populate element list structure prior to call to ElementList.encodeInit() */
/* NOTE: the element names and elementListNum values used for this example may not correspond to actual
   name values */
/* indicate that standard data will be encoded and that elementListNum is included */
elemList.applyHasStandardData();
elemList.checkHasInfo();
```

```

/* populate elementListNum with info needed to cache */
elemList.elementListNum(5);

/* begin encoding of element list - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
if ((retCode = elemList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with ElementList.encodeInit. Error Text:
        %s\n", error.errorId(), error.sysError(), error.text());
}
else
{
    /* elementListInit encoding was successful */
    /* create a single ElementEntry and reuse for each entry */
    ElementEntry elemEntry = CodecFactory.createElementEntry();
    /* stack allocate a time and populate {hour, minute, second, millisecond} */
    Time time = CodecFactory.createTime();
    time.hour(10);
    time.minute(21);
    time.second(16);
    time.millisecond(777);
    Buffer elementEntryName = CodecFactory.createBuffer();

    /* FIRST Element Entry: encode entry from the Time primitive type */
    /* populate and encode element entry with name and dataType information for this element */
    elementEntryName.data("Element1 - Primitive");
    elemEntry.name(elementEntryName);
    elemEntry.dataType(DataTypes.TIME);
    retCode = elemEntry.encode(encIter, time);

    /* SECOND Element Entry: encode entry from preencoded buffer containing an encoded UInt type */
    /* populate and encode element entry with name and dataType information for this element */
    /* because we are re-populating all values on ElementEntry, there is no need to clear it */
    elementEntryName.data("Element2 - Pre-Encoded");
    elemEntry.name(elementEntryName);
    elemEntry.dataType(DataTypes.UINT);
    /* assuming encUInt is a Buffer with length and data properly populated */
    elemEntry.encodedData(encUInt);
    /* no data parameter is passed in because pre-encoded data is set on ElementEntry itself */
    retCode = elemEntry.encode(encIter);

    /* THIRD Element Entry: encode entry as a blank Real primitive type */
    /* populate and encode element entry with name and dataType information for this element */
    elementEntryName.data("Element3 - Blank");
    elemEntry.name(elementEntryName);
    elemEntry.dataType(DataTypes.REAL);
    retCode = elemEntry.encodeBlank(encIter);
}

```

```

/* FOURTH Element Entry: encode entry as a container type, FieldList */
/* populate and encode element entry with name and dataType information for this element */
/* need to ensure that ElementEntry is appropriately cleared - clearing will ensure that encData
   is properly emptied */
elemEntry.clear();
elementEntryName.data("Element4 - Container");
elemEntry.name(elementEntryName);
elemEntry.dataType(DataTypes.FIELD_LIST);
/* begin complex element entry encoding, we are not sure of the approximate max encoding length */
retCode = elemEntry.encodeInit(encIter, 0);
{
    /* now encode nested container using its own specific encode methods */
    /* begin encoding of field list - using same encIterator as element list */
    fieldList.applyHasStandardData();

    if ((retCode = fieldList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)

        /*----- Continue encoding field entries. See example in Section Section 11.3.1 ---- */

        /* Complete nested container encoding */
        retCode = fieldList.encodeComplete(encIter, success);
}
/* complete encoding of complex element entry. If any field list encoding failed, success is false */
retCode = elemEntry.encodeComplete(encIter, success);
}
/* complete elementList encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = elemList.encodeComplete(encIter, success);

```

Code Example 27: ElementList Encoding Example

11.3.2.5 ElementList Decoding Examples

The following sample demonstrates how to decode an **ElementList** and is structured to decode each entry to its contained value. This example uses a switch statement to invoke the specific decoder for the contained type, however for sample clarity, unnecessary cases have been omitted. This example uses the same **DecodeIterator** when calling the primitive decoder method. An application could optionally use a new **DecodeIterator** by setting the **encodedData** on a new iterator. For simplification, the example omits some error handling.

```

/* decode into the element list structure */
if ((retCode = elemList.decode(decIter, localSetDefs)) >= CodecReturnCodes.SUCCESS)
{
    /* decode each element entry */
    while ((retCode = elemEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with ElementEntry.decode. Error
                Text: %s\n", error.errorId(), error.sysError(), error.text());
        }
        else
        {
            /* use elemEntry.dataType to call correct primitive decode method */
            switch (elemEntry.dataType())
            {
                case DataTypes.REAL:
                    retCode = real.decode(decIter);
                    break;
                case DataTypes.TIME:
                    retCode = time.decode(decIter);
                    break;
                /* full switch statement omitted to shorten sample code */
            }
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with ElementList.decode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}

```

Code Example 28: ElementList Decoding Example

11.3.3 Map

The **Map** is a uniform container type of associated key-value pair entries. Each entry, known as a **MapEntry**, contains an entry key, which is a base primitive type (Section 11.2) and value. A **Map** can contain zero to N^7 entries, where zero entries indicate an empty **Map**.

11.3.3.1 Map Methods

A **Map** structure contains the following Methods:

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) to indicate the presence of optional Map content. For more information about MapFlags values, refer to Section 11.3.3.2. - You can use the following convenient methods to set specific MapFlags: applyHasKeyFieldId, applyHasPerEntryPermData, applyHasSetDefs, applyHasSummaryData, applyHasTotalCountHint. - You can use the following convenient methods to check whether specific MapFlags are set: checkHasKeyFieldId, checkHasPerEntryPermData, checkHasSetDefs, checkHasSummaryData, checkHasTotalCountHint.
keyPrimitiveType	Sets or gets the value (keyPrimitiveType) that describes the base primitive type of each MapEntry's key. keyPrimitiveType accepts primitive DataTypes (values between 1 and 63), cannot be specified as blank, and cannot be the DataTypes.ARRAY or DataTypes.UNKNOWN primitive types. For more information about base primitive types, refer to Section 11.2.
keyFieldId	(Optional) Sets or gets a fieldId associated with the entry key information. This is mainly used as an optimization to avoid inclusion of redundant data. In situations where key information is also a member of the entry payload (e.g., Order Id for Market By Order domain type), this allows removal of data from each entry's payload prior to encoding as it is already present via the key and keyFieldId. keyFieldId has an allowable range of -32,768 to 32,767 where positive values are LSEG-defined and negative values are user-defined.
containerType	Sets or gets the value (DataTypes) that describes the container type of each MapEntry 's payload.
totalCountHint	Sets or gets a four-byte unsigned integer (totalCountHint) that indicates an approximate total number of entries associated with this stream. This is typically used when multiple Map containers are spread across multiple parts of a refresh message (for more information about message fragmentation and multi-part message handling, refer to Section 13.1). totalCountHint provides an approximation of the total number of entries sent across all maps on all parts of the refresh message. This information is useful when determining the amount of resources to allocate for caching or displaying all expected entries. totalCountHint values have a range of 0 to 1,073,741,824.

Table 104: Map Methods

7. A **Map** currently has a maximum entry count of 65,535. This type has an approximate maximum encoded length of 5 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of a **MapEntry** has a maximum encoded length of 65,535 bytes. These limitations could be changed in subsequent releases.

METHOD	DESCRIPTION
encodedSummaryData	Sets or gets the encodedSummaryData, which is a TransportBuffer (with position and length) that contains the encoded summary data, if any, contained in the message. If populated, summary data contains information that applies to every entry encoded in the Map (e.g., currency type). The container type of summary data should match the containerType specified on the Map. If encodedSummaryData is populated while encoding, contents are used as pre-encoded summary data. Encoded summary data has maximum allowed length of 32,767 bytes. For more information, refer to Section 11.5.
encodedSetDef	Sets or gets the encodedSetDef, which is a Buffer (with position and length) that contains the encoded local set definitions, if any, contained in the message. If populated, these definitions correspond to data contained within the Map's entries and are used for encoding or decoding their contents. Encoded local set definitions have a maximum allowed length of 32,767 bytes. For more information, refer to Section 11.6.
encodedEntries	Returns the encodedEntries, which is a Buffer (with position and length) that contains the length and pointer to the all encoded key-value pair data, if any, contained in the message. This would refer to encoded Map payload and length information.
EncodeInit	Begins encoding a Map which can include summary data (Section 11.5) and Local Set Definitions (Section 11.6). - If summary data and set definitions are pre-encoded, you can populate them on the encodedSummaryData and encodedSetDef prior to calling Map.EncodeInit. Additional work is not needed to complete encoding this content. - If summary data and set definitions are not pre-encoded, Map.EncodeInit performs the Init for these values. You must call the corresponding Complete method after this content is encoded. - You can reserve the appropriate amount of space while encoding by passing in summary data and set definition encoded length hint values to this method. If either is not being encoded or the approximate encoded length is unknown, you can pass in a value of 0. This is required only when content is not pre-encoded.
EncodeComplete	Completes the encoding of a Map. This method expects the same EncodeIterator that was used with Map.EncodeInit, any summary data, set data, and all entries. - If encoding was successful, the boolean success parameter should be set to true to finish encoding. - If any component failed to encode, the boolean success parameter should be set to false which rolls back the encoding process to the last previously successful encoded point in the contents. Encode all map content prior to this call.
EncodeSummaryDataComplete	Completes encoding of any non-pre-encoded Map summary data. If MapFlags.HAS_SUMMARY_DATA is set and EncodeSummaryData is not populated, summary data is expected after Map.EncodeInit or Map.EncodeSetDefsComplete returns. This method expects the same EncodeIterator used with previous map encoding methods. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If encoding fails, the boolean success parameter should be set to false to roll back to the last previously successful encoded point in the contents. If both MapFlags.HAS_SUMMARY_DATA and MapFlags.HAS_SET_DEFS are present, then set definitions are expected first, and summary data is encoded after the call to Map.EncodeSetDefsComplete.

Table 104: Map Methods (Continued)

METHOD	DESCRIPTION
EncodeSetDefsComplete	Completes encoding of any non pre-encoded local set definition data. If MapFlags.HAS_SET_DEFS is set and encodedSetDef is not populated, local set definition data is expected after Map.EncodeInit returns. This method expects the same EncodeIterator used with Map.EncodeInit. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If encoding fails, the boolean success parameter should be set to false to roll back to the last previously successful encoded point in the contents. If both MapFlags.HAS_SUMMARY_DATA and MapFlags.HAS_SET_DEFS are present, set definitions are expected first, while any summary data is encoded after the call to Map.EncodeSetDefsComplete.
decode	Begins decoding a Map . This method will decode from the Buffer to which the passed-in DecodeIterator refers.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse Map without using clear.

Table 104: Map Methods (Continued)

11.3.3.2 MapFlags Values

FLAG ENUMERATION	MEANING
MapFlags.HAS_KEY_FIELD_ID	Indicates the presence of the keyFieldId member. keyFieldId should be provided if the key information is also a field that would be contained in the entry payload. This optimization allows keyFieldId to be included once instead of in every entry's payload.
MapFlags.HAS_TOTAL_COUNT_HINT	Indicates the presence of the totalCountHint member. This member can provide an approximation of the total number of entries sent across all maps on all parts of the refresh message. This information is useful when determining the amount of resources to allocate for caching or displaying all expected entries.
MapFlags.HAS_PER_ENTRY_PERM_DATA	Indicates that permission information is included with some map entries. The Map encoding functionality sets this flag value on the user's behalf if any entry is encoded with its own permData . A decoding application can check this flag to determine if any contained entry has permData , often useful for fan out devices (if an entry does not have permData , the fan out device can likely pass on data and not worry about special permissioning for the entry). Each entry will also indicate the presence of permission data via the use of MapEntryFlag.HAS_PERM_DATA .
MapFlags.HAS_SUMMARY_DATA	Indicates that the Map contains summary data. If this flag is set while encoding, summary data must be provided by encoding or populating encodedSummaryData with pre-encoded information. If this flag is set while decoding, summary data is contained as part of the Map and the user can choose whether to decode it.
MapFlags.HAS_SET_DEFS	Indicates that the Map contains local set definition information. Local set definitions correspond to data contained within this Map 's entries and are used for encoding or decoding their contents. For more information, refer to Section 11.6.

Table 105: MapFlags Values

11.3.3.3 MapEntry Methods

MapEntrys can house only other container types. **Map** is a uniform type, where the **Map.containerType** indicates the single type housed in each entry. Each entry has an associated action which informs the user of how to apply the information contained in the entry.

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values to indicate the presence of any optional MapEntry content. For more information about MapEntryFlags values, refer to Table 11.3.3.4. - You can use the following convenient methods to set specific MapEntryFlags: applyHasPermData. - You can use the following convenient methods to check whether specific MapEntryFlags are set: checkHasPermData.
action	Sets or gets the entry action which helps to manage change processing rules and tells the consumer how to apply the information contained in the entry. For specific information about possible action's associated with a MapEntry, refer to Table 11.3.3.5.
encodedKey	Sets or gets the encodedKey, which is a Buffer (with position and length) that contains the encoded map entry key information. The encoded type of the key corresponds to the Map's keyPrimitiveType. The key value must be a base primitive type and cannot be blank, DataTypes.ARRAY, or DataTypes.UNKNOWN primitive types. If populated on encode functions, this indicates that the key is pre-encoded and encodedKey will be copied while encoding. While decoding, this would contain only this encoded MapEntry key's payload and length information.
permData	(Optional) Sets or gets authorization information for this specific entry. If present, MapEntryFlag.HAS_PERM_DATA should be set. permData has a maximum allowed length of 32,767 bytes. - For more information on permissioning, refer to Section 11.4. - For more information about MapEntryFlags values, refer to Table 11.3.3.4.
encodedData	Sets or gets encodedData, which is a Buffer (with position and length) that contains the encoded content of this MapEntry. If populated on encode methods, this indicates that data is pre-encoded, and encodedData will be copied while encoding. While decoding, this would refer to this encoded MapEntry's payload and length information.MapEntryFlag.
Encode(w/primitiveType)	Encodes a MapEntry with a primitive data type (e.g. UInt). This method expects the same EncodeIterator used with Map.EncodeInit and is called after Map.EncodeInit and after completing any summary data and local set definition data encoding. Call this method for each primitiveType entry you want to encode.
Encode	Encodes a MapEntry from pre-encoded data. This method expects the same EncodeIterator used with Map.EncodeInit. You must set the pre-encoded map entry payload via the MapEntry.encodedData method prior to calling this method. This method is called after Map.EncodeInit and after completing any summary data and local set definition data encoding. Call this method for each pre-encoded entry you want to encode.

Table 106: MapEntry Methods

METHOD	DESCRIPTION
EncodeInit(w/keyPrimitiveType)	Encodes a MapEntry from a container type. This method expects the same EncodeIterator used with Map.EncodeInit. After this call, you can use housed-type encode methods to encode contained types. The keyPrimitiveType accepts primitive DataTypes (values between 1 and 63), cannot be specified as blank and cannot be the DataTypes.ARRAY or DataTypes.UNKNOWN primitive types. For more information about base primitive types, refer to Section 11.2. You call this method after Map.EncodeInit and after encoding any summary data and local set definition data. To reserve the appropriate amount of space for encoding, you can pass in a max-length hint value, associated with the expected maximum encoded length of this entry. If the approximate encoded length is unknown, you can pass in a value of 0.
EncodeInit	Encodes a MapEntry with pre-encoded primitive key. This method expects the same EncodeIterator used with Map.EncodeInit. After this call, you can use housed-type encode methods to encode contained types. Call this method after Map.EncodeInit and after encoding any summary data and local set definition data. To reserve the appropriate amount of space for encoding, you can pass in a max-length hint value, associated with the expected maximum encoded length of this entry. If the approximate encoded length is unknown, you can pass in a value of 0. Set Map.encodedKey with pre-encoded data before calling this method.
EncodeComplete	Completes the encoding of a MapEntry. This method expects the same EncodeIterator used with Map.EncodeInit, MapEntry.EncodeInit, and all other encoding for this container. - If this specific map entry is encoded successfully, the boolean success parameter should be set to true to finish entry encoding. - If this specific entry fails to encode, the boolean success parameter should be set to false to roll back the encoding of only this MapEntry.
decode(keyData)	Decodes a MapEntry and can optionally decode the MapEntry.encodedKey. This method expects the same DecodeIterator used with Map.decode. This populates encodedData with encoded entry contents and encodedKey with the encoded entry key. After this method returns, you can use the Map.containerType to invoke the correct contained-type's decode methods. Calling MapEntry.decode again continues the decoding of the next entry in the Map until no more entries are available. keyData can be any valid keyPrimitiveType primitive (e.g. UInt). If keyData is non NULL, the entry key will also be decoded into the specified keyData. As entries are received, the action dictates how to apply contents.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse MapEntry without using clear.

Table 106: MapEntry Methods (Continued)

11.3.3.4 MapEntry Flag Enumeration Value

FLAG ENUMERATION	MEANING
MapEntryFlag.HAS_PERM_DATA	Indicates that the container entry includes a permData member and also specifies any authorization information for this entry. For more information, refer to Section 11.4.

Table 107: MapEntryFlags Values

11.3.3.5 MapEntry Action Enumeration Values

ACTION ENUMERATION	MEANING
ADD	Indicates that the consumer should add the entry. An add action typically occurs when an entry is initially provided. It is possible for multiple add actions to occur for the same entry. If this occurs, any previously received data associated with the entry should be replaced with the newly added information.
UPDATE	Indicates that the consumer should update any previously stored or displayed information with the contents of this entry. An update action typically occurs when an entry has already been added and changes to the contents need to be conveyed. If an update action occurs prior to the add action for the same entry, the update action should be ignored.
DELETE	Indicates that the consumer should remove any stored or displayed information associated with the entry. No map entry payload is included when the action is delete.

Table 108: MapEntryActions Values

11.3.3.6 MapEntry Encoding Example

The following sample illustrates the encoding of a **Map** containing **FieldList** values. The example encodes three **MapEntry** values as well as summary data:

- The first entry is encoded with an update action type and a passed in key value.
- The second entry is encoded with an add action type, pre-encoded data, and pre-encoded key.
- The third entry is encoded with a delete action type.

This example also demonstrates error handling for the initial encode method. To simplify the example, additional error handling is omitted, though it should be performed.

```

/* populate map structure prior to call to Map.encodeInit() */
/* NOTE: the key names used for this example may not correspond to actual name values */

/* indicate that summary data and a total count hint will be encoded */
map.applyHasSummaryData();
map.applyHasTotalCountHint();
/* populate maps keyPrimitiveType and containerType */
map.containerType(DataTypes.FIELD_LIST);
map.keyPrimitiveType(DataTypes.UINT);
/* populate total count hint with approximate expected entry count */
map.totalCountHint(3);

/* begin encoding of map - assumes that encIter is already populated with buffer and version information,
   store return value to determine success or failure */
/* expect summary data of approx. 100 bytes, no set definition data */
if ((retCode = map.encodeInit(encIter, 100, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Map.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}

```

```

}
else
{
    /* mapInit encoding was successful */
    /* create a single MapEntry and FieldList and reuse for each entry */
    UInt entryKeyUInt = CodecFactory.createUInt();

    /* encode expected summary data, init for this was done by Map.encodeInit - this type should
       match map.containerType */
    {
        /* now encode nested container using its own specific encode methods */
        /* begin encoding of field list - using same encIterator as map list */
        fieldList.applyHasStandardData();

        if ((retCode = fieldList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)

            /*----- Continue encoding field entries. See example in Section 11.3.1.5 ----- */

            /* Complete nested container encoding */
            retCode = fieldList.encodeComplete(encIter, success);
    }
    /* complete encoding of summary data. If any field list encoding failed, success is false */
    retCode = map.encodeSummaryDataComplete(encIter, success);

    /* FIRST Map Entry: encode entry from non pre-encoded data and key. Approx. encoded length unknown */
    mapEntry.action(MapEntryActions.UPDATE);
    entryKeyUInt.value(1);
    retCode = mapEntry.encodeInit(encIter, entryKeyUInt, 0);
    /* encode contained field list - this type should match map.containerType */
    {
        /* now encode nested container using its own specific encode methods */
        /* clear, then begin encoding of field list - using same encIterator as map */
        fieldList.clear();
        fieldList.applyHasStandardData();

        if ((retCode = fieldList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)

            /*----- Continue encoding field entries. See example in Section 11.3.1.5 ----- */

            /* Complete nested container encoding */
            retCode = fieldList.encodeComplete(encIter, success);
    }
    retCode = mapEntry.encodeComplete(encIter, success);

    /* SECOND Map Entry: encode entry from pre-encoded buffer containing an encoded FieldList */
    /* because we are re-populating all values on MapEntry, there is no need to clear it */
    mapEntry.action(MapEntryActions.ADD);
    /* assuming encUInt Buffer contains the pre-encoded key with length and data properly populated */
    mapEntry.encodedKey(encUInt);
    /* assuming encFieldList Buffer contains the pre-encoded payload with data and length populated */

```

```

mapEntry.encodedData(encFieldList);

/* no keyData parameter is passed in because pre-encoded key is set on MapEntry itself */
retCode = mapEntry.encode(encIter);

/* THIRD Map Entry: encode entry with delete action. Delete actions have no payload */
/* need to ensure that MapEntry is appropriately cleared - clearing will ensure that encData and
   encKey are properly emptied */
mapEntry.clear();
mapEntry.action(MapEntryActions.DELETE);
entryKeyUInt.value(3);
/* entryKeyUInt parameter is passed in for key primitive value. encodedData is empty for delete */
retCode = mapEntry.encode(encIter, entryKeyUInt);
}
/* complete map encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = map.encodeComplete(encIter, success);

```

Code Example 29: MapEntry Encoding Example

11.3.3.7 MapEntry Decoding Example

The following sample demonstrates the decoding of a **Map** and is structured to decode each entry to the contained value. This sample assumes that the housed container type is a **FieldList** and that the **keyPrimitiveType** is **DataTypes.INT**. This sample also uses the **MapEntry.decode** method to perform key decoding. Typically an application would invoke the specific container-type decoder for the housed type or use a switch statement to allow for a more generic map entry decoder. This example uses the same **DecodeIterator** when calling the content's decoder method. An application could optionally use a new **DecodeIterator** by setting the **encodedData** on a new iterator. To simplify the sample, some error handling is omitted.

```
/* decode contents into the map structure */
if ((retCode = map.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* create primitive value to have key decoded into and a single map entry to reuse */
    Int tempInt = CodecFactory.createInt();

    /* if summary data is present, invoking decoder for that type (instead of DecodeEntry)
       indicates to ETA that user wants to decode summary data */
    if (map.checkHasSummaryData())
    {
        /* summary data is present. Its type should be that of map.containerType */
        retCode = fieldList.decode(decIter, null);
        /* Continue decoding field entries. See example in Section 11.3.1.6 */
    }

    /* decode each map entry, passing keyPrimitiveType decodes mapEntry key as well */
    while ((retCode = mapEntry.decode(decIter, tempInt)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with MapEntry.decode. Error Text:
                               %s\n", error.errorId(), error.sysError(), error.text());
        }
        else
        {
            retCode = fieldList.decode(decIter, null);
            /* Continue decoding field entries. See example in Section 11.3.1.6 */
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with Map.decode. Error Text: %s\n",
                      error.errorId(), error.sysError(), error.text());
}
```

Code Example 30: Map Decoding Example

11.3.4 Series

The **Series** is a uniform container type. Each entry, known as an **SeriesEntry**, contains only encoded data. This container is often used to represent table-based information, where no explicit indexing is present or required. A **Series** can contain zero to N^8 entries, where zero entries indicates an empty **Series**.

11.3.4.1 Series Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicates the presence of optional Series content. For more information about flag values, refer to Section 11.3.4.2. - You can use the following convenient methods to set specific SeriesFlags: applyHasSetDefs, applyHasSummaryData, applyHasTotalCountHint. - You can use the following convenient methods to check whether specific SeriesFlags are set: checkHasSetDefs, checkHasSummaryData, checkHasTotalCountHint.
containerType	Sets or gets containerType , which is a DataTypes value that describes the container type of each SeriesEntry 's payload.
totalCountHint	Sets or gets a four-byte unsigned integer (totalCountHint) that indicates an approximate total number of entries associated with this stream. This is typically used when multiple Series containers are spread across multiple parts of a refresh message (For more information about message fragmentation and multi-part message handling, refer to Section 13.1). The totalCountHint provides an approximation of the total number of entries sent across all series on all parts of the refresh message. This information is useful when determining the amount of resources to allocate for caching or displaying all expected entries. totalCountHint values have a range of 0 to 1,073,741,824.
encodedSummaryData	Sets or gets encodedSummaryData, which is a TransportBuffer (with position and length) that contains the encoded summary data, if any, contained in the message. If populated, summary data contains information that applies to every entry encoded in the Series (e.g., currency type). The container type of summary data should match the containerType specified on the Series. If encodedSummaryData is populated while encoding, the contents will be used as pre-encoded summary data. For more information, refer to Section 11.5. Encoded summary data a maximum allowed length of 32,767 bytes.
encodedSetDef	Sets or gets encodedSetDefs, which is a TransportBuffer (with position and length) that contains the encoded local set definitions, if any, contained in the message. If populated, these definitions correspond to data contained within this Series's entries and are used to encode or decode their contents. For more information, refer to Section 11.6. Encoded local set definitions have a maximum allowed length of 32,767 bytes.
encodedEntries	Returns encodedEntries , which is a Buffer (with position and length) that contains all encoded key-value pair encoded data, if any, contained in the message. This refers to encoded Series payload and length data.

Table 109: Series Methods

8. A **Series** currently has a maximum entry count of 65,535. This type has an approximate maximum encoded length of 4 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of an **SeriesEntry** has a maximum encoded length of 65,535 bytes. These limitations can change in subsequent releases.

METHOD	DESCRIPTION
EncodeInit	Starts encoding a Series and allows for the encoding of summary data (for details, refer to Section 11.5) and Local Set Definitions (for details, refer to Section 11.6). You can encode additional summary data, set definitions, or entries after this method returns. - If summary data or set definitions are pre-encoded, you populate them on the encodedSummaryData and encodedSetDefs prior to calling EncodeInit. No additional work is needed to complete the encoding of this content. - If summary data or set definitions are not pre-encoded, EncodeInit will perform the Init for these components. After this content is encoded, you must call the corresponding Complete methods. To reserve space while encoding, you can pass in summary data and set definition encoded length hint values to this method. If either is not being encoded or the approximate encoded length is unknown, a value of 0 can be passed in. This is only needed when the content is not pre-encoded.
EncodeComplete	Completes the encoding of a Series. This method expects the same EncodeIterator used with Series.EncodeInit, any summary data, set data, and all entries. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If encoding fails, the boolean success parameter should be false to roll back the encoding to the last previously successful encoded point in the contents. Encode all series content prior to this call.
EncodeSummaryDataComplete	Completes the encoding of any non-pre-encoded Series summary data. If the SeriesFlags.HAS_SUMMARY_DATA flag is set and encodedSummaryData is not populated, summary data is expected after Series.EncodeInit or Series.EncodeSetDefsComplete returns. This method expects the same EncodeIterator used with previous series encoding methods. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If encoding fails, the boolean success parameter should be false to roll back the encoding prior to summary data. If both SeriesFlags.HAS_SUMMARY_DATA and SeriesFlags.HAS_SET_DEFS are present, set definitions are expected first, while any summary data is encoded after the call to EncodeSetDefsComplete.
EncodeSetDefsComplete	Completes the encoding of any non pre-encoded local set definition data. If the SeriesFlags.HAS_SET_DEFS flag is set and encodedSetDefs is not populated, local set definition data is expected after Series.EncodeInit returns. This method expects the same EncodeIterator used with Series.EncodeInit. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If encoding fails, the boolean success parameter should be false to roll back the encoding prior to the set definition data. If both SeriesFlags.HAS_SUMMARY_DATA and SeriesFlags.HAS_SET_DEFS are present, set definitions are expected first, while any summary data is encoded after the call to Series.EncodeSetDefsComplete.
decode	Begins decoding a Series from the TransportBuffer specified by DecodeIterator .
clear	Clears the object, so that you can reuse it.  TIP: When decoding, you can reuse Series without using clear.

Table 109: Series Methods (Continued)

11.3.4.2 SeriesFlags Values

SERIES FLAG	MEANING
NONE	Indicates that optional flags are not set.
HAS_TOTAL_COUNT_HINT	Indicates the presence of the totalCountHint member, which can provide an approximation of the total number of entries sent across maps on all parts of the refresh message. Such information is useful when determining resource allocation for caching or displaying all expected entries.
HAS_SUMMARY_DATA	Indicates that the Series contains summary data. - If set while encoding, summary data must be provided by encoding or populating encodedSummaryData with pre-encoded information. - If set while decoding, summary data is contained as part of Series and the user can choose to decode it.
HAS_SET_DEFS	Indicates that the Series contains local set definition information. Local set definitions correspond to data contained in this Series 's entries and encode or decode their contents. For more information, refer to Section 11.6.

Table 110: SeriesFlags Values

11.3.4.3 SeriesEntry Methods

Each **SeriesEntry** can house other Container Types only. **Series** is a uniform type, where **Series.containerType** indicates the single type housed in each entry. As entries are received, they are appended to any previously received entries.

METHOD	DESCRIPTION
encodedData	Sets or gets encodedData, which is a Buffer (with position and length) that contains the encoded content of this SeriesEntry. - If populated on encode methods, this indicates that data is pre-encoded and encodedData will be copied while encoding. - If populated while decoding, this refers to this encoded SeriesEntry's payload and length data.
Encode	Encodes a SeriesEntry from pre-encoded data. This method expects the same EncodeIterator used with Series.EncodeInit. You can pass in the pre-encoded series entry payload via SeriesEntry.encodedData. SeriesEntry.Encode is called after Series.EncodeInit and any summary data and local set definition data is encoded.
EncodeInit	Encodes a SeriesEntry from a container type. SeriesEntry.EncodeInit expects the same EncodeIterator used with Series.EncodeInit. After this call, you can use housed-type encode methods to encode the contained type. The contained type's encode method would be called after Series.EncodeInit and any summary data and local set definition data encoding has been completed. To reserve space while encoding, you can pass in a max-length hint value to this method. If the approximate encoded length is unknown, You can pass in a value of 0.
EncodeComplete	Completes the encoding of a SeriesEntry. This method expects the same EncodeIterator used with Series.EncodeInit, SeriesEntry.EncodeInit, and all other encoding for this container. - If encoding succeeds, the boolean success parameter should be true to finish entry encoding. - If encoding of this specific entry fails, the boolean success parameter should be false to roll back the encoding of only this SeriesEntry.
decode	Decodes a SeriesEntry. This method expects the same DecodeIterator used with Series.decode and populates encodedData with encoded entry. After SeriesEntry.decode returns, you can use Series.containerType to invoke the correct contained type's decode methods. Calling SeriesEntry.decode again decodes the next entry in the Series until no more entries are available. As entries are received, they are appended to previously received entries.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse SeriesEntry without using clear.

Table 111: SeriesEntry Methods

11.3.4.4 Series Encoding Example

The following sample illustrates how to encode an **Series** containing **ElementList** values. The example encodes two **SeriesEntry** values as well as summary data.

- The first entry is encoded from an unencoded element list.
- The second entry is encoded from a buffer containing a pre-encoded element list.

The example demonstrates error handling for the initial encode method. To simplify the example, additional error handling is omitted, though it should be performed.

```
/* populate series structure prior to call to Series.encodeInit() */

/* indicate that summary data and a total count hint will be encoded */
series.applyHasSummaryData();
series.applyHasTotalCountHint();
/* populate containerType and total count hint */
series.containerType(DataTypes.ELEMENT_LIST);
series.totalCountHint(2);

/* begin encoding of series - assumes that encIter is already populated with buffer and version
   information, store return value to determine success or failure */
/* summary data approximate encoded length is unknown, pass in 0 */
if ((retCode = series.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Series.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
    /* series init encoding was successful */
    /* create a single SeriesEntry and ElementList and reuse for each entry */
    SeriesEntry seriesEntry = CodecFactory.createSeriesEntry();
    ElementList elementList = CodecFactory.createElementList();

    /* encode expected summary data, init for this was done by Series.encodeInit - this type should match
       series.containerType */
    {
        /* now encode nested container using its own specific encode methods */
        /* begin encoding of element list - using same encIterator as series */
        elementList.applyHasStandardData();

        if ((retCode = elementList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)

            /*----- Continue encoding element entries. See example in Section 11.3.2 ----- */

        /* Complete nested container encoding */
        retCode = elementList.encodeComplete(encIter, success);
    }
}
```

```

/* complete encoding of summary data. If any element list encoding failed, success is false */
retCode = series.encodeSummaryDataComplete(encIter, success);

/* FIRST Series Entry: encode entry from unencoded data. Approx. encoded length unknown */
retCode = seriesEntry.encodeInit(encIter, 0);
/* encode contained element list - this type should match series.containerType */
{
    /* now encode nested container using its own specific encode methods */
    /* clear, then begin encoding of element list - using same encIterator as series */
    elementList.clear();
    elementList.applyHasStandardData();

    if ((retCode = elementList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)

        /*----- Continue encoding element entries. See example in Section 11.3.2 ---- */

        /* Complete nested container encoding */
        retCode = elementList.encodeComplete(encIter, success);
}
retCode = seriesEntry.encodeComplete(encIter, success);

/* SECOND Series Entry: encode entry from pre-encoded buffer containing an encoded ElementList */
/* assuming encElementList Buffer contains the pre-encoded payload with data and length populated */
seriesEntry.encodedData(encElementList);

retCode = seriesEntry.encode(encIter);
}
/* complete series encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = series.encodeComplete(encIter, success);

```

Code Example 31: Series Encoding Example

11.3.4.5 Series Decoding Example

The following sample illustrates how to decode a **Series** and is structured to decode each entry to the contained value. The sample code assumes the housed container type is an **ElementList**. Typically an application invokes the specific container-type decoder for the housed type or uses a switch statement to allow for a more generic series entry decoder. This example uses the same **DecodeIterator** when calling the content's decoder method. An application could optionally use a new **DecodeIterator** by setting **encodedData** on a new iterator. To simplify the sample, some error handling is omitted.

```

/* decode contents into the series structure */
if ((retCode = series.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* create single series entry and reuse while decoding each entry */
    SeriesEntry seriesEntry = CodecFactory.createSeriesEntry();
    /* if summary data is present, invoking decoder for that type (instead of DecodeEntry)
    indicates to the Transport API that user wants to decode summary data */
    if (series.checkHasSummaryData())
    {
        /* summary data is present. Its type should be that of series.containerType */
        ElementList elementList = CodecFactory.createElementList();
        retCode = elementList.decode(decIter, null);
        /* Continue decoding element entries. See example in Section 11.3.2 */
    }

    /* decode each series entry until there are no more left */
    while ((retCode = seriesEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with SeriesEntry.decode.
                Error Text: %s\n", error.errorId(), error.sysError(), error.text());
        }
        else
        {
            ElementList elementList = CodecFactory.createElementList();
            retCode = elementList.decode(decIter, null);
            /* Continue decoding element entries. See example in Section 11.3.2 */
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with Series.decode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}

```

Code Example 32: Series Decoding Example

11.3.5 Vector

The **Vector** is a uniform container type of **index**-value pair entries. Each entry, known as an **VectorEntry**, contains an index that correlates to the entry's position in the information stream and value. A **Vector** can contain zero to N^9 entries (zero entries indicates an empty **Vector**).

11.3.5.1 Vector Structure Members

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate special behaviors and whether optional Vector content is present. For more information about flag values, refer to Section 11.3.5.2. - You can use the following convenient methods to set specific VectorFlags: applyHasPerEntryPermData, applyHasSetDefs, applyHasSummaryData, applyHasTotalCountHint, applySupportsSorting. - You can use the following convenient methods to check whether specific VectorFlags are set: checkHasPerEntryPermData, checkHasSetDefs, checkHasSummaryData, checkHasTotalCountHint, checkSupportsSorting.
containerType	Sets or gets the container type (containerType ; a DataTypes value) of each VectorEntry 's payload.
totalCountHint	Sets or gets a four-byte, unsigned integer (totalCountHint) that indicates the approximate total number of entries sent across all vectors on all parts of the refresh message. totalCountHint is typically used when multiple Vector containers are spread across multiple parts of a refresh message (for more information about message fragmentation and multi-part message handling, refer to Section 13.1). Such information helps in determining the amount of resources to allocate for caching or displaying all expected entries. totalCountHint values have a range of 0 to 1,073,741,824.
encodedSummaryData	Sets or gets the encodedSummaryData, which is a Buffer (with position and length) containing the encoded summary data contained in the message. If populated, summary data contains information that applies to every entry encoded in the Vector (e.g. currency type). The container type of summary data must match the containerType specified on the Vector. If encodedSummaryData is populated while encoding, contents are used as pre-encoded summary data. Encoded summary data a maximum allowed length of 32,767 bytes. For more information, refer to Section 11.5.
encodedSetDef	Sets or gets the encodedSetDefs, which is a Buffer (with position and length) containing the encoded local set definitions contained in the message. If populated, these definitions correspond to data contained within this Vector's entries and are used to encode or decode their contents. Encoded local set definitions have a maximum allowed length of 32,767 bytes. For more information, refer to Section 11.6.
encodedEntries	Returns the encodedEntries , which is a Buffer (with position and length) containing the encoded index -value pair encoded data contained in the message. This would refer to encoded Vector payload and length information.

Table 112: Vector Methods

9. A **Vector** currently has a maximum entry count of 65,535. This type has an approximate maximum encoded length of 4 gigabytes but may be limited to 65,535 bytes if housed inside of a container entry. The content of a **VectorEntry** has a maximum encoded length of 65,535 bytes. These limitations can change in future releases.

METHOD	DESCRIPTION
EncodeInit	Begins encoding a Vector. Using this method, you can encode summary data (Section 11.5) and local set definitions (Section 11.6). Further summary data, set definitions, and/or entries can be encoded after this method returns. - If summary data and set definitions are pre-encoded, they can be populated on the encodedSummaryData and encodedSetDefs prior to calling Vector.EncodeInit. No additional work is needed to complete the encoding of this content. - If summary data and set definitions are not pre-encoded, Vector.EncodeInit will perform the Init for these components. After encoding this content, the corresponding Complete methods must be called. - To allow extra space while encoding, you can pass in summary data and set definition encoded length hint values to this method. If either is not being encoded or the approximate encoded length is unknown, a value of 0 can be passed in. This is only needed when the content is not pre-encoded.
EncodeComplete	Completes the encoding of a Vector. This method expects the same EncodeIterator used with Vector.EncodeInit, any summary data, set data, and all entries. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If any component fails to encode, the boolean success parameter should be false to roll back encoding to the last successfully-encoded point in the contents. Vector content should be encoded prior to this call.
EncodeSummaryDataComplete	Completes the encoding of Vector summary data. If VectorFlags.HAS_SUMMARY_DATA is set and encodedSummaryData is not populated, summary data is expected after Vector.EncodeInit or Vector.EncodeSetDefsComplete returns. This method expects the same EncodeIterator used with previous vector encoding methods. - If encoding succeeds, the boolean success parameter should be true to finish encoding. - If any data fail to encode, the boolean success parameter should be false to roll back to the last successfully-encoded point prior to summary data. - If both VectorFlags.HAS_SUMMARY_DATA and VectorFlags.HAS_SET_DEFS are present, set definitions are expected first, while summary data is encoded after the call to Vector.EncodeSetDefsComplete.
EncodeSetDefsComplete	Completes the encoding of local set definition data. If VectorFlags.HAS_SET_DEFS is set and encodedSetDefs is not populated, local set definition data is expected after Vector.EncodeInit returns. This method expects the same EncodeIterator used with Vector.EncodeInit. - If set definition data encodes successfully, the boolean success parameter should be true to finish encoding. - If set definition data fails to encode, the boolean success parameter should be false to roll back to the last successfully-encoded point prior to set definition data. - If both VectorFlags.HAS_SUMMARY_DATA and VectorFlags.HAS_SET_DEFS are present, set definitions are expected first, and then any summary data is encoded after the call to Vector.EncodeSetDefsComplete.
decode	Begins decoding a Vector . This method decodes from the TransportBuffer to which the passed-in DecodeIterator refers.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse Vector without using clear .

Table 112: Vector Methods (Continued)

11.3.5.2 Vector Flag Enumeration Values

VECTOR FLAG	MEANING
NONE	Indicates that optional flags are not set.
HAS_TOTAL_COUNT_HINT	Indicates that the totalCountHint member is present. totalCountHint can provide an approximation of the total number of entries sent across all vectors on all parts of the refresh message. Such information is useful in determining the amount of resources to allocate for caching or displaying all expected entries.
HAS_PER_ENTRY_PERM_DATA	Indicates that permission information is included with some vector entries. The Vector encoding functionality sets this flag value on the user's behalf if an entry is encoded with its own permData . A decoding application can check this flag to determine whether a contained entry has permData and is often useful for fan out devices (if an entry does not have permData , the fan out device can likely pass on data and not worry about special permissioning for the entry). Each entry also indicates the presence of permission data via the use of VectorEntryFlags.HAS_PERM_DATA . Refer to Section 11.3.5.4.
HAS_SUMMARY_DATA	Indicates that the Vector contains summary data. - If this flag is set while encoding, summary data must be provided by encoding or populating encodedSummaryData with pre-encoded data. - If this flag is set while decoding, summary data is contained as part of Vector and the user can choose whether to decode it.
HAS_SET_DEFS	Indicates that the Vector contains local set definition information. Local set definitions correspond to data contained in this Vector 's entries and are used for encoding or decoding their contents. For more information, refer to Section 11.6.
SUPPORTS_SORTING	Indicates that the Vector may leverage sortable action types. If an Vector is sortable, all components must properly handle changing index values based on insert and delete actions. If a component does not properly handle these action types, it can result in the corruption of the Vector 's contents. For more information on proper handling, refer to Section 11.3.5.5.

Table 113: VectorFlags Values

11.3.5.3 VectorEntry Structure Members

Each **VectorEntry** can house other Container Types only. **Vector** is a uniform type, whereas **Vector.containerType** indicates the single-type housed in each entry. Each entry has an associated action which informs the user of how to apply the data contained in the entry.

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate whether optional VectorEntry content is present. For more information about VectorEntryFlags values, refer to Section 11.3.5.4. - You can use the convenient method applyHasPermData to set specific VectorEntryFlags. - You can use the convenient method checkHasPermData to check whether specific VectorEntryFlags are set.
action	Sets or gets action, which helps to manage change processing rules and informs the consumer of how to apply the entry's data. For specific information about possible action's associated with an VectorEntry, refer to Section 11.3.5.5.
index	Sets or gets the entry's position (index) in the Vector. This value can change over time based on other VectorEntryActions. index has an allowable range of 0 to 1,073,741,823.
permData	(Optional) Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for this specific entry. If present, the VectorEntryFlags.HAS_PERM_DATA flag should be set. - For more information, refer to Section 11.4. - For more information about VectorEntryFlags, refer to Section 11.3.5.4. permData has a maximum allowed length of 32,767 bytes.
encodedData	Sets or gets encodedData, which is a Buffer (with position and length) that contains this VectorEntry's encoded content. - If populated using encode methods, this indicates that data is pre-encoded and encodedData is copied while encoding. - If populated while decoding, this refers to this encoded VectorEntry's payload and length information.
Encode	Encodes a VectorEntry from pre-encoded data. This method expects the same EncodeIterator used with Vector.EncodeInit. The pre-encoded vector entry payload can be passed in via VectorEntry.encodedData. This method is called after Vector.EncodeInit and after encoding any summary data and local set definition data.
EncodeInit	Encodes a VectorEntry from a container type. This method expects the same EncodeIterator used with Vector.EncodeInit. After this call, housed-type encode methods can encode the contained type. This method is called after Vector.EncodeInit and after encoding any summary and local set definition data. To reserve space for encoding, pass in a maximum length hint value (associated with the expected maximum encoded length of this entry). If you do not know the approximate encoded set data length, you can pass in a value of 0.
EncodeComplete	Completes the encoding of a VectorEntry. This method expects the same EncodeIterator used with Vector.EncodeInit, VectorEntry.EncodeInit and all other encoding for this container. - If encoding succeeds, the boolean success parameter should be true to finish entry encoding. - If encoding fails, the boolean success parameter should be false to roll back the encoding of this VectorEntry only.

Table 114: VectorEntry Methods

METHOD	DESCRIPTION
decode	Decodes a VectorEntry . This method expects the same DecodeIterator used with Vector.decode and populates encodedData with an encoded entry. After this method returns, you can use the Vector.containerType to invoke the correct contained type's decode methods. Calling VectorEntry.decode again will continue to decode subsequent entries in Vector until no more entries are available. As entries are received, the action will indicate how to apply their contents.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse VectorEntry without using clear .

Table 114: VectorEntry Methods (Continued)

11.3.5.4 VectorEntry Flag Enumeration Value

VECTOR ENTRY FLAG	MEANING
NONE	Indicates that optional flags are not set.
VectorEntryFlags.HAS_PERM_DATA	Indicates the presence of the permData member in this container entry and indicates authorization information for this entry. For more information, refer to Section 11.4.

Table 115: VectorEntryFlags Values

11.3.5.5 VectorEntryActions Values

ACTION	MEANING
VectorEntryActions.SET	Indicates that the consumer should set the entry at this index position. A set action typically occurs when an entry is initially provided. It is possible for multiple set actions to target the same entry. If this occurs, any previously received data associated with the entry should be replaced with the newly-added information. VectorEntryActions.SET_ENTRY can apply to both sortable and non-sortable vectors.
VectorEntryActions.UPDATE	Indicates that the consumer should update any previously stored or displayed information with the contents of this entry. An update action typically occurs when an entry is already set or inserted and changes to the contents are required. If an update action occurs prior to the set or insert action for the same entry, the update action should be ignored. VectorEntryActions.UPDATE_ENTRY can apply to both sortable and non-sortable vectors.
VectorEntryActions.CLEAR	Indicates that the consumer should remove any stored or displayed information associated with this entry's index position. VectorEntryActions.CLEAR_ENTRY can apply to both sortable and non-sortable vectors. No entry payload is included when the action is a 'clear.'
VectorEntryActions.INSERT	Applies only to a sortable vector. The consumer should insert this entry at the index position. Any higher order index positions are incremented by one (e.g., if inserting at index position 5 the existing position 5 becomes 6, existing position 6 becomes 7, and so forth).
VectorEntryActions.DELETE	Applies only to a sortable vector. The consumer should remove any stored or displayed data associated with this entry's index position. Any higher order index positions are decremented by one (e.g., if deleting at index position 5 the existing position 5 is removed, position 6 becomes 5, position 7 becomes 6, and so forth). No entry payload is included when the action is a 'delete.'

Table 116: VectorEntryActions Values

11.3.5.6 Vector Encoding Example

The following sample demonstrates how to encode an **Vector** containing **Series** values. The example encodes three **VectorEntry** values as well as summary data:

- The first entry is encoded from an unencoded series
- The second entry is encoded from a buffer containing a pre-encoded series and has perm data
- The third is a clear action type with no payload.

This example demonstrates error handling for the initial encode method. To simplify the example, additional error handling is omitted (though it should be performed).

```
/* populate vector structure prior to call to Vector.encodeInit() */

/* indicate that summary data and a total count hint will be encoded */
vector.applyHasSummaryData();
vector.applyHasTotalCountHint();
vector.applyHasPerEntryPermData();
/* populate containerType and total count hint */
vector.containerType(DataTypes.SERIES);
vector.totalCountHint(3);

/* begin encoding of vector - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
/* summary data approximate encoded length is 50 bytes */
if ((retCode = vector.encodeInit(encIter, 50, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Vector.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
    /* vector init encoding was successful */
    /* create a single VectorEntry and Series and reuse for each entry */
    VectorEntry vectorEntry = CodecFactory.createVectorEntry();
    Series series = CodecFactory.createSeries();

    /* encode expected summary data, init for this was done by Vector.encodeInit
       - this type should match vector.containerType */
    {
        /* now encode nested container using its own specific encode methods */
        /* begin encoding of series - using same encIterator as vector */
        if ((retCode = series.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)

            /*----- Continue encoding series entries. See example in Section 11.3.4.4 */

        /* Complete nested container encoding */
        retCode = series.encodeComplete(encIter, success);
    }
}
```

```

}
/* complete encoding of summary data.  If any series entry encoding failed, success is false */
retCode = vector.encodeSummaryDataComplete(encIter, success);

/* FIRST Vector Entry: encode entry from unencoded data. Approx. encoded length 90 bytes */
/* populate index and action, no perm data on this entry */
vectorEntry.index(1);
vectorEntry.action(VectorEntryActions.UPDATE);
retCode = vectorEntry.encodeInit(encIter, 90);
/* encode contained series - this type should match vector.containerType */
{
    /* now encode nested container using its own specific encode methods */
    /* clear, then begin encoding of series - using same encIterator as vector */
    series.clear();
    if ((retCode = series.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)

        /*----- Continue encoding series entries. See example in Section 11.3.4.4 ----- */

        /* Complete nested container encoding */
        retCode = series.encodeComplete(encIter, success);
}
retCode = vectorEntry.encodeComplete(encIter, success);

/* SECOND Vector Entry: encode entry from pre-encoded buffer containing an encoded Series */
/* assuming encSeries Buffer contains the pre-encoded payload with data and length populated
   and permData contains permission data information */
vectorEntry.index(2);
/* by passing permData on an entry, the map encoding functionality will implicitly set the
   VectorFlags.HAS_PER_ENTRY_PERM_DATA flag */
vectorEntry.applyHasPermData();
vectorEntry.action(VectorEntryActions.SET);
vectorEntry.permData(permData);
vectorEntry.encodedData(encSeries);

retCode = vectorEntry.encode(encIter);

/* THIRD Vector Entry: encode entry with clear action, no payload on clear */
/* Should clear entry for safety, this will set flags to NONE */
vectorEntry.clear();
vectorEntry.index(3);
vectorEntry.action(VectorEntryActions.CLEAR);

retCode = vectorEntry.encode(encIter);
}
/* complete vector encoding.  If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = vector.encodeComplete(encIter, success);

```

Code Example 33: Vector Encoding Example

11.3.5.7 Vector Decoding Example

The following sample illustrates how to decode a **Vector** and is structured to decode each entry to the contained value. This sample code assumes the housed container type is a **Series**. Typically an application would invoke the specific container type decoder for the housed type or use a switch statement to allow a more generic series entry decoder. This example uses the same **DecodeIterator** when calling the content's decoder function. Optionally, an application could use a new **DecodeIterator** by setting the **encodedData** on a new iterator. To simplify the sample, some error handling is omitted.

```
/* decode contents into the vector structure */
if ((retCode = vector.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* create single vector entry and reuse while decoding each entry */
    VectorEntry vectorEntry = CodecFactory.createVectorEntry();
    /* if summary data is present, invoking decoder for that type (instead of DecodeEntry)
       indicates to the Transport API that the user wants to decode summary data */
    if (vector.checkHasSummaryData())
    {
        /* summary data is present. Its type should be that of vector.containerType */
        retCode = series.decode(decIter);
        /* Continue decoding series entries. See the example in Section 11.3.4.5 */
    }

    /* decode each vector entry until there are no more left */
    while ((retCode = vectorEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with VectorEntry.decode. Error
                               Text: %s\n", error.errorId(), error.sysError(), error.text());
        }
        else
        {
            retCode = series.decode(decIter);
            /* Continue decoding series entries. See example in Section 11.3.4 */
        }
    }
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with Vector.decode. Error Text: %s\n",
                      error.errorId(), error.sysError(), error.text());
}
```

Code Example 34: Vector Decoding Example

11.3.6 FilterList

The **FilterList** is a non-uniform container type of **filterId**-value pair entries. Each entry, known as a **FilterEntry**, contains an **id** corresponding to one of 32 possible bit-value identifiers. These identifiers are typically defined by a domain model specification and can indicate interest in or the presence of specific entries through the inclusion of the **filterId** in the message key's **filter** member. A **FilterList** can contain zero to N^{10} entries, where zero indicates an empty **FilterList**, though this type is typically limited by the number of available of **filterId** values.

11.3.6.1 FilterList Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) to indicate presence of optional FilterList content. For more information about FilterListFlags values, refer to Section 11.3.6.2. - You can use the following convenient methods to set specific FilterListFlags: applyHasPerEntryPermData, applyHasTotalCountHint. - You can use the following convenient methods to check whether specific FilterListFlags are set: checkHasPerEntryPermData, checkHasTotalHintCount.
containerType	Sets or gets a containerType, which is a DataTypes enumeration value that, for most efficient bandwidth use, should describe the most common container type across all housed filter entries. All housed entries may match this type, though one or more entries may differ. If an entry differs, the entry specifies its own type via the FilterEntry.containerType member.
totalCountHint	Sets or gets a four-byte unsigned integer (totalCountHint) that indicates an approximate total number of entries associated with this stream. totalCountHint is used typically when multiple FilterList containers are spread across multiple parts of a refresh message (for more information about message fragmentation and multi-part message handling, refer to Section 13.1). totalCountHint is useful in determining the amount of resources to allocate for caching or displaying all expected entries. totalCountHint values have a range of 0 to 1,073,741,824, though the FilterList is typically limited by available filterId values.
encodedEntries	Returns the encodedEntries, which is a Buffer (with position and length) that contains the filterId-value pair encoded data, if any, contained in the message. This would refer to the encoded FilterList payload and length information.
EncodeInit	Begins encoding a FilterList. containerType should define the most common entry type.
EncodeComplete	Completes the encoding of a FilterList. This method expects the same EncodeIterator used with FilterList.EncodeInit. - If encoding succeeds, the boolean success parameter should be set to true to finish encoding. - If any entry fails to encode, the boolean success parameter should be set to false to roll back to the last successfully encoded point in the contents. Encode all entries prior to this call.
decode	Begins decoding a FilterList. This method decodes from the Buffer specified in DecodeIterator.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse FilterList without using clear.

Table 117: FilterList Methods

10. A **FilterList** currently has a maximum entry count of 65,535, though due to the allowable range of id values, this typically does not exceed 32. If all entry count values are allowed, this type has an approximate maximum encoded length of 4 GB but may be limited to 65,535 bytes if housed inside a container entry. The content of an **FilterEntry** has a maximum encoded length of 65,535 bytes. These limitations can change in future releases.

11.3.6.2 FilterList Flag Enumeration Values

FILTER LIST FLAG	MEANING
NONE	Indicates that optional flags are not set.
FilterListFlags.HAS_TOTAL_COUNT_HINT	Indicates the presence of the totalCountHint member. totalCountHint provides an approximation of the total number of entries sent across all filter lists on all parts of the refresh message. This information is useful in determining the amount of resources to allocate for caching or displaying all expected entries.
FilterListFlags.HAS_PER_ENTRY_PERM_DATA	Indicates some filter entries include permission information. The FilterList encoding functionality sets this flag value on the user's behalf if any entry is encoded with its own permData . A decoding application can check this flag to determine whether any contained entry has permData , often useful for fan out devices (if entries do not have permData , the fan out device can pass along the data and not worry about special permissioning for an entry). Each entry will also indicate permission data presence via the use of the FilterEntryFlags.HAS_PERM_DATA flag. Refer to Section 11.3.6.4.

Table 118: FilterListFlags Values

11.3.6.3 FilterEntry Methods

Each **FilterEntry** can house only other container types. **FilterList** is a non-uniform type, where the **FilterList.containerType** should indicate the most common type housed in each entry. Entries that differ from this type must specify their own type via **FilterEntry.containerType**.

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate the presence of optional FilterEntry content. For more information about FilterEntryFlags values, refer to Section 11.3.6.4. - You can use the following convenient methods to set specific FilterEntryFlags: applyHasContainerType, applyHasPermData. - You can use the following convenient methods to check whether specific FilterEntryFlags are set: checkHasContainerType, checkHasPermData.
action	Sets or gets action, which helps manage change processing rules and informs the consumer how to apply the information contained in the entry. For specific information about possible action's associated with an FilterEntry, refer to Section 11.3.6.5.
id	Sets or gets the ID (id) associated with the entry. Each possible id corresponds to a bit-value that can be used with the message key's filter member. This bit-value can be specified on the filter to indicate interest in the id when present in an RequestMsg or to indicate presence of the id when present in other messages. For additional information about the filter, refer to Section 12.1.2. id has a range of 1 to 32. A value of 0 is not valid as it cannot correlate to a bit-value for use with the message key filter.
containerType	Sets or gets containerType, a DataTypes value describing the type of this FilterEntry. If containerType is present, the user should set the FilterEntry flag (FilterEntryFlags.HAS_CONTAINER_TYPE). For more information about FilterEntry flag values, refer to Section 11.3.6.4.

Table 119: FilterEntry Methods

METHOD	DESCRIPTION
permData	(Optional) Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for this entry. If permData is present, the user should set the FilterEntryFlags.HAS_PERM_DATA flag. permData has a maximum allowed length of 32,767 bytes. - For more information about FilterEntry flag values, refer to Section 11.3.6.4. - For more information, refer to Section 11.4.
encodedData	Sets or gets encodedData, which is a Buffer (with position and length) containing the FilterEntry's encoded content. - If populated on encode functions, encodedData indicates that data is pre-encoded, and encodedData will be copied while encoding. - If populated while decoding, this refers to this encoded FilterEntry's payload and length information.
Encode	Encodes a FilterEntry from pre-encoded data. Encode expects the same EncodeIterator used with FilterList.EncodeInit. The pre-encoded filter entry payload can be passed in via FilterEntry.EncodeData. This method can be called after FilterList.EncodeInit completes. If this filter entry houses a type other than what is specified in FilterList.containerType, the entry's containerType should be populated to indicate the difference.
EncodeInit	Encodes a FilterEntry from a container type. EncodeInit expects the same EncodeIterator used with FilterList.EncodeInit. After this call, the housed type encode Method can begin to encode the contained type. This method can be called after FilterList.EncodeInit is completed. - To reserve space for encoding, pass in a maximum length hint value (associated with the expected maximum encoded length of this entry) to FilterEntry.EncodeInit. If you do not know the approximate encoded length, you can pass in a value of 0. - If this filter entry houses a type other than that specified in FilterList.containerType, the entry's containerType value must indicate the difference.
EncodeComplete	Completes the encoding of a FilterEntry. EncodeComplete expects the same EncodeIterator used with FilterList.EncodeInit, FilterEntry.EncodeInit and all other encoding for this container. - If encoding succeeds, the boolean success parameter should be set to true to finish entry encoding. - If encoding fails, the boolean success parameter should be set to false to roll back the encoding of this FilterEntry.
decode	Decodes a FilterEntry. decode expects the same DecodeIterator used with FilterList.decode. This populates encodedData with an encoded entry. As an entry is received, its action indicates how to apply contents. After this method returns, the FilterList.containerType (or FilterEntry.containerType if present) can invoke the correct contained type's decode methods. Calling FilterEntry.decode again decodes the remaining entries in the FilterList.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse FilterEntry without using clear.

Table 119: FilterEntry Methods (Continued)

11.3.6.4 FilterEntry Flag Enumeration Values

FILTER ENTRY FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
FilterEntryFlags.HAS_PERM_DATA	Indicates the presence of permData in this container entry and indicates authorization information for this entry. For more information, refer to Section 11.4.
FilterEntryFlags.HAS_CONTAINER_TYPE	Indicates the presence of containerType in this entry. This flag is used when the entry's containerType differs from the specified FilterList.containerType .

Table 120: FilterEntryFlags Values

11.3.6.5 FilterEntryActions Values

Each entry has an associated **action** which informs the user of how to apply the entry's contents.

ACTION ENUMERATION	MEANING
SET	Indicates that the consumer should set the entry corresponding to this id . A set action typically occurs when an entry is initially provided. Multiple set actions can occur for the same entry id , in which case, any previously received data associated with the entry id should be replaced with the newly-added information.
UPDATE	Indicates that the consumer should update any previously stored or displayed information with the contents of this entry. An update action typically occurs when an entry is set and changes to the contents need to be conveyed. An update action can occur prior to the set action for the same entry id , in which case, the update action should be ignored.
CLEAR	Indicates that the consumer should remove any stored or displayed information associated with this entry's id . No entry payload is included when the action is a clear.

Table 121: FilterEntryActions Values

11.3.6.6 FilterEntry Encoding Example

The following sample illustrates how to encode an **FilterList** containing a mixture of housed types. The example encodes three **FilterEntry** values:

- The first is encoded from an unencoded element list.
- The second is encoded from a buffer containing a pre-encoded element list.
- The third is encoded from an unencoded map value.

This example demonstrates error handling only for the initial encode function, and to simplify the example, omits additional error handling (though it should be performed).

```
/* populate filterList structure prior to call to FilterList.encodeInit() */

/* populate containerType. Because two element lists exist, this is most common so specify that type */
filterList.containerType(DataTypes.ELEMENT_LIST);

/* begin encoding of filterList - assumes that encIter is already populated with buffer and version
   information, store return value to determine success or failure */
```

```

if ((retCode = filterList.encodeInit(encIter)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with FilterList.encodeInit. Error Text:
        %s\n", error.errorId(), error.sysError(), error.text());
}
else
{
    /* filterList init encoding was successful */
    /* create a single FilterEntry and reuse for each entry */
    FilterEntry filterEntry = CodecFactory.createFilterEntry();

    /* FIRST Filter Entry: encode entry from unencoded data. Approx. encoded length 350 bytes */
    /* populate id and action */
    filterEntry.id(1);
    filterEntry.action(FilterEntryActions.SET);
    retCode = filterEntry.encodeInit(encIter, 350);
    /* encode contained element list */
    {
        ElementList elementList = CodecFactory.createElementList();
        elementList.applyHasStandardData();
        /* now encode nested container using its own specific encode methods */
        if ((retCode = elementList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)
        /*----- Continue encoding element entries. See example in Section 11.3.2.4----- */
        /* Complete nested container encoding */
        retCode = elementList.encodeComplete(encIter, success);
    }
    retCode = filterEntry.encodeComplete(encIter, success);

    /* SECOND Filter Entry: encode entry from pre-encoded buffer containing an encoded element list */
    /* assuming encElemList Buffer contains the pre-encoded payload with data and length populated */
    filterEntry.id(2);
    filterEntry.action(FilterEntryActions.UPDATE);

    filterEntry.encodedData(encElemList);

    retCode = filterEntry.encode(encIter);

    /* THIRD Filter Entry: encode entry from an unencoded map */
    filterEntry.id(3);
    filterEntry.action(FilterEntryActions.UPDATE);
    /* because type is different from filterList.containerType, we need to specify on entry */
    filterEntry.applyHasContainerType();
    filterEntry.containerType(DataTypes.MAP);

    retCode = filterEntry.encodeInit(encIter, 0);
    /* encode contained map */
    {

```

```

map.keyPrimitiveType(DataTypes.ASCII_STRING);
map.containerType(DataTypes.FIELD_LIST);
/* now encode nested container using its own specific encode methods */
if ((retCode = map.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)
/*----- Continue encoding map entries. See example in Section 11.3.3.6 ----- */
/* Complete nested container encoding */
retCode = map.encodeComplete(encIter, success);
}
retCode = filterEntry.encodeComplete(encIter, success);

}
/* complete filterList encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = filterList.encodeComplete(encIter, success);

```

Code Example 35: FilterList Encoding Example

11.3.6.7 FilterEntry Decoding Example

The following sample illustrates how to decode an **FilterList** and is structured to decode each entry to its contained value. The sample code uses a switch statement to decode the contents of each filter entry. Typically an application invokes the specific container type decoder for the housed type or uses a switch statement to use a more generic series entry decoder. This example uses the same **DecodeIterator** when calling the content's decoder function. Optionally, an application could use a new **DecodeIterator** by setting the **encodedData** on a new iterator. To simplify the example, some error handling is omitted.

```
/* decode contents into the filter list structure */
if ((retCode = filterList.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* create single filter entry and reuse while decoding each entry */
    FilterEntry filterEntry = CodecFactory.createFilterEntry();

    /* decode each filter entry until there are no more left */
    while ((retCode = filterEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
    {
        if (retCode < CodecReturnCodes.SUCCESS)
        {
            /* decoding failure tends to be unrecoverable */
            System.out.printf("Error (%d) (errno: %d) encountered with FilterEntry.decode. Error
                Text: %s\n", error.errorId(), error.sysError(), error.text());
        }
        else
        {
            /* if filterEntry.containerType is present, switch on that,
                Otherwise switch on filterList.containerType */
            int cType;

            if (filterEntry.checkHasContainerType())
                cType = filterEntry.containerType();
            else
                cType = filterList.containerType();

            switch (cType)
            {
                case DataTypes.MAP:
                    retCode = map.decode(decIter);
                    /* Continue decoding map entries. See example in Section 11.3.3.7 */
                    break;
                case DataTypes.ELEMENT_LIST:
                    retCode = elemList.decode(decIter, null);
                    /* Continue decoding element entries. See example in Section 11.3.2.5 */
                    break;
                /* full switch statement omitted to shorten sample code */
            }
        }
    }
}
else
{

```

```
/* decoding failure tends to be unrecoverable */  
System.out.printf("Error (%d) (errno: %d) encountered with FilterList.decode. Error Text: %s\n",  
    error.errorId(), error.sysError(), error.text());  
}
```

Code Example 36: FilterList Decoding Example

11.3.7 Non-LSEG Rssl Wire Format Container Types

Enterprise Transport API messages and container entries allow non-LSEG Rssl Wire Format content. Non-LSEG Rssl Wire Format content can be:

- A specific type of formatted data such as ANSI Page or XML, where a **DataTypes** value aids in identifying the type.
- A type of customized, user-defined information. You can use **DataTypes**'s range of **225 - 255** to define custom types.

11.3.7.1 Non-LSEG Rssl Wire Format Encode Functions

The Transport API provides utility methods to help encode non-LSEG Rssl Wire Format types. These methods work in conjunction with **EncodeIterator** to provide appropriate encoding position and length data to the user, which can then be used with specific methods for the non-LSEG Rssl Wire Format type being encoded.

METHOD	DESCRIPTION
EncodeIterator.EncodeNonRWFFinit	Uses the EncodeIterator to populate a Buffer with encoding information for the user. Buffer.data contains the backing ByteBuffer . Buffer.position contains the position where encoding begins. Buffer.length contains the number of available bytes for encoding. After this method returns successfully, you can populate this buffer using non-LSEG Rssl Wire Format encode methods.
EncodeIterator.EncodeNonRWFFComplete	Integrates content encoded into Buffer with other pre-encoded information. Buffer.data.position should be set to the position of the last byte encoded prior to this method being called.

Table 122: Non-LSEG Rssl Wire Format Type Encode Methods

11.3.7.2 Non-LSEG Rssl Wire Format Encoding Example

NOTE: Do not change the value of **Buffer.data** between calls to **EncodeIterator.encodeNonRWFFinit** and **EncodeIterator.encodeNonRWFFComplete**.

The following sample demonstrates how to encode an **Series** containing a non-LSEG Rssl Wire Format type of ANSI Page. This example demonstrates error handling for the initial encode method while omitting additional error handling (though it should be performed).

```
/* populate containerType with the ANSI dataType enumerated value; this could be any non-RWF type enum */
series.containerType(DataTypes.ANSI_PAGE);
/* begin encoding of series - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
if ((retCode = series.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Series.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
```



```

/* series init encoding was successful */
/* begin our series entry and then nest ANSI Page inside of it using non-RWF encode methods */
SeriesEntry seriesEntry = CodecFactory.createSeriesEntry();
/* create an empty buffer for information to be populated into */
Buffer nonRWFBuffer = CodecFactory.createBuffer();

retCode = seriesEntry.encodeInit(encIter, 0);
/* encode contained non-RWF type using non-RWF encode methods */
{
    retCode = encIter.encodeNonRWFFinit(nonRWFBuffer);
    /* now encode nested container using its own specific encode methods -
       Ensure that we do not exceed nonRWFBuffer.length */
    /* we could copy into the nonRWFBuffer or use it with other encode methods */
    /* The encAnsiBuffer shown here is expected to be populated with data from an
       external ANSI encoder. The native ANSI encode methods could be called, instead
       of a copy with pre-encoded ANSI content, to directly encode into the nonRWFBuffer */
    nonRWFBuffer.data().put(encAnsiBuffer.data());
    retCode = encIter.encodeNonRWFFComplete(nonRWFBuffer, success);
}
retCode = seriesEntry.encodeComplete(encIter, success);
}
/* complete series encoding. If success parameter is true, this will finalize encoding.
   If success parameter is false, this will roll back encoding prior to encodeInit */
retCode = series.encodeComplete(encIter, success);

```

Code Example 37: Non-LSEG Rssl Wire Format Type Encoding Example

11.3.7.3 Decoding Non-LSEG Rssl Wire Format Types

When decoding, the user can obtain non-LSEG Rssl Wire Format data via the **encodedData** member and use this with methods specific to the non-LSEG Rssl Wire Format type being decoded.

11.4 Permission Data

Permission Data is optional authorization information. The DACS Lock API provides functionality for creating and manipulating permissioning information. For more information on Data Access Control System usage and permission data creation, refer to the *Enterprise Transport API DACS LOCK Library Reference Manual*.

Permission data can be specified in some Enterprise Transport API messages. When permission data is included in a **RefreshMsg** or a **StatusMsg**, this generally defines authorization information associated with all content on the stream. You can change permission data on an existing stream by sending a subsequent **StatusMsg** or **RefreshMsg** which contains the new permission data. When permission data is included in an **UpdateMsg**, this generally defines authorization information that applies only to that specific **UpdateMsg**.

Permission data can also be specified in some container entries. When a container entry includes permission data, it generally defines authorization information that applies only to that specific container entry. Specific usage and inclusion of permissioning information can be further defined within a domain model specification.

Permission data typically ensures that only entitled parties can access restricted content. On LSEG Real-Time Distribution System, all content is restricted (or filtered) based on user permissions.

When content is contributed, permission data in a **PostMsg** is used to permission the user who posts the information. If the payload of the **PostMsg** is another message type with permission data (i.e., **RefreshMsg**), the nested message's permissions can change the permission expression associated with the posted item. If permission data for the nested message is the same as permission data on the **PostMsg**, the nested message does not need permission data.

11.5 Summary Data

Some Enterprise Transport API container types allow summary data. **Summary data** conveys information that applies to every entry housed in the container. Using summary data ensures data is sent only once, instead of repetitively including data in each entry. An example of summary data is the currency type because it is likely that all entries in the container share the same currency. Summary data is optional and applications can determine when to employ it.

Specific domain model definitions typically indicate whether summary data should be present, along with information on its content. When included, the **containerType** of the summary data is expected to match the **containerType** of the payload information (e.g., if summary data is present on a **Vector**, the **Vector.containerType** defines the type of summary data and **VectorEntry** payload).

11.6 Set Definitions and Set-Defined Data

A **Set-Defined Primitive Type** is similar to a primitive type (described in Section 11.2) with several key differences. While primitive types can be encoded as a variable number of bytes, most set-defined primitive types use a fixed-length encoding. Fixed-length encoding can help reduce the number of bytes required to contain the encoded primitive type. **DataTypes** values between **64** and **127** are set-defined primitive types and set fixed-length encodings for many base primitive types (e.g., **DataTypes.INT_1** is a one-byte fixed-length encoding of **DataTypes.INT**). Whereas all primitive types can represent blank data, only several set-defined primitive types can do so. All encoding and decoding continues to use primitive type definitions and should continue to function in the same manner as described in the previous sections. The **DataTypes** enumeration exposes values that define each set-defined primitive, though these values are only used inside of a set definition. When using set-defined primitive types, a set definition is required to encode or decode content.

A **Set Definition** can define the contents of an **FieldList** or an **ElementList** and allow additional optimizations. Use of a set definition can reduce overall encoded content by eliminating repetitive type and length information.

- A set definition describing an **FieldList** contains **fieldId** and type information specified in the same order as the contents are arranged in the encoded field list.
- A set definition describing an **ElementList** contains element **name** and type information specified in the same order as the contents are arranged in the encoded element list.

When encoding, in addition to providing set definition information, an application encodes the field list or element list content. Internally the encoder uses the provided set definition to perform type encoding specific to the definition and omit redundant information needed only in the definition.

When decoding, in addition to providing set definition information, an application decodes the field list or element list content. Internally, the decoder uses the provided set definition to decode any type-specific optimizations and to reintroduce redundant information omitted during the encoding.

Instead of including multiple instances of the same content, you can use a set definition (i.e., a **Map** containing **FieldList** content in each entry). In this case, a set definition can be provided once as part of the **Map** to define the layout of repetitive field list information contained in the **MapEntry** (i.e., **fieldId**). When encoding each **FieldList**, this content will be omitted because it is included in the set definition.

A set definition can contain primitive type enumerations (Section 11.2), set-defined primitive type enumerations, and container type enumerations (Section 11.3). Encoding and decoding occurs exactly the same as primitive type and container type encoding or decoding.

11.6.1 Set-Defined Primitive Types

Set primitive types do not use separate interface methods for encoding or decoding. Decoding uses the same primitive type decoder used when decoding the primitive type. Because these types can only be contained in a **FieldList** or **ElementList**, encoding occurs as usual by calling **FieldEntry.encode** or **ElementEntry.encode**. When calling these methods, populate the field or element entry using the base primitive type. The table below provides a brief description of each set-defined primitive type, along with its corresponding base primitive type enumeration and its respective decode interface.

SET-DEFINED PRIMITIVE DATATYPE	BASE PRIMITIVE DATATYPE	PRIMITIVE TYPE	DECODE INTERFACE	TYPE DESCRIPTION
DataTypes.INT_1	DataTypes.INT	Int	Int.decode	A signed, one-byte integer type that represents a value up to 7 bits with a one-bit sign (positive or negative). Allowable range is (-2^7) to $(2^7 - 1)$. This type cannot be represented as blank.
DataTypes.INT_2	DataTypes.INT	Int	Int.decode	A signed, two-byte integer type that represents a value up to 15 bits with a one-bit sign (positive or negative). Allowable range is (-2^{15}) to $(2^{15} - 1)$. This type cannot be represented as blank.
DataTypes.INT_4	DataTypes.INT	Int	Int.decode	A signed, four-byte integer type that represents a value up to 31 bits with a one-bit sign (positive or negative). Allowable range is (-2^{31}) to $(2^{31} - 1)$. This type cannot be represented as blank.
DataTypes.INT_8	DataTypes.INT	Int	Int.decode	A signed, eight-byte integer type that represents a value up to 63 bits with a one-bit sign (positive or negative). Allowable range is (-2^{63}) to $(2^{63} - 1)$. This type cannot be represented as blank.
DataTypes.UINT_1	DataTypes.UINT	UInt	UInt.decode	An unsigned, one-byte integer type that represents an unsigned value with precision of up to 8 bits. Allowable range is 0 to $(2^8 - 1)$. This type cannot be represented as blank.
DataTypes.UINT_2	DataTypes.UINT	UInt	UInt.decode	An unsigned, two-byte integer type that represents an unsigned value with precision of up to 16 bits. Allowable range is 0 to $(2^{16} - 1)$. This type cannot be represented as blank.
DataTypes.UINT_4	DataTypes.UINT	UInt	UInt.decode	An unsigned, four-byte integer type that represents an unsigned value with precision of up to 32 bits. Allowable range is 0 to $(2^{32} - 1)$. This type cannot be represented as blank.

Table 123: Set-Defined Primitive Types

SET-DEFINED PRIMITIVE DATATYPE	BASE PRIMITIVE DATATYPE	PRIMITIVE TYPE	DECODE INTERFACE	TYPE DESCRIPTION
DataTypes.UINT_8	DataTypes.UINT	UInt	UInt.decode	An unsigned, eight-byte integer type that represents an unsigned value with precision of up to 64 bits. Allowable range is 0 to $(2^{64} - 1)$. This set-defined primitive type cannot be represented as blank.
DataTypes.FLOAT_4	DataTypes.FLOAT	Float	Float.decode	A four-byte, floating point type that represents the same range of values allowed by the system float type. Follows the IEEE 754 specification. This type cannot be represented as blank.
DataTypes.DOUBLE_8	DataTypes.DOUBLE	Double	Double.decode	An eight-byte, floating point type that represents the same range of values allowed by the system double type. Follows the IEEE 754 specification. This type cannot be represented as blank.
DataTypes.REAL_4RB	DataTypes.REAL	Real	Real.decode	An optimized Rssl Wire Format representation of a decimal or fractional value which typically requires less bytes on the wire than float or double types. This type allows up to a four-byte value, with a hint value (for converting to decimal or fractional representation), which can add or remove up to seven trailing zeros, ten decimal places, or fractional denominators up to 256. Allowable range is (-2^{31}) to $(2^{31} - 1)$. This type can be represented as blank. For more details on this type, refer to Section 11.2.2.
DataTypes.REAL_8RB	DataTypes.REAL	Real	Real.decode	An optimized Rssl Wire Format representation of a decimal or fractional value which typically requires less bytes on the wire than float or double types. This type allows up to an eight byte value, with a hint value (for converting to decimal or fractional representation), which can add or remove up to seven trailing zeros, 14 decimal places, or fractional denominators up to 256. Allowable range is (-2^{63}) to $(2^{63} - 1)$. This type can be represented as blank. For more details on this type, refer to Section 11.2.2.
DataTypes.DATE_4	DataTypes.DATE	Date	Date.decode	Representation of a date containing month, day, and year values. This value can be represented as blank. For more details on this type, refer to Section 11.2.3.
DataTypes.TIME_3	DataTypes.TIME	Time	Time.decode	Representation of a time containing hour, minute, and second values. This value can be represented as blank. For more details on this type, refer to Section 11.2.4.

Table 123: Set-Defined Primitive Types (Continued)

SET-DEFINED PRIMITIVE DATATYPE	BASE PRIMITIVE DATATYPE	PRIMITIVE TYPE	DECODE INTERFACE	TYPE DESCRIPTION
DataTypes.TIME_5	DataTypes.TIME	Time	Time.decode	Representation of a time containing hour, minute, second, and millisecond values. This value can be represented as blank. For more details on this type, refer to Section 11.2.4.
DataTypes.DATETIME_7	DataTypes.DATETIME	DateTime	DateTime.decode	Combined representation of date and time. Contains all members of DataTypes.DATE and hour, minute, and second from DataTypes.TIME . This value can be represented as blank. For more details on this type, refer to Section 11.2.5.
DataTypes.DATETIME_9	DataTypes.DATETIME	DateTime	DateTime.decode	Combined representation of date and time. Contains all members of DataTypes.DATE and all members of DataTypes.TIME . This value can be represented as blank. For more details on this type, refer to Section 11.2.5.

Table 123: Set-Defined Primitive Types (Continued)

11.6.2 Set Definition Use

In the Enterprise Transport API, an application can leverage local set definitions. A **local set definition** is a set definition sent along with the content it defines. Local set definitions are valid only within the scope of the container of which they are a part and apply only to the information in the container on which they are specified (e.g., a **Map**'s set definition content applies only to the payload within the map's entries). Set definitions are divided into two concrete types

- **Field set definition:** A set definition that defines **FieldList** content
- **Element set definition:** A set definition that defines **ElementList** content

Set definitions can contain multiple entries, each defining a specific encoding type for a **FieldEntry** or **ElementEntry**.

11.6.2.1 FieldSetDef Methods

The following table defines **FieldSetDef** Methods. **FieldSetDef** represents a single field set definition and can define the contents of multiple entries in an **FieldList**.

METHOD	DESCRIPTION
setId	Sets or gets the field set definition's identifier value (setId). Any field list content that leverages this definition should have FieldList.setId match this identifier. setId values have an allowed range of 0 to 32,767. However, only values 0 to 15 are valid for local set definition content. For more information, refer to Section 11.6. For more details on how FieldList indicates the use of a set definition, refer to Section 11.3.1
count	Sets or gets the number (count) of FieldSetDefEntry s contained in this definition. Each entry defines how a FieldEntry is encoded or decoded. A set definition is limited to 255 entries. For more information, refer to Section 11.6.2.2
entries	Sets or gets entries, which is an array of FieldSetDefEntry s. Each entry defines how an FieldEntry is encoded or decoded. For more information, refer to Section 11.6.2.2.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse FieldSetDef without using clear .

Table 124: FieldSetDef Methods

11.6.2.2 FieldSetDefEntry Structure Members

METHOD	DESCRIPTION
fieldId	Set or get the fieldId value that corresponds to this entry in the set-defined FieldList content. fieldId is a signed, two-byte value that refers to specific name and type information defined by an external field dictionary, such as the RDMFieldDictionary. Negative fieldId values typically refer to user-defined values while positive fieldId values typically refer to LSEG-defined values. When encoding, the FieldEntry.fieldId should match the value that the set definition expects. When decoding, the FieldEntry.fieldId is populated with the fieldId value indicated in the set definition. fieldId has an allowable range of -32,768 to 32,767 where positive values are LSEG-defined and negative values are user-defined. The fieldId value of 0 is reserved to indicate dictionaryId changes, where the type of fieldId 0 is an Int.
DataTypes	Set or get the DataTypes, which defines the DataTypes value of the entry as it encodes or decodes when using this set definition. This can be a base primitive type, a set-defined primitive type, or a container type. - While encoding, populate the FieldEntry.DataTypes with the base primitive type or container type value that corresponds to the type contained in this definition. - While decoding, FieldEntry.DataTypes is populated with the specific DataTypes information as indicated by the Set Definition, where any set-defined primitive type is converted to the corresponding base primitive type. For a map of set-defined primitive types and their corresponding base primitive types, refer to Section 11.6.1.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, you can reuse FieldSetDefEntry without using clear.

Table 125: FieldSetDefEntry Methods

11.6.2.3 ElementSetDef Methods

The following table defines **ElementSetDef** Methods. **ElementSetDef** represents a single element set definition, and can define content for multiple entries in an **ElementList**.

METHOD	DESCRIPTION
setId	Sets or gets the field set definition's identifier value (setId). Any element list content that leverages this definition should have the ElementList.setId matching this identifier. Though setId values have an allowed range of 0 to 32,767, the only values valid for local set definition content are 0 - 15. These indicate locally defined set definition use. For more information, refer to Section 11.6. For more information about how an ElementList indicates use of a set definition, refer to Section 11.3.2.
count	Sets or gets the count, which is the number of ElementSetDefEntrys contained in this definition. Each entry defines how to encode or decode an ElementEntry. A set definition is limited to 255 entries. For more information, refer to Section 11.6.2.4.
entries	Set or get entries, which is an array of ElementSetDefEntrys. Each entry defines how to encode or decode an ElementEntry. For more information, refer to Section 11.6.2.4.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse ElementSetDef without using clear.

Table 126: ElementSetDef Methods

11.6.2.4 ElementSetDefEntry Methods

METHOD	DESCRIPTION
name	Sets or gets the name, which is a Buffer (with position and length) that corresponds to this set-defined element. Element names are defined outside of the Transport API, typically as part of a domain model specification or dictionary. When encoding, you can optionally populate ElementEntry.name with the name expected in the set definition. If name is not used, validation checking is not provided and information might be encoded that does not properly correspond to the definition. When decoding, ElementEntry.name is populated with the information indicated in the set definition. The name buffer allows content length ranging from 0 bytes to 32,767 bytes.
DataTypes	Sets or gets DataTypes. When encoding or decoding an entry using this set definition, DataTypes defines the entry's DataTypes. This can be a base primitive type, a set-defined primitive type, or a container type. - While encoding, populate ElementEntry.DataTypes with the base primitive type or container type value that corresponds to the type contained in this definition. - While decoding, populate ElementEntry.DataTypes with the specific DataTypes information as indicated by set definition, where any set-defined primitive type is converted to the corresponding base primitive type. For a map of set-defined primitive types and their corresponding base primitive types, refer to Section 11.6.1.
clear	Clears this object, so that you can reuse it.
	 TIP: When decoding, you can reuse ElementSetDefEntry without using clear.

Table 127: ElementSetDefEntry Methods

11.6.3 Set Definition Database

A **set definition database** can group definitions together. Using a database can be helpful when the content leverages multiple definitions; the database provides an easy way to pass around all set definitions necessary to encode or decode information. For instance, a **Vector** can contain multiple set definitions via a set definition database with the contents of each **VectorEntry** requiring a different definition from the database.

11.6.3.1 LocalFieldSetDefDb Methods

LocalFieldSetDefDb represents multiple local field set definitions and uses the following Methods.

METHOD	DESCRIPTION
definitions	Returns an array containing up to fifteen FieldSetDefs . Each contained field set definition defines a unique setId for use in the container. This memory is created by the Transport API and should not be overwritten otherwise a garbage collection (GC) will occur. For suggested use, refer to the encoding example in Section 11.6.3.5.
entries	A FieldSetDefEntry that helps manage memory associated with set definition entries for each FieldSetDef . This memory is created by the Enterprise Transport API and should not be overwritten or a GC will occur. - When decoding, the Transport API assigns entries to definitions, according to the Set Definitions being decoded. - When encoding, you can assign entries to definitions, according to the Set Definitions being encoded. Refer to the encoding example in Section 11.6.3.5 for suggested use.
clear	Clears this object, so that you can reuse it.

Table 128: LocalFieldSetDefDb Methods

11.6.3.2 LocalElementSetDefDb Methods

LocalElementSetDefDb (which represents multiple local element set definitions) has the following methods:

METHOD	DESCRIPTION
definitions	An array containing up to fifteen ElementSetDefs . Each contained element set definition defines a unique setId for use within the container on which this is present. Refer to the encoding example in Section 11.6.3.7 for suggested use.  WARNING! This memory is created by the Enterprise Transport API. Do not overwrite this memory or a GC will occur.
entries	An ElementSetDefEntry that helps manage memory associated with set definition entries for each ElementSetDef . - When decoding, the Enterprise Transport API assigns entries to definitions, according to the Set Definitions being decoded. - When encoding, the user can assign entries to definitions, according to the Set Definitions being encoded. Refer to the encoding example in Section 11.6.3.7 for suggested use.  WARNING! This memory is created by the Enterprise Transport API. Do not overwrite this memory or a GC will occur.
clear	Clears this object, so that you can reuse it.

Table 129: LocalElementSetDefDb Methods

11.6.3.3 Local Set Definition Database Encoding Interfaces

Applications can send or receive local set definitions while using the **Map**, **Vector**, or **Series** container types. To provide local set definition information, an application can use the **encodedSetDefs** method with a pre-encoded set definition database, or encode this using the Enterprise Transport API-provided methods described in this section.

The following table describes all available encoding methods required to provide set definition database content on a **Map**, **Vector**, or **Series**. When present, this information should apply to any **FieldList** or **ElementList** content within the types' entries. When encoding set-defined field or element list content, the application must pass **LocalFieldSetDefDb** or **LocalElementSetDefDb** into the **FieldList.EncodeInit** and **ElementList.EncodeInit** methods.

ENCODE INTERFACE	DESCRIPTION
LocalFieldSetDefDb.encode	Encodes a non-pre-encoded local field set definition database into its own buffer for use with encodedSetDefs or directly into a Map , Vector , or Series . After the container's EncodeInit method, local set definition encoding is expected prior to any summary data or container entries.
LocalElementSetDefDb.encode	Encodes a non-pre-encoded local element set definition database into its own buffer for use with encodedSetDefs or directly into a Map , Vector , or Series . After the containers EncodeInit method, local set definition encoding is expected prior to any summary data or container entries.
Map.EncodeSetDefsComplete	Completes encoding non-pre-encoded element or field set definition database content. This applies to local set definition database content on a Map , refer to Section 11.3.3.
Series.EncodeSetDefsComplete	Completes encoding non-pre-encoded element or field set definition database content. This applies to local set definition database content on a Series , refer to Section 11.3.4.
Vector.EncodeSetDefsComplete	Completes encoding non-pre-encoded element or field set definition database content. This applies to local set definition database content on a Vector , refer to Section 11.3.5.

Table 130: Local Set Definition Database Encode Methods

11.6.3.4 Local Set Definition Database Decoding Interfaces

The following table describes decoding methods for use with a local set definition database. When decoding set-defined content, the application can pass the **LocalFieldSetDefDb** or **LocalElementSetDefDb** into the **FieldList.decode** and **ElementList.decode** methods. If this information is not provided, Enterprise Transport API skips decoding set-defined content.

DECODE INTERFACE	DESCRIPTION
LocalFieldSetDefDb.decode	Decodes encodedSetDef into a local field set definition database for use when decoding contained FieldList information.
LocalElementSetDefDb.decode	Decodes encodedSetDef into a local field set definition database for use when decoding contained ElementList information.

Table 131: Local Set Definition Database Decode Methods

11.6.3.5 Field Set Definition Database Encoding Example

The following example demonstrates encoding of a field set definition database into an **Map**. The field set definition database contains one definition, made up of three field set definition entries. After set-defined content encoding is completed, an additional standard data field entry is encoded.

```

/* Create the fieldSetDefDb */
LocalFieldSetDefDb fieldSetDefDb = CodecFactory.createLocalFieldSetDefDb();

/* create entries arrays */
FieldSetDefEntry[] fieldSetDefEntries = new FieldSetDefEntry[3];

/* Contains BID as Real */
fieldSetDefEntries[0] = CodecFactory.createFieldSetDefEntry();
fieldSetDefEntries[0].dataType(DataTypes.REAL);
fieldSetDefEntries[0].fieldId(22);

/* Contains ASK as an optimized Real */
fieldSetDefEntries[1] = CodecFactory.createFieldSetDefEntry();
fieldSetDefEntries[1].dataType(DataTypes.REAL_8RB);
fieldSetDefEntries[1].fieldId(25);

/* Contains TRADE TIME as an optimized Time */
fieldSetDefEntries[2] = CodecFactory.createFieldSetDefEntry();
fieldSetDefEntries[2].dataType(DataTypes.TIME_3);
fieldSetDefEntries[2].fieldId(18);

/* Now populate the entries into the set definition Db. If there were more than one definition, all
   required defs would be populated into the same Db */
/* Structure must be cleared first */
fieldSetDefDb.clear();
/* set the definition into the slot that corresponds to its ID */
/* since this definition is ID 5, it goes into definitions array position 5 */
fieldSetDefDb.definitions()[5].setId(5);
fieldSetDefDb.definitions()[5].count(3);
fieldSetDefDb.definitions()[5].entries(fieldSetDefEntries);

/* begin encoding of map that will contain set def DB - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
map.applyHasSetDefs();
map.containerType(DataTypes.FIELD_LIST);
map.keyPrimitiveType(DataTypes.UINT);
if ((retCode = map.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Map.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else

```

```

{
    /* map init encoding was successful */

    /* It expects the local set definition database to be encoded next */
    /* because we are encoding a local field set definition database, we have to call the correct method
       */
    retCode = fieldSetDefDb.encode(encIter);
    /* Our set definition db is now encoded into the map, we must complete the map portion of this
       encoding and then begin encoding entries */
    retCode = map.encodeSetDefsComplete(encIter, true);
    /* begin encoding of map entry - this contains a field list using the set definition encoded above */
    mapEntry.action(MapEntryActions.ADD);
    uInt.value(100212); /* populate map entry key */
    retCode = mapEntry.encodeInit(encIter, uInt, 0);
    /* set field list flags - this has a setId and set defined data - we can also have standard data after
       set defined data is encoded */
    fieldList.applyHasSetId();
    fieldList.applyHasSetData();
    fieldList.applyHasStandardData();
    fieldList.setId(5); /* this field list will use the set definition from above */
    /* when encoding set defined data, the database containing the necessary definitions must be passed
       in */
    retCode = fieldList.encodeInit(encIter, fieldSetDefDb, 0);
    /* for each field entry we encode that is set defined, the Transport API encoder verifies that the
       correct fieldId and content type are passed in. Order must match definition */

    /* Encode FIRST field in set definition */
    fieldEntry.fieldId(22); /* fieldId of the first set definition entry */
    fieldEntry.dataType(DataTypes.REAL); /* base primitive type of the first set definition entry */
    real.value(227, RealHints.EXPONENT_2);
    /* encode the first entry - this matches the fieldId and type specified in the first definition entry
       */
    retCode = fieldEntry.encode(encIter, real);

    /* Encode SECOND field in set definition */
    fieldEntry.fieldId(25); /* fieldId of the second set definition entry */
    fieldEntry.dataType(DataTypes.REAL); /* base primitive type of the second set definition entry */
    real.value(22801, RealHints.EXPONENT_4);
    /* encode the second entry - this matches the fieldId and type specified in the first definition
       entry */
    retCode = fieldEntry.encode(encIter, real);

    /* Encode THIRD field in set definition */
    fieldEntry.fieldId(18); /* fieldId of the third set definition entry */
    fieldEntry.dataType(DataTypes.TIME); /* base primitive type of the third set definition entry */
    time.hour(8);
    time.minute(39);
    time.second(24);
    /* encode the third entry - this matches the fieldId and type specified in the first definition entry
       */
}

```

```

retCode = fieldEntry.encode(encIter, time);

/* Encode standard data after field set definition is complete */
fieldEntry.fieldId(2); /* fieldId of the first standard data entry after set definition is
complete*/
fieldEntry.dataType(DataTypes.UINT); /* base primitive type of the first set definition entry */
/* encode the standard data in the message after set data is complete */
retCode = fieldEntry.encode(encIter, uInt);

/* complete encoding of the content */
retCode = fieldList.encodeComplete(encIter, true);
retCode = mapEntry.encodeComplete(encIter, true);
retCode = map.encodeComplete(encIter, true);
}

```

Code Example 38: Field Set Definition Database Encoding Example

11.6.3.6 Field Set Definition Database Decoding Example

The following example illustrates how to decode a field set definition database from an **Map**. After decoding the database, it can be passed in while decoding **FieldList** content.

```
/* Decode the map */
retCode = map.decode(decIter);
/* If the map flags indicate that set definition content is present, decode the set def db */
if (map.checkHasSetDefs())
{
    /* must ensure it is the correct type - if map contents are field list, this is a field set definition
       db */
    if (map.containerType() == DataTypes.FIELD_LIST)
    {
        fieldSetDefDb.clear();
        retCode = fieldSetDefDb.decode(decIter);
    }
    /* If map contents are an element list, this is an element set definition db */
    if (map.containerType() == DataTypes.ELEMENT_LIST)
    {
        /* this is an element list set definition db */
    }
}

/* decode map entries */
while ((retCode = mapEntry.decode(decIter, uInt)) != CodecReturnCodes.END_OF_CONTAINER)
{
    if (retCode < CodecReturnCodes.SUCCESS)
    {
        /* decoding failure tends to be unrecoverable */
        System.out.printf("Error (%d) (errno: %d) encountered with MapEntry.decode. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
    }
    else
    {
        /* entries contain field lists - since there were definitions provided they should be passed
           in for field list decoding. Any set defined content will use the definition when
           decoding. If set definition db is not passed in, any set content will not be decoded */
        retCode = fieldList.decode(decIter, fieldSetDefDb);
        /* Continue decoding field entries. See example in Section 11.3.1.6 */
    }
}
```

Code Example 39: Field Set Definition Database Decoding Example

11.6.3.7 Element Set Definition Database Encoding Example

The following example illustrates how to encode an element set definition database into a **Series**. The database contains one element set definition with three element set definition entries. After encoding is completed, the sample encodes an additional standard data element entry.

```
/* Create the elementSetDefDb and element set definition */
LocalElementSetDefDb elementSetDefDb = CodecFactory.createLocalElementSetDefDb();
/* create entries arrays */
ElementSetDefEntry elementSetDefEntries[] = new ElementSetDefEntry[3];

/* Contains BID as a Real */
elementSetDefEntries[0] = CodecFactory.createElementSetDefEntry();
elementSetDefEntries[0].dataType(DataTypes.REAL);
Buffer bidBuffer = CodecFactory.createBuffer();
bidBuffer.data("BID");
elementSetDefEntries[0].name(bidBuffer);

/* Contains ASK as an optimized Real */
elementSetDefEntries[1] = CodecFactory.createElementSetDefEntry();
elementSetDefEntries[1].dataType(DataTypes.REAL_8RB);
Buffer askBuffer = CodecFactory.createBuffer();
askBuffer.data("ASK");
elementSetDefEntries[1].name(askBuffer);

/* Contains TRADE TIME as an optimized Time */
elementSetDefEntries[2] = CodecFactory.createElementSetDefEntry();
elementSetDefEntries[2].dataType(DataTypes.TIME_3);
Buffer tradeTimeBuffer = CodecFactory.createBuffer();
tradeTimeBuffer.data("TRADE TIME");
elementSetDefEntries[2].name(tradeTimeBuffer);

/* Now populate the entries into the set definition Db. If there were more than one definition,
   all required defs would be populated into the same Db */
/* Structure must be cleared first */
elementSetDefDb.clear();
/* set the definition into the slot that corresponds to its ID */
/* since this definition is ID 10, it goes into definitions array position 10 */
elementSetDefDb.definitions()[10].setId(10);
elementSetDefDb.definitions()[10].count(3);
elementSetDefDb.definitions()[10].entries(elementSetDefEntries);

/* begin encoding of series that will contain set def DB - assumes that encIter is already populated with
   buffer and version information, store return value to determine success or failure */
series.applyHasSetDefs();
series.containerType(DataTypes.ELEMENT_LIST);
if ((retCode = series.encodeInit(encIter, 0, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
}
```

```

        System.out.printf("Error (%d) (errno: %d) encountered with Series.encodeInit. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
    }
else
{
    /* series init encoding was successful */
    SeriesEntry seriesEntry = CodecFactory.createSeriesEntry();
    ElementList elementList = CodecFactory.createElementList();
    ElementEntry elementEntry = CodecFactory.createElementEntry();

    /* It expects the local set definition database to be encoded next */
    /* because we are encoding a local element set definition database, we have to call the correct
       method */
    retCode = elementSetDefDb.encode(encIter);
    /* Our set definition db is now encoded into the series, we must complete the series portion of this
       encoding and then begin encoding entries */
    retCode = series.encodeSetDefsComplete(encIter, true);
    /* begin encoding of series entry - this contains an element list using the set definition encoded
       above */
    retCode = seriesEntry.encodeInit(encIter, 0);
    /* set element list flags - this has a setId and set defined data - we can also have standard data
       after set defined data is encoded */
    elementList.applyHasSetId();
    elementList.applyHasSetData();
    elementList.applyHasStandardData();
    elementList.setIds(10); /* this element list will use the set definition from above */
    /* when encoding set defined data, the database containing the necessary definitions must be passed
       in */
    retCode = elementList.encodeInit(encIter, elementSetDefDb, 0);
    /* for each element entry we encode that is set defined, the Transport API encoder verifies that the
       correct element name and content type are passed in. Order must match definition */

    /* Encode FIRST element in set definition */
    elementEntry.name(bidBuffer); /* name of the first set definition entry */
    elementEntry.dataType(DataTypes.REAL); /* base primitive type of the first set definition entry */
    real.value(227, RealHints.EXPONENT_2);
    /* encode the first entry - this matches the name and type specified in the first definition entry */
    retCode = elementEntry.encode(encIter, real);

    /* Encode SECOND element in set definition */
    elementEntry.name(askBuffer); /* name of the second set definition entry */
    elementEntry.dataType(DataTypes.REAL); /* base primitive type of the second set definition entry */
    real.value(22801, RealHints.EXPONENT_4);
    /* encode the second entry - this matches the name and type specified in the second definition entry
       */
    retCode = elementEntry.encode(encIter, real);

    /* Encode THIRD field in set definition */
    elementEntry.name(tradeTimeBuffer); /* name of the third set definition entry */
    elementEntry.dataType(DataTypes.TIME); /* base primitive type of the third set definition entry */

```

```

time.hour(8);
time.minute(39);
time.second(24);
/* encode the third entry - this matches the name and type specified in the third definition entry */
retCode = elementEntry.encode(encIter, time);

/* Encode standard data after element set definition is complete */
Buffer displayTemplateBuffer = CodecFactory.createBuffer();
displayTemplateBuffer.data("DISPLAYTEMPLATE");
elementEntry.name(displayTemplateBuffer); /* name of the first standard data entry after
      set definition is complete*/
elementEntry.dataType(DataTypes.UINT); /* base primitive type of the first set definition entry */
uInt.value(2112);
/* encode the standard data in the message after set data is complete */
retCode = elementEntry.encode(encIter, uInt);

/* complete encoding of the content */
retCode = elementList.encodeComplete(encIter, true);
retCode = seriesEntry.encodeComplete(encIter, true);
retCode = series.encodeComplete(encIter, true);
}

```

Code Example 40: Element Set Definition Database Encoding Example

11.6.3.8 Element Set Definition Database Decoding Example

The following example illustrates how to decode an element set definition database from a **Series**. After decoding the database, it can be passed in while decoding **ElementList** content.

```

/* Decode the series */
retCode = series.decode(decIter);
/* If the series flags indicate that set definition content is present, decode the set def db */
if (series.checkHasSetDefs())
{
    /* must ensure it is the correct type - if series contents are element list,
       this is an element set definition db */
    if (series.containerType() == DataTypes.ELEMENT_LIST)
    {
        elementSetDefDb.clear();
        retCode = elementSetDefDb.decode(decIter);
    }
    /* If map contents are an field list, this is a field set definition db */
    if (series.containerType() == DataTypes.FIELD_LIST)
    {
        /* this is a field list set definition db */
    }
}

/* decode series entries */
while ((retCode = seriesEntry.decode(decIter)) != CodecReturnCodes.END_OF_CONTAINER)
{
    if (retCode < CodecReturnCodes.SUCCESS)
    {
        /* decoding failure tends to be unrecoverable */
        System.out.printf("Error (%d) (errno: %d) encountered with Series.decode. Error Text: %s\n",
            error.errorId(), error.sysError(), error.text());
    }
    else
    {
        /* entries contain element lists - since there were definitions provided they should be passed
           in for element list decoding. Any set defined content will use the definition when
           decoding. If set definition db is not passed in, any set content will not be decoded */
        retCode = elementList.decode(decIter, elementSetDefDb);
        /* Continue decoding element entries. See example in Section 11.3.2.5 */
    }
}

```

Code Example 41: Element Set Definition Database Decoding Example

12 Message Package Detailed View

12.1 Concepts

Messages communicate data between system components: to exchange information, indicate status, permission users and access, and for a variety of other purposes. Many messages have associated semantics for efficient use in market data systems to request information, respond to information, or provide updated information. Other messages have relatively loose semantics, allowing for a more dynamic use either inside or outside market data systems.

An individual flow of related messages within a connection is typically referred to as a **stream**, and the message package allows multiple simultaneous streams to coexist in a connection. An information stream is instantiated between a consuming application and a providing application when the consumer issues a **RequestMsg** followed by the provider responding with a **RefreshMsg** or **StatusMsg**. At this point the stream is established and allows other messages to flow within the stream. The remainder of this chapter discusses streams, stream identification, and stream uniqueness.

The Codec Package offers a suite of message definitions; each optimized to communicate a specific set of information. There are constructs to allow for communication stream identification and to determine uniqueness of streams within a connection. The following sections describe the various constructs, concepts, and processes involved with use of Messages in the Transport API Codec.

12.1.1 Common Message Interface

Each Enterprise Transport API message consists of both unique members and common message methods. The common methods form the **Msg** portion of the message structure, which all other Message interfaces extend.

12.1.1.1 Msg Methods

METHOD	DESCRIPTION
MsgClass	Required on all messages. Sets or gets the MsgClass , which identifies the specific type of a message (e.g. UpdateMsg , RequestMsg). MsgClass allows a range from 0 to 31 , with all values reserved for use by LSEG. For more details about the various message classes, refer to Section 12.1.1.2.
domainType	Required on all messages. Sets or gets the domainType , which identifies the specific domain message model type. domainType allows a range from 0 to 255 , where LSEG-defined values are between 0 and 127 and user-defined values are between 128 and 255 . The domain model definition is decoupled from the API and domain models are typically defined in a specification document. Domain models defined by LSEG are specified in the <i>Transport API LSEG Domain Model Usage Guide</i> , and Domain types are defined in com.refinitiv.eta.rdm.DomainTypes .
containerType	Required on all messages. Sets or gets the containerType , which identifies the type of message payload content and indicates the presence of a container type (value 129 - 224), some type of customer-defined, or non-LSEG Rssl Wire Format container type (225 - 255), or no message payload (128). For more details about container type definitions and use, refer to Section 11.3.
msgKey	Required on an RequestMsg and optional on RefreshMsg , StatusMsg , UpdateMsg , GenericMsg , PostMsg , and AckMsg . Returns the MsgKey , which houses various attributes that help identify contents flowing within a stream. The MsgKey , in conjunction with QoS and domainType , uniquely identifies the stream. The key typically includes naming and service-related information. For more information about the message key and stream identification, refer to Section 12.1.2 and Section 12.1.3.

Table 132: Msg Methods

METHOD	DESCRIPTION
streamId	Required on all messages. Sets or gets streamId, which specifies a unique, signed-integer identifier associated with all messages flowing within a stream. streamId allows a range from -2,147,483,648 to 2,147,483,647, where: - Positive values indicate a consumer-instantiated stream (typically via RequestMsg). - Negative values indicate a provider-instantiated stream (often associated with non-interactive providers). For more information about stream identification and streamId use, refer to Section 12.1.3.
encodedDataBody	Set or get the encodedDataBody, which is a Buffer (with position and length) containing any encoded data contained in the message. If populated, the content type is described by containerType. encodedDataBody would contain only encoded message payload and length information. encodedDataBody can represent up to 4,294,967,295 bytes of payload. This payload length is typically limited by the contained type's specification. - When encoding, encodedDataBody refers to any pre-encoded message payload. - When decoding, encodedDataBody refers to any encoded message payload.
encodedMsgBuffer	Returns the encodedMsgBuffer, which is a Buffer (with position and length) containing the entire encoding of the message. encodedMsgBuffer would contain both encoded message header and encoded message payload. encodedMsgBuffer is typically populated only while decoding, and refers to the entire encoded message header and payload.
extendedHeader	Available for domain-specific user-specified header information. Contents and formatting are defined by the domain model specification. This data is not used in determining stream uniqueness and may not pass through all components. To determine support, refer to the relevant component documentation.
validateMsg	Performs a basic validation on the populated Msg structure (useful when encoding), ensuring that optional members indicated as present are correctly populated (e.g., that length and data are both populated).
isFinalMsg	Returns true if the message is the last message received on a stream, such as: - The final response to non-streaming requests. - A message with a streamState indicating a closed stream (refer to Section 11.2.7). - A message that explicitly closes the stream (e.g. closed with a CloseMsg). Returns false if data is to continue streaming.
copy	Performs a deep copy of a Msg structure. Expects all memory to be owned and managed by the user. - If the memory for the Buffers (i.e. name, attrib, ect.) is not provided, it will be created. - If memory is passed in by the user, the user is responsible for managing the memory.
clear	Clears this object, so that you can reuse it.

Table 132: Msg Methods (Continued)

12.1.1.2 MsgClasses Values

MSG CLASS VALUE	MESSAGE INTERFACE NAME	DESCRIPTION
REQUEST	RequestMsg	Consumers use RequestMsg to express interest in a new stream or modify some parameters on an existing stream; typically results in the delivery of an RefreshMsg or StatusMsg . For more information, refer to Section 12.2.1.
REFRESH	RefreshMsg	The Interactive Provider can use this class to respond to a consumer's request for information (solicited) or provide a data resynchronization point (unsolicited). The non-interactive provider can use this class to initiate a data flow on a new item stream. Conveys state information, QoS, stream permissioning information, and group information in addition to payload. For more information, refer to Section 12.2.2.
UPDATE	UpdateMsg	Providers (of either type) use the UpdateMsg to convey changes to information on a stream. Update messages typically flow on a stream after delivery of a refresh. For more information, refer to Section 12.2.3.
STATUS	StatusMsg	Indicates changes to the stream or data properties. A provider uses StatusMsg to close streams and to indicate successful establishment of a stream when there is no data to convey. For more information, refer to Section 12.2.4. This message can indicate changes: • In streamState or dataState • In a stream's permissioning information • To the item group to which the stream belongs
CLOSE	CloseMsg	A consumer uses CloseMsg to indicate no further interest in a stream. As a result, the stream should be closed. For more information, refer to Section 12.2.5.
GENERIC	GenericMsg	A bi-directional message that does not have any implicit interaction semantics associated with it, thus the name generic. For more information, refer to Section 12.2.6. After a stream is established via a request-refresh/status interaction: • A consumer can send this message to a provider. • A provider can send this message to a consumer. • A non-interactive provider can send this message to the LSEG Real-Time Advanced Distribution Hub.
POST	PostMsg	A consumer uses PostMsg to push content upstream. This information can be applied to an LSEG Real-Time Distribution System cache or routed further upstream to a data source. After receiving posted data, upstream components can republish it to downstream consumers. For more information, refer to Section 12.2.7.
ACK	AckMsg	A provider uses AckMsg to inform a consumer of success or failure for a specific PostMsg or CloseMsg . For more information, refer to Section 12.2.8.

Table 133: MsgClasses Values

12.1.1.3 MsgClasses Methods

METHOD	DESCRIPTION
toString	Returns a Java String representation of a message interface.
	 WARNING! This method creates garbage.

Table 134: MsgClasses Methods

12.1.2 Message Key

The **Message Key** (**msgKey**) houses a variety of attributes that help identify content that flows in a particular stream. A data stream is uniquely identified by the **domainType**, Quality of Service data, and message key.

12.1.2.1 MsgKey Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) to indicate the presence of optional MsgKey members. For more information about flag values, refer to Section 12.1.2.2. - You can use the following convenient methods to set specific MsgKeyFlags: applyHasAttrib, applyHasFilter, applyHasIdentifier, applyHasName, applyHasNameType, applyHasServiceId. - You can use the following convenient methods to check whether specific MsgKeyFlags are set: checkHasAttrib, checkHasFilter, checkHasIdentifier, checkHasName, checkHasNameType, checkHasServiceId.
Id	Sets or gets a service's two-byte, unsigned integer identifier (serviceId); a logical mechanism that provides or enables access to a set of capabilities. serviceId allows a range from 0 to 65,535, with 0 being reserved. This value should correspond to the service content being requested or provided. In the Enterprise Transport API, a service corresponds to a subset of content provided by a component, where the Source Directory domain defines specific attributes associated with each service. These attributes include information such as Quality of Service, the specific domain types available, and any dictionaries required to consume information from the service. The Source Directory domain model can obtain this and other types of information. For details, refer to the <i>Transport API LSEG Domain Model Usage Guide</i>.
nameType	Sets or gets a numeric value (nameType), typically enumerated, that indicates the type of the name member. Examples are User Name or RIC (i.e., the Instrument Code). nameTypes are defined on a per-domain model basis. nameType allows a range from 0 to 255. Name type values and rules are defined within domain message model specifications. Values associated with LSEG domain models can be found in com.refinitiv.eta.rdm.InstrumentNameTypes.
name	Sets or gets the name, which is a Buffer (with position and length) containing the name associated with the contents of the stream. Specific name type and contents should comply with the rules associated with the nameType member. name is a Buffer type that allows for a name of up to 255 bytes.

Table 135: msgKey Methods

METHOD	DESCRIPTION
filter	Sets or gets a filter; a combination of up to 32 unique filterId bit-values (where each filterId corresponds to a filter bit-value) that describe content for domain model types with an FilterList payload. Filter identifier values are defined by the corresponding domain model specification. - When specified in a RequestMsg, filter conveys information which entries to include in responses. - When specified on a message housing an FilterList payload, filter conveys information about which filter entries are present. For more information, refer to Section 11.3.6.
addFilterId	Converts a filterId value into the bit-value representation and adds bit-value to the MsgKey.filter member. Used with FilterList container types. For more information, refer to Section 11.3.6.
checkFilterId	Converts a filterId value into the bit-value representation and checks for the bit-value presence in the msgKey.filter member. Used with FilterList container types. For more information, refer to Section 11.3.6.
identifier	User-specified numeric identifier defined on a per-domain model basis. identifier allows a range from -2,147,483,648 to 2,147,483,647. NOTE: More information should be present as part of the specific domain model definition.
attribContainerType	Sets or gets the content type (attribContainerType) of the msgKey.encodedAttrib information. Can indicate the presence of a container type (value 129 - 224) or some type of customer-defined container type (225 - 255). For more details about container type definitions and use, refer to Section 11.3.
encodedAttrib	Sets or gets name, which is a Buffer (with position and length) containing additional, encoded, message key attribute information. If populated, contents are described by the attribContainerType member. Additional attribute information typically allows for further uniqueness in the identification of a stream. encodedAttrib is a Buffer that can represent up to 32,767 bytes of information.
equals	Compares this msgKey to another MsgKey, to determine whether they are the same. Returns true if the keys match; false otherwise.
copy	Performs a deep copy of a MsgKey. Expects all memory to be owned and managed by user. If the memory for the Buffers (i.e. name, attrib) are not provided, they will be created.
clear	Clears this object, so that you can reuse it.  TIP: When decoding, the MsgKey object can be reused without using clear.

Table 135: msgKey Methods (Continued)

12.1.2.2 Message Key Flag Enumeration Values

MSG KEY FLAG	MEANING
MsgKeyFlags.HAS_SERVICE_ID	Indicates the presence of the serviceId member.
MsgKeyFlags.HAS_NAME	Indicates the presence of the name member.
MsgKeyFlags.HAS_NAME_TYPE	Indicates the presence of the nameType member.
MsgKeyFlags.HAS_FILTER	Indicates the presence of the filter member.
MsgKeyFlags.HAS_IDENTIFIER	Indicates the presence of the identifier member.
MsgKeyFlags.HAS_ATTRIB	Indicates the presence of the attribContainerType and encodedAttrib members.

Table 136: MsgKeyFlags Values

12.1.3 Stream Identification

The Enterprise Transport API allows users to simultaneously interact across multiple, independent data streams within a single network connection. Each data stream can be uniquely identified by the specified **domainType**¹, QoS, and **msgKey** contents. The **msgKey** contains a variety of attributes used in defining a stream. To avoid repeatedly sending **msgKey** and QoS on all messages in a stream², a signed integer (referred to as a **StreamId** or stream identifier) is used. This **streamId** can convey all of the same stream identification information, but consumes only a small, fixed-size (four bytes). A positive value **streamId** indicates a consumer-instantiated stream while a negative value **streamId** indicates a provider-instantiated stream, usually, but not always, associated with a non-interactive provider application.

For a consumer application, a positive value **streamId** should be specified on any **RequestMsg**, along with the **domainType**, **msgKey** and additional key attributes, and desired QoS information. An interactive provider application should provide a response, typically a **RefreshMsg**, which contains the same **streamId**, **domainType**, and message key information. If the request specified a QoS range, this response will also contain the concrete or actual QoS being provided for the stream. For more information about QoS, refer to Section 11.2.6.

For a non-interactive provider, the initial **RefreshMsg** published for each item should contain **domainType**, message key information, and the QoS being provided for the stream. In addition, the non-interactive provider should specify a negative value **streamId** to be associated with the stream for the remainder of the run-time.

12.1.3.1 Stream Comparison

To most efficiently use a connection's bandwidth, LSEG recommends that you combine like streams when possible. Two streams are identical when all identifying aspects match - that is the two streams have the same **domainType**, provided QoS, and all **msgKey** members. When these message members match, a new stream should not be established, rather the existing stream and **streamId** should be leveraged to consume or provide this content.

A consumer application can issue a subsequent **RequestMsg** using the existing **streamId**, referred to as a **reissue**. This allows the consumer application to obtain an additional refresh, if desired, and to indicate a change in the priority of the stream. The additional solicited **RefreshMsg** can satisfy the additional request, and any **StatusMsg**, **UpdateMsg**, and **GenericMsg** content can be provided to both requestors, if different. This behavior is called fan-out and is the responsibility of the consumer application when combining multiple like-streams into a single stream.

A provider application can choose to allow multiple like-streams to be simultaneously established or, more commonly, it can reject any subsequent requests on a different **streamId** using an **StatusMsg**. In this case, the **StatusMsg** would contain a **streamState** of **StreamStates.CLOSED_RECOVER**, a **dataState** of **DataStates.SUSPECT**, and a state **code** of **StateCodes.ALREADY_OPEN**. This status message informs the consumer that they already have a stream open for this information and that they should use the existing **streamId** when re-requesting this content. For more details about the state information, refer to Section 11.2.7.

1. When off-stream posting, it is possible for the post messages sent on the Login stream to contain a different **domainType**. This is a specialized use case and more information is available in Section 13.9.
2. **domainType** is present on all messages and cannot be optimized out like quality of service and **msgKey** information.

12.1.3.2 Private Streams

The Enterprise Transport API provides *private stream* functionality, an easy way to ensure delivery of content only between a stream's two endpoints. Private streams behave in a manner similar to standard streams, with the following exceptions:

- All data on a private stream flow between the end provider and the end consumer of the stream.
- Intermediate components do not fan out content (i.e., do not distribute it to other consumers).
- Intermediate components should not cache content.
- In the event of connection or data loss, intermediate components do not recover content. All private stream recovery is the responsibility of the consumer application.

These behaviors ensure that only the two endpoints of the private stream send or receive content associated with the stream. As a result, a private stream can exchange identifying information so the provider can validate the consumer, even through multiple intermediate components (such as might exist in an LSEG Real-Time Distribution System deployment). After a private stream is established, content can flow freely within the stream, following either existing market data semantics (i.e., private Market Price domain) or any other user-defined semantics (i.e., bidirectional exchange of **GenericMsgs**).

For more information about private stream instantiation, refer to Section 13.13.

12.1.3.3 Changeable Stream Attributes

A select number of attributes may change during the life of a stream. A consumer can change attributes via a subsequent **RequestMsg** that uses the same **streamId** as previous requests. A provider of either type can change attributes via a subsequent solicited or unsolicited **RefreshMsg**.

The message key's **filter** member, though not typical, can change between the consumer request and provider response. A change is likely due to a difference between the filter entries for which the consumer asks and the filter entries that the provider can provide. If this behavior is allowed within a domain, it is defined on a per-domain model basis. More information should be present as part of the specific domain model definition.

Contents of the message key's **encodedAttrib** may change. If this behavior is allowed within a domain, it is defined on a per-domain model basis. More information should be present as part of the specific domain model definition.

A consumer can change the **priorityClass** or **priorityCount** via a subsequent **RequestMsg** to indicate more or less interest in a stream. For more information, refer to Section 13.2.

If a Quality of Service range is requested, the provided **RefreshMsg** includes only the concrete Quality of Service, which may be different from the best and worst specified. If a **dynamic** Quality of Service is supported, Quality of Service may occasionally change over the life of the stream, however this should stay within the range requested in **RequestMsg**.

An item's identification might also change, which can result in changes to multiple **msgKey** members. Such a case can occur via a **redirect**, a **RefreshMsg** or **StatusMsg** with a **streamState** of **StreamStates.REDIRECTED** (for more information on the redirected state value, refer to see Section 11.2.7.2). The user can determine the original item identification from the **msgKey** information previously associated with the **streamId** contained in the redirect message. The new item identification that should be requested is provided via the redirect's **msgKey** member. When a redirect occurs, the stream closes. At this point, the user can open a new stream and continue the flow of data by issuing a new **RequestMsg**, containing the redirected **msgKey**.

Some **RequestMsg.flag** values are allowed to change over the life of a stream. These values include the **RequestMsgFlags.PAUSE** and **RequestMsgFlags.STREAMING** flags, used when pausing or resuming content flow on a stream. For more details, refer to Section 13.6. Additionally, the **RequestMsgFlags.NO_REFRESH** flag can be changed. This allows subsequent reissue requests to be performed where the user does not require a response - this can be useful for a reissue to change the priority of a stream.

12.2 Messages

12.2.1 Request Message Interface

The **RequestMsg** interface extends the **Msg** interface. A consumer uses a **RequestMsg** to express interest in a particular information stream. The request's **msgKey** members help identify the stream and priority information can be used to indicate the stream's importance to the consumer. QoS information can be used to express either a specific desired QoS or a range of acceptable qualities of service that can satisfy the request (refer to Section 13.3).

When a **RequestMsg** is issued with a new **streamId**, this is considered a request to open the stream. If requested information is available and the consumer is entitled to receive the information, this typically results in a **RefreshMsg** being delivered to the consumer, though a **StatusMsg** is also possible - either message can be used to indicate a stream is open. If information is not available or the user is not entitled, a **StatusMsg** is typically delivered to provide more detailed information to the consumer.

Issuing a **RequestMsg** on an existing stream allows a consumer to modify some parameters associated with the stream (also refer to Section 12.1.3.2). Also known as a *reissue*, this can be used to pause or resume a stream (also refer to Section 13.6), change a Dynamic View (also refer to Section 13.8), increase or decrease the stream's priority (also refer to Section 13.2) or request a new refresh.

12.2.1.1 RequestMsg Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) to indicate special behaviors and the presence of optional RequestMsg content. For more information about flag values, refer to Section 12.2.1.2. - You can use the following convenient methods to set specific RequestMsgFlags: applyConfInfoInUpdates, applyHasBatch, applyHasExtendedHdr, applyHasPriority, applyHasQos, applyHasView, applyHasWorstQos, applyMsgKeyInUpdates, applyNoRefresh, applyPause, applyPrivateStream, applyStreaming. - You can use the following convenient methods to check whether specific RequestMsgFlags are set: checkConfInfoInUpdates, checkHasBatch, checkHasExtendedHdr, checkHasPriority, checkHasQos, checkHasView, checkHasWorstQos, checkMsgKeyInUpdates, checkNoRefresh, checkPause, checkPrivateStream, checkStreaming.
priority	Returns a Priority object which you can use to set or get the priorityClass and priorityCount. Priority.priorityClass can contain values ranging from 0 to 255. Priority.priorityCount can contain values ranging from 0 to 65,535. For more information about Priority and its uses, refer to Section 13.2.

Table 137: RequestMsg Methods

METHOD	DESCRIPTION
Qos	Returns a Qos object which you can use to set or get the allowable QoS for the requested stream. - When specified without a worstQos member, this is the only allowable QoS for the requested stream. If this QoS is unavailable, the stream is not opened. - When specified with a worstQos, this is the best in the range of allowable QoSs. When a QoS range is specified, any QoS within the range is acceptable for servicing the stream. - If neither Qos nor worstQos are present on the request, this indicates that any available QoS will satisfy the request. Some components may require Qos on initial request and reissue messages. See specific component documentation for details. - For more information, refer to Section 11.2.6. - For specific handling information, refer to Section 13.3.
worstQos	Returns a Qos object which you can use to set or get the least acceptable QoS for the requested stream. When specified with a Qos value, this is the worst in the range of allowable QoSs. When a QoS range is specified, any QoS within the range is acceptable for servicing the stream. - For more information, refer to Section 11.2.6. - For specific handling information, refer to Section 13.3.

Table 137: RequestMsg Methods (Continued)

12.2.1.2 RequestMsgFlags Values

REQUEST MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
RequestMsgFlags.STREAMING	Indicates whether the request is for streaming data. - If present, the consumer wants to continue to receive changes to information after the initial refresh is complete. - If absent, the consumer wants to receive only the refresh, after which the provider should close the stream. Such a request is typically referred to as a non-streaming or snapshot data request. Because a refresh can be split into multiple parts, it is possible for updates to occur between the first and last part of the refresh, even as part of a non-streaming request. For more information about multi-part message handling, refer to Section 13.1.
RequestMsgFlags.NO_REFRESH	Indicates that the consumer application does not require a refresh for this request. This typically occurs after an initial request handshake is completed, usually to change stream attributes (e.g., priority). In some instances, a provider might still deliver a refresh message (but if the consumer does not explicitly ask for it, the message is unsolicited).
RequestMsgFlags.PAUSE	Indicates that the consumer would like to pause the stream, though this does not guarantee that the stream will pause. To resume data flow, the consumer must send a subsequent request message with the RequestMsgFlags.STREAMING flag set. For more information, refer to Section 13.6.
RequestMsgFlags.HAS_PRIORITY	Indicates the presence of the priority member, which contains priorityClass and priorityCount members. For more information about using priority, refer to Section 13.2.

Table 138: RequestMsgFlags Values

REQUEST MSG FLAG	MEANING
RequestMsgFlags.HAS_QOS	Indicates the presence of the Qos member. - For more information, refer to Section 12.2.1.1 and Section 11.2.6. - For specific handling information, refer to Section 13.3.
RequestMsgFlags.HAS_WORST_QOS	Indicates the presence of the worstQos member. - For more information, refer to Section 12.2.1.1 and Section 11.2.6. - For specific handling information, refer to Section 13.3.
RequestMsgFlags.HAS_VIEW	Indicates that the request message payload might contain a dynamic view, specifying information the application wishes to receive (or that the application wishes to continue receiving a previously specified view). If this flag is not present, any previously specified view is discarded and a full view is provided. For more information about using dynamic views, refer to Section 13.8.
RequestMsgFlags.HAS_BATCH	Indicates that the request message payload contains a list of items of interest, all with matching msgKey information. For more information on using batch requests, refer to Section 13.7.
RequestMsgFlags.HAS_EXTENDED_HEADER	Indicates that the extendedHeader member is present. Information in the extendedHeader is defined outside of the scope of the Enterprise Transport API.
RequestMsgFlags.MSG_KEY_IN_UPDATES	Indicates that the consumer wants to receive the full msgKey in update messages. This flag does not guarantee that the msgKey is present in an update message. Instead, the provider application determines whether this information is present (the consumer should be written to handle either the presence or absence of msgKey in any UpdateMsg). When specified on a request to an LSEG Real-Time Advanced Distribution Server, the server fulfills the request.
RequestMsgFlags.CONF_INFO_IN_UPDATES	Indicates that the consumer wants to receive conflation information in update messages delivered on this stream. This flag does not guarantee that conflation information is present in update messages. Instead, the provider application determines whether this information is present (the consumer should be capable of handling conflation information in any UpdateMsg). For details about conflation information on update messages, refer to Section 12.2.3.
RequestMsgFlags.PRIVATE_STREAM	Requests that the stream be opened as private. For details, refer to Section 13.13.

Table 138: RequestMsgFlags Values (Continued)

12.2.2 Refresh Message Interface

The **RefreshMsg** interface extends the **Msg** interface. **RefreshMsg** is often provided as an initial response or when an upstream source requires a data resynchronization point. A **RefreshMsg** contains payload information along with state, Quality of Service, permissioning, and group information.

- If provided as a response to a **RequestMsg**, the refresh is a **solicited refresh**. Typically, solicited refresh messages are delivered only to the requesting consumer application
- If some kind of information change occurs (e.g., some kind of error is detected on a stream), an upstream provider can push out an **RefreshMsg** to downstream consumers. This type of refresh is an **unsolicited refresh**. Typically, unsolicited refresh messages are delivered to all consumers using each consumer's respective stream.

When an OMM interactive provider sends a **RefreshMsg**, the **streamId** should match the **streamId** on the corresponding **RequestMsg**. The **msgKey** should be populated with the appropriate stream identifying information, and often matches the **msgKey** of the request. When a non-interactive provider sends a **RefreshMsg**, the provider should assign a negative **streamId** (when establishing a new stream, the **streamId** should be unique). In this scenario, the **msgKey** should define the information that the stream provides.

Using **RefreshMsg**, an application can fragment the contents of a message payload and deliver the content across multiple messages, with the final message indicating that the refresh is complete. This is useful when providing large sets of content that may require multiple cache look-ups or be too large for an underlying transport layer. Additionally, an application receiving multiple parts of a response can potentially begin processing received portions of data before all content has been received. For more details on multi-part message handling, refer to Section 13.1.

12.2.2.1 RefreshMsg Methods

METHOD	DESCRIPTION
flags	Set or get flags, which is a combination of bit values that indicate special behaviors and the presence of optional RefreshMsg content. For more information about flag values, refer to Section 12.2.2.2. • You can use the following convenient methods to set specific RefreshMsgFlags: applyClearCache, applyDoNotCache, applyHasExtendedHdr, applyHasMsgKey, applyHasPartNum, applyHasPermData, applyHasPostUserInfo, applyHasQos, applyHasSeqNum, applyPrivateStream, applyRefreshComplete, applySolicited. • You can use the following convenient methods to check whether specific RefreshMsgFlags are set: checkClearCache, checkDoNotCache, checkHasExtendedHdr, checkHasMsgKey, checkHasPartNum, checkHasPermData, checkHasPostUserInfo, checkHasQos, checkHasSeqNum, checkPrivateStream, checkRefreshComplete, checkSolicited.
partNum	Sets or gets the part number (partNum) of this refresh. partNum can contain values ranging from 0 to 32,767 where a value of 0 indicates the initial part of a refresh. • On multi-part refresh messages, partNum should start at 0 (to indicate the initial part) and increment by 1 for each subsequent message in the multi-part message. • If sent on a single-part refresh, a partNum of 0 should be used.
seqNum	Sets or gets a user-defined sequence number (seqNum), which allows for values ranging from 0 to 4,294,967,295. seqNum should typically increase to help with temporal ordering, but may have gaps depending on the sequencing algorithm in use. Details about sequence number use should be defined within the domain model specification or any documentation for products which require the use of seqNum.

Table 139: RefreshMsg Methods

METHOD	DESCRIPTION
State	Returns a State object which you can use to set or get stream and data state information, which can change over time via subsequent refresh, status messages, or group status notifications. - For details about state information, refer to Section 11.2.7. - For a decision table that provides example behavior for various state combinations, refer to Appendix A.
qos	Returns a Quality of Service object which you can use to set or get the concrete Quality of Service of the stream. If a range was requested by the RequestMsg, the qos should fall somewhere in this range, otherwise qos should exactly match what was requested. - For more details on Quality of Service, refer to Section 11.2.6. - For specific handling information, refer to Section 13.3.
permData	Optional. Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for this stream. permData has a maximum allowed length of 32,767 bytes. When permData is specified on an RefreshMsg, this indicates authorization information for all content on the stream, unless additional permission information is provided with specific content (e.g., MapEntry.permData). For more information, refer to Section 11.4.
groupId	Sets or gets groupId, which is a Buffer (with position and length) containing information about the item group to which this stream belongs. The groupId Buffer has a maximum allowed length of 255 bytes. You can change the associated groupId via a subsequent StatusMsg or RefreshMsg. Group status notifications can change the state of an entire group of items. For more information about item groups, refer to Section 13.4.
postUserInfo	Optional. Returns a PostUserInfo object which can be used to set or get information that identifies the user posting this information. If present on an RefreshMsg, this implies that the refresh was posted to the system by the user described in postUserInfo. - For more information about posting, refer to Section 13.9. - For more information about the Visible Publisher Identifier, refer to Section 13.11.

Table 139: RefreshMsg Methods (Continued)

12.2.2.2 RefreshMsgFlags Values

REFRESH MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
RequestMsgFlags.REFRESH_COMPLETE	Indicates that the message is the final part of the RefreshMsg . This flag value should be set when: - The message is a single-part refresh (i.e., atomic refresh). - The message is the final part of a multi-part refresh. For more information about multi-part message handling, refer to Section 13.1.
RequestMsgFlags.SOLICITED	Indicates that the refresh is sent as a response to a request, referred to as a solicited refresh. A refresh sent to inform a consumer of an upstream change in information (i.e., an unsolicited refresh) must not include this flag.
RequestMsgFlags.DO_NOT_CACHE	Indicates that the message's payload information should not be cached. This flag value applies only to the message on which it is present.
RequestMsgFlags.CLEAR_CACHE	Indicates that the stream's stored payload information should be cleared. This is typically set by providers when: - Sending the initial solicited RefreshMsg. - Sending the first part of a multi-part RefreshMsg. - Some portion of data is known to be invalid.
RequestMsgFlags.HAS_MSG_KEY	Indicates that the RefreshMsg contains a populated msgKey . This can aid in associating a request with its corresponding refresh or identify an item sent from a non-interactive provider application.
RequestMsgFlags.HAS_QOS	Indicates the presence of the Qos member. For specific handling information, refer to Section 13.3.
RequestMsgFlags.HAS_SEQ_NUM	Indicates the presence of the seqNum member.
RequestMsgFlags.HAS_PART_NUM	Indicates the presence of the partNum member.
RequestMsgFlags.HAS_PERM_DATA	Indicates the presence of the permData member.
RequestMsgFlags.HAS_POST_USER_INFO	Indicates that this message includes postUserInfo , implying that this RefreshMsg was posted by the user described in postUserInfo .
RequestMsgFlags.HAS_EXTENDED_HEADER	Indicates the presence of the extendedHeader member.
RequestMsgFlags.PRIVATE_STREAM	Acknowledges the initial establishment of a private stream or, when combined with a streamState value of StreamStates.REDIRECTED , indicates that a stream can only be opened as private. For details, refer to Section 13.13.

Table 140: RefreshMsgFlags Values

12.2.3 Update Message Interface

The **UpdateMsg** interface extends the **Msg** interface. Providers (both interactive and non-interactive) use **UpdateMsg** to convey changes to data associated with an item stream. When streaming, update messages typically flow after the delivery of an initial refresh. Update messages can be delivered between parts of a multi-part refresh message, even in response to a non-streaming request. For more information on multi-part message handling, refer to Section 13.1.

Some providers can aggregate the information from multiple update messages into a single update message using a technique called conflation. Conflation typically occurs if a conflated QoS is requested (refer to Section 11.2.6), a stream is paused (refer to Section 13.6), or if a consuming application is unable to keep up with a stream's data rates. If conflation is used, specific information can be provided with **UpdateMsg** via optional conflation information.

12.2.3.1 UpdateMsg Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate special behaviors and the presence of optional content. For more information about flag values, refer to Section 12.2.3.2. - You can use the following convenient methods to set specific UpdateMsgFlags: applyDiscardable, applyDoNotCache, applyDoNotConflate, applyDoNotRipple, applyHasConfInfo, applyHasExtendedHdr, applyHasMsgKey, applyHasPermData, applyHasPostUserInfo, applyHasSeqNum. - You can use the following convenient methods to check whether specific UpdateMsgFlags are set: checkDiscardable, checkDoNotCache, checkDoNotConflate, checkDoNotRipple, checkHasConfInfo, checkHasExtendedHdr, checkHasMsgKey, checkHasPermData, checkHasPostUserInfo, checkHasSeqNum.
updateType	Sets or gets the type of data (updateType) in the UpdateMsg, where values are typically defined in an enumeration (valid values range from 0 to 255). Examples of possible update types include: Trade, Quote, or Closing Run. - Domain message model specifications define available update types. - For domain models provided by LSEG, com.refinitiv.eta.rdm.UpdateEventTypes defines available update types.
seqNum	Sets or gets a user-defined sequence number (seqNum), which can range in value from 0 to 4,294,967,295. To help with temporal ordering, seqNum should increase across messages, but can have gaps depending on the sequencing algorithm in use. Details about sequence number use should be defined within the domain model specification or any documentation for products which require the use of seqNum.
conflationCount	Sets or gets the conflationCount. When conflating data, this value indicates the number of updates conflated or aggregated into this UpdateMsg. conflationCount allows for values ranging from 1 to 32,767.
conflationTime	Sets or gets the conflationTime. When conflating data, this value indicates the period of time over which individual updates were conflated or aggregated into this UpdateMsg (typically in milliseconds; for further details, refer to specific component documentation). conflationTime allows for values ranging from 1 to 65,535.

Table 141: UpdateMsg Methods

METHOD	DESCRIPTION
permData	Optional. Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for this stream. When specified, permData indicates authorization information for only the content within this message, though this can be overridden for specific content within the message (e.g., MapEntry.permData). permData has a maximum allowed length of 32,767 bytes. For more information, refer to Section 11.4.
postUserInfo	Optional. Returns a postUserInfo object that you can use to set or get information that identifies - For more information about posting, refer to Section 13.9. - For more information about the Visible Publisher Identifier, refer to Section 13.11.

Table 141: UpdateMsg Methods (Continued)

12.2.3.2 UpdateMsgFlags Values

UPDATE MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
UpdateMsgFlags.DISCARDABLE	Indicates that this update can be discarded. Common for options with no open interest.
UpdateMsgFlags.DO_NOT_CACHE	Indicates that payload information associated with this message should not be cached. UpdateMsgFlags.DO_NOT_CACHE applies only to the message on which it is present.
UpdateMsgFlags.DO_NOT_CONFLATE	Indicates that this message should not be conflated. This flag value only applies to the message on which it is present.
UpdateMsgFlags.DO_NOT_RIPPLE	Indicates that the contents of this message should not be rippled. Rippling is typically associated with a FieldList . For additional information, refer to Section 11.3.1.4.
UpdateMsgFlags.HAS_MSG_KEY	Indicates that the UpdateMsg contains a populated msgKey . The additional key information can help associate a request with updates or identify an item being sent from a non-interactive provider application. This information is typically not necessary in an UpdateMsg as the streamId can be used to determine the same information with less bandwidth cost.
UpdateMsgFlags.HAS_SEQ_NUM	Indicates the presence of the seqNum member.
UpdateMsgFlags.HAS_CONF_INFO	Indicates the presence of conflationTime and conflationCount information.
UpdateMsgFlags.HAS_PERM_DATA	Indicates the presence of the permData member.
UpdateMsgFlags.HAS_POST_USER_INFO	Indicates that this message includes postUserInfo , implying that this UpdateMsg was posted by the user described in the postUserInfo .
UpdateMsgFlags.HAS_EXTENDED_HEADER	Indicates the presence of the extendedHeader member.

Table 142: UpdateMsgFlags Values

12.2.4 Status Message Interface

The **StatusMsg** interface extends the **Msg** interface. A **StatusMsg** can convey changes in **streamState** or **dataState** (refer to Section 11.2.7), changes in a stream's permissioning information (refer to Section 9.4), or changes to the item group of which the stream is a part (refer to Section 13.4). A Provider application uses **StatusMsg** to close streams to a consumer, in conjunction with an initial request or later after the stream has been established. A **StatusMsg** can also indicate the successful establishment of a stream, though the message might not contain data (useful in establishing a stream solely to exchange bi-directional **GenericMsgs**).

12.2.4.1 StatusMsg Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) indicating special behaviors and the presence of optional content. For more information about flag values, refer to Section 12.2.4.2. - You can use the following convenient methods to set specific StatusMsgFlags: applyClearCache, applyHasExtendedHdr, applyHasGroupId, applyHasMsgKey, applyHasPermData, applyHasPostUserInfo, applyHasState, applyPrivateStream. - You can use the following convenient methods to check whether specific StatusMsgFlags are set: checkClearCache, checkHasExtendedHdr, checkHasGroupId, checkHasMsgKey, checkHasPermData, checkHasPostUserInfo, checkHasState, checkHasPrivateStream.
state	Returns a State object that you can use to set or get stream and data state information, which can change over time via subsequent refresh or status messages or group status notifications. - For details about state information, refer to Section 11.2.7. - For a decision table that provides example behavior for various state combinations, refer to Appendix A.
permData	Optional. Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for this stream, unless additional permission information is provided with specific content (e.g., MapEntry.permData). permData allows a maximum length of 32,767 bytes. For more information, refer to Section 11.4.
groupId	Sets or gets the groupId, which is a Buffer (with position and length) with a maximum allowed length of 255 bytes that contains information about the item group to which this stream belongs. A subsequent StatusMsg or RefreshMsg can change the item group's associated groupId, while group status notifications can change the state of an entire group of items. For more information about item groups, refer to Section 13.4.
postUserInfo	Optional. Returns a PostUserInfo object that you can use to set or get information that identifies the user who posted this information. - For more information about posting, refer to Section 13.9. - For more information about Visible Publisher Identifier, refer to Section 13.11.

Table 143: StatusMsg Methods

12.2.4.2 StatusMsgFlags Values

STATUS MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
StatusMsgFlags.CLEAR_CACHE	Indicates that the application should clear stored header or payload information associated with the stream. This can happen if some portion of data is invalid.
StatusMsgFlags.HAS_MSG_KEY	Indicates that the StatusMsg contains a populated MsgKey . The MsgKey can be used to aid in associating a request to a status message or identify an item sent from an non-interactive provider application.
StatusMsgFlags.HAS_STATE	Indicates the presence of State information. If State information is not present, the message might be changing the stream's permission information or groupId .
StatusMsgFlags.HAS_PERM_DATA	Indicates the presence of permData . When present, the message might be changing the stream's permission information.
StatusMsgFlags.HAS_GROUP_ID	Indicates the presence of groupId . When present, the message might be changing the stream's groupId .
StatusMsgFlags.HAS_POST_USER_INFO	Indicates the presence of postUserInfo , which identifies the user who posted the StatusMsg .
StatusMsgFlags.HAS_EXTENDED_HEADER	Indicates the presence of extendedHeader .
StatusMsgFlags.PRIVATE_STREAM	Acknowledges the establishment of a private stream, or when combined with a streamState value of StreamStates.REDIRECTED , indicates that a stream can be opened only as private. For details, refer to Section 13.13.

Table 144: Flags Values

12.2.5 Close Message Interface

The **CloseMsg** interface extends the **Msg** interface. A consumer uses **CloseMsg** to indicate no further interest in an item stream and to close the stream. The **streamId** indicates the item stream to which **CloseMsg** applies.

12.2.5.1 CloseMsg Methods

METHOD	DESCRIPTION
flags apply* check*	Sets or gets a combination of bit values (flags) that indicate special behaviors and the presence of optional content. For available flag values, refer to CloseMsgFlags in Section 12.2.5.2. - You can use the following convenient methods to set specific StatusMsgFlags: applyAck, applyHasExtendedHdr. - You can use the following convenient methods to check whether specific StatusMsgFlags are set: checkAck, checkHasExtendedHdr.

Table 145: CloseMsg Methods

12.2.5.2 CloseMsgFlags Values

CLOSE MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
CloseMsgFlags.ACK	If present, the consumer wants the provider to send an AckMsg to indicate that the CloseMsg has been processed properly and the stream is properly closed. This functionality might not be available with some components; for details, refer to the component's documentation.
CloseMsgFlags.HAS_EXTENDED_HEADER	Indicates the presence of extendedHeader .

Table 146: CloseMsgFlags Values

12.2.6 Generic Message Class

The **GenericMsg** interface extends the **Msg** interface. **GenericMsg** is a bi-directional message without any implicit interaction semantics associated with it, hence the name generic. After a stream is established via a request-refresh/status interaction, both consumers and providers can send **GenericMsgs** to one another, and non-interactive provider applications can leverage them. Generic messages are transient and typically not cached by LSEG Real-Time Distribution System components.

The **msgKey** of an **GenericMsg** does not need to match the **msgKey** information of the stream over which the generic message flows. Thus, key information can be used independently within the stream. A domain message model specification typically defines any specific message usage, **msgKey** usage, expected interactions, and handling instructions.

12.2.6.1 GenericMsg Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate special behaviors and the presence of optional content. For more information about flag values, refer to Section 12.2.6.2. - You can use the following convenient methods to set specific GenericMsgFlags: applyHasExtendedHdr, applyHasMsgKey, applyHasPartNum, applyHasPermData, applyHasSecondarySeqNum, applyHasSeqNum, applyMessageComplete. - You can use the following convenient methods to check whether specific GenericMsgFlags are set: checkHasExtendedHdr, checkHasMsgKey, checkHasPartNum, checkHasPermData, checkHasSecondarySeqNum, checkHasSeqNum, checkMessageComplete.
partNum	Sets or gets the part number (partNum) of this generic message, typically used with multi-part generic messages. partNum can contain values ranging from 0 to 32,767, where a value of 0 indicates the initial part of a refresh. - If sent on a single-part post message, use a partNum of 0. - On multi-part post messages, use a partNum of 0 on the initial part and increment partNum in each subsequent part by 1.
seqNum	Sets or gets a user-defined sequence number (seqNum) ranging in value from 0 to 4,294,967,295. A seqNum typically corresponds to the sequencing of this message. To help with temporal ordering, seqNum should increase across messages, but can have gaps depending on the sequencing algorithm in use. Details about using seqNum should be defined in the domain model specification or the documentation for products that must use seqNum.
secondarySeqNum	Sets or gets an additional user-defined sequence number (secondarySeqNum) ranging in value from 0 to 4,294,967,295. When using GenericMsg on a stream in a bi-directional manner, secondarySeqNum is often used as an acknowledgment sequence number. For example, a consumer sends a generic message with seqNum populated to indicate the sequence of this message in the stream and secondarySeqNum set to the seqNum last received from the provider. This effectively acknowledges all messages received up to that point while still sending additional information. Sequence number use should be defined within the domain model specification or any documentation for products that use secondarySeqNum.
permData	Optional. Sets or gets permData, which is a Buffer (with position and length) that indicates authorization information for content within this message only, though this can be overridden for specific content within the message (e.g. MapEntry.permData). permData allows a maximum length of 32,767 bytes. For more information, refer to Section 11.4.

Table 147: GenericMsg Methods

12.2.6.2 GenericMsgFlags Values

GENERIC MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
GenericMsgFlags.MESSAGE_COMPLETE	When set, this flag indicates that the message is the final part of an GenericMsg . This flag should be set on: - Single-part generic messages (i.e., an atomic generic message). - The last message (final part) in a multi-part generic message. For more information on handling multi-part messages, refer to Section 13.1.
GenericMsgFlags.HAS_MSG_KEY	Indicates the presence of a populated msgKey . Use of a msgKey differentiates a generic message from the msgKey information specified for other messages within the stream. Contents and semantics associated with an GenericMsg.msgKey should be defined by the domain model specification that employs them.
GenericMsgFlags.HAS_SEQ_NUM	Indicates the presence of the seqNum member.
GenericMsgFlags.HAS_SECONDARY_SEQ_NUM	Indicates the presence of the secondarySeqNum member.
GenericMsgFlags.HAS_PART_NUM	Indicates the presence of the partNum member.
GenericMsgFlags.HAS_PERM_DATA	Indicates the presence of the permData member.
GenericMsgFlags.HAS_EXTENDED_HEADER	Indicates presence of the extendedHeader member.

Table 148: GenericMsgFlagsValues

12.2.7 Post Message Interface

The **PostMsg** interface extends the **IMsg** interface. A consumer application uses **PostMsg** to push content to upstream components. Such content can be applied to an LSEG Real-Time Distribution System cache or routed further upstream to the source of data. After upstream components receive the content, the components can republish the data to their downstream consumers.

Post messages can be routed along a specific item stream, referred to as **on-stream** posting, or along a user's Login stream, referred to as **off-stream** posting. **PostMsg** can contain any container type, including other messages. User identification information can be associated with a post message and be provided along with posted content. For more details, refer to Section 13.9.

12.2.7.1 Post Msg Methods

METHOD	DESCRIPTION
flags	Sets or gets a combination of bit values (flags) that indicate special behaviors and the presence of optional content. For more information about flag values, refer to Section 12.2.7.2. - You can use the following convenient methods to set specific PostMsgFlags: applyAck, applyHasExtendedHdr, applyHasMsgKey, applyHasPartNum, applyHasPermData, applyHasPostId, applyHasPostUserRights, applyHasSeqNum, applyPostComplete. - You can use the following convenient methods to check whether specific PostMsgFlags are set: checkAck, checkHasExtendedHdr, checkHasMsgKey, checkHasPartNum, checkHasPermData, checkHasPostId, checkHasPostUserRights, checkHasSeqNum, checkPostComplete.
partNum	Sets or gets the part number for this post message, typically used with multi-part post messages. partNum can contain values ranging from 0 to 32,767, where a value of 0 indicates the initial part of a refresh. - If sent on a single-part post message, use a partNum of 0. - On multi-part post messages, use a partNum of 0 on the initial part and in each subsequent part, increment partNum part by 1.
postId	Sets or gets the consumer-assigned identifier (postId), which can range in value from 0 to 4,294,967,295. postId distinguishes different post messages. In multi-part post messages, each part must use the same postId value.
seqNum	Sets or gets a user-defined sequence number (seqNum), typically corresponding to the sequencing of the message. seqNum allows for values ranging from 0 to 4,294,967,295. To help with temporal ordering, seqNum should increase, though gaps might exist depending on the sequencing algorithm in use. Details about seqNum use should be defined in the domain model specification or any documentation for products that use seqNum. When acknowledgments are requested, the seqNum will be provided back in the AckMsg to help identify the PostMsg being acknowledged.
permData	Optional. Sets or gets permData, which is a Buffer (with position and length) that specifies authorization information for content in this message only. permData can be overridden for specific content within the message (e.g. MapEntry.permData). permData allows a maximum length of 32,767 bytes. For more information, refer to Section 11.4.

Table 149: PostMsg Methods

METHOD	DESCRIPTION
postUserInfo	Returns a PostUserInfo object which can set or get information that identifies the posting user. postUserInfo can optionally be provided along with posted content via a RefreshMsg , UpdateMsg , and StatusMsg . - For more information about posting, refer to Section 13.9. - For more information about Visible Publisher Identifier, refer to Section 13.11.
postUserRights	Conveys the rights or abilities of the user posting this content, which can indicate whether the user is permissioned to: - Create items in the cache of record, - Delete items from the cache of record, or - Modify the permData on items already present in the cache of record. For details about different rights, refer to Section 12.2.7.3.

Table 149: PostMsg Methods (Continued)

12.2.7.2 PostMsgFlags Values

POST MSG FLAG	MEANING
NONE	Indicates that none of the optional flags are set.
PostMsgFlags.POST_COMPLETE	Indicates that this is the final part of the PostMsg . This flag should be set on: - Single-part post messages (i.e., an atomic post message). - The final part of a multi-part post message. For more information about multi-part message handling, refer to Section 13.1.
PostMsgFlags.ACK	Specifies that the consumer wants the provider to send an AckMsg to indicate that the PostMsg was processed properly. When acknowledging a PostMsg , the provider must include the postId in the ackId and communicate any associated seqNum .
PostMsgFlags.HAS_MSG_KEY	Indicates that the PostMsg contains a populated msgKey that identifies the stream on which the information is posted. An msgKey is typically required for off-stream posting and is not necessary when on-stream posting. For more detailed information about posting, refer to Section 13.9.
PostMsgFlags.HAS_SEQ_NUM	Indicates the presence of the seqNum member.
PostMsgFlags.HAS_POST_ID	Indicates the presence of the postId member.
PostMsgFlags.HAS_POST_USER_RIGHTS	Indicates the presence of the postUserRights member.
PostMsgFlags.HAS_PART_NUM	Indicates the presence of the partNum member.
PostMsgFlags.HAS_PERM_DATA	Indicates the presence of the permData member.
PostMsgFlags.HAS_EXTENDED_HEADER	Indicates the presence of the extendedHeader member.

Table 150: Flags Values

12.2.7.3 PostUserRights Values

POST USER RIGHT	MEANING
PostUserRights.NONE	The user has no additional posting abilities.
PostUserRights.CREATE	The user is allowed to create items in the cache of record.
PostUserRights.DELETE	The user is allowed to remove items from the cache of record.
PostUserRights.MODIFY_PERM	The user is allowed to modify the permData associated with items already in the cache of record.

Table 151: PostUserRights Values

12.2.7.4 PostUserInfo Methods

METHOD	DESCRIPTION
userId	Sets or gets the userId , which identifies the specific user that posted this data.
userAddr	Sets or gets the IP Address (userAddr) of the user that posted this data. Though the address can be specified as either a long or String (e.g., "127.0.0.1"), if it is specified as a String , it will be converted to its integer equivalent.
userAddrToString	Converts an IP address in integer format to its string equivalent.
clear	Clears the object, so that it can be reused.

Table 152: PostUserRights Methods

12.2.8 Acknowledgment Message Interface

The **AckMsg** interface extends the **Msg** interface. A provider can send an **AckMsg** to a consumer to indicate receipt of a specific message. The acknowledgment carries success or failure (i.e., a negative acknowledgment or 'NAK') information to the consumer. Currently, a consumer can request acknowledgment for a **PostMsg** or **CloseMsg**.

12.2.8.1 AckMsg Methods

METHOD	DESCRIPTION
flags	Sets or gets flags, which is a combination of bit values indicating special behaviors and the presence of optional content. For more information about flag values, refer to Section 12.2.8.2. - You can use the following convenient methods to set specific AckMsgFlags: applyHasExtendedHdr, applyHasMsgKey, applyHasNakCode, applyHasSeqNum, applyHasText, applyPrivateStream. - You can use the following convenient methods to check whether specific AckMsgFlags are set: checkHasExtendedHdr, checkHasMsgKey, checkHasNakCode, checkHasSeqNum, checkHasText, checkPrivateStream.
ackId	Sets or gets ackId, which associates the AckMsg with the message it acknowledges. ackId allows for values ranging from 0 to 4,294,967,295. When acknowledging a PostMsg, ackId typically matches the post message's postId.
seqNum	Sets or gets seqNum, which specifies a user-defined sequence number, ranging in value from 0 to 4,294,967,295. To help with temporal ordering, seqNum should increase, though gaps might exist depending on the sequencing algorithm in use. The acknowledgment message may populate this with the seqNum of the PostMsg being acknowledged. This helps correlate the message being acknowledged when the postId alone is not sufficient (e.g., multi-part post messages).
nakCode	Sets or gets nakCode. If present, this message indicates a NAK. The nakCode is an enumerated code value (ranging in value from 1 to 255) that provides additional information about the reason for the NAK. nakCode values are defined in Section 12.2.8.3
text	Optional. Sets or gets text, which is a Buffer (with position and length) that provides additional information about the acceptance or rejection of the message being acknowledged. text has a maximum allowed length of 65,535 bytes.

Table 153: AckMsg Methods

12.2.8.2 AckMsgFlags Values

ACK MSG VALUE	MEANING
NONE	Indicates that none of the optional flags are set.
AckMsgFlags.HAS_MSG_KEY	Indicates the presence of a populated msgKey . When present, this is typically populated to match the information being acknowledged.
AckMsgFlags.HAS_SEQ_NUM	Indicates the presence of the sequm member.
AckMsgFlags.HAS_NAK_CODE	Indicates the presence of the nakCode member.
AckMsgFlags.HAS_TEXT	Indicates the presence of the text member.
AckMsgFlags.HAS_EXTENDED_HEADER	Indicates presence of the extendedHeader member.
AckMsgFlags.PRIVATE_STREAM	Acknowledges the initial establishment of a private stream. For details, refer to Section 13.13.

Table 154: Flags Values

12.2.8.3 NakCodes Values

NAK CODE VALUE	DESCRIPTION
NONE	Indicates that none of the optional flags are set.
NakCodes.ACCESS_DENIED	The user is not permissioned to post on the item or service.
NakCodes.DENIED_BY_SRC	The source being posted to has denied accepting this post message.
NakCodes.SOURCE_DOWN	The source being posted to is down or unavailable.
NakCodes.SOURCE_UNKNOWN	The source being posted to is unknown and unreachable.
NakCodes.NO_RESOURCES	Some component along the path of the post message does not have appropriate resources available to continue processing the post.
NakCodes.NO_RESPONSE	There is no response from the source being posted to. This may mean that the source is unavailable or that there is a delay in processing the posted information.
NakCodes.GATEWAY_DOWN	A gateway device for handling posted or contributed information is down or unavailable.
NakCodes.SYMBOL_UNKNOWN	The system does not recognize the item information provided with the post message. This may be an invalid item.
NakCodes.NOT_OPEN	The item being posted to does not have an available stream.
NakCodes.INVALID_CONTENT	The content of the post message is invalid (it does not match the expected formatting) and cannot be posted.

Table 155: AckMsgNakCodes Values

12.2.9 Msg Encoding and Decoding

All message interfaces (e.g. **RequestMsg**, **RefreshMsg**, etc.) extend the **Msg** interface.

12.2.9.1 Msg Encoding Interfaces

When encoding, any message interfaces can call **Msg** encoding methods without the need to explicitly cast to the **Msg** interface. For simplicity, this encoding section will refer to the **Msg** interface.

An **Msg** can be encoded from pre-encoded data or by encoding individual pieces of data as they are provided.

ENCODE INTERFACE	DESCRIPTION
Encode	Encodes a message where all message content is pre-encoded. • msgKey attribute information should be encoded and populated on msgKey.encodedAttrib prior to this call. • extendedHeader information should be encoded and populated on the message's extendedHeader member prior to this call. • Message payload information should be encoded and populated on the encodedDataBody member prior to this call.
EncodeInit	Begins encoding of an Msg. All message header elements should be properly populated. The containerType member should be populated with the specific type of message payload. • If encoding msgKey attribute information: pre-encoded msgKey attribute information should be populated in msgKey.encodedAttrib. Unencoded msgKey attribute information should be encoded after Msg.encodeInit returns, followed by encodeKeyAttribComplete. • If encoding extendedHeader information: pre-encoded extendedHeader information should be populated in the extendedHeader member of the message. Unencoded extendedHeader information should be encoded after the call to Msg.encodeInit and after msgKey attribute information is encoded. When extendedHeader encoding is completed, call encodeExtendedHeaderComplete.
encodeComplete	Completes encoding of an Msg. All message content should be encoded prior to this call. This function expects the same EncodeIterator that was used with Msg.encodeInit. • If the content (i.e., payload, msgKey attrib, and extendedHeader) encodes successfully, the Boolean success parameter should be set to true to finish encoding. • If any of the content fails to encode, the boolean success parameter should be set to false to roll back the encoding of the message.

Table 156: Msg Encode Methods

ENCODE INTERFACE	DESCRIPTION
encodeKeyAttribComplete	Completes encoding of any non-pre-encoded msgKey attribute information. Can be used only when message encoding leverages Msg.encodeInit. If the MsgKeyFlags.HAS_ATTRIB flag is set and msgKey.encodedAttrib is not populated, msgKey attribute information is expected after Msg.encodeInit returns, with the specific attribContainerType methods being used to encode it. This method expects the same EncodeIterator used with Msg.encodeInit. • If encoding of the msgKey attribute information succeeds, the Boolean success parameter should be set to true to finish attribute encoding. • If encoding of attributes fails, the Boolean success parameter should be set to false to roll back encoding prior to msgKey attributes. If both msgKey attributes and extendedHeader information are being encoded, msgKey attributes are expected first with extendedHeader being encoded after the call to encodeKeyAttribComplete.
encodeExtendedHeaderComplete	Completes encoding of any non-pre-encoded extendedHeader information. Can be used only when the message encoding leverages Msg.encodeInit. If the specific message's HAS_EXTENDED_HEADER flag is set and extendedHeader is not populated, this information is expected after Msg.encodeInit (and encodeKeyAttribComplete if encoding msgKey attributes) returns. This function expects the same EncodeIterator used with previous message encoding functions. • If encoding of extendedHeader succeeds, the Boolean success parameter should be set to true to finish encoding. • If encoding of extendedHeader fails, the Boolean success parameter should be set to false to roll back to encoding prior to extendedHeader. If both msgKey attributes and extendedHeader information are being encoded, msgKey attributes are expected first, while extendedHeader should be encoded after the call to encodeKeyAttribComplete.

Table 156: Msg Encode Methods (Continued)

12.2.9.2 Msg Encoding Example 1

The following code sample demonstrates **Msg** encoding, showing the use of **encodeInit** with **encodeComplete** and includes unencoded **msgKey** attribute information, unencoded payload, and unencoded **extendedHeader** information. While this example demonstrates error handling for the initial encode method, it omits additional error handling to simplify the example (though it should still be performed).

```
/* EXAMPLE 1 - Msg.encodeInit/Complete with unencoded msgKey attribute, payload, and extendedHeader */

/* Populate and encode a requestMsg */
RequestMsg reqMsg = (RequestMsg)CodecFactory.createMsg();
reqMsg.msgClass(MsgClasses.REQUEST); /* message is a request */
reqMsg.domainType(DomainTypes.MARKET_PRICE);
reqMsg.containerType(DataTypes.ELEMENT_LIST);
/* Choose a stream Id that is not in use if this is a new request, otherwise reuse associated id */
reqMsg.streamId(6);
/* Populate flags for request message members and behavior - our message is for a streaming request,
   will specify a quality of service range, priority, contains an extended header and payload is a
   dynamic view request */
reqMsg.applyStreaming();
reqMsg.applyHasPriority();
reqMsg.applyHasQos();
```

```

reqMsg.applyHasWorstQos();
reqMsg.applyHasExtendedHdr();
reqMsg.applyHasView();

/* Populate qos range and priority */
reqMsg.priority().priorityClass(2);
reqMsg.priority().count(1);
/* Populate best qos allowed */
reqMsg.qos().rate(QosRates.TICK_BY_TICK);
reqMsg.qos().timeliness(QosTimeliness.REALTIME);
/* Populate worst qos allowed, rate and timeliness values allow for rateInfo and timeInfo to be sent */
reqMsg.worstQos().rate(QosRates.TIME_CONFLATED);
reqMsg.worstQos().rateInfo(1500);
reqMsg.worstQos().timeliness(QosTimeliness.DELAYED);
reqMsg.worstQos().timeInfo(20);

/* Populate msgKey to specify a serviceId, a name with type of RIC (which is default nameType) and attrib
*/
reqMsg.msgKey().applyHasServiceId();
reqMsg.msgKey().applyHasName();
reqMsg.msgKey().applyHasAttrib();
reqMsg.msgKey().serviceId(1);
/* Specify name and length of name. Because this is a RIC, no nameType is required. */
reqMsg.msgKey().name().data("TRI");
/* Msg Key attribute info will be encoded after Msg.encodeInit returns */
reqMsg.msgKey().attribContainerType(DataTypes.ELEMENT_LIST);

/* begin encoding of message - assumes that encIter is already populated with
    buffer and version information, store return value to determine success or failure */
/* data max encoded size is unknown so 0 is used */
if ((retCode = reqMsg.encodeInit(encIter, 0)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Msg.encodeInit. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
else
{
    Buffer nonRWFBBuffer = CodecFactory.createBuffer();
    /* retCode should be CodecReturnCodes.ENCODE_MSG_KEY_OPAQUE */
    /* encode msgKey attrib as element list to match setting of attribContainerType */
    {
        elementList.applyHasStandardData();
        /* now encode nested container using its own specific encode methods */
        if ((retCode = elementList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)
        /*----- Continue encoding element entries. See example in Section 11.3.2 ----- */
        /* Complete nested container encoding */
        retCode = elementList.encodeComplete(encIter, success);
    }
}

```



```

}
/* now that it is done, complete msgKey attrib encoding. */
retCode = reqMsg.encodeKeyAttribComplete(encIter, success);

/* retCode should be CodecReturnCodes.ENCODE_EXTENDED_HEADER */
/* encode extended header as non-RWF type using non-RWF encode methods */
{
    retCode = encIter.encodeNonRWFInit(nonRWFBuffer);
    /* now encode extended header using its own specific encode methods -
       Ensure that we do not exceed nonRWFBuffer.length */
    /* we could copy into the nonRWFBuffer or use it with other encode methods */
    nonRWFBuffer.data().put(encExtendedHeader.data());
    retCode = encIter.encodeNonRWFComplete(nonRWFBuffer, success);
}
retCode = reqMsg.encodeExtendedHeaderComplete(encIter, success);

/* retCode should be CodecReturnCodes.ENCODE_CONTAINER */
/* encode message payload to match containerType */
{
    elementList.applyHasStandardData();
    /* now encode nested container using its own specific encode methods */
    if ((retCode = elementList.encodeInit(encIter, null, 0)) < CodecReturnCodes.SUCCESS)
        /*----- Continue encoding element entries. See example in Section 11.3.2 ----- */
        /* Complete nested container encoding */
        retCode = elementList.encodeComplete(encIter, success);
}
/* now that specified msgKey attrib, extendedHeader and payload are done, complete message encoding.
*/
retCode = reqMsg.encodeComplete(encIter, success);
}

```

Code Example 42: Msg Encoding Example #1, encodeInit / encodeComplete Use

12.2.9.3 Msg Encoding Example 2

The following code sample demonstrates **Msg** encoding and shows the use of **encode** with pre-encoded **msgKey** attribute information and payload. While this example demonstrates error handling for the initial encode function, it omits additional error handling to simplify the example (though it should still be performed).

```

/* EXAMPLE 2 - EncodeMsg with pre-encoded msgKey.attrib and pre-encoded payload, no extendedHeader */

/* Populate and encode a refreshMsg */
RefreshMsg refreshMsg = (RefreshMsg)CodecFactory.createMsg();
refreshMsg.msgClass(MsgClasses.REFRESH); /* message is a refresh */
refreshMsg.domainType(DomainTypes.MARKET_PRICE);
refreshMsg.containerType(DataTypes.FIELD_LIST);
/* Use the stream Id corresponding to the request, because it is in reply to a request, it's solicited */
refreshMsg.streamId(6);
/* Populate stream and data state information. This is required on an RefreshMsg */
refreshMsg.state().streamState(StreamStates.OPEN);

```

```

refreshMsg.state().dataState(DataStates.OK);
/* Populate flags for refresh message members and behavior - because this in response to a request
   This should be solicited, msgKey should be present, single part refresh so it is complete,
   and also want the concrete qos of the stream */
refreshMsg.applySolicited();
refreshMsg.applyHasMsgKey();
refreshMsg.applyRefreshComplete();
refreshMsg.applyHasQos();
refreshMsg.applyClearCache();
/* Populate msgKey to specify a serviceId, a name with type of RIC (which is default nameType) and attrib
   */
refreshMsg.msgKey().applyHasServiceId();
refreshMsg.msgKey().applyHasName();
refreshMsg.msgKey().applyHasAttrib();
refreshMsg.msgKey().serviceId(1);
/* Specify name and length of name. Because this is a RIC, no nameType is required. */
refreshMsg.msgKey().name().data("TRI");
/* Msg Key attribute info is pre-encoded, should be set in encAttrib */
refreshMsg.msgKey().attribContainerType(DataTypes.ELEMENT_LIST);
/* assuming encodedAttrib Buffer contains the pre-encoded msgKey attribute info with data and length
   populated */
refreshMsg.msgKey().encodedAttrib(encodedAttrib);
/* assuming encodedPayload Buffer contains the pre-encoded payload information with data and length
   populated */
refreshMsg.encodedDataBody(encodedPayload);

/* encode message - assumes that encIter is already populated with buffer and version information,
   store return value to determine success or failure */
/* Because this method expects all portions to be populated and pre-encoded, all Message encoding is
   complete after this returns. */
if ((retCode = refreshMsg.encode(encIter)) < CodecReturnCodes.SUCCESS)
{
    /* error condition - switch our success value to false so we can roll back */
    success = false;
    /* print out message with return value string, value, and text */
    System.out.printf("Error (%d) (errno: %d) encountered with Msg.encode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}

```

Code Example 43: Msg Encoding Example #2, encode Use

12.2.9.4 Msg Decoding Interfaces

Msg contains common members that can identify the specific message class or domain type. When decoding, you must use the **Msg** interface (because the **MsgClass** is not known until after the message is decoded). Once decoded, the **Msg** can be cast to the appropriate message interface. Because **msgKey** is optional and specified on a per-message class basis, do not use **Msg.msgKey** until the specific message class flags are consulted to determine whether the **msgKey** is present.

A decoded **Msg** structure provides access to the encoded content of the message. You can further decode the message's content by invoking the specific contained type's decode function.

DECODE INTERFACE	DESCRIPTION
decode	Decodes Msg header members. Any msgKey attribute information remains encoded unless the user chooses to decode it. This can be accomplished by setting the encodedAttrib buffer on a separate DecodeIterator or by calling Msg.decodeKeyAttrib followed by decode functions for the specified attribContainerType. Any message payload content will be described by the message's containerType member and will be present in the encodedDataBody. This can be decoded by calling the containerType's specific decode methods using the same DecodeIterator or by setting the encodedDataBody on a new decode iterator. Any extendedHeader information is expected to be decoded by using a separate DecodeIterator. This method will decode from the Buffer to which the passed in DecodeIterator refers.
decodeKeyAttrib	Prepares the DecodeIterator to decode Msg.msgKey.encodedAttrib information. This method expects the same DecodeIterator as was used with Msg.decode and the Msg.msgKey member that was populated by calling Msg.msgKey. This populates encodedData with an encoded entry. After this method returns, you can call the msgKey.attribContainerType decode methods to decode attribute information. If you do not want to decode msgKey attribute information, you can decode the payload by using the containerType's decode methods after Msg.decode returns.

Table 157: Msg Decode Methods

12.2.9.5 Msg Decoding Example

The following code sample demonstrates how to decode an **Msg**. This sample code uses a switch statement to decode the message's content. Typically an application would invoke the specific container type decoder for the housed type or use a switch statement to allow for a more generic message decoding. The example uses the same **DecodeIterator** when decoding the **msgKey.encodedAttrib** and the message payload. An application could optionally use a new **DecodeIterator** by setting the **encodedAttrib** or **encodedDataBody** on a new iterator. To simplify the following sample code, some error handling is omitted.

```

/* decode contents into the Msg structure */
if ((retCode = msg.decode(decIter)) >= CodecReturnCodes.SUCCESS)
{
    /* We can cast to the appropriate message class for convenience or use the accessor methods */
    /* Use the ease of use accessor to get the msgKey if it exists on whatever msgClass this is */
    MsgKey key = msg.msgKey();
    /* If we have a key and it has attribute information, decode it */
    if (key != null && key.checkHasAttrib())
    {
        /* Need to set up the decodeIterator to expect decoding of attribute information, otherwise
           it assumes we are decoding the payload */
        retCode = msg.decodeKeyAttrib(decIter, key);

        switch (key.attribContainerType())

```

```

{
    case DataTypes.FIELD_LIST:
        retCode = fieldList.decode(decIter, null);
        /* Continue decoding field entries. See example in Section 11.3.1 */
        break;
    case DataTypes.ELEMENT_LIST:
        retCode = elementList.decode(decIter, null);
        /* Continue decoding element entries. See example in Section 11.3.2 */
        break;
    /* full switch statement omitted to shorten sample code */
}
}

/* Decode any contained payload information */
switch (msg.containerType())
{
    case DataTypes.NO_DATA:
        System.out.println("No payload contained in message.");
        break;
    case DataTypes.FIELD_LIST:
        retCode = fieldList.decode(decIter, null);
        /* Continue decoding field entries. See example in Section 11.3.1 */
        break;
    case DataTypes.ELEMENT_LIST:
        retCode = elementList.decode(decIter, null);
        /* Continue decoding element entries. See example in Section 11.3.2 */
        break;
    /* full switch statement omitted to shorten sample code */
}
}
else
{
    /* decoding failure tends to be unrecoverable */
    System.out.printf("Error (%d) (errno: %d) encountered with Msg.decode. Error Text: %s\n",
        error.errorId(), error.sysError(), error.text());
}
}

```

Code Example 44: Msg Decoding Example

12.2.9.6 EncoderIterator Utility Methods

The Enterprise Transport API provides the following **EncoderIterator** utility methods for use with the **Msg**.

METHOD	DESCRIPTION
replaceStreamId	Takes an encoded message and replaces the streamId without re-encoding the message. For more details on the streamId , refer to Section 12.1.3.
replaceSeqNum	Takes an encoded message and replaces the seqNum without re-encoding the message.
replaceGroupId	Takes an encoded message and replaces the groupId without re-encoding the message. For more information about group use, refer to Section 13.4.
replacePostId	Takes an encoded message and replaces the postId without re-encoding the message. For more information, refer to Section 13.9.
replaceStreamState	Takes an encoded message and replaces the streamState without re-encoding the message. For more information about state values, refer to Section 11.2.7.
replaceDataState	Takes an encoded message and replaces the dataState without re-encoding the message. For more information about state values, refer to Section 11.2.7.
replaceStateCode	Takes an encoded message and replaces the State.code without re-encoding the message. For more information about state values, refer to Section 11.2.7.
setConfInfoInUpdatesFlag unsetConfInfoInUpdatesFlag	Sets or unsets the RefreshMsgFlags.CONF_INFO_IN_UPDATES flag on an encoded buffer.
setGenericCompleteFlag unsetGenericCompleteFlag	Sets or unsets the GenericMsg.MESSAGE_COMPLETE flag on an encoded buffer.
setMsgKeyInUpdatesFlag unsetMsgKeyInUpdatesFlag	Sets or unsets the RefreshMsgFlags.MSG_KEY_IN_UPDATES flag on an encoded buffer.
setNoRefreshFlag unsetNoRefreshFlag	Sets or unsets the RefreshMsgFlags.NO_REFRESH flag on an encoded buffer.
setRefreshCompleteFlag unsetRefreshCompleteFlag	Sets or unsets the RefreshMsgFlags.REFRESH_COMPLETE flag on an encoded buffer.
setSolicitedFlag unsetSolicitedFlag	Sets or unsets the RefreshMsgFlags.SOLICITED flag on an encoded buffer.
setStreamingFlag unsetStreamingFlag	Sets or unsets the RefreshMsgFlags.STREAMING flag on an encoded buffer.

Table 158: EncoderIterator Utility Methods

12.2.9.7 Decodelterator Utility Methods

The Enterprise Transport API provides the following **Decodelterator** utility methods for use with the **Msg**.

NOTE: Multiple **extract*** calls on the same encoded message will likely be less efficient than a single call to **Msg.decode**.

METHOD	DESCRIPTION
extractMsgClass	Takes an encoded message and returns the msgClass information without fully decoding the message header.
extractDomainType	Takes an encoded message and returns the domainType information without fully decoding the message header.
extractStreamId	Takes an encoded message and returns the streamId information without fully decoding the message header. For more details on the streamId , refer to Section 12.1.3.
extractSeqNum	Takes an encoded message and returns the seqNum information without fully decoding the message header.
extractGroupId	Takes an encoded message and returns the groupId information without fully decoding the message header. For more information about group use, refer to Section 13.4.
extractPostId	Takes an encoded message and returns the postId information without fully decoding the message header. For more information, refer to Section 13.9.

Table 159: DecodeIterator Utility Methods

13 Advanced Messaging Concepts

13.1 Multi-Part Message Handling

RefreshMsg, **PostMsg**, and **GenericMsg** all support splitting payload content across multiple message parts, commonly referred to as *message fragmentation*. Each message part includes relevant message header information along with the part's payload, where payload can be combined by following the modification semantics associated with the specific **containerType** (for specific container details, refer to Section 11.3). Message fragmentation is typically used to split large payload information into smaller, more manageable pieces. The size of each message part can vary, and is controlled by the application that performs the fragmentation. Often, sizes are chosen based on a specific transport layer frame or packet size.

When sending a multi-part message, several message members can convey additional part information. Each message class that supports fragmentation has an optional **partNum** member that can order and ensure receipt of every part of the message. For consistency and compatibility with LSEG Real-Time Distribution System components, **partNum** should begin with **0** and increment by one for each subsequent part. Several container types have an optional **totalCountHint** value. This can convey information about the expected entry count across all message parts, and often helps size needed storage or display for the message contents.

These message classes have an associated **COMPLETE** flag value (specifically **RequestMsgFlags.REFRESH_COMPLETE**, **PostMsgFlags.POST_COMPLETE**, and **GenericMsgFlags.MESSAGE_COMPLETE**). A flag value of **COMPLETE** indicates the final part of a multi-part message (or that the message is a single-part and no subsequent parts will be delivered).

For both streaming and non-streaming information, other messages might arrive between parts of a fragmented message. For example, it is expected that update messages be received between individual parts of a multi-part refresh message. Such updates indicate changes to data being received on the stream and should be applied according to the modification semantics associated with the **containerType** of the payload. If non-streaming, no additional messages should be delivered after the final part.

If a transport layer is used, messages can fan out in the order in which they are received. On a transport where reliability is not guaranteed and the order can be determined by a sequence number, special rules should be used by consumers when processing a multi-part message. The following description explains how a multi-part refresh message can be handled. After the request is issued, any messages received on the stream should be stored and properly ordered based on sequence number. When an application encounters the first part of the **RefreshMsg**, the application should process the part and note its sequence number. The application can drop (i.e., not process) stored messages with earlier sequence numbers. When the application encounters the next part of the **RefreshMsg**, the application should first process any stored message with a sequence number intermediate between this refresh part and the previous part then the application should process the refresh part. This process should continue until the final part of the **RefreshMsg** is encountered, at which time any remaining stored messages with a later sequence number should be processed and the stream's data flow can continue as normal.

13.2 Stream Priority

Consumers use **RequestMsg** to indicate the stream's level of importance, conveyed by the priority information. When a consumer is aggregating streams on behalf of multiple users, the priority typically corresponds to the number of users interested in the particular stream. A consumer can increase or decrease a stream's associated priority information by issuing a subsequent request message on an already open stream.

A Provider application tracks the priority of each of its open streams. If the consumer reaches some kind of item count limitation (i.e., the maximum allowable number of streams), the provider can employ a preemption algorithm. Specific details must be defined by the provider application. The LSEG Real-Time Advanced Distribution Hub uses the combination of **priorityCount** and **priorityClass** to preempt items when the user's allowable cache list size is exceeded. LSEG Real-Time Advanced Distribution Hub always preempts the item with the lowest **priorityCount** within the **priorityClass** and then provides an **StatusMsg** with a **streamState** of **StreamStates.CLOSED_RECOVER** for the item.

Priority is represented by a **priorityClass** value and a **priorityCount** value.

- The **priority class** indicates the general importance of the stream to the consumer.
- The **priority count** indicates the stream's specific importance within the priority class.

The **priorityClass** value takes precedence over any **priorityCount** value. For example, a stream with a **priorityClass** of 5 and **priorityCount** of 1 has a higher overall priority than a stream with a **priorityClass** of 3 and a **priorityCount** of 10,000.

Because priority information is optional on a **RequestMsg**:

- If priority information is not present on an initial request to open a stream, it is assumed that the stream has a **priorityClass** and a **priorityCount** of 1.
- If priority information is not present on a subsequent request message on an open stream, this means that the priority has not changed and previously stored priority information continues to apply.

If a consumer aggregates identical streams, the consumer should use the highest **priorityClass** value. Individual **priorityCount** values are always combined on a per-**priorityClass** basis.

For example, if a consumer application combines three identical streams:

- One with **priorityClass** 3 and **priorityCount** 5
- One with **priorityClass** 2 and **priorityCount** 10
- One with **priorityClass** 3 and **priorityCount** of 1

In this case, the aggregate priority information would be **priorityClass** 3 (i.e., the highest **priorityClass**) and **priorityCount** of 6 (the combined **priorityCount** values for that class level).

13.3 Stream Quality of Service

A consumer can use **RequestMsg** to indicate the desired QoS for its streams. This can be a request for a specific QoS or a range of qualities of service, where any value within the range will satisfy the request. The **RefreshMsg** includes the QoS used to indicate the QoS being provided for a stream. When issuing a request, the QoS specified on the request typically matches the advertised QoS of the service, as conveyed via the Source Directory domain model. For more information, refer to the *Enterprise Transport API Java Edition LSEG Domain Model Usage Guide*.

- An initial request containing only **RequestMsg.Qos** indicates a request for the specified QoS. If a provider cannot satisfy this QoS, the request should be rejected.
- An initial request containing both **RequestMsg.Qos** and **RequestMsg.worstQos** sets the range of acceptable QoSs. Any QoS within the range, inclusive of the specified **Qos** and **worstQos**, will satisfy the request. If a provider cannot provide a QoS within the range, the provider should reject the request.

When a provider responds to an initial request, the **RefreshMsg.Qos** should contain the actual QoS being provided for the stream. Subsequent requests issued on the stream should not specify a range as the QoS has been established for the stream.

Because QoS information is optional on an **RequestMsg**, some special handling is required when it is absent.

- If neither **Qos** nor **worstQos** are specified on an initial request to open a stream, it is assumed that any QoS will satisfy the request.
- If QoS information is absent on a subsequent reissue request, it is assumed that QoS, timeliness, and rate conform to the stream's currently established settings.
- If QoS information is absent in an initial **RefreshMsg**, this should be assumed to have a **timeliness** of **QosTimeliness.REALTIME** and a **rate** of **QosRates.TICK_BY_TICK**. On any subsequent solicited or unsolicited refresh, this should be assumed to match any QoS already established by the initial **RefreshMsg**.

To determine whether components require QoS information on initial and reissue requests, refer to the documentation for the specific component.

The following example demonstrates how to set Qos for the Request message:

```
RequestMsg msg = (RequestMsg) CodecFactory.createMsg();
msg.msgClass(MsgClasses.REQUEST);
msg.applyHasQos();
msg.qos().timeliness(QosTimeliness.REALTIME);
msg.qos().rate(QosRates.TICK_BY_TICK);
/* If the user application wants to set the range of Qos, the worstQos property should be set
/* in the following manner */
msg.applyHasWorstQos();
msg.worstQos().timeliness(QosTimeliness.DELAYED);
msg.worstQos().timeInfo(65532);
msg.worstQos().rate(QosRates.TIME_CONFLATED);
msg.worstQos().rateInfo(65533);
```

Code Example 45: Setting Qos for Request message

13.4 Item Group Use

You can use item groups to efficiently update the state for multiple item streams via a single group status message (instead of using multiple, individual item status messages). Each open data stream is assigned an item group. This information is associated with the stream through the **RefreshMsg.groupId** (refer to Section 12.2.2) or **StatusMsg.groupId** (refer to Section 12.2.4) members. Once established, item group information can be modified via a subsequent **StatusMsg** or **RefreshMsg** containing a different **groupId** affiliation.

Item groups are defined on a per-service basis. While two item groups can have the same **groupId**, each group's **serviceId** will be unique. A consumer application should track **serviceId-groupId** pairings to ensure the correct sets of items are modified whenever group status messages are received. A provider can establish item group assignments according to the application's needs, but must maintain the uniqueness of each item group within a service. For example, a provider that aggregates multiple upstream services into a single downstream service might establish a different item group for each aggregated service. Thus, should an upstream service become unavailable, the provider can mark all items as being suspect while items from other upstream services remain in their prior state.

13.4.1 Item Group Buffer Contents

The consuming application should treat data (which may be of varying length) contained in the **groupId** buffer as opaque. A simple memory comparison operation can determine whether two groups are equivalent. The actual data contained in the **groupId** buffer is a collection of one or more unsigned two-byte, unsigned integer values, where each two-byte value is appended to the end of the current **groupId Buffer**. Providers that combine multiple data sources must ensure that the item groups in the resulting service are unique, which can be accomplished by appending an additional two-byte value to each on-passed **groupId**.

For example, the following figure depicts two non-interactive provider applications, each publishing item streams belonging to specific services and item groups.

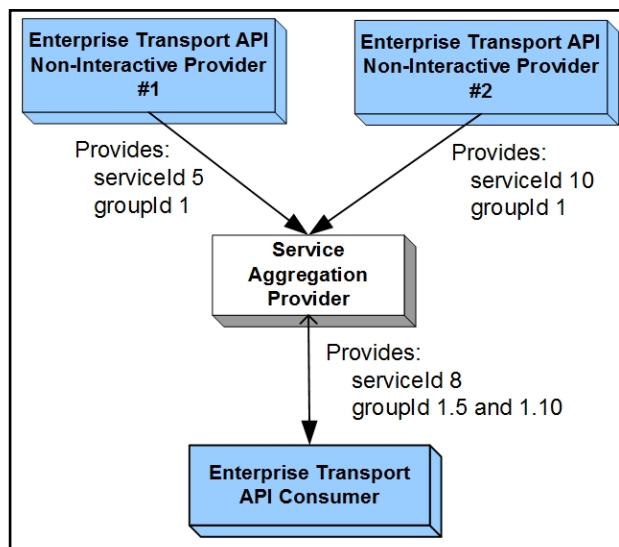


Figure 38. Item Group Example

Though the providers in this diagram use the same **groupId** for an item, using different **serviceIds** makes items unique. Both providers communicate with an application that consumes data from both services, aggregates the data into a single service, and then distributes the data to consumer applications. To ensure uniqueness to downstream components, the service aggregation provider appends additional identifiers to the group information it receives from the provider applications. In this example, the aggregation device modifies **serviceId 5**, **groupId 1** into a **groupId** of **1.5** and **serviceId 10**, **groupId 1** into a **groupId** of **1.10**. If for any reason non-interactive provider #1's service becomes unavailable, the aggregation device can send a single group status message to inform the consumer that all items belonging to **groupId 1.5** are suspect. This would have no impact to any items belonging to **groupId 1.10**.

13.4.2 Item Group Utility Functions

The Transport API provides the following utility methods for use with and modification of the **groupId Buffer**.

METHOD	DESCRIPTION
CodecUtils.addGroupId	Appends a two-byte, unsigned integer to existing groupId content. Useful when modifying groupId buffers to ensure uniqueness.
groupId (from RefreshMsg and StatusMsg)	Takes a populated Msg structure, determines if groupId information is present and if available, returns it; NULL otherwise.
DecodeIterator.extractGroupId	Takes an encoded message and returns the groupId without fully decoding the message header. NOTE: Multiple DecodeIterator.extract* calls on the same encoded message will likely be less efficient than a single call to Msg.decode .
EncodeIterator.replaceGroupId	Takes an encoded message and replaces the groupId without re-encoding the message.

Table 160: **groupId Buffer** Utility Methods

13.4.3 Group Status Message Information

Information regarding state changes and the merging of item groups occurs via group status messages. A group status message is communicated via the Source Directory domain message model. Specific group information is contained in the Directory's Group **FilterEntry** which corresponds to the specific service associated with the group.

- For more specific information, refer to the Source Directory Domain section in the *Transport API Java Edition LSEG Domain Model Usage Guide*.
- For a decision table providing example behavior for various state combinations, refer to Appendix A.

NOTE: If an application does not subscribe to the Source Directory's group filter, the application will not receive group status messages. This can result in potentially incorrect item state information, as relevant status information might be missed.

13.4.4 Group Status Responsibilities by Application Type

Dissemination and handling of group status information is distributed across providers and consumers. This section discusses responsibilities by application type.

A provider application (interactive or non-interactive) is responsible for:

- Assigning and providing item group id values. This is accomplished by specifying the **RefreshMsg.groupId** or **StatusMsg.groupId** for all provided content¹.
- If a group of items becomes unavailable (i.e., an upstream service or provider goes down), group status messages should be sent out for all affected item groups. These are sent via the Source Directory domain.

For more information about group status messages (including specific message content and formatting), refer to the *Transport API Java Edition LSEG Domain Model Usage Guide*.

- If items become available again, recovery should occur and items' states should be updated via a subsequent **RefreshMsg** or **StatusMsg** provided to any downstream components interested in the item.

A consumer application is responsible for:

- Subscribing to the item group filter when requesting Source Directory information.

1. This does not include administrative domains such as Login, Source Directory, and Dictionary.

For more information about the item group filter and group status messages (including specific message content and formatting), refer to the *Transport API Java Edition LSEG Domain Model Usage Guide*.

- If group status changes are received, the state change should be propagated to all items associated with the indicated group, as noted by the **RefreshMsg.groupId** or **StatusMsg.groupId** provided with the item stream.
- Any recovery should follow **SingleOpen** and **AllowSuspectData** rules, as described in the *Transport API Java Edition LSEG Domain Model Usage Guide*.

13.5 Single Open and Allow Suspect Data Behavior

A consumer application can specify desired item recovery and state transition information on its Login domain **RequestMsg** using the **SingleOpen** and **AllowSuspectData msgKey** attributes. A providing application can acknowledge support for the behavior in the Login domain **RefreshMsg**, in which case the provider performs certain state transitions. This section offers a high-level description of item recovery and state transition behavior modifications.

- **Single open** behavior allows a consumer application to open an item stream once and have an upstream component handle stream recovery (if needed). With single open enabled, a consumer should not receive a **streamState** of **CLOSED_RECOVER**, as the providing application should convert to **SUSPECT** and attempt to recover on the consumer's behalf. If a stream is **CLOSED**, this will be propagated to the consumer application.
- **Allow suspect data** behavior indicates whether an application can tolerate an open stream with a **dataState** of **SUSPECT**, or if it is preferable to have the stream closed. If an application indicates that it does not wish to allow **SUSPECT** streams to remain open, the providing application should transition the **streamState** to **CLOSED_RECOVER**.

If the providing application does not support either behavior, the application should indicate such a restriction in the Login domain's **RefreshMsg**. For additional information, including on the **DomainTypes.LOGIN** domain definition, refer to the *Enterprise Transport API Java Edition LSEG Domain Model Usage Guide*.

The following table shows how a provider can convert messages to correspond with the consumer's **SingleOpen** and **AllowSuspectData** settings. The first column in the table shows the actual **streamState** and **dataState**. Each subsequent column shows how this state information can be modified to follow the column's specific **SingleOpen** and **AllowSuspectData** settings. If a **SingleOpen** and **AllowSuspectData** configuration causes a behavioral contradiction (e.g., **SingleOpen** indicates that the provider should handle recovery, but **AllowSuspectData** indicates that the consumer does not want to receive suspect status), the **SingleOpen** configuration takes precedence.

NOTE: The Transport API does not perform special processing based on the **SingleOpen** and **AllowSuspectData** settings. The provider application must perform any necessary conversion.

ACTUAL STATE INFORMATION	CONVERSION WHEN: SINGLEOPEN = 1 ALLOWSUSPECTDATA = 1	CONVERSION WHEN: SINGLEOPEN = 1 ALLOWSUSPECTDATA = 0	CONVERSION WHEN: SINGLEOPEN = 0 ALLOWSUSPECTDATA = 1	CONVERSION WHEN: SINGLEOPEN = 0 ALLOWSUSPECTDATA = 0
streamState = OPEN dataState = SUSPECT	No conversion required	No conversion required	No conversion required	streamState = CLOSED_RECOVER dataState = SUSPECT
streamState = CLOSED_RECOVER dataState = SUSPECT	streamState = OPEN dataState = SUSPECT	streamState = OPEN dataState = SUSPECT	No conversion required	No conversion required

Table 161: SingleOpen and AllowSuspectData Effects

13.6 Pause and Resume

The Transport API allows applications to send or receive requests to pause or resume content flow on a stream.

- Issuing a **pause** on a stream can result in the temporary stop of **UpdateMsg** flow.
- Issuing a **resume** on a paused stream restarts the **UpdateMsg** flow.

Pause and resume can help optimize bandwidth by pausing streams that are only temporarily not of interest, instead of closing and re-requesting a stream. Though a pause request may be issued on a stream, it does not guarantee that the contents of the stream will actually be paused. Additionally, if the contents of the stream are paused, state-conveying messages can still be delivered (i.e., status messages and unsolicited refresh messages). Pause and resume is only valid for data streams instantiated as streaming (**RequestMsgFlags.STREAMING**). The consumer application is responsible for continuing to handle all delivered messages, even after the issuance of a pause request.

A consumer application can request to pause an individual item stream by issuing **RequestMsg** with the **RequestMsgFlags.PAUSE** flag set. This can occur on the initial **RequestMsg** or via a subsequent **RequestMsg** on an established stream (i.e., a reissue). If a pause is issued on the initial request, it should always result in the delivery of the initial **RefreshMsg** (this conveys initial state, permissioning, Quality of Service, and group association information necessary for the stream). A paused stream remains paused until a resume request is issued. To resume data flow on a stream a consumer application can issue a subsequent **RequestMsg** with the **RequestMsgFlags.STREAMING** flag set.

If a provider application receives a pause request from a consumer, it can choose to pause the content flow or continue delivering information. When pausing a stream, where possible, the provider should aggregate information updates until the consumer application resumes the stream. When resuming, an aggregate update message should be delivered to synchronize the consumer's information to the current content. However, if data cannot be aggregated, resuming the stream should result in a full, unsolicited **RefreshMsg** to synchronize the consumer application's information to a current state.

A pause request issued on the **streamId** associated with a user's login is interpreted as a request to **pause all** streams associated with the user. A pause all request is only valid for use on an already established login stream and cannot be issued on the initial login request. A 'pause all' request affects open streams only. Thus, newly-requested streams begin in a resumed state. After a pause all request, the application can choose to either resume individual item streams or resume all streams. A **resume all** will result in all paused streams being transitioned to a resumed state. This is performed by issuing a subsequent **RequestMsg** with the **RequestMsgFlags.STREAMING** flag set using the **streamId** associated with the applications login.

For more information about the **RequestMsg** and the **RequestMsgFlags.PAUSE** or **RequestMsgFlags.STREAMING** flag values, refer to Section 12.2.1.

A provider application can indicate support for pause and resume behavior by sending the **msgKey** attribute **supportOptimizedPauseResume** in the Login domain **RefreshMsg**. For more details on the Login **domainType** (**DomainTypes.LOGIN**), refer to the *Transport API Java Edition LSEG Domain Model Usage Guide*.

13.7 Batch Requesting

Applications can use the Enterprise Transport API to send and / or receive batch requests.

- Consumers use a **batch request** to indicate interest in multiple like-item streams with a single **RequestMsg**.
- Providers should respond by providing a status on the batch request stream itself and with new individual streams for each item in the batch.

13.7.1 Batch Request Usage

Batch requesting can be leveraged across all non-administrative² domain model types, where specific usage and support should be indicated in the model definition. If an item requested as part of a batch is not available, the provider should send a **StatusMsg** on the stream (this is handled in the same manner as an individual item request).

A consumer application can issue a batch request by using a **RequestMsg** with the **RequestMsgFlags.HAS_BATCH** flag set and including a specifically formatted payload. The payload should contain an **ElementList** along with an **ElementEntry** named **:ItemList**. Because payload content can include customer-defined portions and LSEG-defined portions, the Transport API uses a name-spacing scheme. Any content in an element **name** prior to **:** is used as name space information (e.g., **Customer:Element**). LSEG reserves the empty name space (e.g., **:Element**). The **com.refinitiv.eta.rdm.ElementNames** defines batch request-related enumeration and element name buffer constant.

The **:ItemList** contains an **Array**, where the **Array.primitiveType** is **DataTypes.ASCII_STRING**. Each contained string (populated in a **Buffer**) corresponds to a requested name. The **msgKey** contents will be applied to all names in the list, and a **msgKey.name** (or **MsgKeyFlags.HAS_NAME_TYPE**) should not be present.

When a provider application receives a batch request, it should respond on the same stream with a **StatusMsg** that acknowledges receipt of the batch by indicating the **dataState** is **DataStates.OK** and **streamState** is **StreamStates.CLOSED**. The stream on which the batch request was sent (i.e., the 'batch stream') then closes, because all additional responses are provided on individual streams, and thus no reissuing is possible on a batch stream. The **:ItemList** should be traversed to obtain each requested name and the batch **RequestMsg.msgKey** content should be associated with each item. If any request cannot be fulfilled, the provider should send a **StatusMsg** to close the stream and indicate the reason, using the stream that corresponds to that particular item (for further details, refer to Section 12.2.4).

Assignment of **streamId** values for all requested items is sequential, beginning with **(1 + streamId)** of the batch **RequestMsg**. Because a consumer requests the batch, positive **streamId** values should be assigned. For example, if the batch request uses **streamId 20** and requests ten items, the **StatusMsg** response to the batch request would be delivered on **streamId 20**, then the first item in the list receives a response with **streamId 21**, the second item with **streamId 22**, etc. By setting the initial **streamId**, the consumer application can control the resultant **streamId** range, ensuring enough available **streamId** values exist to allocate identifiers for all requested items.

Any view information (described in Section 13.8) included in a batch request should be applied for each item in the request. If a consumer application wants to reissue any item that was requested as part of a batch, the application can issue a subsequent **RequestMsg** on that item's **streamId**.

A provider application can indicate support for batch request handling by sending the **msgKey** attribute **supportBatchRequests** in the Login domain **RefreshMsg**.

- For an example of encoding a batch request, refer to Section 13.7.2.
- For more information about **RequestMsg** and **RequestMsgFlags.HAS_BATCH** flag values, refer to Section 12.2.1.
- For more information about **ElementList**, refer to Section 11.3.2.
- For more details on the Login **domainType** (**DomainTypes.LOGIN**) and batch request use in general, see the *Enterprise Transport API Java Edition LSEG Domain Model Usage Guide*.

2. Administrative domain types are considered to be the Login, Directory, and Dictionary domain models. All other domains are considered non-administrative.

13.7.2 Batch RequestMsg Encoding Example

The following example demonstrates how to encode a batch **RequestMsg**. The request is sent using a **streamId** of **10** and contains an **:ItemList** of three items. Such a message should result in four responses:

- A **StatusMsg** delivered on **streamId 10** which indicates that the batch is being processed and closes the stream.
- Three **RefreshMsgs** are delivered, where the first item returns on **streamId 11**, the second on **streamId 12**, and the third on **streamId 13**.

To simplify the example, some error handling has been omitted; though applications should perform all appropriate error handling.

```
/* Example assumes encode iterator is properly initialized */
/* Create and populate request message with information pertaining to all items in batch, set batch flag
   */
reqMsg.msgClass(MsgClasses.REQUEST); /* message is a request */
reqMsg.domainType(DomainTypes.MARKET_PRICE);
/* Set RequestMsgFlags.HAS_BATCH so provider application is alerted to batch payload */
reqMsg.applyHasQos();
reqMsg.applyStreaming();
reqMsg.applyHasBatch();
reqMsg.qos().timeliness(QosTimeliness.REALTIME);
/* Populate msgKey - no name should be provided as all names should be in payload */
reqMsg.msgKey().applyHasNameType();
reqMsg.msgKey().applyHasServiceId();
reqMsg.msgKey().nameType(InstrumentNameTypes.RIC);
reqMsg.msgKey().serviceId(5);
/* Payload type is an element list */
reqMsg.containerType(DataTypes.ELEMENT_LIST);
/* Populate streamId with value to start streamId assignment */
reqMsg.streamId(10); /* Batch status response should be delivered using streamId 10 */
/* Begin message encoding */
retCode = reqMsg.encodeInit(encIter, 0);
{
    Array nameList = CodecFactory.createArray();
    ArrayEntry nameEntry = CodecFactory.createArrayEntry();
    elementList.applyHasStandardData();
    /* now encode nested container using its own specific encode methods */
    retCode = elementList.encodeInit(encIter, null, 0);
    /* Batch requests require an element with the name of :ItemList */
    elemEntry.name().data(":ItemList");
    elemEntry.dataType(DataTypes.ARRAY);
    /* encode array of item names in the element entry */
    retCode = elemEntry.encodeInit(encIter, 0);
    {
        Buffer nameBuf = CodecFactory.createBuffer();
        /* Encode the array and the names */
        nameList.primitiveType(DataTypes.ASCII_STRING);
        nameList.itemLength(0); /* Array will have variable length entries */
        retCode = nameList.encodeInit(encIter);
        /* Populate first name in the list. This should use streamId 11 when the response comes */
        nameBuf.data("TRI");
        nameEntry.clear();
    }
}
```



```

nameEntry.encode(encIter, nameBuf);
/* Populate the second name in the list. This should use streamId 12 when the response comes */
nameBuf.data("GOOG.O");
nameEntry.clear();
nameEntry.encode(encIter, nameBuf);
/* Populate the third name in the list. This should use streamId 13 when the response comes */
nameBuf.data("AAPL.O");
nameEntry.clear();
nameEntry.encode(encIter, nameBuf);
/* List is complete, finish encoding array */
retCode = nameList.encodeComplete(encIter, true);
}
/* Complete the element encoding and then the element list */
retCode = elemEntry.encodeComplete(encIter, true);
retCode = elementList.encodeComplete(encIter, success);
}
/* now that :ItemList is encoded in the payload, complete the message encoding */
retCode = reqMsg.encodeComplete(encIter, success);

```

Code Example 46: Batch Request Encoding Example

13.8 Dynamic View Use

Applications can use the Enterprise Transport API to send or receive requests for a dynamic view of a stream's content. A consumer application uses a **dynamic view** to specify a subset of data in which the application has interest. A provider can choose to supply only this requested subset of content across all response messages. Filtering content in this manner can reduce the volume of data that flows across the connection. View use can be leveraged across all non-administrative³ domain model types, where the model definition should specify associated usage and support. Though a consumer might request a specific view, the provider might still send additional content and/or content might be unavailable (and not provided).

A consumer application can request a view through an **RequestMsg** with the **RequestMsgFlags.HAS_VIEW** flag set and by including a specially-formatted payload. The payload should contain an **ElementList** along with:

- An **ElementEntry** for **:ViewType** which contains a **DataTypes.UINT** value indicating the specific type of view requested. Section 13.8.1 describes the currently defined **:ViewType** values.
- An **ElementEntry** for **:ViewData** which contains an **Array** populated with the content being requested. For instance, when specifying a **fieldId** list, the array would contain two-byte fixed length **DataTypes.INT** entries. The specific contents of the **:ViewData** are indicated in the definition of the **:ViewType**.

Because payload content can include customer-defined portions and LSEG-defined portions, the Enterprise Transport API uses a name-spacing scheme. Any content in the **name** member prior to the colon (:) is used as name space information (e.g., **Customer:Element**). LSEG reserves the empty name space (e.g., **:Element**). View-related enumerations and element name string constants are defined in **com.refinitiv.eta.rdm.ElementNames**.

If a consumer application wishes to change a previously-specified view, the same process can be followed by issuing a subsequent **RequestMsg** using the same **streamId** (a reissue). In this case, **:ViewData** would contain the newly desired view. If a reissue is required and the consumer wants to continue using the same view, the **RequestMsg** should continue to include the **RequestMsgFlags.HAS_VIEW** flag, **:ViewType** or **:ViewData** are not required. Sending a **RequestMsg** without the **RequestMsgFlags.HAS_VIEW** flag removes any view associated with a stream.

A provider application can receive a view request and determine an appropriate way to respond. Response content can be filtered to abide by the view specification, or the provider can send full/additional content. Several **State.code** values are available to convey view-related status. If a view's possible content changes (e.g., a previously requested field becomes available), a **RefreshMsg** should be provided to convey such a change to the data. This refresh should follow the rules associated with solicited or unsolicited refresh messages.

A provider application can indicate support for dynamic view handling by sending the **msgKey** attribute **supportViewRequests** in the Login domain **RefreshMsg**.

- For details on **State.code** values, refer to Section 11.2.7.6.
- For details on the **RequestMsg** and **RequestMsgFlags.HAS_VIEW** flag values, refer to Section 12.2.1.
- For details on the **ElementList**, refer to Section 11.3.2.
- For rules associated with refresh messages, refer to Section 12.2.2.
- For details on the Login **domainType** (**DomainTypes.LOGIN**) and general view use, refer to the *Transport API LSEG Domain Model Usage Guide*.

3. Administrative domain types are considered to be the Login, Directory, and Dictionary domain models. Other domains are considered non-administrative.

13.8.1 RDM ViewTypes Names

The following table defines the `com.refinitiv.eta.rdm.ViewTypes`.

VIEW TYPE	DESCRIPTION
FIELD_ID_LIST	Indicates that <code>:ViewData</code> contains an array populated with <code>fieldId</code> values. The array should specify a <code>primitiveType</code> of <code>DataTypes.INT</code> and a fixed two-byte <code>itemLength</code> . For specific details about the Array , refer to Section 11.2.8.
ELEMENT_NAME_LIST	Indicates that <code>:ViewData</code> contains an array populated with element <code>name</code> values. The array should specify a <code>primitiveType</code> corresponding to the type used for the domain model's element names (e.g. <code>DataTypes.ASCII_STRING</code>). For specific details about the Array , refer to Section 11.2.8.

Table 162: LSEG Domain Model Viewtypes Values

13.8.2 Dynamic View RequestMsg Encoding Example

The following example demonstrates how to encode an `RequestMsg` which specifies a `fieldId`-based view. The request asks for two fields, though it is possible that more will be delivered. For the sake of simplicity, some error handling is omitted from the example; though applications should perform all appropriate error handling.

```
/* Example assumes encode iterator is properly initialized */
/* Create and populate request message, set view flag */
reqMsg.msgClass(MsgClasses.REQUEST); /* message is a request */
reqMsg.domainType(DomainTypes.MARKET_PRICE);
/* Set RequestMsgFlags.HAS_VIEW so provider application is alerted to view payload */
reqMsg.applyHasQos();
reqMsg.applyStreaming();
reqMsg.applyHasView();
reqMsg.streamId(15);
reqMsg.qos().timeliness(QosTimeliness.REALTIME);
/* Populate msgKey */
reqMsg.msgKey().applyHasName();
reqMsg.msgKey().applyHasNameType();
reqMsg.msgKey().applyHasServiceId();
reqMsg.msgKey().nameType(InstrumentNameTypes.RIC);
reqMsg.msgKey().name().data("TRI");
reqMsg.msgKey().serviceId(5);
/* Payload type is an element list */
reqMsg.containerType(DataTypes.ELEMENT_LIST);
/* Begin message encoding */
retCode = reqMsg.encodeInit(encIter, 0);
{
    UInt viewTypeUInt = CodecFactory.createUInt();
    Array fidList = CodecFactory.createArray();
    ArrayEntry fidEntry = CodecFactory.createArrayEntry();
    elementList.applyHasStandardData();
    /* now encode nested container using its own specific encode methods */
    retCode = elementList.encodeInit(encIter, null, 0);
}
```

```

/* Initial view requests require two elements, one with the name of :ViewType and the other :ViewData
   */
elemEntry.name().data(":ViewType");
elemEntry.dataType(DataTypes.UINT);
viewTypeUInt.value(ViewTypes.FIELD_ID_LIST);
retCode = elemEntry.encode(encIter, viewTypeUInt);
/* encode array of fieldIds in the element entry */
elemEntry.name().data(":ViewType");
elemEntry.dataType(DataTypes.ARRAY);
retCode = elemEntry.encodeInit(encIter, 0);
{
    Int fieldIdInt = CodecFactory.createInt();
    /* Encode the array and the fieldIds. FieldId list should be fixed two byte integers */
    fidList.primitiveType(DataTypes.INT);
    fidList.itemLength(2); /* Array will have fixed 2 byte length entries */
    retCode = fidList.encodeInit(encIter);
    /* Populate first fieldId in the list. */
    /* Passed in as third parameter as data is not pre-encoded */
    fieldIdInt.value(22); /* fieldId for BID */
    fidEntry.clear();
    fidEntry.encode(encIter, fieldIdInt);
    /* Populate the second fieldId in the list */
    fieldIdInt.value(25); /* fieldId for ASK */
    fidEntry.clear();
    fidEntry.encode(encIter, fieldIdInt);
    /* List is complete, finish encoding array */
    retCode = fidList.encodeComplete(encIter, true);
}
/* Complete the element encoding and then the element list */
retCode = elemEntry.encodeComplete(encIter, true);
retCode = elementList.encodeComplete(encIter, success);
}
/* now that :ViewType and :ViewData are encoded in the payload, complete the message encoding */
retCode = reqMsg.encodeComplete(encIter, success);

```

Code Example 47: View Request Encoding Example

13.9 Posting

The Enterprise Transport API provides **posting** functionality: an easy way for consumer applications to publish content to upstream components for further distribution. Posting is similar in concept to unmanaged publications or SSL Inserts, where content originates from a consuming application and flows upstream to some destination component. After arriving at the destination component, content can be incorporated into cache and republished to downstream applications with an acknowledgment issued to the posting application. Via posting, the Enterprise Transport API can push content to all non-administrative⁴ domain model types, where specific usage and support should be indicated in the model definition. **PostMsg** payloads can include any container type; often this is an **Msg** (**DataTypes.MSG**). When payload is a **Msg**, the contained message should be populated with any contributed header and payload information. For additional information on how to encode and decode container types, refer to Section 11.3.

The Transport API offers two types of posting:

- **On-stream posting**, where you send a **PostMsg** on an existing data stream, in which case posted content corresponds to the stream on which it is posted. The upstream route of an on-stream post is determined by the route of the data stream over which it is sent. On-stream posting should be directed towards the provider that sources the item. Because on-stream post messages are flowing on the stream related to the item, a **msgKey** is not required. If the content is republished by the upstream provider, the consumer should receive it on the same stream over which they posted it.
- **Off-stream posting**, where you send a **PostMsg** on the **streamId** associated with the users Login. Thus a consumer application can post data, regardless of whether they have an open stream associated with the post-related item. Post messages issued on this stream must indicate the specific **domainType** and **msgKey** corresponding to the content being posted. Off-stream posting is typically routed by configuration values on the upstream components.

The **PostMsg** contains **Visible Publisher Identifier** information (contained in **PostMsg.postUserInfo**), which identifies the user who posted it. **PostMsg.postUserInfo** must be populated and consists of:

- **postUserId**: which should be an ID associated with the user. For example, a Data Access Control System user ID or if unavailable, a process id)
- **postUserAddr**: which should contain the IP address⁵ of the application posting the content.

Optionally, such information can be carried along with republished **RefreshMsgs**, **UpdateMsgs**, or **StatusMsgs** so that receiving consumers can identify the posting user. For more information about the Visible Publisher Identifier, refer to Section 13.11.

PostMsg.permData permissions the user who posts data. If the payload of the **PostMsg** is another nested message type (i.e., **RefreshMsg**) with permission data, such permission data can change the permission expression of the item being posted. However, if the permission data for the nested message is the same as the permission data on the **PostMsg**, the nested message does not need to include permission data. The permission data is used in conjunction with the **PostMsg.postUserRights**, which indicate:

- Whether the posting user can create or destroy items in the cache of record.
- Whether the user has the ability to change the **permData** associated with an item in the cache of record.

Each independent post message flowing in a stream should use a unique **postId** to distinguish between individual post messages and those used for acknowledgment purposes. The consumer can request an acknowledgment upon the successful receipt and processing of content. When the provider responds, the **AckMsg.ackId** should be populated using the **PostMsg.postId** to match the two messages. **seqNum** information can also be used during acknowledgment.

NOTE: Provider applications that support posting must have the ability to properly acknowledge posted content.

You can split content across multiple **PostMsg** messages. When sending a multi-part **PostMsg**, the **postId** should match all parts of the post. If the consumer requests an acknowledgment, the **seqNum** is also required. Each part should be acknowledged by the receiving component, where each **AckMsg.ackId** is populated using the **PostMsg.postId**, and each **AckMsg.seqNum** is populated using the **PostMsg.seqNum**. Each part of the **PostMsg** should specify a **partNum**, where the first part begins with 0. The final part of a multi-part **PostMsg** should have the **PostMsgFlags.POST_COMPLETE** flag set to indicate that it is the final part.

4. Administrative domain types are considered to be the Login, Directory, and Dictionary domain models. Other domains are considered non-administrative.

5. The **Transport.hostByName** method can be used to help obtain the IP address of the application. Refer to Section 10.14.

A provider application can indicate support for posting and acknowledgment use by sending the **msgKey** attribute **supportOmmPost** in the Login domain **RefreshMsg**.

- For more information on the **PostMsg**, refer to Section 12.2.7.
- For more information on the **AckMsg**, refer to Section 12.2.8.
- For more information on managing multi-part **PostMsgs**, refer to Section 13.1.
- For more details on the Login **domainType** (**DomainTypes.LOGIN**), see the *Transport API LSEG Domain Model Usage Guide*.

13.9.1 Post Message Encoding Example

The following example demonstrates how to encode an off-stream **PostMsg** with a nested **Msg**.

```
/* Example assumes encode iterator is properly initialized */
/* Create and populate post message - since it's off stream, msgKey is required */
PostMsg postMsg = (PostMsg)CodecFactory.createMsg();
postMsg.msgClass(MsgClasses.POST);
postMsg.streamId(1); /* Use streamId of the Login stream for off-stream posting */
postMsg.domainType(DomainTypes.MARKET_PRICE); /* domainType of data being posted */
/* off stream requires key. Post asking for ACK and including postId and seqNum for ack purposes.
   Since it's a single part post, the POST_COMPLETE flag must be set as well */
postMsg.applyHasMsgKey();
postMsg.applyAck();
postMsg.applyHasPostId();
postMsg.applyHasSeqNum();
postMsg.applyPostComplete();
/* Populate msgKey with information about the item being posted to */
postMsg.msgKey().applyHasName();
postMsg.msgKey().applyHasNameType();
postMsg.msgKey().applyHasServiceId();
postMsg.msgKey().nameType(InstrumentNameTypes.RIC);
postMsg.msgKey().name().data("TRI");
postMsg.msgKey().serviceId(5);
/* populate postId with a unique ID for this posting, this and seqNum are used on ack */
postMsg.postId(42);
postMsg.seqNum(124);
/* postUserInfo must be populated, with processId and IP address */
postMsg.postUserInfo().userId(Thread.currentThread().getId());
postMsg.postUserInfo().userAddr(InetAddress.getLocalHost().getHostAddress());

/* put a message in the postMsg */
postMsg.containerType(DataTypes.MSG);
/* Begin message encoding */
retCode = postMsg.encodeInit(encIter, 0);
{
    /* populate the message that is in the payload of the post message */
    UpdateMsg updMsg = (UpdateMsg)CodecFactory.createMsg();
    updMsg.msgClass(MsgClasses.UPDATE);
    updMsg.streamId(1);
    updMsg.domainType(DomainTypes.MARKET_PRICE);
```

```

updMsg.updateType(UpdateEventTypes.QUOTE);
updMsg.containerType(DataTypes.FIELD_LIST);
/* begin encoding of the payload message */
retCode = updMsg.encodeInit(encIter, 0);
/* Continue encoding field list contents of the message - see example in Section 11.3.1 */
/* Complete the postMsg payload messages encoding */
retCode = updMsg.encodeComplete(encIter, true);
}
/* now complete encoding of postMsg */
retCode = postMsg.encodeComplete(encIter, success);

```

Code Example 48: Off-Stream Posting Encoding Example

13.9.2 Post Acknowledgement Encoding Example

The following example demonstrates how to encode an **AckMsg**.

```

/* Example assumes encode iterator is properly initialized */
/* Create and populate ack message with information used to acknowledge the post */
AckMsg ackMsg = (AckMsg)CodecFactory.createMsg();
ackMsg.msgClass(MsgClasses.ACK);
ackMsg.domainType(DomainTypes.MARKET_PRICE);
ackMsg.streamId(1); /* Ack should be sent back on same stream that post came on */
ackMsg.applyHasSeqNum();
/* Acknowledge the post from above, use its postId and seqNum */
ackMsg.ackId(postMsg.postId());
ackMsg.seqNum(postMsg.seqNum());
/* No payload associated with this acknowledgment */
ackMsg.containerType(DataTypes.NO_DATA);
/* Since there is no payload, no need for Init/Complete as everything is in the msg header */
retCode = ackMsg.encode(encIter);

```

Code Example 49: Post Acknowledgement Encoding Example

13.10 Round Trip Time Examples

13.10.1 Round Trip Time Message Sending Example

```
LoginRttInfo loginRttInfo = loginRttInfoList.get(channel);
loginRtt.clear();
loginRtt.updateRTTActualTicks();
TransportBuffer transportBuffer = encodeMsgAndGetBuffer(channel, loginRtt);
writeMsgIntoChnl(channel, transportBuffer, error);
```

13.10.2 Round Trip Time Message Calculation Example

```
if (Objects.equals(DataTypes.ELEMENT_LIST, msg.containerType())) {
    loginRtt.clear();
    ret = loginRtt.decode(dIter, msg);
    /*get rtt in nanos. Method also writes obtained value into rtLatency variable*/
    long calculatedRtt = loginRtt.calculateRTTLatency(TimeUnit.NANOSECONDS);
}
```


13.11 Visible Publisher Identifier

The Enterprise Transport API offers the **Visible Publisher Identifier** feature, which inserts originating publisher data into message payloads. You can use Visible Publisher Identifier data to identify the user ID and user address for users who post, insert, or publish to an interactive service or to a non-interactive service cache on the LSEG Real-Time Advanced Distribution Hub.

Visible Publisher Identifier data is present on Post, Refresh, Update, and Status Messages and is carried in **PostMsg.postUserInfo**, which consists of:

- Post user ID (i.e., publisher ID)
- Post user address (i.e., publisher address)

They can both contain values assigned by and specific to the application.

A **PostMsg** contains data (in **PostMsg.postUserInfo**) that identifies the user who posts content. For this reason, **PostMsg.postUserInfo** must be populated with:

- **PostUserInfo.userId**: An ID associated with the posting user. The application should determine what information to put into this field (e.g., a Data Access Control System user ID).
- **PostUserInfo.userAddr**: The address of the posting user's application that posted the contents. The application should decide what information to put into this field, for example, an IP address.

Optionally, this data can be republished by the provider in **RefreshMsgs**, **UpdateMsgs**, or **StatusMsgs** so that receiving consumers can identify the posting user.

The Enterprise Transport API allows the Visible Publisher Identifier to be populated on Post messages submitted by a consumer application before the post is sent over the network.

Provider applications receive Visible Publisher Identifier data in Post Messages. Additionally, OMM providers can optionally set Visible Publisher Identifier data in their response messages. If the upstream provider is an intermediary device getting data from an upstream source, then the intermediary device will route Visible Publisher Identifier data as set in the **PostMsg** to the upstream source. The final publisher in the upward chain decides whether to set Visible Publisher Identifier data in its published responses.

Visible Publisher Identifier information can also be communicated using Field Identifiers defined in the publisher component. For further details refer to the publishing component's documentation.

13.11.1 Example: Encoding PostUserInfo into a Refresh Message

The following example shows how **PostUserInfo** is set in the **RefreshMsg** by the Provider application.

```
// Setting RefreshMsg PostUserInfo in Provider application
RefreshMsg _refreshMsg = (RefreshMsg)CodecFactory.createMsg();
_refreshMsg.msgClass(MsgClasses.REFRESH);
_refreshMsg.domainType(DomainTypes.MARKET_PRICE);
_refreshMsg.applyHasPostUserInfo();
try
{
    _refreshMsg.postUserInfo().userAddr(InetAddress.getLocalHost().getHostAddress());
}
catch (Exception e)
{
    System.out.println("Populating postUserInfo failed. InetAddress.getLocalHost().
        getHostAddress exception: " + e.getLocalizedMessage());
}
_refreshMsg.postUserInfo().userId(Integer.parseInt(System.getProperty("pid", "1")));
```

Table 163: Setting PostUserInfo in Provider Example

13.11.2 Example: Decoding PostUserInfo from Refresh Message

The following example shows the Consumer application using **PostUserInfo** to obtain Visible Publisher Identifier data in the **processMarketPriceResponse()** method.

```
/* The Visible Publisher Identifier (VPI) can be found within the RsslPostUserInfo.
/* This will provide both the publisher ID and publisher address. Consumer can obtain the
/* information from the msg - The partially decoded message. */
if (_refreshMsg.checkHasPostUserInfo())
{
    System.out.print("\nReceived RefreshMsg for stream " + _refreshMsg.streamId());
    System.out.println(" from publisher with user ID: " + _refreshMsg.postUserInfo().userId() +
        " at user address: " + _refreshMsg.postUserInfo().userAddrToString
            (_refreshMsg.postUserInfo().userAddr()) + "\n");
}
```

Table 164: Getting PostUserInfo in a Consumer Example Sent by Provider

13.11.3 Example: Encoding PostUserInfo into a Post Message

The following example populates Visible Publisher Identifier on Post messages submitted by a Transport API consumer application in the **encodePostWithMsg()** method internally used by the **sendPostMsg()** method. It encodes a **PostMsg** and populates the **PostUserInfo** with the IP address and process ID of the machine running the application.

```
postMsg.clear();

// set-up message
postMsg.msgClass(MsgClasses.POST);
postMsg.streamId(streamId);
postMsg.domainType(DomainTypes.MARKET_PRICE);
postMsg.containerType(DataTypes.MSG);

// Note: post message key not required for on-stream post
postMsg.applyPostComplete();
postMsg.applyAck();
postMsg.applyHasPostId();
postMsg.applyHasSeqNum();
postMsg.applyHasMsgKey();
postMsg.applyHasPostUserRights();
postMsg.postId(nextPostId++);
postMsg.seqNum(nextSeqNum++);

// populate post user info
try
{
    postMsg.postUserInfo().userAddr(InetAddress.getLocalHost().getHostAddress());
}
catch (Exception e)
{
}
```

Table 165: Set PostUserInfo in Consumer Example

```

        System.out.println("Populating postUserInfo failed. InetAddress.getLocalHost().getHostAddress
exception: " + e.getLocalizedMessage());
        return CodecReturnCodes.FAILURE;
    }

postMsg.postUserInfo().userId(Integer.parseInt(System.getProperty("pid", "1")));
postMsg.postUserRights(PostUserRights.CREATE | PostUserRights.DELETE);

```

Table 165: Set PostUserInfo in Consumer Example

13.11.4 Example: Decoding PostUserInfo from Post Message

The following example shows how **PostUserInfo** is set in the Provider application from the message sent by the Consumer.

```

PostMsg postMsg = (PostMsg)msg;

// if the post message contains another message, then use the
// "contained" message as the update/refresh/status
if (postMsg.containerType() == DataTypes.MSG)
{
    _nestedMsg.clear();
    int ret = _nestedMsg.decode(dIter);
    if (ret != CodecReturnCodes.SUCCESS)
    {
        error.text("Unable to decode msg");
        return ret;
    }
    switch (_nestedMsg.msgClass())
    {
        case MsgClasses.REFRESH:
            _nestedMsg.msgClass(MsgClasses.REFRESH);
            int flags = _nestedMsg.flags();
            flags |= RefreshMsgFlags.HAS_POST_USER_INFO;
            flags &= ~RefreshMsgFlags.SOLICITED;
            _nestedMsg.flags(flags);

            ((RefreshMsg)_nestedMsg).postUserInfo().userAddr(postMsg.postUserInfo().userAddr());
            ((RefreshMsg)_nestedMsg).postUserInfo().userId(postMsg.postUserInfo().userId());
            if (updateItemInfoFromPost(itemInfo, _nestedMsg, dIter, error) != CodecReturnCodes.SUCCESS)
            {
                ret = sendAck(chnl, postMsg, NakCodes.INVALID_CONTENT, error.text(), error);
                if (ret != CodecReturnCodes.SUCCESS)
                {
                    return ret;
                }
            }

            break;

        case MsgClasses.UPDATE:

```

Table 166: Getting PostUserInfo from Post Messages in a Provider Example Sent by Consumer

```

        _nestedMsg.msgClass(MsgClasses.UPDATE);
        ((UpdateMsg)_nestedMsg).flags(_nestedMsg.flags() | UpdateMsgFlags.HAS_POST_USER_INFO);
        ((UpdateMsg)_nestedMsg).postUserInfo().userAddr(postMsg.postUserInfo().userAddr());
        ((UpdateMsg)_nestedMsg).postUserInfo().userId(postMsg.postUserInfo().userId());
        if (updateItemInfoFromPost(itemInfo, _nestedMsg, dIter, error) != CodecReturnCodes.SUCCESS)
        {
            ret = sendAck(chnl, postMsg, NakCodes.INVALID_CONTENT, error.text(), error);
            if (ret != CodecReturnCodes.SUCCESS)
            {
                return ret;
            }
        }
        break;

    case MsgClasses.STATUS:
        _nestedMsg.msgClass(MsgClasses.STATUS);
        ((StatusMsg)_nestedMsg).flags(_nestedMsg.flags() | StatusMsgFlags.HAS_POST_USER_INFO);
        ((StatusMsg)_nestedMsg).postUserInfo().userAddr(postMsg.postUserInfo().userAddr());
        ((StatusMsg)_nestedMsg).postUserInfo().userId(postMsg.postUserInfo().userId());
        if (((StatusMsg)_nestedMsg).checkHasState() && ((StatusMsg)_nestedMsg).state().streamState()
            == StreamStates.CLOSED)
        {
            if (postMsg.checkHasPostUserRights() || postMsg.postUserRights() == 0)
            {
                ret = sendAck(chnl, postMsg, NakCodes.INVALID_CONTENT, "client has insufficient rights
to close/delete an item", error);
                if (ret != CodecReturnCodes.SUCCESS)
                {
                    return ret;
                }
            }
        }
        break;
    default:
        break;
}
}

```

Table 166: Getting PostUserInfo from Post Messages in a Provider Example Sent by Consumer

13.12 UserAuthn Authentication

The Enterprise Transport API can use the UserAuthn Authentication feature, which provides enhanced authentication functionality when used with the LSEG Real-Time Distribution System and Data Access Control System. This feature requires LSEG Real-Time Distribution System 3.1 or later.

A consumer or non-interactive provider application can pass a token generated from a token generator based on the user's credentials to LSEG Real-Time Distribution System. LSEG Real-Time Distribution System passes this token to a local token authenticator for verification.

The token must be encoded in the initial login **RequestMsg** with:

- **msgKey.Name** set to one byte of **0x00**, and
- **msgKey.NameType** set to **Login.UserIdTypes.USER_AUTHN_TOKEN**.

The token will be in the **msgKey.attrib**'s **ElementList**, with an **ElementEntry** named **authenticationToken**.

For additional information, refer to the *Transport API LSEG Domain Model Usage Guide* for encoding and decoding Login messages, and the *UserAuthn Authentication User Manual*⁶ for details on setting up the LSEG Real-Time Distribution System and the token generator.

13.13 Private Streams

The Enterprise Transport API provides *private stream* functionality, an easy way to ensure delivery of content only between a stream's two endpoints. Private streams behave in a manner similar to standard streams, with the following exceptions:

- All data on a private stream flow between the end provider and the end consumer of the stream.
- Intermediate components do not fan out content (i.e., do not distribute it to other consumers).
- Intermediate components should not cache content.
- In the event of connection or data loss, intermediate components do not recover content. All private stream recovery is the responsibility of the consumer application.

These behaviors ensure that only the two endpoints of the private stream send or receive content associated with the stream. As a result, a private stream can exchange identifying information so the provider can validate the consumer, even through multiple intermediate components (such as might exist in an LSEG Real-Time Distribution System deployment). After a private stream is established, content can flow freely within the stream, following either existing market data semantics (i.e., private Market Price domain) or any other user-defined semantics (i.e., bidirectional exchange of **GenericMsgs**).

For more information about private stream instantiation, refer to Section 13.13.

In standard streams, if an application attempts to open the same stream using multiple, unique **streamId** values, provider applications reject subsequent requests. With private streams, even if the streams' identifying information (**msgKey**, domain type, etc.) matches, multiple private stream instances can be opened, allowing for the possibility of different user data contained in each private stream.

To establish a private stream, a consumer observes the following general process:

- The consumer application issues a request for the item data it wants on a private stream. This **RequestMsg** should include the **RequestMsgFlags.PRIVATE_STREAM** flag. If user-identifying information is required, it should be described in the respective domain message model definition.
- When a capable OMM provider application receives a request for a private stream, if it can honor the request, the provider application should acknowledge that the stream is established and is private by sending:
 - **RefreshMsg** with the **RequestMsgFlags.PRIVATE_STREAM** flag; typically sent when there is immediate content to provide in the response.
 - **StatusMsg** with the **StatusMsgFlags.PRIVATE_STREAM** flag; typically sent when there is no immediate content to provide in the response but the provider wants to acknowledge the establishment of the private stream.
 - **AckMsg** with the **AckMsgFlags.PRIVATE_STREAM** flag; can be used as an alternative to the **StatusMsg**.

6. For further details on UserAuthn Authentication, refer to the *UserAuthn Authentication User Manual*, accessible on [MyAccount](#) in the Data Access Control System product documentation set.

- When the consumer application receives the above acknowledgment, the private stream is established and content can be exchanged. The **PRIVATE_STREAM** flag is no longer required on any messages exchanged within the stream.
- If the consumer application receives any other message, or the above messages without their respective **PRIVATE_STREAM** flag, the private stream is not established and the consumer should close the stream if it does not want to consume a standard stream.

Some content might be available as both standard stream and private stream delivery mechanisms. In the standard stream case, all users see the same stream content. Because private streams can support user identification, each private stream instance can contain modified or additional content tailored for the specific user.

Some content might be available only as standard streams, in which case the private stream request is ignored or rejected by sending an **StatusMsg** with a **streamState** of **StreamStates.CLOSED** or **StreamStates.CLOSED_RECOVER**, or by responding to the request with a standard stream (e.g., no **PRIVATE_STREAM** flag).

Some content might be available only as a private stream such as restricted datasets requiring user validation. If an OMM provider has private-only content, it can signal this to downstream applications by redirecting standard stream requests.

If a standard stream **RequestMsg** is received for private-only content, a provider can:

- Inform downstream applications that its content is private by sending a message (including the **msgKey**), with a **streamState** of **StreamStates.REDIRECTED** in a:
 - **StatusMsg** including the **StatusMsgFlags.PRIVATE_STREAM** flag; typically sent when there is not any content to provide as part of the redirect.
 - **RefreshMsg** including the **RequestMsgFlags.PRIVATE_STREAM** flag; typically sent when there is some kind of content to provide as part of the redirect.
- If the consumer application sees a **streamState** of **StreamStates.REDIRECTED** and a **PRIVATE_STREAM** flag, it can issue a new **RequestMsg** and use the **RequestMsgFlags.PRIVATE_STREAM** flag. This process follows standard stream redirect logic and the private stream establishment protocol described above.

13.14 Creating a DACSLOCK for Publishing Permission Data

Provider applications can create a DACSLocks and publish it to permission data on the LSEG Real-Time Distribution System. A DACSLock controls access to data by users. For further details on the DACSLock API, refer to the *Transport API Java Edition DACSLock API Developers Guide*.

The following example code illustrates how to create a DACSLock.

```
import com.refinitiv.eta.dacs.*;

/* Generates DACS lock */
JDacsLock _dacsInterface = JDacsLock.createJDacsLock();
DacsError _error = JDacsLock.createDacsError();
char operation = DacsOperations.OR_OPERATION;
int serviceId = 261;
long[] productEntityList = new long[256];
int productEntityListLength = 1;
productEntityList[0] = 1001;

int len = _dacsInterface.calculateLockLength(serviceId, operation, productEntityList,
productEntityListLength, _error);

ByteBuffer lockData = ByteBuffer.allocate(len);
DacsLock lock1 = JDacsLock.createLock();
lock1.data(lockData);

int ret = _dacsInterface.createLock(serviceId, operation, productEntityList,
productEntityListLength, lock1, _error);

if (ret == DacsReturnCodes.NO_ERROR) {
    System.out.println("createLock() - Success");
} else {
    System.err.println("createLock() failed " + _error.errorId() + " - " + _error.text());
}
```

Code Example 50: Creating a DACSLOCK for Publishing Permission Data

Appendix A Item and Group State Decision Table

The following table describes various item and group status combinations and the common results in terms of application behavior. Though applications are not required to follow this behavior, the information is provided as an example of one possible behavior.

- For general information about **State**, refer to Section 11.2.7.
- For general information about Item Groups, refer to Section 13.4.
- For information about group status delivery and formatting, refer to the *Transport API LSEG Domain Model Usage Guide*.
- For information about how item state is conveyed, refer to Section 12.2.2 and Section 12.2.4.

STATUS TYPE	STREAM STATE	DATA STATE	DESCRIPTION	APPLICATION ACTION
Item	StreamStates.OPEN	DataStates.OK	Stream is open and streaming. Data is ok.	No action.
Item	StreamStates.OPEN	DataStates.SUSPECT	Stream is open and streaming. Data is suspect.	No action. Upstream device should recover data and onpass.
Item	StreamStates.NON_STREAMING	DataStates.OK	Stream was opened as non-streaming. Data was provided for item and was OK.	No action.
Item	StreamStates.CLOSED	DataStates.SUSPECT	Stream is closed. Data is suspect.	Application can attempt to recover this or another service or provider.
Item	StreamStates.CLOSED_RECOVER	DataStates.SUSPECT	Stream is closed, but may become available on same service and provider later. Data is suspect.	Application can attempt to recover to this or another service or provider.
Item	StreamStates.CLOSED	DataStates.OK	Stream is closed. Data provided was OK.	Application can attempt to recover to this or another service or provider. This state combination is not common.
Item	StreamStates.CLOSED_RECOVER	DataStates.OK	Stream is closed, but may become available on same service and provider later. Data provided was OK.	Application can attempt to recover to this or another service or provider. This state combination is not common.

Table 167: Item and Group State Decision Table

STATUS TYPE	STREAM STATE	DATA STATE	DESCRIPTION	APPLICATION ACTION
Group	StreamStates.OPEN	DataStates.NO_CHANGE	All streams associated with the group remain open. Previous state communicated via item or group status continues to apply.	No action.
Group	StreamStates.OPEN	DataStates.SUSPECT	All streams associated with the group remain open. Data on all streams associated with the group is suspect.	Application should fan out dataState change to all items that are part of the group. Upstream device should recover data and onpass.
Group	StreamStates.OPEN	DataStates.OK	All streams associated with the group remain open. Data on all streams associated with the group is ok.	Application should fan out dataState change to all items that are part of the group. This state combination is not common. Typically individual item statuses are used to change items from suspect to ok.
Group	StreamStates.CLOSED_RECOVER	DataStates.SUSPECT	All streams associated with the group are closed, but may become available on same service and provider later. Data on all streams associated with the group is suspect.	Application should fan out streamState and dataState change to all items that are part of the group. Application can attempt to recover to this or another service or provider.

Table 167: Item and Group State Decision Table (Continued)

Appendix B RWF/JSON Converter

B.1 Overview

If you use the WebSockets protocol to send and receive data with the Enterprise Transport API using JSON format, you must convert the data using the RWF/JSON converter.

Message conversion takes place if JSON messages follow the [WebSocket protocol specification](https://github.com/Refinitiv/websocket-api/blob/master/WebsocketAPI_ProtocolSpecification.pdf) at https://github.com/Refinitiv/websocket-api/blob/master/WebsocketAPI_ProtocolSpecification.pdf. RWF/JSON conversion supports most of the typical use cases of nested containers within messages, but not all combinations.



WARNING! Violations of the WebSocket protocol specifications will result in errors or the channel being closed.

NOTE: When converting Map or Vector containers which might have a mix of delete and other actions, the converter must traverse the depth of actions to determine whether the container type incurs additional processing overhead versus converting a container with an initial action that is a non-delete action.

B.2 Automatic Conversion using the Transport API Reactor

The Transport API Reactor autoconverts data between JSON and RWF protocols and presents RWF to the application layer. If you use the Reactor to manage the conversion, you can skip this appendix and refer instead to the Enterprise Transport API Java *Value Added Components Developer Guide*.

B.3 Creating the Converter

B.3.1 Create JsonConverterBuilder Instance

The **ConverterFactory** class includes methods that create all basic objects associated with RWF/JSON conversion. To obtain a converter instance, first create a **JsonConverterBuilder** instance by calling the **ConverterFactory.createJsonConverterBuilder()** method.

B.3.2 Setting Converter Properties

Before building a converter instance, pass the **JsonConverterBuilder** instance the necessary converter properties. The properties can be set either one-by-one or in batch mode.

METHOD	DESCRIPTION
setProperty	Sets necessary properties in batch mode using the parameter properties , which specifies a map of propertyId , propertyValue pairs.
setProperty	Sets the value of an individual boolean or integer property. setProperty uses one of the following parameters: int propertyId, boolean enabled: Sets the id of the boolean property and its value or: int propertyId, int propertyValue: sets the ID of the integer property and its value
setServiceConverter	Passes the ServiceNameIdConverter instance to the converter, which then provides callbacks that convert serviceName to service id or the reverse.
setDictionary	Accepts a DataDictionary instance as a parameter. The provided DataDictionary will be used by the Converter while performing transformations.
build	Performs all necessary actions to build a Converter instance and returns it to the user. Accepts the parameter: JsonConverterError error . If the build operation fails, this parameters contains associated error information.

Table 168: JsonConverterBuilder Methods

B.3.3 Properties Supported by JsonConverterBuilder

Properties supported by **JsonConverterBuilder** (stored in the **JsonConverterProperties** class) are as follows:

PROPERTY ID	PROPERTY NAME	DESCRIPTION
2	JSON_CPC_DICTIONARY_LIST	Specifies the DataDictionary property of the converter.
3	JSON_CPC_DEFAULT_SERVICE_ID	Specifies the default serviceId used by the converter.
4	JSON_CPC_USE_DEFAULT_DYNAMIC_QOS	Specifies the default Dynamic QOS used by the converter.
5	JSON_CPC_EXPAND_ENUM_FIELDS	Specifies whether the converter will use the display value for an enum field. Available values are true or false .
7	JSON_CPC_CATCH_UNKNOWN_JSON_KEYS	Specifies whether the converter returns an error if an unknown key is encountered in a JSON message. Available values are true or false .
8	JSON_CPC_CATCH_UNKNOWN_JSON_FIDS	Specifies whether the converter returns an error in case an unknown filed id is encountered in a JSON message. Available values are true or false .
9	JSON_CPC_ALLOW_ENUM_DISPLAY_STRINGS	Specifies whether the converter accepts display values and tries to convert them to the corresponding enums. Available values are true or false .
10	JSON_CPC_PROTOCOL_VERSION	Specifies the protocol version that the converter uses. Currently only simplified JSON protocol is supported.

Table 169: Properties Supported by JsonConverterBuilder

B.3.4 Converter Callbacks

The RWF/JSON Converter uses the following callbacks (if needed) when converting **ServiceName** to **ServiceId** or from **ServiceId** to **ServiceName**. You must use these callbacks when handling service IDs and names, otherwise they cannot be translated.

The callbacks are provided by the **ServiceNameIdConverter** interface, an instance of which is set before building the converter.

CALLBACK	DESCRIPTION
<code>int serviceNameToId(String serviceName, JsonConverterError error)</code>	This callback accepts the following parameters: • serviceName: specifies the service name that the callback looks up to find the appropriate service id. • error: an instance of JsonConverterError that will carry error information in case of unsuccessful conversion The callback returns the service ID that corresponds to the provided service name; otherwise if the service ID is not found, the callback returns CodecReturnCodes.FAILURE or -1.
<code>String serviceIdToName(int id, JsonConverterError error);</code>	This callback accepts the following parameters: • id: specifies the serviceId that the callback will look up to find the appropriate name • error: an instance of JsonConverterError that will carry error information in case of unsuccessful conversion The callback returns the serviceName that corresponds to the provided serviceId; otherwise if the service name is not found, the callback returns null.

Table 170: Converter Callbacks

B.3.5 Creating an RWF/JSON Converter Example

The following code snippet demonstrates creating an RWF/JSON Converter:

```
DataDictionary dictionary = CodecFactory.createDataDictionary();

JsonConverterError convError = ConverterFactory.createJsonConverterError();

JsonConverter converter = ConverterFactory.createJsonConverterBuilder()
    .setProperty(JsonConverterProperties.JSON_CPC_PROTOCOL_VERSION, JsonProtocol.JSON_JPT_JSON2)
    .setProperty(JsonConverterProperties.JSON_CPC_CATCH_UNKNOWN_JSON_KEYS, false)
    .setProperty(JsonConverterProperties.JSON_CPC_EXPAND_ENUM_FIELDS, true)
    .setServiceConverter(new ServiceNameIdTestConverter())
    .setDictionary(dictionary)
    .build(convError);
```

B.4 Converting from JSON to RWF

To convert from JSON to RWF, use the following **JsonConverter**'s methods:

- **parseJsonBuffer()**: To parse JSON data
- **decodeJsonMsg()**: To decode data.

B.4.1 Methods parseJsonBuffer

The **JsonConverter** interface provides two overloaded methods of **parseJsonBuffer()** (that accept either **Buffer** or **TransportBuffer** as the first parameter) to parse JSON message data into a JSON-tree.

Both **parseJsonBuffer()** methods return an integer value equal to one of the following:

- **CodecReturnCodes.SUCCESS**: meaning transformation was successful.
- **CodecReturnCodes.FAILURE** meaning an error occurred during parsing.

PARAMETER	DESCRIPTION
inBuffer	An instance of Buffer / TransportBuffer that carries the input JSON message.
options	An instance of ParseJsonOptions interface, which carries additional options that the converter can use when parsing the input JSON message.
error	The JsonConverterError instance that contains information describing the failure in case the parsing operation is unsuccessful.

Table 171: JsonConverter.parseJsonBuffer() Method Parameters

B.4.2 ParseJsonOptions Interface

The **ParseJsonOptions** Interface can provide additional parameters, which are passed to the converter when parsing a JSON message.

METHOD	DESCRIPTION
getConverterFlags	Returns flags which are passed to Converter
setConverterFlags	Sets the flags which are passed to Converter. Accepts an integer value as a parameter.
getProtocolType	Returns protocol type set for the current options instance
setProtocolType	Sets the value for the current JSON protocol. Accepts an integer value as a parameter.
clear	Clears the options

Table 172: ParseJsonOptions Interface Methods

B.4.3 Method decodeJsonMsg()

Use the `JsonConverter.decodeJsonMsg()` method to convert a JSON-tree obtained after calling the `parseJsonBuffer` method into a RWF message.

- If `JsonConverter.decodeJsonMsg()` succeeds, the method returns one of the following:
 - `CodecReturnCodes.END_OF_CONTAINER` if a single JSON message was supplied to the converter.
 - `CodecReturnCodes.SUCCESS` if the original JSON message contained an array of messages and more messages are waiting for conversion.
- If `JsonConverter.decodeJsonMsg()` fails, the method returns `CodecReturnCodes.FAILURE`.

PARAMETER	DESCRIPTION
jsonMsg	The <code>JsonMsg</code> instance, which will carry the resulting RWF message (along with other information) after the call to the <code>decodeJsonMsg</code> method.
options	The <code>DecodeJsonMsgOptions</code> instance that specifies any additional parameters to the converter.
error	The <code>JsonConverterError</code> instance, which contains information describing the failure in case the decoding operation is unsuccessful.

Table 173: `JsonConverter.decodeJsonMsg()` Parameters

B.4.4 DecodeJsonMsgOptions Interface

The `DecodeJsonMsgOptions` interface carries additional parameters to the converter when decoding a JSON message.

PARAMETER	DESCRIPTION
getRsslProtocolType	Gets the type of the protocol of the RSSL message.
setRsslProtocolType	Sets the type of the protocol of the RSSL message. <code>setRsslProtocolType</code> accepts an input integer parameter
getMajorVersion	Gets the major version of the wire format to encode.
setMajorVersion	Sets the major version of the wire format to encode. <code>setMajorVersion</code> accepts an integer input value as a parameter.
getMinorVersion	Gets the minor version of the wire format to encode.
setMinorVersion	Sets the minor version of the wire format to encode. <code>setMinorVersion</code> accepts an integer input value as a parameter.
getJsonProtocolType	Gets the JSON protocol set for the current options instance.
setJsonProtocolType	Sets the JSON protocol for the current instance of the options. The JSON protocol of the options must be equal to the JSON protocol supported by the current instance of the <code>JsonConverter</code> .
getConverterFlags	Gets the flags to be passed to the converter.
setConverterFlags	Sets the converter flags. <code>setConverterFlags</code> accepts an integer value as a parameter.

Table 174: `DecodeJsonMsgOptions` Interface Methods

B.4.5 Example: Converting from JSON to RWF

The following example illustrates conversion of JSON data to RWF.

```
ParseJsonOptions parseJsonOptions = ConverterFactory.createParseJsonOptions();
JsonConverterError converterError = ConverterFactory.createJsonConverterError();
DecodeJsonMsgOptions decodeJsonMsgOptions =
ConverterFactory.createDecodeJsonMsgOptions();
JsonMsg jsonMsg = ConverterFactory.createJsonMsg();

parseJsonOptions.clear();
parseJsonOptions.setProtocolType(reactorChannel.channel().protocolType());

converterError.clear();
retval = jsonConverter.parseJsonBuffer(msgBuf, parseJsonOptions, converterError);

if(retval == CodecReturnCodes.SUCCESS)
{
    decodeJsonMsgOptions.clear();
    decodeJsonMsgOptions.setJsonProtocolType(JsonProtocol.JSON_JPT_JSON2);
    decodeJsonMsgOptions.setMajorVersion(reactorChannel.channel().majorVersion());
    decodeJsonMsgOptions.setMinorVersion(reactorChannel.channel().minorVersion());

    /* Set the ReactorChannel so that users can get it in the ReactorServiceNameToIdCallback callback */
    if(Objects.nonNull(serviceNameIdConverterClient))
    {
        serviceNameIdConverterClient.setReactorChannel(reactorChannel);
    }

    jsonMsg.clear();

    while ( (retval = jsonConverter.decodeJsonMsg(jsonMsg, decodeJsonMsgOptions, converterError)) !=
        CodecReturnCodes.END_OF_CONTAINER)
    {
        if(retval != CodecReturnCodes.SUCCESS)
        {
            /* Failed to convert a JSON message. */
            break;
        }

        switch(jsonMsg.jsonMsgClass())
        {
            case JsonMsgClasses.RSSI_MESSAGE:
            {
                failedToConvertJSONMsg = false;
                // inspect the converted message and dispatch it to the application.
                retval = processRwfMessage(msgBuf, jsonMsg.rwfMsg().encodedMsgBuffer(),
                    reactorChannel, errorInfo);
                if (retval != ReactorReturnCodes.SUCCESS)
                {

```

```

        return retval;
    }

    break;
}
case JsonMsgClasses.PING:
{
    failedToConvertJSONMsg = false;

    TransportBuffer msgBuffer = reactorChannel.getBuffer(JSON_PONG_MESSAGE.length(),
        false, errorInfo);

    if (Objects.nonNull(msgBuffer))
    {
        msgBuffer.data().put(JSON_PONG_MESSAGE.getBytes());

        /* Reply with JSON PONG message to the sender */
        retval = sendJSONMessage(msgBuffer, reactorChannel, errorInfo);
    }
    else
    {
        retval = ReactorReturnCodes.FAILURE;
    }

    break;
}
case JsonMsgClasses.PONG:
{
    failedToConvertJSONMsg = false;
    break;
}
case JsonMsgClasses.ERROR:
{
    /* xmlString is a StringBuilder object */
    xmlString.setLength(0);
    xmlDumpTrace.dumpBuffer(reactorChannel.channel(), Codec.JSON_PROTOCOL_TYPE,
        msgBuf, null, xmlString, errorInfo.error());
    jsonErrorMsg = xmlString.toString();

    populateErrorInfo(errorInfo, ReactorReturnCodes.FAILURE, "Reactor.performChannelRead",
        "Received JSON error message: " + jsonErrorMsg)

    failedToConvertJSONMsg = false;
    retval = ReactorReturnCodes.FAILURE;
    break;
}
}

if (retval != ReactorReturnCodes.SUCCESS)
    break;

```



```

        failedToConvertJSONMsg = true; /* Reset the flag to its initial state. */
    }
}

```

B.5 Converting from RWF to JSON

To convert an RWF message to JSON format, use the `JsonConverter.convertRWFToJson()` and `JsonConverter.getJsonBuffer()` methods.

B.5.1 Method `JsonConverter`

Call the `JsonConverter.convertRWFToJson()` method to transform a buffer containing an RWF message to JSON. After a successful conversion, you can then use the `JsonConverter.getJsonBuffer()` method to retrieve and modify the buffer.

- If `JsonConverter.convertRWFToJson()` succeeds, the method returns a `CodecReturnCodes.SUCCESS` value
- If `JsonConverter.convertRWFToJson()` fails, the method returns a `CodecReturnCodes.FAILURE` value.

PARAMETER	DESCRIPTION
<code>inMsg</code>	Specifies the message instance that contains the RWF for conversion to JSON.
<code>options</code>	<code>RWFToJsonOptions</code> instance that contains additional parameters which can be used during conversion
<code>outResults</code>	Optional. An instance of <code>ConversionResults</code> that contains output values obtained after conversion (e.g., the length of the obtained JSON message).
	NOTE: There is an overloaded version of the method that does not take this parameter.
<code>error</code>	A <code>JsonConverterError</code> instance that contains error information if the conversion fails.

Table 175: `JsonConverter.convertRWFToJson()` Method Parameters

B.5.2 `RWFToJsonOptions` Interface

The `RWFToJsonOptions` interface passes additional options to the converter when converting from RWF to JSON.

METHOD	DESCRIPTION
<code>getMajorVersion</code>	Gets the major version of the wire format to encode.
<code>setMajorVersion</code>	Sets the major version of the wire format to encode. <code>setMajorVersion</code> accepts an integer input value as a parameter.
<code>getMinorVersion</code>	Gets the minor version of the wire format to encode.
<code>setMinorVersion</code>	Setter for the minor version of the wire format to encode. <code>setMinorVersion</code> accepts an integer input value as a parameter.
<code>getJsonProtocolType</code>	Returns the JSON protocol set for the current options instance.
<code>setJsonProtocolType</code>	Sets the JSON protocol for the current instance of the options. The JSON protocol of the options must be equal to the JSON protocol supported by the current instance of the <code>JsonConverter</code> .
<code>getConverterFlags</code>	Returns flags to be passed to the converter.
<code>setConverterFlags</code>	Sets the Converter flags. <code>setConverterFlags</code> accepts an integer value as a parameter.

Table 176: `RWFToJsonOptions` Interface Methods

B.5.3 Method `JsonConverter.getJsonBuffer()`

Use the `JsonConverter.getJsonBuffer()` method to retrieve a converted buffer and set any needed modifications to facilitate fanout.

PARAMETER	DESCRIPTION
buffer	A Buffer / TransportBuffer instance that contains the resulting JSON message.
options	A GetJsonMsgOptions instance that contains additional parameters for use during buffer retrieval.
error	A JsonConverterError instance that contains error information if the retrieval fails.

Table 177: `JsonConverter.getJsonBuffer()` Method Parameters

B.5.4 `GetJsonMsgOptions` Interface

The `GetJsonMsgOptions` interface passes additional parameters to the converter when retrieving the buffer containing the converted JSON message from the converter.

METHOD	DESCRIPTION
clear	Resets all fields of the current instance to their default values.
streamId	Sets the value of the streamId , which is applied to the message being retrieved. streamId accepts an integer value as a parameter.
transportProtocolType	Sets the current protocol type. transportProtocolType accepts an integer value as a parameter.
jsonProtocolType	Sets the JSON protocol of the current options instance. jsonProtocolType accepts an integer value as a parameter.
isSolicited	Sets the value of the solicited flag. isSolicited accepts a boolean value as a parameter.
isCloseMsg	Sets the value of the flag that determines whether a message is a Close message. isCloseMsg accepts a boolean value as a parameter.
getStreamId	Gets the streamId which is applied to the current message.
getTransportProtocol	Gets the transport protocol value.
getJsonProtocolType	Gets the JSON protocol.
isCloseMsg	Gets the flag that determines whether the message should be treated as a close message.
isSolicited	Gets the solicited flag.

Table 178: `RWFToJsonOptions` Interface Methods

B.5.5 Example: Converting RWF to JSON

```
Msg _msg = CodecFactory.createMsg();
JsonConverterError converterError = ConverterFactory.createJsonConverterError();
RWFToJsonOptions rwfToJsonOptions = ConverterFactory.createRWFToJsonOptions();
ConversionResults conversionResults = ConverterFactory.createConversionResults();
GetJsonMsgOptions getJsonMsgOptions = ConverterFactory.createGetJsonMsgOptions();

_msg.clear();
_dIter.clear();
ret = _dIter.setBufferAndRWFVersion(buffer, reactorChannel.majorVersion(),
    reactorChannel.minorVersion());
```

```

ret = _msg.decode(_dIter);

if(ret == CodecReturnCodes.SUCCESS)
{
    converterError.clear();
    rwfToJsonOptions.clear();
    rwfToJsonOptions.setJsonProtocolType(JsonProtocol.JSON_JPT_JSON2);

    if( jsonConverter.convertRWFToJson(_msg, rwfToJsonOptions, conversionResults, converterError) !=
        CodecReturnCodes.SUCCESS)
    {
        return populateErrorInfo(errorInfo, ReactorReturnCodes.FAILURE,
            "Reactor.submitChannel", "Failed to convert RWF to JSON protocol. Error text: " +
            converterError.getText());
    }

    TransportBuffer jsonBuffer = reactorChannel.getBuffer(conversionResults.getLength(), false,
        errorInfo);

    if(Objects.isNull(jsonBuffer))
    {
        return populateErrorInfo(errorInfo, ReactorReturnCodes.FAILURE,
            "Reactor.submitChannel", "Failed to get a buffer for sending JSON message. Error text: "
            + errorInfo.error().text());
    }

    getJsonMsgOptions.clear();
    getJsonMsgOptions.jsonProtocolType(JsonProtocol.JSON_JPT_JSON2);
    getJsonMsgOptions.streamId(_msg.streamId());
    getJsonMsgOptions.isCloseMsg(_msg.msgClass() == MsgClasses.CLOSE ? true : false );

    if( jsonConverter.getJsonBuffer(jsonBuffer, getJsonMsgOptions, converterError) !=
        CodecReturnCodes.SUCCESS)
    {
        return populateErrorInfo(errorInfo, ReactorReturnCodes.FAILURE,
            "Reactor.submitChannel", "Failed to get converted JSON message. Error text: " +
            converterError.getText());
    }
}

```

© LSEG 2015 - 2026. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any third party names or marks are the trademarks or registered trademarks of the relevant third party.

Document ID: ETAJ3101L1UM.260

Date of issue: September 2026



LSEG DATA &
ANALYTICS